

SDVoE Solution

API Application Note Library

1	PURPOSE.....	10
2	SUMMARY OF NETWORK REQUIREMENTS.....	11
3	MANAGING THE API SERVER.....	12
3.1	DETECTION OF API SERVER	12
3.1.1	<i>Sample SDVoE Discovery Script.....</i>	<i>13</i>
3.2	SUMMARY OF EVENT NOTIFICATIONS	17
3.3	LOADING API COMMAND MODULE	18
4	MANAGING BLUERIVER DEVICE BANDWIDTH.....	20
4.1	CALCULATING BANDWIDTH	20
4.2	VIDEO PROFILES WITH COMPRESSION	20
4.3	REDUCING BANDWIDTH USAGE.....	20
4.3.1	<i>Compression Types.....</i>	<i>20</i>
5	MANAGING BLUERIVER DEVICE QUANTIZATION	22
5.1	SDVOE APPROACH.....	22
5.2	API RULES.....	23
5.2.1	<i>Input Range</i>	<i>23</i>
5.2.2	<i>API Control.....</i>	<i>23</i>
5.2.3	<i>Quantization in Multiview Mode.....</i>	<i>23</i>
5.3	MANAGING QUANTIZATION.....	25
5.3.1	<i>Managing Quantization Mode.....</i>	<i>25</i>
5.3.2	<i>Quantization Level Override</i>	<i>26</i>
5.4	QUANTIZATION MODE TIPS.....	27
5.4.1	<i>“Pluge” Pattern.....</i>	<i>28</i>
6	BLUERIVER DEVICE MULTIVIEW MODE.....	29
6.1	MULTIVIEW FUNCTION	29
6.2	MULTIVIEW LAYOUTS	30
6.3	MANAGING THE 10GBE LINK	31
6.3.1	<i>Multiview Example.....</i>	<i>31</i>
6.3.2	<i>Scaling & Frame Rate Conversion.....</i>	<i>32</i>
6.3.2.1	<i>Decoder (RX) Bandwidth & Scaling</i>	<i>32</i>
6.3.2.2	<i>Scaled Stream (Stream 1)</i>	<i>32</i>
6.3.2.3	<i>Encoder (TX) Bandwidth & Frame Rate Converter.....</i>	<i>33</i>
6.3.2.4	<i>Bandwidth Management Summary</i>	<i>33</i>
6.4	MULTIVIEW MODEL & TERMINOLOGY	34
6.4.1	<i>Streams & Subscriptions</i>	<i>34</i>
6.4.2	<i>Layout.....</i>	<i>34</i>
6.4.3	<i>Window.....</i>	<i>34</i>
6.4.4	<i>Surface.....</i>	<i>34</i>
6.4.5	<i>Tiles</i>	<i>34</i>
6.5	MULTIVIEW TERMINOLOGY.....	35
6.6	DEFAULT MULTIVIEW SURFACE CONFIGURATION.....	36
6.7	MULTIVIEW API REQUEST	37
6.7.1	<i>Handling the Multiview Mode</i>	<i>37</i>
6.7.2	<i>Retrieving Existing Layouts.....</i>	<i>38</i>
6.8	CREATING & MANAGING LAYOUTS	38

6.8.1	Creating Layouts.....	38
6.8.2	Adding Windows to Layout.....	42
6.8.3	Retrieving Details of Layout	45
6.8.3.1	Retrieve List Details of Available Layout from the API Server.....	45
6.8.3.2	Retrieving Details of Layout Currently Loaded on Decoder (RX) Device.....	50
6.8.3.3	Event MULTIVIEW_CHANGED	52
6.8.4	Deleting a Layout	52
6.8.5	Managing Multiview Streams	53
6.8.5.1	Managing Scaled Streams	53
6.8.5.2	Starting Streams	54
6.8.5.3	Stopping Streams	55
6.8.6	Setting Decoder to Multiview Mode	56
6.8.6.1	Managing Decoder Subscriptions.....	58
6.8.7	Predefined Compatibility Layout	59
6.8.8	Managing Surfaces	59
6.9	TASKS TO CREATE & LOAD LAYOUT SUMMARY.....	61
6.10	VIDEO SOURCES.....	63
6.10.1	Interlaced Video Sources	64
6.10.2	Chroma Sub-Sampled Sources (YCbCr 4:2:2 and YCbCr 4:2:0).....	64
6.10.3	10 or 12-bit Sources	64
6.11	MULTIVIEW USE CASE.....	64
6.11.1	Sample Multiview Scripts	64
6.11.2	Sample Use Case.....	65
6.12	MULTIVIEW GUIDELINES & LIMITATIONS.....	67
6.13	MULTIVIEW LAYOUT CHANGES.....	68
6.13.1	Changing Active Layout.....	70
6.13.2	Changing Video Mode	70
6.13.3	Achieving Seamless Transitions	70
6.13.3.1	Use Case 1: Surface Overlap.....	70
6.13.3.2	Use Case 2: Surface Reuse	73
6.13.3.3	Use Case 3: Repositioning Active Surfaces	75
6.13.3.4	Use Case 4: Maximum Layout Size	78
6.13.3.5	Use Case 5: Simple Layout Transitions	78
6.13.3.6	Use Case 5: Simultaneous Source Change	83
6.13.4	Transition Corner Cases & Limitations.....	87
6.13.4.1	Scaled Stream Reconfiguration (Stream 1).....	87
6.13.4.2	Layout Changes Requiring More Than 32 Surfaces.....	91
6.13.4.3	Layout Change with Window Reshuffling	91
7	BLUERIVER DEVICE VIDEO WALL MODE.....	95
7.1	INTRODUCTION TO VIDEO WALL MODE.....	95
7.1.1	Genlock Wall Mode.....	96
7.1.2	Fast Switch Wall Mode.....	97
7.1.3	Genlock & Fast Switch Wall Modes Summary	98
7.2	IMPLEMENTING SOURCE INDEPENDENT VIDEO WALL.....	98
7.2.1	Source Independent Video Wall Requirements	98
7.2.1.1	Calculating Bezel Compensation	100
7.2.1.2	Applying Wall Aspect Ratios.....	101
7.2.2	Additional Configuration of Source Independent Video Wall	102
7.2.3	Source Independent Video Wall API Request	103
7.2.4	Frame Buffer Node Related to SI Wall (AUTO_WALL)	105

7.3	CONFIGURING CUSTOM VIDEO WALL	107
7.3.1	Wall Mode Terms & Concepts	107
7.3.2	Configure Video Wall Tasks	108
7.3.3	Planning a Custom Video Wall	109
7.3.4	API Wall Parameters	109
7.3.4.1	Custom Wall API Set Video Requests	110
7.3.5	Managing Bezel Compensation	112
7.3.6	Managing Different Source & Wall Aspect Ratios	114
7.3.7	Stretch Source to Fit Wall Geometry	115
7.3.8	Crop Source Horizontally or Vertically to Match Wall Aspect Ratio	115
7.3.8.1	Tasks Involved to Crop Source	116
7.3.8.2	Wall Parameters & API Request	116
7.3.8.3	Wall Parameters Computation	117
7.3.8.4	Step 4: Apply Values to Decoders (RX) to Crop Image	119
7.3.9	Applying Viewport to Maintain Source Aspect Ratio	120
7.3.9.1	Viewport Option Overview	120
7.3.9.2	Calculating Wall Parameters	121
7.3.9.3	Wall Parameter Computations	125
7.3.9.4	Applying Viewport Values to API Request	128
7.3.10	Genlock Wall Case Studies	129
7.3.10.1	Case Study 1	129
7.3.10.2	Case Study 2	137
7.4	VIDEO WALL MODE SUMMARY	144
7.5	CREATING A VIDEO WALL WITH MULTIPLE SOURCES	145
7.5.1	Scenario 1: Multiview Layout as Wall Source	146
7.5.2	Scenario 2: Multiview Offset Argument Modified	146
8	SET PROPERTY COMMAND EXAMPLE USE CASES	148
8.1	DEVICE MODE PROPERTY	148
8.2	MANAGING DEVICE NAME	149
8.2.1	Resetting Device Name to MAC Address	151
8.3	MANAGING FRAME RATE DIVIDER	152
8.3.1	Enabling Frame Rate Divider on Native Video Stream	152
8.3.2	Disabling Frame Rate Divider on Native Video Stream	154
8.3.3	Enabling/Disabling Frame Rate Divider on Scaled Video Stream	154
8.4	COMPRESSION OF HDMI:0 ALWAYS ON	157
8.4.1	Verifying Status & Video Compression Version	157
8.5	REMOVING AUDIO FROM HDMI VIDEO STREAM	158
8.6	BLACK VIDEO GENERATOR	159
8.7	HDCP ENCRYPTION	160
8.8	BLUERIVER RECEIVER (HDMI TX 5V) API CONTROL	161
8.8.1	Overview of HDMI TX 5V Control	161
8.8.2	HDMI TX 5V Mirroring	162
9	BLUERIVER DEVICE AUDIO ROUTING	164
9.1	AUDIO ROUTING USE CASES	165
9.1.1	Routing Audio to HDMI Output	165
9.1.1.1	HDMI Audio Routing Use Case	166
9.1.2	Routing Audio to Stereo Output	167
9.1.2.1	Stereo Audio Output Use Case	168
9.1.3	Routing Audio to Multichannel Output	170

9.1.3.1	Multichannel Output Use Case	170
10	SENDING WAKE-ON-LAN PACKET VIA BLUERIVER DEVICES	172
10.1	WAKE-ON-LAN STANDARD OVERVIEW	172
10.1.1	How to Enable Wake-On-LAN on Your Computer.....	172
10.1.2	Magic Packet.....	172
10.1.3	SecureOn™ Password.....	172
10.1.4	WoL Transport	172
10.2	SENDING WOL PACKET VIA AVP FROM API COMMAND	173
11	SECURING CONTROL COMMUNICATION	175
11.1	SECURE CONTROL COMMUNICATION OVERVIEW	175
11.2	INTRODUCTION TO API AUTHENTICATION & ENCRYPTION	176
11.3	KEY EXCHANGES.....	176
11.4	API SERVER CONFIGURATION	177
11.5	AUTH / UNAUTH API REQUESTS	178
11.5.1	API Configuration Node.....	179
11.6	AUTHENTICATION USE CASES	180
11.6.1	Single API Server Deployment	180
11.6.2	Primary-Backup API Server Deployment	181
11.6.3	Segmented Deployment with Shared Authentication	182
11.6.3.1	Claim Use Case	183
11.6.3.2	API Configuration Node.....	184
11.6.3.3	Managing Multicast IP Range for Multiple API Servers.....	185
12	BLUERIVER DEVICE USB-HID ENCRYPTION.....	187
12.1	ACTIVATE USB HID ENCRYPTION TASKS.....	187
12.2	CONFIGURING USB HID ENCRYPTION KEY.....	187
12.3	CONFIGURING USB HID SECURITY LEVEL.....	188
12.3.1	API USB HID Node	190
13	MANAGING BLUERIVER DEVICE'S 802.1X	192
13.1	OVERVIEW.....	192
13.2	CERTIFICATE REQUIREMENTS.....	193
13.3	UPLOADING DEVICE CERTIFICATE.....	194
13.4	READING BACK DEVICE CERTIFICATE.....	195
14	BLUERIVER ERROR CODES	198
15	MANAGING SEMTECH UEX OR ICRON USB 2.0 EXTENDER	199
15.1	SETTING USB 2.0 EXTENDER ROLE	199
15.2	PAIRING & UNPAIRING USB EXTENDERS	200
15.2.1	Simple Pairing using "switch"	200
15.2.1.1	Using "switch" Requests.....	200
15.2.1.2	Using "stop" Request.....	202
15.2.2	Simultaneous User Interaction (SUI) Pairing	203
15.2.2.1	SUI Guidelines & Limitations	203
15.2.3	Pairing using "pair"	204
15.3	USB_EXTENDER STATUS NODE	206
15.4	USB EXTENDER NODE NETWORK CONFIGURATION	209

15.5	USB EXTENDER NODE SETTINGS FILTER	211
15.6	RECOVERY OF UNREACHABLE USB 2.0 EXTENDERS	213
15.6.1	Visibility of USB 2.0 Extenders IP Address	214
15.6.2	Resetting an Unreachable Icron Module	214
16	BLUERIVER NT1000/NT2000 CHIPSET FEATURES.....	216
16.1	NT2000 BITMAP OVERLAY FEATURE.....	216
16.1.1	Bitmap Overlay Guidelines & Limitations	216
16.1.2	Generating Bitmap Overlay	217
16.1.3	API Bitmap Requests	218
16.1.3.1	Applying set overlay.....	218
16.1.3.2	BITMAP_OVERLAY Node.....	221
16.2	THUMBNAIL PREVIEW.....	221
16.2.1	Important Customer Notice	222
16.2.2	Thumbnail Configuration & Setup	222
16.2.3	GStreamer Installation & Configuration	223
16.2.3.1	Thumbnail Stream Video Resolutions	225
16.2.3.2	Configuring Thumbnail Generator	226
16.2.3.3	HDCP Content	227
16.2.3.4	Thumbnail Generator Node	228
16.2.4	Viewing Thumbnail Preview Streams	230
16.2.4.1	Unicast Thumbnail	230
16.2.4.2	Multicast Thumbnail	230
16.2.4.3	Video Scaling	230
16.2.5	Thumbnail Script & Executable Demo	231
16.2.5.1	Using Thumbnail Script & Executable	231
16.2.5.2	Guidelines & Limitations	232
16.2.6	Managing Format Changes	233
16.2.6.1	General Management	233
16.2.6.2	Using GStreamer as Viewer	234
16.2.6.3	Compile Custom GStreamer Pipeline	235
17	REVISION HISTORY	236

List of Figures

Figure 1: Quantization in Multiview Mode.....	24
Figure 2: Quantization Management.....	27
Figure 3: Multiview Receiver Processing.....	29
Figure 4: Example Multiview Layouts.....	30
Figure 5: Multiview Example Use Case with 3 Sources and 2 Layouts.....	31
Figure 6: Two-Source Multiview Layout.....	31
Figure 7: Two Video Source Layout – Illustrates Window Overlap in 5x5 tile grid (25 Tiles).....	35
Figure 8: Illustration of Default Frame Buffer Surface Configuration	36
Figure 9: Example of Surface Assignments.....	37
Figure 10: Example 4x4 Multiview Layout with PiP Overlay	69
Figure 11: Multiview Transitions Use Case 1 – Single Layout and Surface Usage	71
Figure 12: Multiview Transitions Use Case 1 – Quad Layout and Surface Usage	72
Figure 13: Multiview Transitions Use Case 1 – Illustration of Memory Allocation Overlap	73
Figure 14: Multiview Transitions Use Case 2 – Illustration of Memory Allocation of Current Layout.....	73
Figure 15: Multiview Transitions Use Case 2 – Illustration of Memory Allocation of Quad Layout	74
Figure 16: Multiview Transitions Use Case 2 – Illustration of Memory Allocation During Transition	75
Figure 17: Multiview Transitions Use Case 3 – Illustration of Memory Allocation of Current Layout.....	76
Figure 18: Multiview Transitions Use Case 3 – Illustration of Memory Allocation of New Layout.....	77
Figure 19: Multiview Transitions Use Case 3 – Illustration of Memory Reallocation.....	78
Figure 20: Multiview Transitions Use Case 4 – Current Memory Allocation of Single Layout.....	79
Figure 21: Multiview Transitions Use Case 4 – Memory Allocation of PiP Layout.....	80
Figure 22: Multiview Transitions Use Case 4 – Memory Allocation of PiPCorner Layout	81
Figure 23: Multiview Transitions Use Case 4 – Memory Allocation of Quad5 Layout	82
Figure 24: Multiview Transitions Use Case 4 – Memory Allocation of PaP Layout.....	83
Figure 25: Multiview Transitions Use Case 5 – Memory Allocation of Initial Quad Layout.....	85
Figure 26: Multiview Transitions Use Case 5 – Memory Allocation of Quad7 Layout	86
Figure 27: Multiview Transitions Use Case 5 – Memory Allocation of Quad8 Layout	87
Figure 28: Illustration of Solution 1	88
Figure 29: Illustration Initial Layout of Solution 2.....	90
Figure 30: Illustration of Second Layout Configuration for Solution 2	91
Figure 31: Illustration of Window Reshuffling.....	92
Figure 32: Window Repositioning Solution – Memory Allocation of Current Layout.....	93
Figure 33: Window Repositioning Solution – Memory Allocation of New Layout.....	94
Figure 34: Block View of Decoder Processing.....	96
Figure 35: Illustration of Grid Indexes for 3x2 Video Wall	100
Figure 36: Original 16:9 Source Example	101
Figure 37: SI Wall KEEP Illustration.....	101
Figure 38: SI Wall STRETCH Illustration	101
Figure 39: SI Wall CROP Illustration	102
Figure 40: Illustration of Wall Tearing	108
Figure 41: "Per Row Delay" to Reduce Wall Tearing	108
Figure 42: Example of 2x2 Wall Grid.....	109
Figure 43: Illustration of <code>set video</code> Arguments Applied to RX4 of 2x2 Wall.....	111
Figure 44: Example Without Bezel Compensation.....	112
Figure 45: Example with Bezel Compensation.....	112
Figure 46: Example of Bezel Borders to Consider for 3x3 Wall Grid	113

Figure 47: 2x2 Example of Rectangular Wall Grid with Bezel Compensation Applied	113
Figure 48: Example with AOI Size and x/y Coordinates Applied	114
Figure 49: Example of Source Different than Wall Aspect Ratio	114
Figure 50: Horizontal Example of Stretching Source to Fit Wall	115
Figure 51: Example of Cropping the Source	115
Figure 52: Example of Viewport Option Applied.....	120
Figure 53: Viewport Regions Illustrated for Example 3x2 16:9 Wall Grid	120
Figure 54: Viewport Wall Size	122
Figure 55: Illustration of Location for Viewport Parameters.....	123
Figure 56: Illustration of Addition Viewport Parameters.....	123
Figure 57: Individual Viewport Areas Highlighted for Sample 3x2 Wall Grid	124
Figure 58: Example of Decoder 1 (RX1) Viewport and AOI	124
Figure 59: Example of Decoder 2 (RX2) Viewport and AOI	124
Figure 60: Example of Decoder 3 (RX3) Viewport and AOI	125
Figure 61: 3x2 Wall Grid 16:9 Source Shown with Viewport Applied	145
Figure 62: Example of 2x2 Wall Grid and Two Source Layout.....	145
Figure 63: Scenario 1 Multiview Layout as Wall Source	146
Figure 64: Scenario 2 Multiview Layouts Applied	147
Figure 65: Block Diagram of Encoder (TX) Illustrating Frame Division	152
Figure 66: Flow Chart of HDMI TX 5V Behavior	163
Figure 67: Available Audio Routing for AVP2000T	164
Figure 68: Communication Between API Server and Bound-Authenticated Endpoint Device.....	177
Figure 69: Endpoint Device Authentication.....	177
Figure 70: Network Setup with Single API Server Instance.....	181
Figure 71: Use Case – Backup API Server.....	182
Figure 72: Example of API Server/Endpoint Device Segmentation using API Claim Feature	183
Figure 73: Example Leaf Certificate Structure	193
Figure 74: GStreamer System Variable	224
Figure 75: Example of Verifying GStreamer	224
Figure 76: Example of Thumbnail Preview.....	232
Figure 77: Thumbnail Resolution Change	233
Figure 78: Video Size Changes with GStreamer.....	234

List of Tables

Table 1: Multiview Command List.....	37
Table 2: Table Describing the Layout Description Object.....	45
Table 3: Table Describing the Surface Description Object.....	46
Table 4: Table Describing the Window Description Object.....	46
Table 5: Wall Mode Terms and Concepts	107
Table 6: set video Arguments Applicable to Wall Mode.....	110
Table 7: Definition of Parameters Used in Scaling Wall Calculations.....	117
Table 8: Definition of Parameters Used in Wall Calculations	122
Table 9: Values to Calculate for Area of Interest.....	127
Table 10: Valid Device Mode Changes.....	148
Table 11: RFC3986 Reserved Characters.....	150
Table 12: Summary of Audio Output Routing.....	165
Table 13: Terms and Definitions	175
Table 14: 802.1X Feature List.....	192
Table 15: Error Codes.....	198
Table 16: Local USB Extender Status Node Object Members.....	207
Table 17: BITMAP_OVERLAY Node Configuration Members	221
Table 18: Thumbnail Stream Resolutions and Thumbnail Resolution Output	225
Table 19: Thumbnail Generator Node Object Members	228
Table 20: Thumbnail Generator Node Status Members.....	229

1 Purpose

The following application library provides a series of application notes and guidelines related to the SDVoE API; hereafter referred to as “API”.

The target group is software designers, project managers and support teams involved in the design, coding and/or deployment of applications that are to manage devices implementing SDVoE technology.

Suggested documentation and resources include:

Document		Description
Developers Introduction to SDVoE API User Guide	PDS-062488	Introduction to the fundamentals of the API, including review of command return and error structure, device model and video output modes.
SDVoE Developers API Reference Guide	PDS-062489	Outlines API command set, including syntax, arguments, and examples, as well covers device nodes and configurable properties.
SDVoE Control Server Administrators Guide	PDS-062492	Provides details regarding managing the API server configuration.
SDVoE API Application Note Library	PDS-062491	Provides information and use cases on implementing advanced functions, such as wall and multiview modes, as well as managing device properties.
SDVoE Control Server (API)	PRODDOC021754	API executables.

2 Summary of Network Requirements

The 10GbE network switch used with SDVoE endpoints must meet the following requirements:

- Network switch should be a Layer 3 switch (IGMP version 2 built-in) or Layer 2 switch with IGMP version 2.
- IGMP snooping must be enabled.
- Multicast must be supported and enabled on the switch.
- Filter/Drop unregistered multicast traffic also required.
- In addition, Fast Leave on the ports and/or VLANs in use by SDVoE endpoint devices should be enabled.
 - Sometimes Fast Leave is called “Immediate Leave”.

TIP: For Layer 2 switches issues have been reported with them not supporting the Filter/Drop of unregistered Multicast traffic.

For suggested SDVoE network and security guidelines refer to the following documents:

- SDVoE Security Guidelines
- SDVoE Network Guide

3 Managing the API Server

Details of configuring the SDVoE Control Server (API) are outlined in the SDVoE Control Server Administrators Guide, below are some tips regarding the API, architecture and general information related to the API is outlined in the Developers Introduction to the SDVoE API, while the supported API commands are outlined in the SDVoE Developers API Reference Guide.

3.1 Detection of API Server

Starting in API version 3.6 the ability for clients to discover API instances running on the network was added through the ability to send multicast discovery packets using a standard discovery method. This is applicable to both control server API instances and local (embedded) API instances.

This feature uses simple Service Detection Protocol (SSDP) for which the following applies:

- Does not require assistance of DHCP or DNS.
- Intended for use in residential or small office environments.
- Feature based on HTTP over Multicast UDP (HTTPMU), HTTP over unicast UDP (HTTPU) and JavaScript Object Notation (JSON).
- Messages used: NOTIFY (NOTIFY * HTTP/1.1\r\n) and M-SEARCH (M-SEARCH * HTTP/1.1\r\n)

The API is presented as one REST API service (`restapi`).

To retrieve the service description the client issues an HTTP GET request on the location URL found in the 200 OK response to M-SEARCH. The API returns the service description. Service descriptions are templated with JSON syntax.

With SSDP, the API sends periodic advertisements for its service. Each advertisement has a Notification Type (NT) and a Unique Service Name (USN).

Example of NT and USN for an advertisement:

Header	Value
NT	urn:semtech.com:service:restapi:3.6
UDN	uuid:<device-UUID>::urn:semtech.com:service:restapi:3.6

Example of UUID: 123e4567-e89b-12d3-a456-426614174000

Regarding the UUID:

- The UUID value is associated to the server host where the API instance is running.
- The remainder of the string after "::-:" is the USN (service type).
- When the version changes the USN changes, but the UUID part remains the same.
- The USN does not identify a specific instance of the API server but the service type.

Regarding M-SEARCH:

- If a service type is requested with `urn:semtech.com:service:restapi:3.6`, all API (controlserver) instances running version 3.6 or higher respond.
- The UUID may seem redundant regarding the return provided after "::-:". However, the UUID is used to make the reference to a server host unambiguously.

3.1.1 Sample SDVoE Discovery Script

The following is an example of a python script that can detect all the APIs detectable on the network:

```
# This script allows discovery of all instances of controlserver
# on the network supporting the SSDP protocol.
# This script searches on all the available network interfaces on
# the local host. A controlserver will be visible if it is
# connected to one of our networks, and its client_address
# parameter is set to 0.0.0.0. A controlserver may not be
# visible, even if it is physically connected to one of our
# networks, if its client_address parameter is not set to 0.0.0.0
# or to local address of its interface connected to our network.
#
# It is possible to pass a controlserver version as an argument,
# otherwise the default version requested will be 3.3.0.0. All
# controlserver instances supporting SSDP with a version above
# 3.3.0.0 will answer. The first API version supporting SSDP is
# 3.6.0.0. The goal of using 3.3.0.0 as default was to
# demonstrate that all versions above it would answer. Examples:
#     python3 ssdp.py
#     python3 ssdp.py 3.6.0.0
#
# DISCLAIMER:
# This script was coded and tested on Linux only. It has not been
# verified on Windows.

import platform
import socket
import sys
import psutil
import struct

DISCOVERY_GROUP = '224.0.1.188'
DISCOVERY_PORT = 1900
TIMEOUT = 1
PRODUCT_NAME = 'MyProductName'
PRODUCT_VER = '1.1'
API_VERSION = '3.3.0.0'
TARGET = 'urn:semtech.com:service:restapi:'

def get_api_version():
    global API_VERSION
    if len(sys.argv) > 1:
        version = sys.argv[1]
        parts = version.split('.')
        if len(parts) != 4:
            sys.exit('Invalid API version, not of the form
1.2.3.4')
        for part in parts:
            if not part.isdigit():
                sys.exit('Invalid API version, all parts must be
```

```

decimal digits')
    API_VERSION = version
    print('API version: ', API_VERSION)

def get_interface_list():
    iface_list = []
    addrs = psutil.net_if_addrs()
    interfaces_info = psutil.net_if_stats()
    for name in addrs.keys():
        addr_list = addrs[name]
        for addr in addr_list:
            if addr.family == 2: # AF_INET
                if interfaces_info[name].isup:
                    iface_list.append(addr.address)
                break
    return iface_list

def receive_init(iface_list):
    sock_list = []
    for iface in iface_list:
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM,
socket.IPPROTO_UDP)
        sock.settimeout(TIMEOUT)
        sock.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
        sock.bind((iface, DISCOVERY_PORT))
        sock_list.append(sock)
    return sock_list

def ssdp_rcv(sock):
    try:
        while True:
            data = sock.recv(1024)
            yield data
    except socket.timeout:
        pass
    return

def parse_headers(response, convert_to_lowercase=True):
    """
    Receives an HTTP response/request bytes object and parses the
    HTTP headers.
    Return a dict of all headers.
    If convert_to_lowercase is true, all headers will be saved in
    lowercase form.
    """
    valid_headers = (
        b"HTTP/1.1 200 OK\r\n",
    )
    headers = {}

```

```

        if not any([response.startswith(x) for x in valid_headers]):
            raise ValueError(
                "Invalid header: Should start with one of:
{}".format(valid_headers)
            )

        lines = response.split(b"\r\n")
        # Skip the first line since it's just the HTTP return code
        for line in lines[1:]:
            if not line:
                break # Headers and content are separated by a blank
line
            if b":" not in line:
                raise ValueError("Invalid header: {}".format(line))
            header_name, header_value = line.split(b":", 1)
            headers[header_name.decode("utf-8").lower().strip()] =
header_value.decode(
                "utf-8"
            ).strip()
        return headers

def create_msearch_payload(host, st, mx=1):
    """
    Create an M-SEARCH packet using the given parameters.
    :return: A bytes object containing the generated M-SEARCH
payload.
    """
    data = (
        "M-SEARCH * HTTP/1.1\r\n"
        "HOST:{}\r\n"
        'MAN: "ssdp:discover"\r\n'
        "MX:{}\r\n"
        "ST:{}\r\n"
        "USER-AGENT: {}/{ } { }/{ }"
        "\r\n"
    ).format(
        host,
        mx,
        st + API_VERSION,
        platform.system(),
        platform.release(),
        PRODUCT_NAME,
        PRODUCT_VER)
    return data.encode("utf-8")

def m_search(iface_list, sock_list):
    """
    Send an M-SEARCH request and gather responses.
    :return: A list of all discovered SSDP services. Each service
is represented by a dict, with the keys being the lowercase
equivalents of the response headers.

```

```

"""
    send_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM,
socket.IPPROTO_UDP)
    send_sock.setsockopt(socket.IPPROTO_IP,
socket.IP_MULTICAST_TTL, 1)
    send_sock.settimeout(TIMEOUT)

    host = "{}:{}".format(DISCOVERY_GROUP, DISCOVERY_PORT)
    data = create_msearch_payload(host, TARGET, 1)

    # Send M-SEARCH on all the interfaces
    address = (DISCOVERY_GROUP, DISCOVERY_PORT)
    for iface in iface_list:
        send_sock.setsockopt(socket.IPPROTO_IP,
socket.IP_MULTICAST_IF, socket.inet_aton(iface))
        mreq = struct.pack("4s4s",
socket.inet_aton(DISCOVERY_GROUP), socket.inet_aton(iface))
        send_sock.setsockopt(socket.IPPROTO_IP,
socket.IP_ADD_MEMBERSHIP, mreq)
        send_sock.sendto(data, address)
    parsed_responses = []
    for sock in sock_list:
        responses = [x for x in ssdp_recv(sock)]
        for response in responses:
            try:
                headers = parse_headers(response)
                parsed_responses.append(headers)
            except ValueError:
                # Invalid response, do nothing.
                # TODO: Log dropped responses
                pass
    return parsed_responses

def main():
    get_api_version()

    print('Enumerating all the interfaces:')
    iface_list = get_interface_list()
    for iface in iface_list:
        print(iface)

    sock_list = receive_init(iface_list)
    responses = m_search(iface_list, sock_list)
    for resp in responses:
        print("USN: {}, ST: {}, Location:
{}".format( resp['usn'], resp['st'], resp['location']))
    return 0

if __name__ == "__main__":
    sys.exit(main())

```

3.2 Summary of Event Notifications

For clients to receive asynchronous notifications for events and command requests a WebSocket connection required.

Reminder: On a WebSocket connection, notifications are enabled by default. Details available in the Developers Introduction to SDVoE API User Guide and SDVoE Control Server Administrators Guide.

Alternatively, it is possible to poll for events using the `event` command. Events are assigned an “`event type`” and most have further associated data available. Events with more data available are assigned a “`request ID`” that is then used with the `request` command to retrieve additional data.

As reviewed in the Developers Introduction to SDVoE API User Guide, when a device is discovered by the API, an asynchronous event notification `DEVICE_CONNECTED` is generated. In addition, a `DEVICE_INFO_AVAILABLE` event is also generated notifying of the availability of detailed information about the discovered device, such as if it is a receiver or transmitter, firmware version and the chipset type the board is built-on.

Example of events generated when a device is discovered by the API:

```
{
  "status" : "NOTIFICATION",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "801f124307b7",
        "event_id" : 1,
        "event_type" : "DEVICE_CONNECTED",
        "timestamp" : 1601042767,
        "request_id" : 1
      },
      {
        "device_id" : "801f124307b7",
        "event_id" : 2,
        "event_type" : "DEVICE_INFO_AVAILABLE",
        "timestamp" : 1601042767,
        "request_id" : 2
      }
    ],
    "error" : null
  }
}
```

3.3 Loading API Command Module

With a REST API, each request sent from a client is processed with the version it was compiled for, such as 3.4.0.0. Normally this is satisfactory.

However, if it is desired that a request be processed using a specific version of API, i.e. an earlier version of API server, then it is necessary to issue a `require` command specifying the API version before the command request is sent.

Reminder: This is not a common requirement as REST API is typically backwards compatible.

A `require` request is used to specify a specific API version:

```
GET /api?require(module_name)=version
module_name = requested module.
version = version number
```

TIP: To verify the version of module to load, use a `GET /api` request.

Example below loads the API command module 3.4.0.0.

```
GET /api?require(api)=3.4.0.0
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "server" : "Control Server",
    "vendor" : "Semtech",
    "version" : "3.4.0.0",
    "modules" : [
      {
        "name" : "api",
        "version" : "3.6.0.0"
      }
    ]
  },
  "error" : null
}
```

Here is the return example of a request sent for an invalid module:

```
GET /api?require(unsupported)=1.2.3.4
...
{
  "status" : "ERROR",
  "request_id" : null,
  "result" : null,
  "error" : {
    "reason" : "UNSUPPORTED_FEATURE",
    "message" : "Unknown module 'unsupported' in command 'require' URI query argument"
  }
}
```

This example is of a request for an unsupported version of API module:

```
GET /api?require(blueriver_api)=4.9999.0.0
...
{
  "status" : "ERROR",
  "request_id" : null,
  "result" : null,
  "error" : {
    "reason" : "VERSION",
    "message" : "Incompatible version for module api"
  }
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

4 Managing BlueRiver Device Bandwidth

4.1 Calculating Bandwidth

A 10G Bandwidth Usage Calculator is available for download. This tool can be used to calculate the theoretical bandwidth of a transmitter or transceiver endpoint device. This calculator is applicable to all SDVoE endpoint devices, regardless of whether they are based on the BlueRiver AVP or NT1000/NT2000 chipset series.

This tool is useful to network administrators, system integrators as well as others who often need an idea on the estimated network usage of the video and audio streams, as well as auxiliary data being transmitted to allow for better planning and management of resources.

4.2 Video Profiles with Compression

Lightweight Video compression is automatically applied to HDMI streams that surpass 10Gbps bandwidth usage including the following video profiles:

- 4K60 8-bit RGB and YCbCr 4:4:4
- 4K60 10-bit YCbCr 4:2:2
- 4K60 12-bit YCbCr 4:2:2
- 4K60 12-bit YCbCr 4:2:0
- 4K30 12-bit RGB and YCbCr 4:4:4

When an BlueRiver ASIC endpoint device is operating at 2.5GBaseT or 5GBaseT link rates, compression is always enabled for all video formats.

4.3 Reducing Bandwidth Usage

To help reduce overall bandwidth usage, light compression can be applied to all HDMI video signals.

- Refer to section 8.4 Compression of HDMI:0 Always On for an example of applying compression to the Native stream (stream 0).

Reminder, a compressed video source cannot be used by decoders (RX) operating in multiview mode. However, frame rate conversion can be applied on the encoder (TX) side. Refer to section 8.3 Managing Frame Rate Divider for information on applying this property.

- It is also possible to remove the HDMI audio from the native HDMI video stream. Refer to section 8.5 Removing Audio from HDMI Video Stream.

TIP: For additional information on these and other configurable properties, refer to the SDVoE Developers API Reference Guide, section `Command set property`.

4.3.1 Compression Types

The BlueRiver chipsets support two types of compression:

Compression Version 2	Recommended. Select for best quality in all situations, the proprietary light compression engine was improved.
Compression Version 1	Original compression version, apply if backwards compatibility is of concern. Applicable generally only for NT1000/NT2000 chipset-based devices running API version 2.18 or earlier.

When the factory default settings are configured one of these two compression types is applied as the default. The API supports changing the compression type through an API request.

Automatic decoding of both compression versions 1 and 2 was added in ASIC firmware 1.5.0.1. Requires both the transmitter (TX) and receiver (RX) devices to be updated to firmware 1.5 or later. The transmitter (TX) flags which compression version is being used and the receiver (RX) automatically decodes the flagged version.

Warning! Decoder (RX) devices can subscribe only to HDMI:0 encoder (TX) streams which are of the same compression type.

As an example, if an encoder (TX) is set to Compression Version 2, for a decoder (RX) to successfully subscribe to the native video stream (stream 0) of that encoder it too must be configured to operate in Compression Version 2. If not set to the same compression version, then the output quality is impacted and macroblocking likely to occur.

Reminder: For details on enabling compression to always be applied regardless of the video format, refer to section 8.4 Compression of HDMI:0 Always On.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

5 Managing BlueRiver Device Quantization

RGB quantization range has three possible corresponding AVI InfoFrame values:

DEFAULT
LIMITED
FULL

Whereas an YCbCr AVI InfoFrame has two possible values:

LIMITED
FULL

This adds complexity when managing RGB quantization.

Most displays look at the AVI InfoFrame (LIMITED or FULL) and when set to DEFAULT make a decision based on the CEA-861F (video (CE) or graphics (IT) resolution). This is the preferred conduct and does meet the standard-compliance behavior.

However, as has been seen in the field, some displays can tend to be inconsistent in managing quantization and even show behaviors that are not fully standard compliant. Behaviors that have been seen include:

- i. Some displays consider only the resolution it receives and make decisions accordingly.
- ii. Some displays always assume FULL.
- iii. Some displays ignore the resolution and only look at the AVI InfoFrame (LIMITED or FULL) and assume LIMITED when AVI InfoFrame is set to DEFAULT.
- iv. Some displays ignore the resolution and only look at AVI InfoFrame (LIMITED or FULL) and assume FULL when the AVI InfoFrame is set to DEFAULT.

Due to these potential behaviors, no matter the range implemented by default, it is necessary to implement additional API control to deal with displays not fully complying with the CEA-861F standard.

5.1 SDVoE Approach

When a decoder (RX) is operating in Fast Switch, Genlock Scaling or Wall processing modes, the quantization range is programmed according to a series of rules. This section outlines the behavior of quantization in relation to the SDVoE endpoint devices.

Color ranges:

Limited Range = 16 – 235

Full Range = 0 – 255

Definition of quantization modes (CEA-861-F standard) in relation to the API:

STANDARD

The output quantization range is set according to the video format (as per CEA-861-F standard section 5.1). The AVI InfoFrame is set to DEFAULT (0) as per the same standard.

AUTO

The output quantization range is set according to video format (as per CEA-861-F standard section 5.1) but the AVI InfoFrame is set explicitly to LIMITED (1) or FULL (2) depending on the output quantization for this output format.

LIMITED

The output quantization range and the AVI InfoFrame are always set to LIMITED (1) .

FULL

The output quantization range and the AVI InfoFrame are always set to FULL (2) .

5.2 API Rules

5.2.1 Input Range

In all cases the following applies:

- i. When a source reports FULL or LIMITED, the output of SDVoE endpoint follows the source.
- ii. When a source reports DEFAULT and is an RGB source then:
 - a. Assume LIMITED if the input resolution is a video (CE) resolution.
 - b. Assume FULL if the input resolution is graphics (IT) resolution.
- iii. Above is not applicable to the multiview mode. Refer to section 5.2.3 Quantization in Multiview Mode.
- iv. If the source is DVI, the input format range is always assumed to be RGB Full.

5.2.2 API Control

When the API quantization option is set to:

STANDARD (default option) the behavior is as described above in section 5.1, specifically:

- i. AVI InfoFrame set to DEFAULT.
- ii. If output resolution is a video resolution (CE), then the range set to LIMITED.
- iii. If output resolution is a graphics resolution (IT), then the range is FULL.

In regards to AUTO, again the behavior is as described above but to be specific the following occurs:

- i. If the output resolution is a video resolution (CE), the output range is set to LIMITED and the AVI InfoFrame is also set to LIMITED.
- ii. If the output resolution is a graphics resolution (IT), then the output range is set to FULL and the AVI InfoFrame is set to FULL.

When LIMITED or FULL range are applied, they behave as outlined above in section 5.1 SDVoE Approach, that is range and AVI InfoFrame set to LIMITED or FULL respectively.

5.2.3 Quantization in Multiview Mode

When the decoder (RX) is operating in multiview mode, this is a special case where the decoder can receive up to 32 video streams and each could have a different quantization range.

The HDMI Source of an encoder (TX) device can be BT.2020 or BT.709, HDR or SDR, RGB or YCbCr, Limited or Full.

Note: If the output of the scaled stream (stream 1) is required to be FULL range (versus LIMITED) it is necessary that the source stream be configured as FULL range and not set to DEFAULT. This is because the BlueRiver AVP chipset doesn't convert the original source from LIMITED to FULL when in multiview mode.

TIP: Sources can be forced to output FULL range through the use of the EDID file presented to it.

The scaled stream (stream 1) is then output to the subscribed decoder (RX) device(s) where it is composited into the layout and scaled according to the output resolution. These settings are managed through an API video `set` request, refer to the SDVoE Developers API Reference Guide for details.

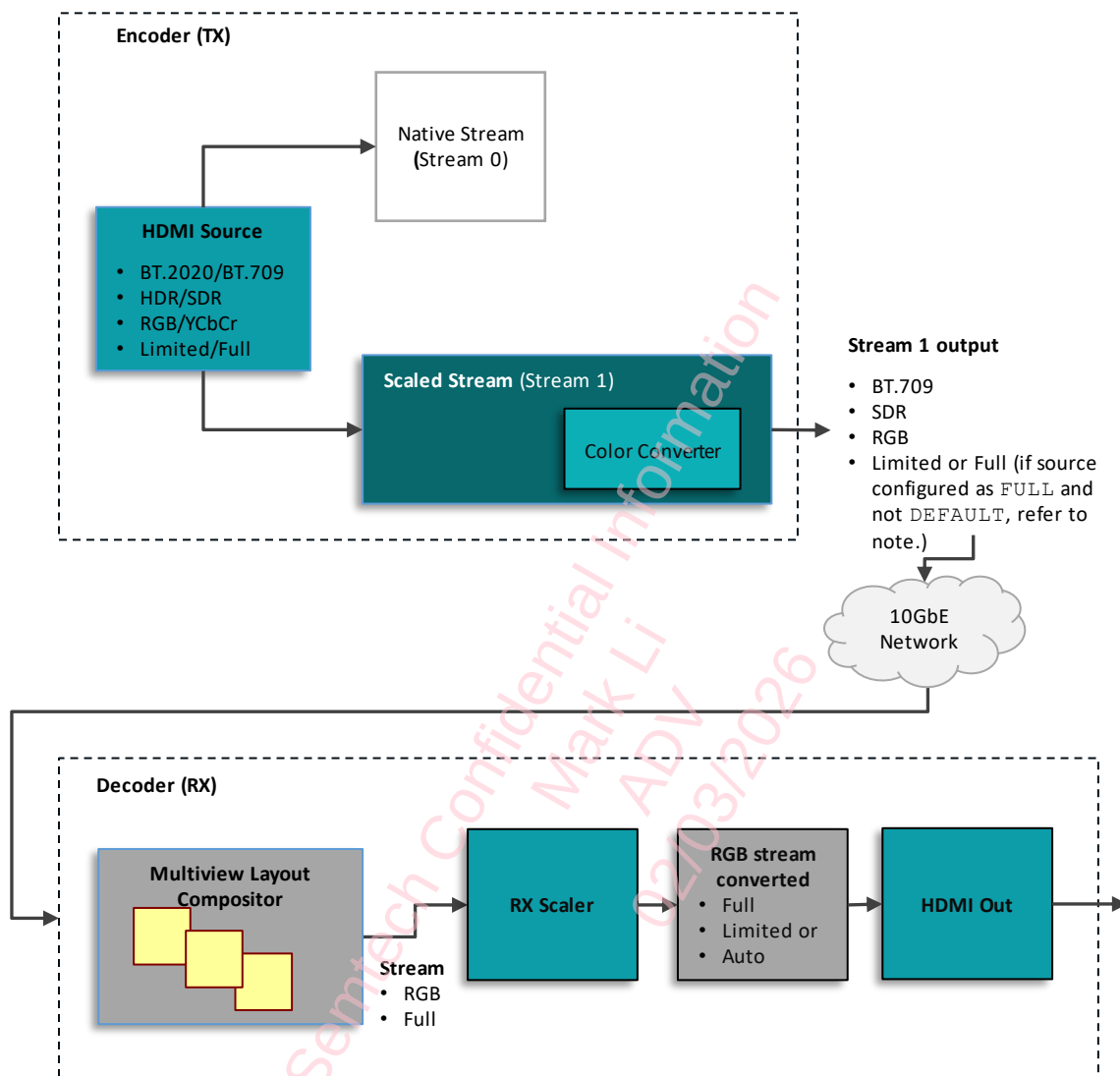


Figure 1: Quantization in Multiview Mode

5.3 Managing Quantization

SDVoE allows for management of quantization, this section reviews the API settings that are provided.

5.3.1 Managing Quantization Mode

The quantization range is managed by applying the video format argument, `quantization_range` to a `set video` request when specifying the decoder (RX) video mode.

Command syntax:

```
POST /api/device/{target}
{
  "op" : "set:video",
  "display_mode" : String,
  "video" : VideoFormatArgument (conditional),
}
```

The "quantization_range" option is part of the video arguments (VideoFormatArgument) and the supported modes are AUTO, FULL, LIMITED and STANDARD. If not specified, the behavior is same as if STANDARD was specified. Refer to the SDVoE Developers API Reference Guide for more information than provided in this application note.

The first example shown here sets a receiver in Fast Switch mode, with an output resolution of 1920x1080@60 and the quantization range to FULL.

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "quantization_range" : "FULL"
  }
}
```

The second example shown here sets a receiver in Fast Switch mode, also with an output resolution of 1920x1080@60 but the quantization range is set to AUTO.

```
POST /api/device/010203040506 HTTP/1.1
...
{
  "op" : "set:video",
  "display_mode" : "FAST_SWITCHED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "quantization_range" : "AUTO"
  }
}
```

5.3.2 Quantization Level Override

In some circumstances, the Quantization level for HDMI sources is wrongly set in the AVI InfoFrame. Issue related to DP to HDMI conversion chips commonly used in popular cables and adapters.

An override is supported for these specific use cases, which can apply an InfoFrame correction on HDMI output of receiver (RX) devices to reflect the real quantization level of the source. The override is applied only when quantization is not specified in InfoFrame (default value).

A `set property` request is used to override the quantization level with the following key:

```
nodes[HDMI_DECODER:0].configuration.default_quantization_level_override
```

The supported values are:

`disabled` No default quantization level override applied, the default (0) is kept.
`full` Default quantization level override set to FULL range.
`limited` Default quantization level override set to LIMITED range.

The syntax for a `set property` command is as follows:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : "String",
  "value" : Varies
}
```

The `target` argument is either the device ID of a single receiver device or the name of one of the following device groups:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All receiver devices

For example, to set the quantization override level for the receiver (RX) device 010203040506 to `full`, the API request would be:

```
POST /api/device/010203040506
...
{
  "op" : "set:property",
  "key" : "
nodes[HDMI_DECODER:0].configuration.default_quantization_level_override "
  "value" : "full"
}
```

5.4 Quantization Mode Tips

Reminder when a decoder is operating in genlock mode (scaling disabled), the pixels are not modified nor are the AV Info Frames associated to the original stream. Therefore, what is received by the decoder is passed through as-is. It is basically the same result as being connected directly to a display.

When a decoder is operating in Fast Switch mode because transformations are done to the pixels, the devices processing engine needs to understand these pixels (color space, pixel depth, range, encryption, etc.). More specifically, if the source device is reporting `LIMITED` range, SDVoE endpoints expand it to process it.

The figure provided below shows graphically what chipsets execute:

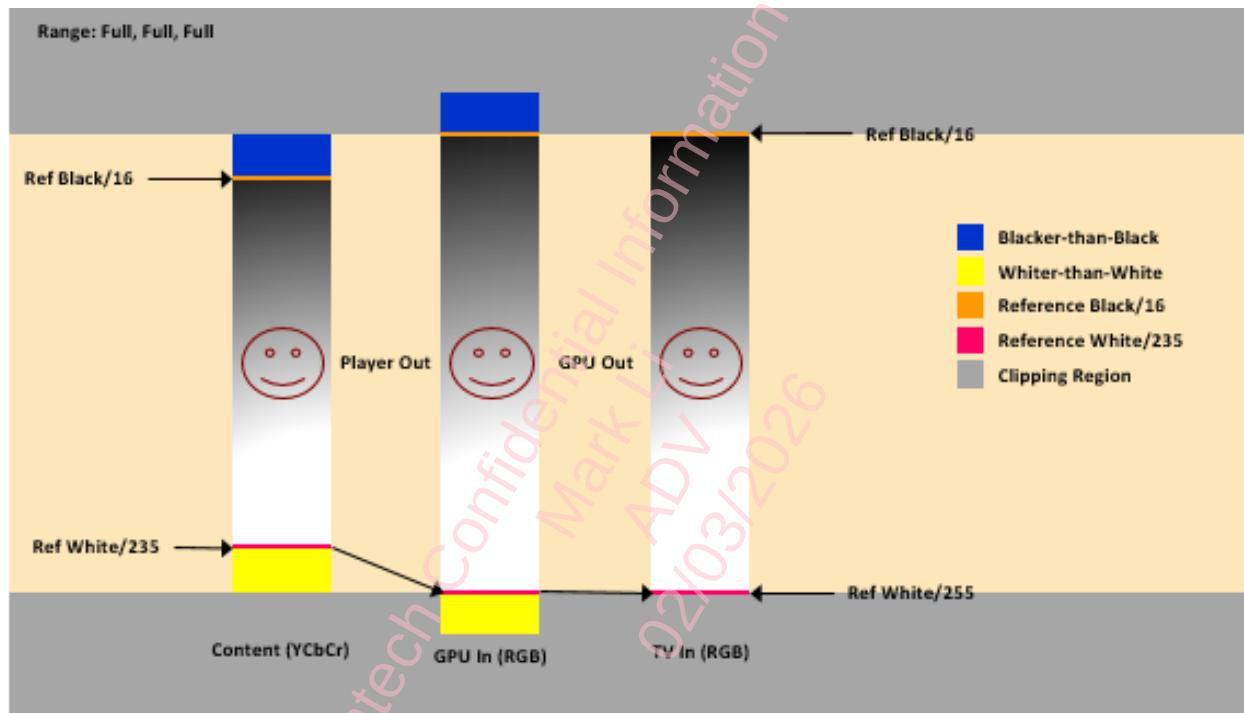


Figure 2: Quantization Management

After processing but prior to sending the pixel out on HDMI, it is possible to restore the `LIMITED` range by forcing the `LIMITED` range through the API. At this point the `SUPER BLACK` and the `SUPER WHITE` is removed, which is the intended behavior. When set to `LIMITED` mode, source devices (except perhaps a test pattern generator) do not use codes 0-15 and 236-255.

TIPS:

- i. The difference between the quantization `AUTO` and `STANDARD` modes is that, while `STANDARD` puts the value `DEFAULT` ($Q=0$) in the AVI InfoFrame, `AUTO` instead puts either `LIMITED` or `FULL` in the AVI InfoFrame depending on whether the video format is a CE format or not. `AUTO` is intended as a workaround for displays that take the wrong decision when they see `DEFAULT` in the AVI InfoFrame.
- ii. The decoder always sends the AVI InfoFrame.
 - Unless specified otherwise through the API (`LIMITED`, `FULL` or `AUTO` values of the quantization argument using set video request), the Q field is set to value 0 "DEFAULT".

- This indicates to the monitor/sink that the quantization range is the default one (depending on whether the video format is a CE format or not).
- iii. Since scaling is supported, it enables use cases where the input is an IT format but the output is a CE format.
 - On the decoder side, CEA-861 default operation means deciding on the quantization range based on the requested output format (CE or IT) and then converting it if the source has a different quantization range.
- iv. The API video arguments include the quantization option, which allows the control of the output quantization range.
 - By default, meaning if the quantization argument is omitted, the quantization range is selected in a CEA-861-compliant manner based on the output video format.
 - The API client could force something else, but please note that it is then the responsibility of that API client to look at the EDID beforehand (reported by the API) to confirm whether it is supported.

5.4.1 “Pluge” Pattern

The “pluge” pattern is a special pattern used to help in quantifying the video range, where:

- The brightest white band is a white with a code higher than 235, therefore “whiter than white” in LIMITED. Absolute White is 235
- The darkest black band is a black with a code lower than 16, therefore “darker than black” in LIMITED. Absolute Black is 16.

When a monitor is displaying the “pluge” pattern it should:

When operating in LIMITED range:

1. Clip the SUPER Black to Black.
2. Clip the SUPER White to White.

When operating in FULL range:

1. Show all Blacks with different intensity.
2. Show all Whites with different intensity.

6 BlueRiver Device Multiview Mode

The API includes multiview setup and control functionality. Multiview is an advanced feature and supported by endpoint decoder devices that include the AV Processing Engine. This includes devices based on either the BlueRiver NT2000 or AVP2000(T) chipset series.

The multiview mode is a powerful feature but it involves managing bandwidth, creating and applying layouts and assigning sources to the layout. This section assists with understanding these components and it specifically covers the following topics:

1. Provides an overview of the Multiview mode and how it works.
2. Reviews the Multiview model and terminology involved.
3. Managing the 10GbE bandwidth pipe in relation to decoder (RX) and encoder (TX) devices.
4. Outline information on the default surface configuration supplied and assignment of these surfaces.
5. In-depth look at the API command set and arguments that apply to the Multiview mode.
6. Review of sample scripts used to build and load multiview layouts, as well as manage impacted streams and subscriptions.

6.1 Multiview Function

Definition:

Multiview is the ability to take multiple independent sources and composite them into a single output stream that is then displayed on a single display.

- When set in Multiview mode, the decoder (RX) adjusts and/or scales source signals, placing them into a predefined layout of a specified dimension.
- It then composites the scene and generates a video signal of the required resolution and framerate, which is then sent out to the connected display.
- Sources can have different resolutions and/or frame rates.

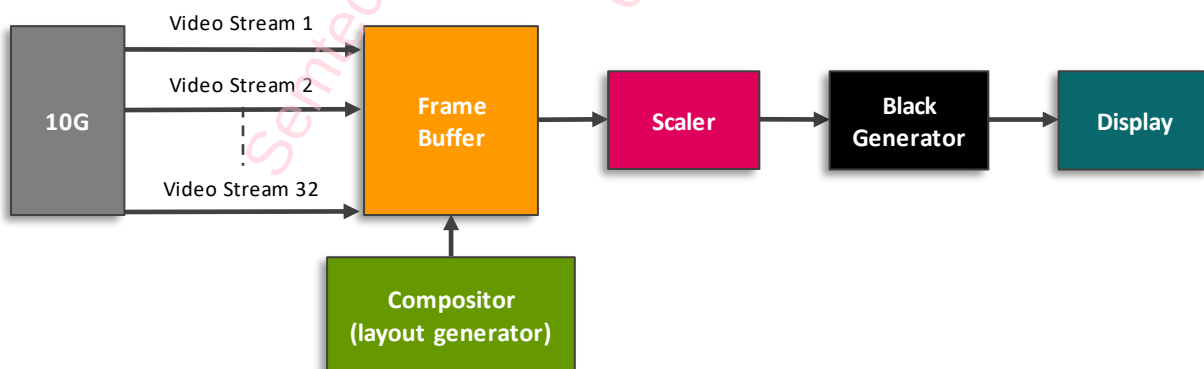


Figure 3: Multiview Receiver Processing

6.2 Multiview Layouts

The multiview mode provided through SDVoE endpoint devices is flexible, supporting many layout configurations, including:

- Picture-in-Picture (PiP)
- Picture-and-Picture (PaP)
- Matrix grid, such as 2x2 (Quad) or 3x3 layouts
- L-Shape, such as 1+5 and 1 + 7
- I-Shape, such as 2+8

In addition, it also supports arbitrary layouts, where sources are placed arbitrarily on the screen.

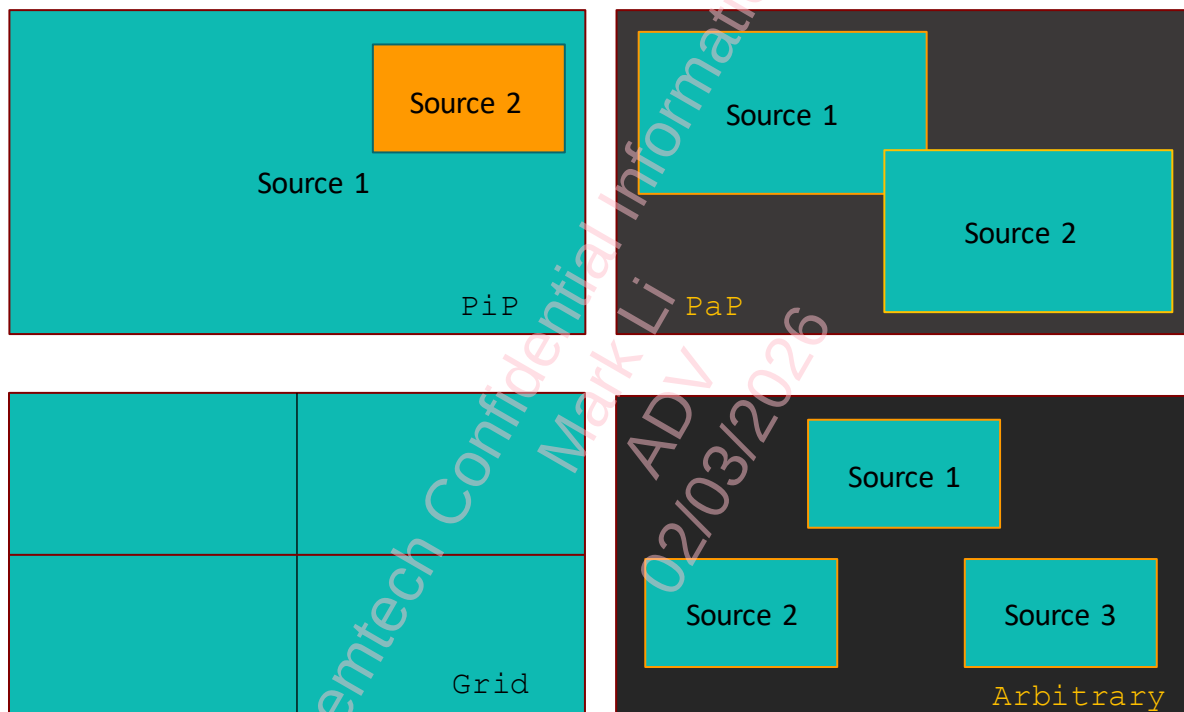


Figure 4: Example Multiview Layouts

Note: An SDVoE multiview implementation supports any layout style, so long as the total subscribed bandwidth on any network link does not exceed the 10GbE network pipe. This is the implicit bandwidth restriction for any 10GbE network.

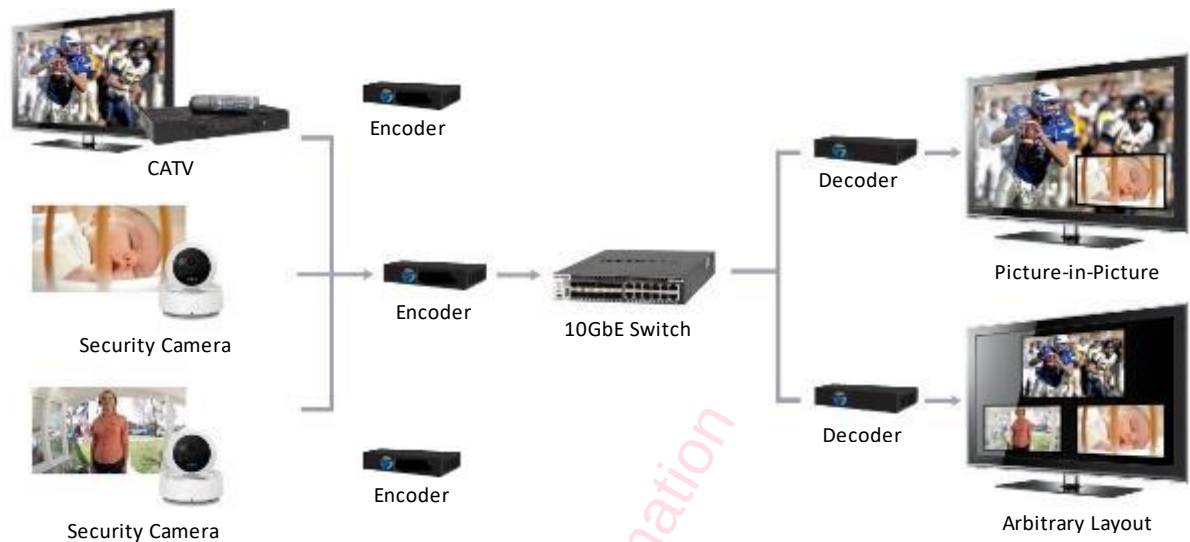


Figure 5: Multiview Example Use Case with 3 Sources and 2 Layouts

6.3 Managing the 10GbE Link

A key to successfully implementing multiview layouts is to respect and correctly manage the 10GbE bandwidth link. This applies to a decoder (RX) where multiple streams are received to create the multiview composition, as well as to an encoder (TX) where the native stream (stream 0) and scaled stream (stream 1) coexist.

6.3.1 Multiview Example

The scenario below illustrates a multiview layout, where two sources are combined into a single scene.

- Encoder 1 is connected to a computer outputting the Ski-racer video.
- Encoder 2 is connected to a media player outputting the Race car video.

The two HDMI streams are subscribed to by a single decoder (RX) where they are combined into a single layout composition. The scene is then rendered into a new video stream that in turn is sent to the output display attached to the decoder (RX).

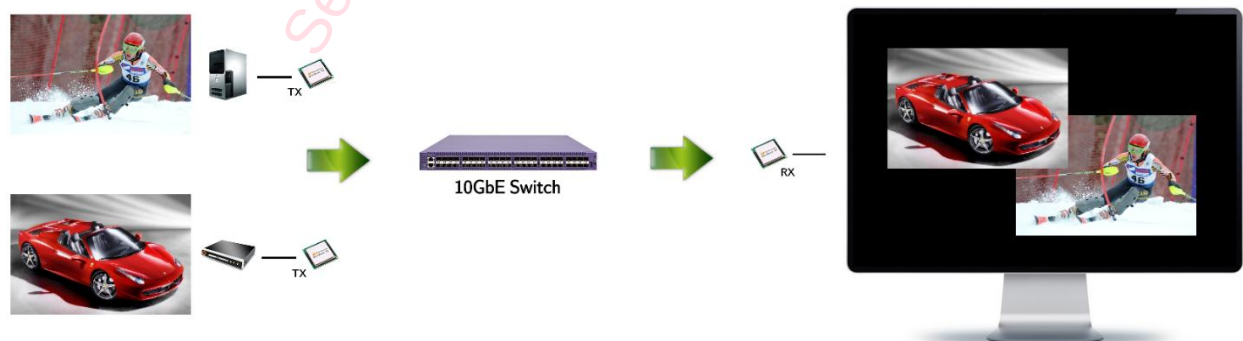


Figure 6: Two-Source Multiview Layout

6.3.2 Scaling & Frame Rate Conversion

6.3.2.1 Decoder (RX) Bandwidth & Scaling

In the above example, with the ski-racer and race car, if the two input streams being fed to the associated encoders (TX) are 1080P60 RGB 8-bit and the output is sent to a 4K display, then there is no issue with them being successfully transmitted and displayed.

Each 1080P60 stream consumes approximately 3.2 Gbps of network bandwidth for a total of 6.4 Gbps. This is within the 10GbE network link. In addition, there is also ample bandwidth left for other signals, such as audio, IR/RS-232, USB or Gigabit Ethernet, to be carried.

What if however, the source signals are 4K30 RGB 8 bit?

In this case, the combined bandwidth of the two video streams exceeds the bandwidth capacity 10GbE link going into the decoder (RX).

To support this type of scenario, an encoder (TX) based on a chipset supporting the processing mode can output two video streams:

- Native stream (stream 0) – original video stream fed into the encoder (TX).
- Scaled stream (stream 1) – a copy of the native stream, scaled to requested resolution.

The Native stream, Stream 0, is the same resolution as the HDMI input of the decoder and is available to all decoders (RX) on the network that are operating in any video mode, Multiview, Genlock, Fast Switch, Genlock Scaling or Wall mode.

The scaled stream (Stream 1) is at the resolution configured by the user and is available to all decoders (RX) operating in either Multiview or Fast Switch mode. The scaled stream is not however, available to decoders operating in Genlock, Genlock Scaling or Genlock Wall modes.

6.3.2.2 Scaled Stream (Stream 1)

Using the scaled stream (stream 1) to scale down the original video expands the capabilities of the multiview feature.

However, the following must be kept in mind when considering the Scaled stream (Stream 1):

- It is always in progressive format, even when original input source is interlaced, such as 1080i.
- If the source is RGB format, then Stream 1 remains in RGB format. If it is 10 or 12-bit, it is truncated to 8-bit.
- If the source is YCbCr then Stream 1 remains in YCbCr but is Chroma resampled to 4:4:4 and if 10 or 12-bit is truncated to 8-bit.
- Stream 1 is always 8-bit. If the original source has higher pixel depth (i.e. 10 or 12 bit) it is converted to 8-bit before it is scaled.
- Finally, Stream 1 is always in uncompressed format.

Going back to the case where the source signals are 4K30 RGB 8-bit, the two encoders (TX) now output the native 4K30 streams and in addition a downscaled 1080P copy of the stream which can be used to create the Multiview layout demonstrated above in Figure 6: Two-Source Multiview Layout.

Using the downscaled streams, allows the total bandwidth used on the network link being fed into a decoder (RX) to remain within the required 10GbE network link.

If this multiview is to be displayed on a standard HD display (1920x1080) rather than a 4K display, the downscaled stream could also be set to a lower resolution such as 720P. This would reduce the total

bandwidth to within the 10GbE pipe and allow both streams to be rendered successfully into a stream that can then be output to the 1920x1080 display.

Note: The scaled stream (Stream 1) can also be used to “upscale” the input source. An example of when this would be helpful is when creating a Picture-in-Picture layout for a 4K display, both sources combined remain within the 4K resolution, as well as the 10Gbps link.

6.3.2.3 Encoder (TX) Bandwidth & Frame Rate Converter

Previous section explained how outputting a second downscaled copy of the native source stream solves the bandwidth issue on the decoder (RX) side. Applying this solution does however potentially impact other elements.

Specifically, adding the second output stream to an encoder (TX) can under certain conditions increase the total required bandwidth beyond 10Gbps.

Let's again look at the initial example, assuming now that the native source is 4K60 RGB 8 bit. Even when compressed, this signal requires close to 8.5 Gbps, leaving little room for the second downscaled stream. In fact, if other signals are to be carried as well, such as RS-232, IR or CEC control, USB and/or Gigabit Ethernet data, there would not be enough room in the 10GbE link.

To avoid this limitation, **frame rate conversion** is provided. Reducing the frame rate for both the native stream (Stream 0) and the scaled stream (Stream 1) by a factor of 2x, allows for both to fit into 10GbE along with additional data.

The Factor 2x down frame rate conversion can be applied to:

- Native stream (Stream 0) only.
- Scaled stream (Stream 1) only.
- Both the Native and Scaled streams simultaneously.

Note: When the 2x down rate conversion is applied to the Native stream (stream 0), it can no longer be used by a decoder (RX) set in Genlock, Genlock Scaling or Wall mode. It can however, still be used by a decoder (RX) set to Fast Switch or Multiview mode.

6.3.2.4 Bandwidth Management Summary

When using the multiview mode, managing the amount of network bandwidth used is key to the successful application of loading multiview layouts.

The following two rules must always be respected:

- The total bandwidth received by a given decoder (RX) device must fit within the 10Gbps link; this includes all video streams needed for a specific multiview layout, along with any additional auxiliary and audio signals.
- The combined bandwidth of the native stream, downscaled stream and other signal data going out of a given encoder (TX) device must also fit within the 10Gbps link.

6.4 Multiview Model & Terminology

When a decoder (RX) is operating in multiview mode, the screen is divided into a grid consisting of a series of tiles.

- Each tile displays video from a rectangular area inside the decoder's frame buffer.
- Alternatively, a tile can be black.
- One or more tiles form a window that in turn displays a given source.

It is important to understand general concepts and terminology that are used to define and program multiview layouts through the API server. This section reviews common terminology associated with the multiview mode.

6.4.1 Streams & Subscriptions

Streams are a sending endpoint while subscriptions are a receiving endpoint.

Each stream or subscription has an assigned type and only streams and subscriptions of the same type can be associated/joined together.

6.4.2 Layout

A layout represents the multiview configuration that is to be composited by a decoder (RX). It has an assigned name which is used to identify it when a multiview API request is issued. It also specifies the total output resolution of the layout.

6.4.3 Window

A window is a high-level abstraction provided by the API. It is a rectangular area on the screen that contains either the rectangular area of a surface or black. Windows can overlap, when this occurs the content of the window with the lowest index is shown in the overlapping area.

6.4.4 Surface

A surface is a rectangular area within a decoder's (RX) frame buffer reserved for use by a specific video source. Each decoder (RX) running in multiview mode, supports up to 32 surfaces and each surface is associated with the HDMI subscription with the same index.

Example, the video from HDMI subscription 4 is forwarded into surface 4.

6.4.5 Tiles

The chipset divides a screen into tiles, which represent the smallest unit on the screen to which content can be assigned. Each tile contains either a rectangular area of a surface or is black.

Multiple tiles may be required to represent an individual window (example when windows overlap), as well as black areas that are not directly a part of any window. Refer to the Figure 7: Two Video Source Layout – Illustrates Window Overlap in 5x5 tile grid (25 Tiles) for an illustration of tiles.

6.5 Multiview Terminology

The figure shown below, illustrates multiview terminology and concepts. The example shown illustrates a layout containing two windows which are assigned to our sample streams:

- Race car.
- Ski-racer.

The window locations define the horizontal (green numbers) and vertical (red numbers) transitions in the example scene, these in turn define the tiles (blue numbers).

The example layout has a 5x5 tile grid, equaling 25 tiles in total.

- Four tiles form the “Race car” window (tiles 7, 8, 12 and 13).
- Three tiles form the visible portion of the “Ski-racer” window (tiles 14, 18 and 19).
- All other tiles are black and part of the background.

The “Race car” window has a lower index than the “Ski-racer” window because it is on top.

For example, the “Race car” window could be index 0, while the “Ski-racer” window could be index 1.

The “Race car” window refers to the surface displaying the “Race car” video, while the “Ski-racer” window refers to the surface that displays the “Ski-racer” video.

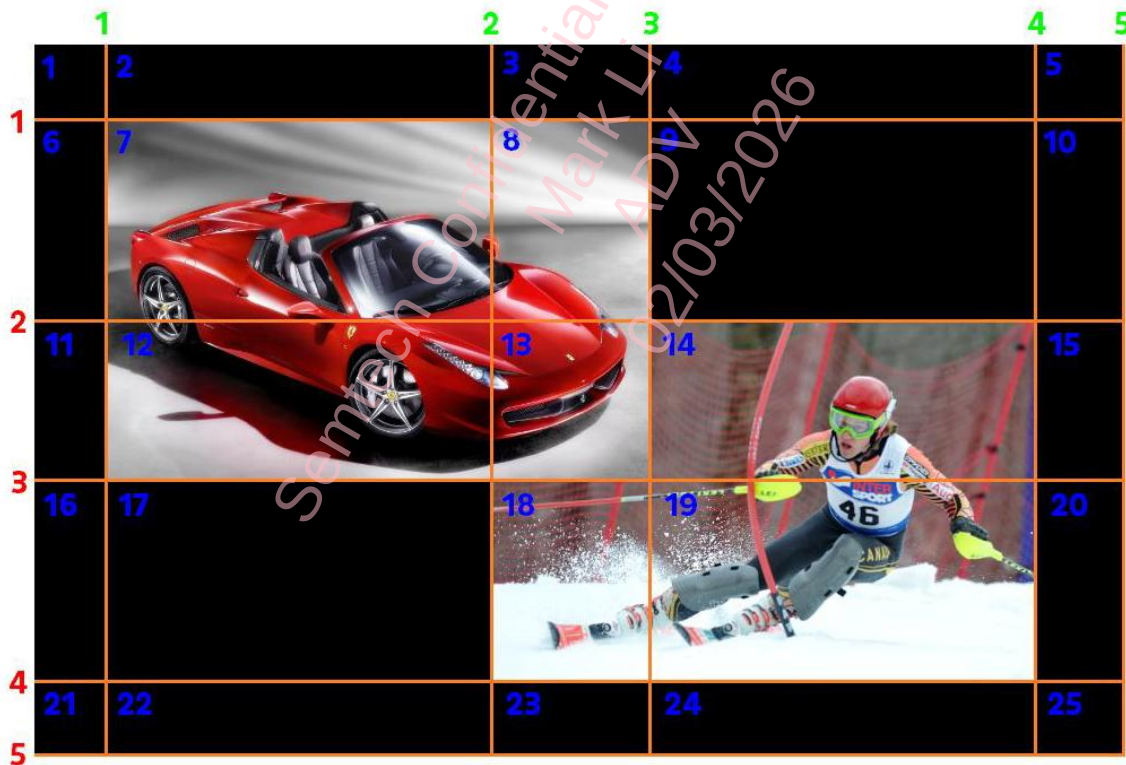


Figure 7: Two Video Source Layout – Illustrates Window Overlap in 5x5 tile grid (25 Tiles)

6.6 Default Multiview Surface Configuration

Reminder: A surface is a rectangular area in a decoder's (RX) frame buffer that is allocated for a specific video source. Each decoder supports up to 32 surfaces and each surface is associated with the HDMI subscription that has the same index.

When a layout is created (or reset) using a `layout create` request, a default surface configuration is applied. This default configuration is sufficient for the majority of multiview compositions.

The default surface configuration applied when a layout is created is as follows:

Surface 0 -- Can receive video up to a resolution of 4096×2160.
Surfaces 1 to 11 -- Can receive video up to a resolution of 2048×1080.
Surfaces 12 to 16 -- Can receive video up to a resolution of 2048×856.
Surfaces 17 to 20 -- Can receive video up to a resolution of 2048×1080.

Notes:

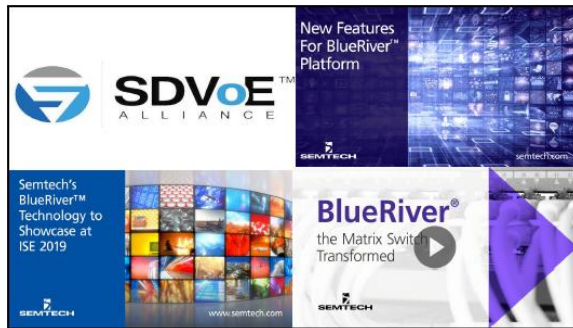
- Surfaces 17 to 20 use the same area of the frame buffer as Surface 0.
- This means surfaces 17 to 20 can be used **instead** of surface 0 if the multiview layout does not contain a source with a resolution over 2048×1080.
- However, surfaces 17 to 20 cannot be used at the same time as surface 0 is used.

The figure below shows the default surface allocation assigned when a layout is created, note that surface 0 (S0) uses the same part of the frame buffer as surfaces 17 through 20 (S17–S20). Also note that the resolutions associated to the various surfaces differ so when managing subscriptions this needs to be considered.

4096 x 2160 S17 S18 S0		2048 x 1080 S1	2048 x 1080 S2	2048 x 1080 S3
S19 S20		2048 x 1080 S4	2048 x 1080 S5	2048 x 1080 S6
2048 x 1080 S7	2048 x 1080 S8	2048 x 1080 S9	2048 x 1080 S10	2048 x 1080 S11
2048 x 856 S12	2048 x 856 S13	2048 x 856 S14	2048 x 856 S15	2048 x 856 S16

Figure 8: Illustration of Default Frame Buffer Surface Configuration

Illustrated here is a quad layout example, where the layout contains four windows. Each window is assigned to an individual surface. In this example, surfaces 0 through 3 are assigned to the windows contained within the layout.



		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 9: Example of Surface Assignments

For situations where the default configuration is not adequate, the allocation of the surfaces in a decoder (RX) frame buffer can be customized using `layout surface` requests. Refer to section 6.8.8 Managing Surfaces for more information on adding or removing surfaces.

6.7 Multiview API Request

API requests are used to create, configure and apply multiview layouts.

6.7.1 Handling the Multiview Mode

The table below shows the API command list that is relevant to multiview. The commands outlined here are used to implement multiview configurations, such as picture-in-picture (PiP) or a matrix grid.

Table 1: Multiview Command List

How to...	Multiview API command
List all layouts	<code>list layout</code>
Create a layout	<code>layout create</code>
Describe a layout	<code>layout describe</code>
Specify surfaces	<code>layout surface</code>
Specify window/tile content	<code>layout window</code>
Delete a layout	<code>layout delete</code>
Apply a layout to decoder (RX)	<code>set multiview</code>
Set scaler resolution of a source	<code>set scaler</code>
Join a source	<code>join</code>

The following sections review the multiview commands deeper. For further information, refer to the SDVoE Developers API Reference Guide.

6.7.2 Retrieving Existing Layouts

It is possible to view a list of layouts that currently exist from the API server using a list layout request.

Request syntax: `GET /api/multiview/layout`

Example of retrieving a listing of currently existing layouts:

```
GET /api/multiview/layout
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "layout": [
      {
        "name": "compatibility_4k_2x2"
      }
    ]
  },
  "error": null
}
```

Two return values are provided:

- First member is "layout" which contains the `LayoutItemObject`.
An array of the object, each representing a multiview layout.
- Second is a string providing the layout name.

6.8 Creating & Managing Layouts

When planning and implementing multiview layouts there is a series of tasks required. The following section reviews these tasks as well as some additional options that are applicable to configuring and implementing multiview layouts.

6.8.1 Creating Layouts

To create a new multiview layout or reset an existing layout with the specified name, a `layout create` request is used.

The syntax for creating a layout request is shown below:

```
POST /api/multiview/layout/{name}
{
  "op" : "create",
  "width" : Integer,
  "height" : Integer
}
```

The `name` argument specifies the name of the multiview layout. While the `width` and `height` arguments respectively specify the total width and height of the multiview composition.

Note: The width argument must be a multiple of 8.

Potential Errors

ILLEGAL_ARGUMENT Occurs when the arguments entered are invalid.

MULTIVIEW_LAYOUT_STATE Occurs if specified layout already exists and is read-only.

Example of creating a layout named `some_layout` with a total size of 3840 x 2160:

```
POST /api/multiview/layout/some_layout
{
  "op" : "create",
  "width" : 3840,
  "height" : 2160
}
```

Command return:

```
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "layout_description": {
      "name": "some_layout",
      "width": 3840,
      "height": 2160,
      "read_only": false,
      "surfaces": [
        {
          "index": 0,
          "horizontal_position": 0,
          "vertical_position": 0,
          "width": 4096,
          "height": 2160
        },
        {
          "index": 1,
          "horizontal_position": 4160,
          "vertical_position": 0,
          "width": 2048,
          "height": 1080
        },
        {
          "index": 2,
          "horizontal_position": 6240,
          "vertical_position": 0,
          "width": 2048,
          "height": 1080
        },
        {
          "index": 3,
          "horizontal_position": 8320,
          "vertical_position": 0,

```



```

        "width": 2048,
        "height": 1080
    },
    {
        "index": 4,
        "horizontal_position": 4160,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 5,
        "horizontal_position": 6240,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 6,
        "horizontal_position": 8320,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 7,
        "horizontal_position": 0,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 8,
        "horizontal_position": 2080,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 9,
        "horizontal_position": 4160,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 10,
        "horizontal_position": 6240,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    }

```

```

    },
    {
        "index": 11,
        "horizontal_position": 8320,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 12,
        "horizontal_position": 0,
        "vertical_position": 3240,
        "width": 2048,
        "height": 856
    },
    {
        "index": 13,
        "horizontal_position": 2080,
        "vertical_position": 3240,
        "width": 2048,
        "height": 856
    },
    {
        "index": 14,
        "horizontal_position": 4160,
        "vertical_position": 3240,
        "width": 2048,
        "height": 856
    },
    {
        "index": 15,
        "horizontal_position": 6240,
        "vertical_position": 3240,
        "width": 2048,
        "height": 856
    },
    {
        "index": 16,
        "horizontal_position": 8320,
        "vertical_position": 3240,
        "width": 2048,
        "height": 856
    },
    {
        "index": 17,
        "horizontal_position": 0,
        "vertical_position": 0,
        "width": 2048,
        "height": 1080
    },
    {

```

```

        "index": 18,
        "horizontal_position": 2080,
        "vertical_position": 0,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 19,
        "horizontal_position": 0,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 20,
        "horizontal_position": 2080,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    }
],
"windows": []
},
},
"error": null
}

```

This example created a layout, `some_layout` and applied the default surface configuration. It did not however add layout windows. Refer to section 6.8.2 Adding Windows to Layout for details on adding windows. For details on the default surface refer to section 6.6 Default Multiview Surface Configuration.

6.8.2 Adding Windows to Layout

Once a layout is created, a `layout window` request is used to add windows to the layout. This request is also used to modify or delete windows.

```

POST /api/multiview/layout/{name}/window/{index}
{
    "op" : "create",
    "horizontal_position" : Integer,
    "vertical_position" : Integer,
    "width" : Integer,
    "height" : Integer,
    "horizontal_offset" : Integer (optional),
    "vertical_offset" : Integer (optional),
    "target_surface" : Integer (optional)
}

```

The `name` argument is the name of the multiview layout.

The `index` argument represents the index of the window itself and must be between 0 and 31.

The `horizontal_position` and `vertical_position` arguments are respectively the horizontal and vertical start positions of the window on the screen, in pixels.

Note: The `horizontal_position` argument cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `width` and `height` arguments are respectively width and height of window in pixels.

Note: The `width` argument also cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `horizontal_offset` and `vertical_offset` arguments are respectively horizontal and vertical start positions of a window relative to start of a target surface, in pixels. For simple use cases, these are typically zero. These arguments can be omitted, which is equivalent to specifying zero.

Note: The `horizontal_offset` argument must also be a minimum of 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `target_surface` argument (optional) is the index of the surface to which the window refers. If this `target_surface` argument is omitted or null, the window is black.

Typically all arguments are present when creating a window. There are however two exceptions, the first is the `offset` argument which can be assumed to be zero (horizontal and vertical) when a new window is created and the second is the `target_surface` argument which is optional.

There is a second method of this request `DELETE`, which is used to remove the specified window from an existing layout. The syntax for this request is as follows:

```
DELETE /api/multiview/layout/{name}/window/{index}
```

Potential Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when arguments entered is invalid.
<code>MULTIVIEW_LAYOUT_STATE</code>	Occurs if specified layout is read-only.
<code>RESOURCE_NOT_FOUND</code>	Occurs when the specified layout does not exist.

Command examples:

Example 1

This example shows the request used to add window 0 to the layout named `some_layout`, also sets the position and size of the window, as well as the target surface index to 3.

```
POST /api/multiview/layout/some_layout/window/0
{
  "op" : "create",
  "horizontal_position" : 480,
  "vertical_position" : 270,
  "width" : 960,
  "height" : 540,
  "horizontal_offset" : 0,
  "vertical_offset" : 0,
  "target_surface" : 3
}
```

```

{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "layout_window": {
      "index": 0,
      "horizontal_position": 480,
      "vertical_position": 270,
      "width": 960,
      "height": 540,
      "horizontal_offset": 0,
      "vertical_offset": 0,
      "content": "VIDEO",
      "target_surface": 3
    }
  },
  "error": null
}

```

This example modifies a window and fills it with black (target_surface is omitted):

```
POST /api/multiview/layout/some_layout/window/0
```

```

{
  "op" : "create",
  "horizontal_position" : 480,
  "vertical_position" : 270,
  "width" : 1920,
  "height" : 1080,
  "horizontal_offset" : 0,
  "vertical_offset" : 0
}

{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "layout_window": {
      "index": 0,
      "horizontal_position": 480,
      "vertical_position": 270,
      "width": 1920,
      "height": 1080,
      "horizontal_offset": 0,
      "vertical_offset": 0,
      "content": "BLACK",
      "target_surface": 0
    }
  },
  "error": null
}

```

Example of deleting a window:

```
DELETE /api/multiview/layout/some_layout/window/0

{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

6.8.3 Retrieving Details of Layout

It is possible to retrieve information related to a given layout in one of two methods. The first is to retrieve details of a layout currently stored within the API server and the second to retrieve details of a layout currently loaded on a decoder (RX) device operating in multiview mode. Below reviews the two independent API commands used for these purposes. Also, in this subsection introduction to the MULTIVIEW_CHANGED event included.

6.8.3.1 Retrieve List Details of Available Layout from the API Server

To retrieve details from the API about an existing layout a `layout describe` request is used. Data provided includes total layout size along with the information for each window including their position, surface, target and size. Three tables are provided below outlining details on the provided data.

Request syntax: `GET /api/multiview/layout/{name}`

The `name` argument is name of the multiview layout.

The structure of the Layout description object (LayoutDescription) is as follows:

```
{
  "name" : String
  "width" : Integer,
  "height" : Integer,
  "read_only" : Boolean,
  "surfaces" : [SurfaceDescription, ...],
  "windows" : [WindowDescription, ...]
}
```

The table below provides a description of the data received describing the layout description object.

Table 2: Table Describing the Layout Description Object

Member Name	Type	Description
name	String	The name of the layout.
width	Integer	Display width in pixels.
height	Integer	Display height in pixels.
read only	Boolean	Whether the layout is protected against modifications. Currently, only the predefined layout, compatibility_4k_2x2 layout, is read-only.
surfaces	Array of SurfaceDescription	Description of all surfaces in the layout.
windows	Array of WindowDescription	Description of all windows in the layout.

Here the structure of the surface description object (`SurfaceDescription`) is shown:

```
{
  "index" : Integer,
  "horizontal_position" : Integer,
  "vertical_position" : Integer,
  "width" : Integer,
  "height" : Integer
}
```

This table describes the structure of the surface description object.

Table 3: Table Describing the Surface Description Object

Member Name	Type	Description
index	Integer	The surface index.
horizontal_position	Integer	Horizontal start position of surface in pixels.
vertical_position	Integer	Vertical start position of surface in pixels.
width	Integer	Surface width in pixels.
height	Integer	Surface height in pixels

Window description object (`WindowDescription`) has the following structure:

```
{
  "index" : Integer,
  "horizontal_position" : Integer,
  "vertical_position" : Integer,
  "width" : Integer,
  "height" : Integer,
  "horizontal_offset" : Integer,
  "vertical_offset" : Integer,
  "content" : String,
  "target_surface" : Integer,
}
```

Table 4: Table Describing the Window Description Object

Member name	Type	Description
index	Integer	Surface index.
horizontal_position	Integer	Horizontal start position of window on screen, in pixels.
vertical_position	Integer	Vertical start position of window on screen, in pixels.
width	Integer	Window width in pixels.
height	Integer	Window height in pixels
horizontal_offset	Integer	Horizontal offset from the start of the surface.
vertical_offset	Integer	Vertical offset from the start of the surface.
content	Integer	Type of content of this window, either: VIDEO window has video content. BLACK window is black.
target_surface	Integer	Index of the surface to which the window refers. Only meaningful when content member is VIDEO.

Following shows an example request for retrieving details for the `some_layout` example:

- The layout description includes the `name` and `size` of the layout.
- Surfaces list each of the surfaces that exist in the layout, includes details of their index

number, position and size. The one included here is the surface configuration applied by default to new layouts.

- iii. `Windows` list each of the windows that exist in the layout, includes details of their index number, position, size and offset.
- iv. The `target_surface` is the index of the surface to which the window refers. It is only meaningful if the content member is `VIDEO`.

```
GET /api/multiview/layout/some_layout
```

```
{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "layout_description": {
      "name": "some_layout",
      "width": 3840,
      "height": 2160,
      "read_only": false,
      "surfaces": [
        {
          "index": 0,
          "horizontal_position": 0,
          "vertical_position": 0,
          "width": 4096,
          "height": 2160
        },
        {
          "index": 1,
          "horizontal_position": 4160,
          "vertical_position": 0,
          "width": 2048,
          "height": 1080
        },
        {
          "index": 2,
          "horizontal_position": 6240,
          "vertical_position": 0,
          "width": 2048,
          "height": 1080
        },
        {
          "index": 3,
          "horizontal_position": 8320,
          "vertical_position": 0,
          "width": 2048,
          "height": 1080
        },
        {
          "index": 4,
          "horizontal_position": 4160,
          "vertical_position": 1080,
          "width": 2048,
```

```

        "height": 1080
    },
    {
        "index": 5,
        "horizontal_position": 6240,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 6,
        "horizontal_position": 8320,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 7,
        "horizontal_position": 0,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 8,
        "horizontal_position": 2080,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 9,
        "horizontal_position": 4160,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 10,
        "horizontal_position": 6240,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 11,
        "horizontal_position": 8320,
        "vertical_position": 2160,
        "width": 2048,
        "height": 1080
    },
    },

```



```

{
  "index": 12,
  "horizontal_position": 0,
  "vertical_position": 3240,
  "width": 2048,
  "height": 856
},
{
  "index": 13,
  "horizontal_position": 2080,
  "vertical_position": 3240,
  "width": 2048,
  "height": 856
},
{
  "index": 14,
  "horizontal_position": 4160,
  "vertical_position": 3240,
  "width": 2048,
  "height": 856
},
{
  "index": 15,
  "horizontal_position": 6240,
  "vertical_position": 3240,
  "width": 2048,
  "height": 856
},
{
  "index": 16,
  "horizontal_position": 8320,
  "vertical_position": 3240,
  "width": 2048,
  "height": 856
},
{
  "index": 17,
  "horizontal_position": 0,
  "vertical_position": 0,
  "width": 2048,
  "height": 1080
},
{
  "index": 18,
  "horizontal_position": 2080,
  "vertical_position": 0,
  "width": 2048,
  "height": 1080
},
{
  "index": 19,

```

```

        "horizontal_position": 0,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    },
    {
        "index": 20,
        "horizontal_position": 2080,
        "vertical_position": 1080,
        "width": 2048,
        "height": 1080
    }
],
"windows": [
    {
        "index": 0,
        "horizontal_position": 480,
        "vertical_position": 270,
        "width": 1920,
        "height": 1080,
        "horizontal_offset": 0,
        "vertical_offset": 0,
        "content": "VIDEO",
        "target_surface": 3
    }
]
},
"error": null
}

```

6.8.3.2 Retrieving Details of Layout Currently Loaded on Decoder (RX) Device

Starting in API 3.4, it is possible to request details of the layout currently loaded on a decoder (RX). This request returns the same data as a layout describe request, including the layout name, size, window locations and size as well as the index surface assignments.

This request is:

```

POST /api/device/{target}

{
  "op" : "get",
  "subset" : "multiview"
}

```

The target argument is either the device ID of a single decoder (RX) or a device group:

ALL	All devices. Only affects decoders (RX).
ALL_RX	All decoders (RX) are set to multiview mode and the specified layout is loaded.

The information returned by a `get multiview` request is the same as for a layout describe request, refer to section 6.8.3.1 Retrieve List Details of Available Layout from the API Server for details.

The layout description resides within the `FRAME_BUFFER` node of the returned JSON device array and is part of the `multiview_layout` JSON object. Therefore, when present the description data is also returned as part of a `get_device` request.

Outline of the structure of the JSON return:

```
{
  "status" : "SUCCESS",
  "request_id" : request_ID,
  "result" : {
    devices: [
      {
        "device_id": "target",
        "status": {
          "active": true
        },
        "nodes" : [
          {
            "type": "FRAME_BUFFER",
            "index": 0,
            "configuration": {
              "display_mode": "MULTIVIEW",
              "multiview_layout": {
                "name" : String
                "width" : Integer,
                "height" : Integer,
                "read_only" : Boolean,
                "surfaces" : [SurfaceDescription, ...],
                "windows" : [WindowDescription, ...]
              }
            }
          },
          ...
        ]
      },
      ...
    ],
    "error" : null
  }
}
```

Potential Errors

<code>IO_ERROR</code>	Occurs when API not able to retrieve layout information from the devices memory.
<code>INVALID_STATE</code>	Returned if device is not currently set to multiview mode.

Due to the volume of data returned example not included with this article. Refer to the SDVoE Developers API Reference Guide for more information.

6.8.3.3 Event MULTIVIEW_CHANGED

Starting in API 3.4 a new event type, the `MULTIVIEW_CHANGE` event, was introduced. This allows for client applications to monitor changes to layouts currently loaded on a decoder (RX) device. The API generates a `MULTIVIEW_CHANGED` event if one or more of the following configuration changes occur:

- layout name and/or layout size
- window
 - position (horizontal & vertical)
 - size (horizontal & vertical)
 - target (i.e. attached surface)
- offset (horizontal & vertical)
- surface
 - position (horizontal & vertical)
 - size (horizontal & vertical)

Note: Subscription changes do not trigger a `MULTIVIEW_CHANGED` event as they are included in `SETTING_CHANGED` events.

Example of a `MULTIVIEW_CHANGE` event:

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : "d8deadbeef07",
        "event_id" : 69053,
        "event_type" : " MULTIVIEW_CHANGED",
        "timestamp" : 1638995235,
        "request_id" : 123}
    ]
  },
  "error" : null
}
```

For more information on events, refer to the Developers Introduction to SDVoE API User Guide.

6.8.4 Deleting a Layout

The `layout delete` command is used to remove the entire layout including all its related properties from the API server.

Command syntax: `DELETE /api/multiview/layout/{name}`

A delete request removes the layout completely, including the window and surface configuration.

The `name` argument represents the name of the multiview layout that is to be removed/deleted.

Potential Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when arguments entered invalid.
<code>MULTIVIEW_LAYOUT_STATE</code>	Occurs if specified layout is read-only.

Example:

```
DELETE /api/multiview/layout/some_layout

{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

6.8.5 Managing Multiview Streams

To successfully stream a layout from a decoder to its attached display besides creating the layout, there are some additional actions required. Specifically, from the encoder side this includes configuring the scaled streams as well as managing the status of both the native and scaled streams (start/stop); while at the decoder side need to manage subscriptions and set the video display mode to multiview at the desired output resolution.

6.8.5.1 Managing Scaled Streams

When creating layouts containing multiple windows, the assigned destination streams are usually the HDMI scaled stream (stream 1) of an encoder (TX). The size of the destination stream must match the specified window size of the layout. The scaled stream is managed using a `set_scaler` request.

TIP: The scaler resolution should be set before starting the scaled video stream.

Syntax for a `set_scaler` request is as follows:

```
POST /api/device/{target}

{
  "op" : "set:scaler",
  "width" : Integer,
  "height" : Integer
}
```

The `target` argument is either the device ID of a single encoder (TX) or the name of a device group. The supported device groups are:

ALL	All devices
ALL_TX	All transmitter devices

The `width` and `height` arguments respectively specify width and height of the scaled video, in pixels.

Example of setting the HDMI scaled stream (stream 1) of encoder `d88039e83991` to an output resolution of `640 x 480`:

```
POST /api/device/d88039e83991

{
  "op" : "set:scaler",
  "width" : 640,
  "height" : 480
}
```

```
{
  "status" : "PROCESSING",
  "request_id" : 42,
  "result" : null,
  "error" : null
}
```

For more information on the `set_scaler` command, refer to the SDVoE Developers API Reference Guide.

6.8.5.2 Starting Streams

When loading a layout, the streams contained within a layout must be currently running to be successfully be incorporated into the multiview output stream. This task is completed using a `start` request specifying the stream type and the appropriate stream index. In relation to multiview, the stream type is HDMI and the index is either 0 for the native stream or 1 for the scaled stream.

Syntax for a `start` request:

```
POST /api/device/{target}

{
  "op" : "start",
  "stream_type" : String,
  "stream_index" : Integer,
  "dest_address" : String (optional),
}
```

Reminder: When referring to stream 1, it is available for subscription to only decoders (RX) operating in multiview, fast switch or fast switch wall modes.

For a `start` request, the `target` argument is either the device ID of a single encoder (TX) or the name of a device group. The supported device groups are:

ALL All devices. Only affects encoders (TX).
ALL_TX All transmitter devices.

The `stream_type` argument represents the type of stream that is to be started. As mentioned above, in regard to multiview, the HDMI stream (index 0 or 1) is the stream type of interest, although the following audio stream types may also be started using a `start` request:

STEREO_AUDIO for a stereo (Single I²S) audio stream.
HDMI_AUDIO for an HDMI audio stream.
MULTICH_AUDIO for a multichannel (Quad I²S) audio stream.

Reminders: A decoder (RX) must respect the 10Gbps bandwidth on the HDMI input, subscribing to the scaled streams allows for up to 32 video streams to be incorporated into a given layout.

The HDMI output of an encoder (TX) must also respect this bandwidth link. For an encoder, useful options for managing bandwidth include stopping the native stream when it is not required or applying the frame rate divider to the native and/or scaled streams.

For more information on the `start` command, refer to the SDVoE Developers API Reference Guide.

Example of starting the HDMI scaled stream (stream 1) for an individual encoder (TX) with the device ID of 010203040506:

```
POST /api/device/010203040506
{
  "op" : "start",
  "stream_type" : "HDMI",
  "stream_index" : 1
}
```

Manually Managing Multicast IP Allocation

When it is preferred to manually manage the multicast IP address allocation, this is completed by specifying an IP address using the `dest_address` argument of the `start` command.

When manually managing the multicast IP allocation, the IP entered must be an available unused multicast IP address within the allocation range configured in the configuration file of the API server. The default range is [224.1.1.1 to 224.1.3.255].

The following are invalid in regards to the `dest_address` argument:

- 224.1.1.253 is the default address reserved for the “ALL_TX” multicast IP address. The default may be modified through the API server configuration file.
- 224.1.1.254 is the default address reserved for the “ALL_RX” multicast IP address. The default may be modified through the API server configuration file.
- Multicast IP address 225.225.225.225.
- Any multicast address in the following ranges defined in IETF RFC 3171:
 - An address in the Local Network Control Block (224.0.0.0/24).
 - An address in the Internetwork Control Block (224.0.1.0/24).
- Any of the following special use IPv4 addresses as defined in IETF RFC 5735:
 - An address on the “This” network (0.0.0.0/8).
 - A loopback address (127.0.0.0/8).

TIP: For more information on managing default multicast ranges, refer to the SDVoE Control Server Administrators Guide.

If `dest_address` is not specified, the API server uses either the address already assigned to each stream by a previous invocation of the `start` command that has not been released by a `stop` request, otherwise it allocates an unused multicast address.

6.8.5.3 Stopping Streams

As referred to earlier, stopping an HDMI stream is useful in managing the output bandwidth of an encoder (TX). It should however be kept in mind that when a stream is stopped it is no longer available to any subscribed decoder (RX) device.

A `stop` request is used to stop one or more stream(s) on one or more devices and optionally free the multicast address associated with each stopped stream. The syntax of a stop request is as follows:

```

POST /api/device/{target}
{
  "op" : "stop",
  "stream_type" : String (optional),
  "stream_index" : Integer (optional),
  "free" : Boolean
}

```

Reminder: Freeing the multicast address allows it to be reassigned to another stream.

The example shown stops the HDMI native stream (stream 0) of the encoder (TX) 010203040506.

```

POST /api/device/010203040506
{
  "op" : "stop",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "free" : false}

```

For more information on the `stop` command, refer to the SDVoE Developers API Reference Guide.

6.8.6 Setting Decoder to Multiview Mode

A `set multiview` request is used to set one or more decoders (RX) to operate in multiview mode, as well as to load the specified layout.

The request syntax for this action is as follows:

```

POST /api/device/{target}
{
  "op" : "set:multiview",
  "layout" : String,
  "video" : VideoFormatArguments (conditional),
  "subscriptions" : [SubscriptionObject,...] (optional)
}

```

The `target` argument is either the device ID of a single decoder (RX) or the name of a device group. The supported device groups are:

<code>ALL</code>	All devices. Only affects decoders (RX).
<code>ALL_RX</code>	All decoders (RX) are set to multiview mode, and the specified layout is loaded.

The `layout` argument specifies the multiview layout to load.

The `video` argument specifies video format arguments, in regard to managing layouts this argument must be implicitly specified as `multiview`. Refer to the SDVoE API Developer Reference Guide, section Specifying a Video Output Format for more information on video output formats.

It is possible to specify the output resolution to be applied, which can either be the same resolution as the Multiview layout size or a different resolution (in the latter case, the decoder (RX) will scale the full Multiview layout composition to the desired output resolution). The latter case can be useful if the same Multiview is to be displayed to different monitors supporting different native resolution (for example, 1080p60 and 4K60 monitors) and can also be useful to ensure seamless transitions between different Multiview layouts that do not have the same layout size. However, if the output resolution (video arguments) is not specified, the total layout size is applied.

Reminder: It is possible to specify the output resolution either by using video arguments of `size width height fps` or specifying the Video Identification Code (VIC) or the HDMI VIC. For list of supported VIC and HDMI VIC codes, refer to the SDVoE Developers API Reference Guide.

Potential Errors

ILLEGAL_ARGUMENT	Occurs when target argument entered is not a valid device ID or group name.
ILLEGAL_ARGUMENT	Also occurs in the following cases: • When no subscriptions in the subscription array (i.e. empty list). • A subscription source is not a valid device ID. • A subscription source does not exist, device disconnected or is not a transmitter or transceiver device. • The source stream index is missing or invalid. • The subscription index is missing or invalid. • A source stream or a subscription index is duplicated.
RESOURCE_NOT_FOUND	Occurs when the specified layout does not exist.
MULTIVIEW_LAYOUT_STATE	Occurs when the specified layout does not satisfy either the hardware or API limitations. Refer to section 6.12 Multiview Guidelines & Limitations for details.

The request may also fail for individual devices for the following reasons:

DEVICE_TYPE	This error occurs if the specified device is not a receiver device.
DEVICE_TYPE	Also occurs when the device does not support the full multiview capabilities, such as an AVP1000 or NT1000 device. In addition, this error is received if device is a legacy BlueRiver NT device and the specified layout is not the <code>compatibility_4k_2x2</code> layout.
INVALID_STATE	Indicates that the source stream of a subscription is stopped.

Reminder: The AVP1000 and NT1000 chipsets do not support the multiview mode.

Command example loading the `some_layout` multiview layout of any size to an output resolution of 1920x1080 with a frame rate of 60:

```
POST /api/device/010203040506
{
  "op" : "set:multiview",
  "layout" : "some_layout",
  "video" :
  {
    "width" : 1920,
    "height" : 1080,
```

```

        "fps" : 60
    }
}
{
    "status" : "PROCESSING",
    "request_id" : 41,
    "result" : null,
    "error" : null
}

```

The return values for the `set` command are the same as for the `get settings` command. Refer to Appendix A of the SDVoE Developers API Reference Guide for an example. For more information on the `set` command refer to the SDVoE Developers API Reference Guide.

6.8.6.1 Managing Decoder Subscriptions

When loading a layout, it is necessary to subscribe each target surface to the appropriate encoder (TX) stream. This can be done using an individual `join` request or by combining a `set multiview` request with the `join` request.

TIP: Completing the two tasks in a single request reduces the load time for a multiview request so we will review this option. For details on a join request, refer to the SDVoE Developers API Reference Guide.

```

POST /api/device/{target}
{
    "op" : "set:multiview",
    "layout" : String,
    "video" : VideoFormatArguments (conditional),
    "subscriptions" : [SubscriptionObject,...] (optional)
}

```

The layout and video arguments were reviewed previously, now the optional `subscriptions` argument is described. This optional argument specifies an array of HDMI subscription objects (`SubscriptionObject`) to be applied to the multiview layout.

Reminder: The `target` argument must be either the device ID of a single decoder (RX) or the name of a device group.

Each item in `SubscriptionObject` has the following structure:

```

{
    "source_device" : String,
    "stream_index" : Integer,
    "subscription_index" : Integer
}

```

Where these objects are defined as follows:

<code>source_device</code>	Is the device ID of the source encoder (TX).
<code>stream_index</code>	Is the index of the source HDMI stream, is either 0 for the native stream or 1 for the scaled stream.
<code>subscription_index</code>	Is the index of the destination subscription.

TIP: The `source_device` argument represents the device ID of the encoder (TX) supplying the HDMI source stream. The same stream types are supported as for the `set` command (HDMI, HDMI_AUDIO, MULTICH_AUDIO and STEREO_AUDIO) but as this section is focused on multiview, only the HDMI stream is applicable. Refer to the SDVoE Developers API Reference Guide for more information on the `join` command.

For examples of this argument, refer to the multiview case studies in section 6.11 Multiview Use Case.

6.8.7 Predefined Compatibility Layout

The API server provides a predefined layout named `compatibility_4k_2x2`.

Note: This is the only layout that can be applied to legacy BlueRiver NT devices.

As the name indicates, this layout is a 4K (3840×2160) layout with four 1080p sources arranged in a 2×2 grid. Frame rate needs to be considered, respecting the 10GbE bandwidth pipe. For example, one layout source can be 60Hz and others could be 24 Hz or all could be 30 Hz.

The `compatibility_4k_2x2` layout can be applied to any devices that support the `multiview` mode.

TIP: The `compatibility_4k_2x2` layout cannot be modified, it is predefined with read-only rights. Trying to modify this layout with an API layout request results in it failing with the error `MULTIVIEW_LAYOUT_STATE`.

6.8.8 Managing Surfaces

In the event that the default surface configuration does not meet the needs of a given layout, such as the layout contains more windows than the default surface configuration contains, it is possible to modify the default surface configuration.

TIP: The frame buffer has a limited physical buffering capacity of 10880x4096 pixels.

A `surface` request is used to create, modify, or delete the surface configuration of a layout.

Syntax for creating/modifying a surface:

```
POST /api/multiview/layout/{name}/surface/{index}
{
  "op" : "create",
  "horizontal_position" : Integer,
  "vertical_position" : Integer,
  "width" : Integer,
  "height" : Integer
}
```

Syntax for deleting a surface:

```
DELETE /api/multiview/layout/{name}/surface/{index}
```

The `name` argument is the name of the multiview layout.

The `index` argument is the index of the surface and must be a numerical number between 0 and 31.

The `horizontal_position` and `vertical_position` arguments are respectively the horizontal and vertical start positions of the surface in the frame buffer, in pixels.

Note: The `horiz_position` argument cannot be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

The `width` and `height` arguments are respectively the width and height of the surface, in pixels.

Note: The `width` argument can also not be less than 8-pixels and is applied in increments of two pixels, so for example 8, 10, 12, etc.

If the method selected is `DELETE`, then if the surface specified exists it is deleted.

Potential Errors

<code>ILLEGAL_ARGUMENT</code>	Occurs when the arguments entered are invalid.
<code>MULTIVIEW_LAYOUT_STATE</code>	Occurs when the specified layout exists but is read-only.
<code>RESOURCE_NOT_FOUND</code>	Occurs when the specified layout does not exist.

Example specifying the surface, position and size of surface 23 for the layout named `some_layout`:

```
POST /api/multiview/layout/some_layout/surface/23
{
  "op" : "create",
  "horizontal_position" : 3872,
  "vertical_position" : 0,
  "width" : 1920,
  "height" : 1080
}
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "layout_surface" : {
      "index" : 23,
      "horizontal_position" : 3872,
      "vertical_position" : 0,
      "width" : 1920,
      "height" : 1080
    }
  },
  "error" : null
}
```

This second example deletes surface 23 from the layout named `some_layout`.

```
DELETE /api/multiview/layout/some_layout/surface/23
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : null,
  "error" : null
}
```

Refer to the SDVoE Developers API Reference Guide for more information on the `surface` command.

6.9 Tasks to Create & Load Layout Summary

To use the Multiview feature, a layout must be created and configured. This layout is then applied to one or more decoder (RX) devices. The output resolution to configure in the decoder (RX) devices while applying the multiview does not need to be the same as the multiview layout size, although it is preferable to keep the same aspect ratio. The following section illustrates a simple example of the steps required when creating, activating, and loading a PiP (picture-in-picture) layout.

Reminder: If not currently running, it is necessary to load the appropriate `blueriver_api` module. Refer to section 3.3 Loading API Command Module for details on this task.

1. The first step in configuring a multiview layout is to create the layout itself.

For this example, a multiview layout called `PiP_Sample` is created with a layout size of 3840x2160. The default surface configuration assigned when the layout is created is applied.

The request to enter is:

```
multiview/layout/PiP_Sample
{
    "op" : "create",
    "width" : 3840,
    "height" : 2160
}
```

2. Next the layout windows are added to the layout created in step 1. Reminder, that the position, size and surface are specified when creating a layout window.

For this PiP example, two requests are required, one for the larger window (window 1) and a second for the smaller window (window 0) that is to be overlaid on the first.

```
multiview/layout/PiP_Sample/window/1
{
    "op" : "create",
    "horizontal_position" : 0,
    "vertical_position" : 0,
    "width" : 3840,
    "height" : 2160,
    "horizontal_offset" : 0,
    "vertical_offset" : 0,
    "target_surface" : 0
}
```

Note that in the above request, the horizontal and vertical positions are both 0. This is because the source is to cover the entire output display, so its position is specified as the upper most left corner. The width and height specified match the total layout size.

Creating the second window:

```
multiview/layout/PiP_Sample/window/0
{
    "op" : "create",
    "horizontal_position" : 400,
```

```

    "vertical_position" : 400,
    "width" : 960,
    "height" : 540,
    "horizontal_offset" : 0,
    "vertical_offset" : 0,
    "target_surface" : 1
}

```

This second request to be overlaid on the first, so the window index assigned is lower than the window it is to be overlaid on. The position for this window is 400 pixels horizontally and 400 pixels vertically and the window size is 960x540. Reminder, the resolution of the assigned stream must match the size of the window in which it is to be displayed.

3. Manage the streams that are required for the layout created above.
 - a. Set the scaled stream (stream 1) of the appropriate encoder (TX) devices to reflect the required layout window resolutions. Refer to section 6.8.5.1 Managing Scaled Stream for more information on this task. In this specific example, if device `my_device_tx_a` is the device from which the full screen picture comes from and `my_device_tx_b` is the device from which the picture in picture comes from, we would set device `my_device_tx_a` stream 1 to 3840x2160 and device `my_device_tx_b` stream 1 to 960x540.
 - b. Start the scaled streams (stream 1) and/or native streams (stream 0) of the appropriate encoder (TX) devices to be subscribed to by the decoder (RX) that will load the above layout. Refer to section 6.8.5.2 Starting Streams for more information on starting streams.
 - c. Stop the native streams (stream 0) if required to remain within the 10Gbps Ethernet network link. Refer to section 6.8.5.3 Stopping Streams for more information on stopping streams.
4. Once the streams have been configured, proceed to set the decoder (RX) device(s) into multiview mode at the desired output resolution, load the appropriate layout and subscribe to the appropriate streams required to compose the multiview. Refer to section 6.8.6 Setting Decoder to Multiview Mode for more details.

For this PiP example, if the decoder (RX) device is `my_device_rx` and the the output multiview is to be applied to a 4K monitor, the following request can be used to apply the 3840x2160 multiview layout to an output of the same 3840x2160 resolution.

```

device/my_device_rx
{
    "op" : "set:multiview",
    "layout" : "PiP_Sample",
    "video" :
    {
        "width" : 3840,
        "height" : 2160,
        "fps" : 30
    }
    "subscriptions" :
    {
        "source_device" : "my_device_tx_a",

```

```

        "stream_index" : 1,
        "subscription_index" : 0
    },
    {
        "source_device" : "my_device_tx_b",
        "stream_index" : 1,
        "subscription_index" : 1
    }
}

```

Note that the same 3840x2160 multiview layout is not limited to be displayed on a 4K monitor. It can also be applied to a 1080p monitor for example, by scaling the full multiview composition in the decoder (RX) from 3840x2160 (multiview layout size) to 1920x1080 (maximum resolution supported by the monitor), using the request below (same as above but with video arguments set to 1920x1080 rather than 3840x2160).

device/my_device_rx

```

{
    "op" : "set:multiview",
    "layout" : "PiP_Sample",
    "video" :
    {
        "width" : 1920,
        "height" : 1080,
        "fps" : 30
    }
    "subscriptions" :
    {
        "source_device" : "my_device_tx_a",
        "stream_index" : 1,
        "subscription_index" : 0
    },
    {
        "source_device" : "my_device_tx_b",
        "stream_index" : 1,
        "subscription_index" : 1
    }
}

```

6.10 Video Sources

The above case study reviewed the steps involved in creating and loading a PiP layout and applying it to a decoder (RX).

To manage the multiview mode, there are conditions that relate to video sources that need to be kept in mind when working with and configuring multiview layouts. This section reviews these conditions in relation to:

- Interlaced sources
- Chroma sub-sampled sources (YCbCr)
- 10-bit and 12-bit sources

6.10.1 Interlaced Video Sources

As described earlier, all HDMI video streams used by decoders (RX) running in the multiview mode must be in progressive format. This means, if the HDMI source being fed as the native stream (Stream 0) is an interlaced video source it cannot be assigned to a decoder (RX) operating in multiview mode. It is necessary to assign the scaled stream (Stream 1), as it is always in progressive format.

Also, should be noted that in regards to an interlaced video source, the frame rate divider option built into an encoder (TX) cannot be applied as it does not handle interlaced video sources. Refer to section 6.3.2.3 Encoder (TX) Bandwidth & Frame Rate Converter for more information.

Summary when dealing with interlaced video sources:

- When the HDMI source (stream 0) is interlaced video, the video stream fed into a multiview layout must be the scaled stream (Stream 1).
- The frame rate divider cannot be applied to the native stream (stream 0), although it can be applied to the scaled stream (stream 1).

6.10.2 Chroma Sub-Sampled Sources (YCbCr 4:2:2 and YCbCr 4:2:0)

Reminder, all video streams assigned to a multiview layout must be fully color sampled.

So, this means in the case of a YCbCr 4:2:2 or YCbCr 4:2:0 source, use the scaled stream (stream 1) which is always converted to YCbCr 4:4:4 and therefore compatible with the multiview mode.

6.10.3 10 or 12-bit Sources

All video streams used in multiview layouts must be 8-bit. This means that for any 10 or 12-bit video source, the native stream (Stream 0) cannot be used. The scaled stream (Stream 1) must instead be selected as it is always output in 8-bit progressive format.

6.11 Multiview Use Case

To manage and apply a multiview layout requires a series of API commands. It is likely an application client will read a script and send the requests to the API server, who in turn completes the request itself or forwards it to the appropriate endpoint device(s).

6.11.1 Sample Multiview Scripts

Scripts can contain multiview layout requests as well as other API requests. For example, scripts could contain:

Example of commands that could be included in a multiview script:

- Create a layout
- Load a layout
- Create or modify windows
- Assign sources to surfaces
- Join encoders to layout subscriptions

- Start streams
- Scale the native stream
- Reduce video frame rate of a native and/or scaled stream of an encoder (TX)

The following section provides a sample user case.

6.11.2 Sample Use Case

Below is a review of the requests that could be issued to build the sample Picture-in-Picture (PiP) layout outlined earlier, which contained two windows. The larger window filling the entire output display and the smaller overlaid on the first.

Reminder: These requests use the `POST` method.

1. First request issued in this sample script creates the layout `"some_layout"` with the total size set to `3840x2160`.
2. Next the native stream (stream 0) of the encoder device `d88039e83991` is stopped.
TIP: This is to prevent the over subscription regarding the output of the encoder, as for this example the decoder (RX) is subscribing to the scaled streams (stream 1) of the impacted encoder devices (TX). Refer to section 6.3 Managing the 10GbE Link for details and considerations on managing the 10GbE link.
3. The scaled stream (stream 1) of the encoder device `d88039e83991` is set to `3840x2160` and the stream then started. This stream will feed the larger of our two layout windows.
4. Steps 2 and 3 are repeated for the encoder device `d88039e55d9a`, with the scaled stream set to `960x540`.
5. Next, two windows are added to the layout `some_layout`.
 - a. The first window is assigned the index of 1, the horizontal and vertical positions are set to 0 and 0, respectively and the width and height are set to `3840x2160`. The surface target ID is assigned to surface 0, which supports the windows total size.
 - b. The second window is assigned the index of 0, which being lower than the first window ensures this window is overlaid on the first. The horizontal and vertical positions are set to 400 and 400, respectively and the width and height to `960x540`. The surface target ID is assigned to surface 1.

Note: The width and height specified for each of the windows matches the resolutions of the scaled stream.
6. Next entry in the sample request script, set the decoder (RX) `0016c04c46d7` to operate in `multiview` mode. This request also:
 - a. Loads the `"some_layout"` layout, setting the output resolution and frame rate to `3840x2160` and `30 fps` respectively.
 - b. Establishes the appropriate stream subscriptions for the decoder. In this example, this is the scaled streams of encoders `d88039e83991` and `d88039e55d9a`.

Note: Doing multiple tasks in a single request allows for a saving on the implementation time, subscriptions can however also be implemented or changed using a `join` request. Refer to the SDVoE Developers API Reference Guide for more information on

the join command.

Below are the requests referred to in the above explanation:

```
multiview/layout/some_layout
{"op" : "create", "width" : 3840, "height" : 2160}

device/d88039e83991
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free"
: True}

device/d88039e83991
{"op" : "set:scaler", "width" : 3840, "height" : 2160}

device/d88039e83991
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}

device/d88039e55d9a
{"op" : "stop", "stream_type" : "HDMI", "stream_index" : 0, "free"
: True}

device/d88039e55d9a
{"op" : "set:scaler", "width" : 960, "height" : 540}

device/d88039e55d9a
{"op" : "start", "stream_type" : "HDMI", "stream_index" : 1}

multiview/layout/some_layout/window/1
{"op" : "create", "horizontal_position" : 0, "vertical_position" :
0, "width" : 3840, "height" : 2160, "horizontal_offset" : 0,
"vertical_offset" : 0, "target_surface" : 0}

multiview/layout/some_layout/window/0
{"op" : "create", "horizontal_position" : 400, "vertical_position"
: 400, "width" : 960, "height" : 540, "horizontal_offset" : 0,
"vertical_offset" : 0, "target_surface" : 1}

device/0016c04c46d7
{"op" : "set:multiview", "layout" : "some_layout", "video" :
{"width" : 3840, "height" : 2160, "fps" : 30}, "subscriptions" :
[{"source_device" : "d88039e83991",
"stream_index":1,"subscription_index":0},{ "source_device" :
"d88039e55d9a", "stream_index":1,"subscription_index":1}]}
```

When creating or modifying multiview scripts, it is important to note that all entries in the script to be sent the API server must adhere to the required syntax, with the appropriate values entered for supported arguments as outlined in the SDVoE Developers API Reference Guide.

6.12 Multiview Guidelines & Limitations

There are some guidelines and limitations that apply to multiview layouts.

- The maximum number of windows supported is 32.
- The maximum number of distinct video sources (i.e. surfaces) is also 32.
- The number of tiles cannot exceed 15 in either the horizontal or vertical direction, and 16 in the other direction (i.e. a grid of 15×16 or 16×15 tiles are both acceptable but 16×16 is not).
- Tile widths and horizontal read offsets (equivalently window widths, horizontal positions and horizontal offsets) must be even.
- Tile width has a minimum width of 8-pixels.
- A tile width cannot be equal to the following values 1366, 1368, 2732, 2734, 4098 or 4100, in pixels.
- The total size of the frame buffer is 10880×4096.
- Surfaces must be horizontally aligned on a 32-pixel boundary.
- The total bandwidth sent by an individual transmitter and received by an individual receiver is recommended to not exceed 9 Gbps.
- Source video must be progressive and fully Chroma sampled (RGB, YCbCr 4:4:4).

Reminder: The total bandwidth sent by an individual encoder (TX) and received by an individual decoder (RX) must never exceed the 10 Gbps network bandwidth pipe. In fact, for all practical purposes, it is recommended that bandwidth usage never exceed 8.5 Gbps. This allows room for audio, USB, Gigabit Ethernet, control (IR and RS-232) and other auxiliary data to be included in the stream.

Note: When the `receive` path of an AVP2000T Transceiver device is operating in multiview mode, the device cannot subscribe to stream 0 or 1 of the same devices `transmit` path (HDMI local loopback is disabled).

Additional tips applicable when using the multiview mode:

1. For a video stream to be successfully displayed in a multiview layout, it must be in an uncompressed format.
 - For example, a 4K 60Hz RGB 8-bit stream cannot be used in multiview mode because it is compressed to fit into the 10GbE pipe.
2. When the total bandwidth output from an encoder surpasses the 10Gbps link but it is desired to output both the native and scaled streams, this can be managed by applying the frame rate divider option to either or both streams.
 - For example, divide the frame rate of the native stream (stream 0) on the encoder (TX) from 60 Hz to 30Hz.

TIP: The `set property` command is used for this option. Refer to section 8.3 Managing Frame Rate Divider for examples of using this option.

Note: The frame rate divider is only available on the `receive` path of an AVP2000T transceiver device. The implication of this is, when using the transmit path of a transceiver as a multiview source, the HDMI source must be managed such that it meets the image format and bandwidth required for the currently loaded multiview layout.

3. Video streams must be in RGB 8-bit or YCbCr 4:4:4 8-bit format.
 - This implies that when Stream 0 is subscribed to by a decoder operating in a multiview, the source must be either RGB 8-bit or YCbCr 444 8-bit. If it is not in one of these formats then the workaround is to use the scaled stream set to the appropriate resolution.
4. Video streams used in a multiview layout must be progressive.
 - If the source is an interlaced signal, it must be de-interlaced (converted to progressive) on the encoder side using the scaled stream (Stream 1) and this stream then subscribed to by the decoder (RX).
5. Applying both frame rate conversion and de-interlacing on an encoder simultaneously may negatively impact on video quality.

There is a workaround available for this scenario:

- If the video source is interlaced, convert it to a progressive signal on encoder (TX) using scaler (Stream 1).
- Apply the frame rate conversion on the decoder (RX) side.

6.13 Multiview Layout Changes

When a layout change request is issued, generally the transition change is seamless. To ensure a seamless transition occurs, the following conditions must be met:

- The total number of tiles of the current layout and the requested layout remains below 255.
- The total number of tiles of the requested layout must also remain below 128.

In the event the decoder (RX) cannot guarantee a seamless transition, a glitch-less transition of approximately 6-frames of black occurs between the current and requested layout change. When a video source (subscription) change occurs, the decoder (RX) will have a seamless transition within the impacted tiles.

On a video source (subscription) change, unaffected tiles of the decoder (RX) remain unchanged.

The figure below shows an example that would result in the approximate 6-frames of black during the transition of the current and new layouts. In the use case demonstrated here, a 4x4 layout is shown with a PiP window. The sources are 1920x1080 and 960x540 (PiP). The output stream resolution is 3840x2160@60. This layout uses 144 tiles, surpassing the requirements mentioned above.

For information regarding layout transitions, refer to section 6.13.3 Achieving Seamless Transitions.

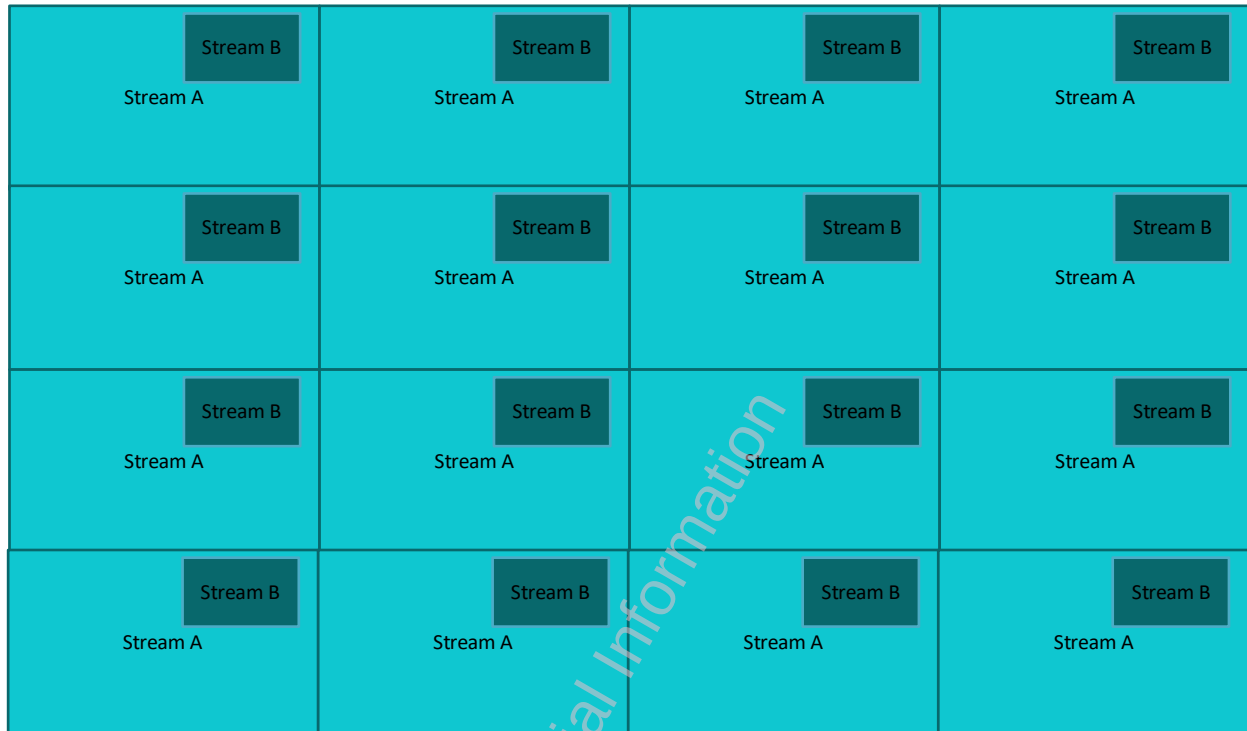


Figure 10: Example 4x4 Multiview Layout with PiP Overlay

When establishing subscriptions for the sources to be composited within a layout, this task can be completed using one of two methods.

1. The first method is to send independent requests, one request to set the decoder to operate in multiview mode and then a separate join request for each required source subscription.
For example, if loading an existing picture-in-picture layout, three API requests are required. One to set the decoder into multiview mode and two join requests, one for each of the sources to be included in the composited layout.
2. The second option, outlined in this application note, is to include a request for each required source subscription with the set video request.
 - Subscriptions can be added with a set multiview request, allowing for a layout to loaded quicker than when individual requests are sent.
 - Refer to section 6.8.6 Setting Decoder to Multiview Mode for details on loading a layout and establishing subscriptions in a single step.

Note: If there are unassigned windows in the layout that is loaded, the windows will contain black until video subscriptions are established.

6.13.1 Changing Active Layout

When a layout request is sent, new subscriptions are established while any existing subscriptions that are no longer needed are unsubscribed.

For example, if a quad layout is presently loaded where four subscriptions are currently established and a request is sent to change to another quad layout with three of the same subscriptions but the fourth window subscription is different, the first three window subscriptions remain as is but the fourth subscription is changed to the newly requested stream and the previous stream is unsubscribed.

Another example would be, a quad layout is currently loaded but a request is sent to change it to a new picture-in-picture layout with two new subscriptions required. These two new subscriptions are established and the original four streams are unsubscribed. The API server manages these assignments to ensure that phantom streams are not running in the background, potentially leading to oversubscription issues.

If it is desired to manually unsubscribe one or more decoders (RX) from an encoder stream, this task is completed using a `leave` request. If the currently active layout contains a window subscription for the stream that was unsubscribed, the window will contain black when the leave request completes. Refer to the SDVoE Developers API Reference Guide for more information on the `leave` command, including potentials errors.

6.13.2 Changing Video Mode

Changing a decoder (RX) from multiview mode to another operating mode, such as genlock or fast switch, is completed using a `set video` request. The API server unsubscribes the decoder (RX) stream subscriptions from the sources, leaving only subscription 0 subscribed, preventing any required action if this source is to remain subscribed. Otherwise, if the subscription is to be changed it is necessary to send a `join` request to subscribe the decoder (RX) to the desired stream. Refer to the SDVoE Developers API Reference Guide for more information on the `join` command.

6.13.3 Achieving Seamless Transitions

As mentioned above, when layouts are loaded the action is typically seamless. However, there are some instances when this is not the case and black is present during the transition period for approximately 6-frames.

To assist with having a good understanding when the above occurs, details on requirements required to have layouts load seamlessly are outlined below. The examples below outline the constraints and limitations that must be respected to achieve seamless transitions.

6.13.3.1 Use Case 1: Surface Overlap

When building a request to load a new multiview layout, it must be ensured that the memory space to be used by the new layout does not physically overlap memory space currently is in use.

The following example illustrates a use case in which the sole factor preventing seamless multiview transition is the overlapping of surfaces within the framebuffer.

Initial Layout Loaded

For this use case, first let's review the details of the current layout that is loaded `Single_Layout` for the decoder with the device ID `0016c04c46d7`. The encoder for this use case example is device ID `d88039e55d9a`, representing Encoder A. This initial layout has a single window using surface 0.

Below shows a summary of the REST requests that would be issued using the POST method:

```
multiview/layout/Single_Window
{"op" : "create", "width" : 3840, "height" : 2160}

multiview/layout/Single_Window/window/0
{"op" : "create", "horizontal_position" : 0, "vertical_position" : 0, "width" : 3840, "height" : 2160, "horizontal_offset" : 0, "vertical_offset" : 0, "target_surface" : 0}

device/0016c04c46d7
{"op" : "set:multiview", "layout" : "Single_Window", "video" : {"width" : 3840, "height" : 2160, "fps" : 30}, "subscriptions" : [{"source_device" : "d88039e83991", "stream_index":1, "subscription_index":0}, {"source_device" : "d88039e55d9a", "stream_index":1, "subscription_index":0}]}
```

Example of the decoder's output stream for the initial layout `Single_Layout`:



Illustration of the surface assigned for the above layout:

		2048 x 1080	2048 x 1080	2048 x 1080
		2048 x 1080	2048 x 1080	2048 x 1080
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 11: Multiview Transitions Use Case 1 – Single Layout and Surface Usage

For this use case example, we will now load a layout with four windows were Window 4 will request surface 17 which uses the same memory space as currently in use by the `Single_Layout` surface 0.

New Layout Request

Summary details of the second layout, referred to here as the `Quad_Layout`:

```
Total size 3840 2160
Window 0 position 0 0 size 1920 1080 target surface 1
Window 1 position 1920 0 size 1920 1080 target surface 2
Window 2 position 0 1080 size 1920 1080 target surface 3
Window 3 position 1920 1080 size 1920 1080 target surface 17
```

When the `set multiview` request is sent for the decoder 0016c04c46d7 to load the new layout `Quad_Layout`, the output resolution is set to 3840 2160 with fps of 30 and the requested subscriptions are as follows:

```
Encoder B: stream index 1 subscription_index (surface) 1
Encoder C: stream index 1 subscription_index (surface) 2
Encoder D: stream index 1 subscription_index (surface) 3
Encoder E: stream index 1 subscription_index (surface) 17
```

Example of the decoder output stream for the `Quad_Layout`:



Illustration of the surface allocation in the decoder's frame buffer:

		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 12: Multiview Transitions Use Case 1 – Quad Layout and Surface Usage

As outlined above, during the layout transition, a frame buffer memory space overlap occurs between surfaces 0 and 17, preventing a seamless layout change.

					
		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 13: Multiview Transitions Use Case 1 – Illustration of Memory Allocation Overlap

6.13.3.2 Use Case 2: Surface Reuse

Another requirement for a seamless transition is that when loading a new multiview layout, it must be ensured that the surfaces requested for the new layout are not already in use by the current layout.

Note that for the following example, a custom surface configuration is in use for the currently loaded layout, `Single_Window`.

Initial Layout Loaded

For this second use case, the summary for the currently loaded layout, `Single_Window`, is as follows:

Total layout size is 3840 2160

Includes a single window, window 0 position 0 0 size 4096 2160 target surface 1

The subscription of the decoder (RX) is to stream index 1 and subscription_index (surface) 0.

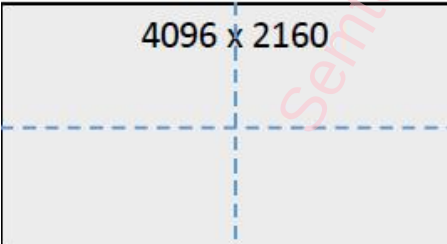
				2048 x 1080	
				2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 14: Multiview Transitions Use Case 2 – Illustration of Memory Allocation of Current Layout

New Layout Request

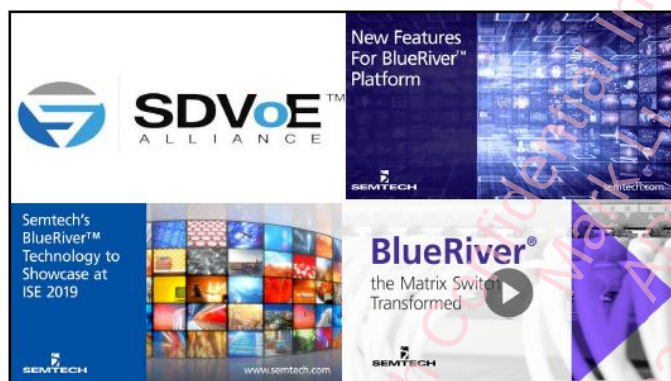
Summary details of the second layout request Quad2_Layout, are as follows:

```
Total size 3840 2160
Window 0 position 0 0 size 1920 1080 target surface 7
Window 1 position 1920 0 size 1920 1080 target surface 0
Window 2 position 0 1080 size 1920 1080 target surface 8
Window 3 position 1920 1080 size 1920 1080 target surface 9
```

When the set multiview request is sent to the decoder for the new layout, Quad2_Layout the output resolution is 3840 2160 fps 30, as with use case 1 of interest is the requested subscriptions but for this layout the requested subscriptions are as follows:

```
Encoder B: stream index 1 subscription_index (surface) 7
Encoder C: stream index 1 subscription_index (surface) 8
Encoder D: stream index 1 subscription_index (surface) 9
Encoder E: stream index 1 subscription_index (surface) 0
```

Example of the decoder's output stream for this quad layout:



Surface assignments and configuration:

		2048 x 1080	2048 x 1080	2048 x 1080	
		2048 x 1080	2048 x 1080	2048 x 1080	
			2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 15: Multiview Transitions Use Case 2 – Illustration of Memory Allocation of Quad Layout

For this use case, the surface allocations of the request do not contain an overlap with current layout. During the layout transition, as illustrated below, there is no memory overlap as with the previous use case. However, since both layouts subscribed to target surface 0, this prevents a seamless transition.

				2048 x 1080
				2048 x 1080
			2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 16: Multiview Transitions Use Case 2 – Illustration of Memory Allocation During Transition

6.13.3.3 Use Case 3: Repositioning Active Surfaces

When requesting a new Multiview Layout, the requester must ensure that surfaces remaining in use do not move inside the frame buffer.

Initial Layout Loaded

The summary of the Quad3_Layout currently loaded on the decoder (RX) is as follows:

Total size 3840 2160

Includes four windows as follows:

Window 0 position 0 0 size 1920 1080 target surface 17
 Window 1 position 1920 0 size 1920 1080 target surface 18
 Window 2 position 0 1080 size 1920 1080 target surface 19
 Window 3 position 1920 1080 size 1920 1080 target surface 20

The decoder which is operating in multiview mode with the Quad3_Layout loaded, has an output resolution of 3840 2160 fps 30 and the following active subscriptions:

Encoder A: stream index 1 subscription_index (surface) 17
 Encoder B: stream index 1 subscription_index (surface) 18
 Encoder C: stream index 1 subscription_index (surface) 19
 Encoder D: stream index 1 subscription_index (surface) 20

Example of the decoder output stream for the Quad3_Layout:



Surface allocation for this quad layout:

		2048 x 1080	2048 x 1080	2048 x 1080	
		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 17: Multiview Transitions Use Case 3 – Illustration of Memory Allocation of Current Layout

New Layout Request

Summary details of the new layout to be loaded, Quad4_Layout:

Total size 3840 2160

Includes four windows as follows:

Window 0 position 0 0 size 1920 1080 target surface 17
 Window 1 position 1920 0 size 1920 1080 target surface 2
 Window 2 position 0 1080 size 1920 1080 target surface 19
 Window 3 position 1920 1080 size 1920 1080 target surface 5

This layout request also includes two surface configuration changes:

Surface 17 position 4096 0 size 2048 1080
 Surface 19 position 4096 1080 size 2048 1080

When the set multiview request is sent for the decoder to load the new layout Quad4_Layout, the total output resolution is 3840 2160 fps 30 and the requested subscriptions are as follows:

Encoder A: stream index 1 subscription_index (surface) 17
Encoder C: stream index 1 subscription_index (surface) 19
Encoder E: stream index 1 subscription_index (surface) 2
Encoder F: stream index 1 subscription_index (surface) 5

Example of the decoder's output stream for the new quad layout:



Illustration of the surface allocation for this layout:

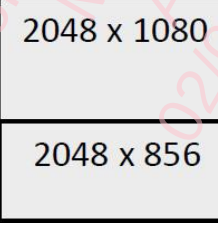
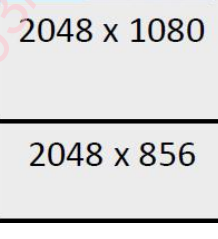
4096 x 2160		 		
		 		2048 x 1080
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 18: Multiview Transitions Use Case 3 – Illustration of Memory Allocation of New Layout

During the multiview transition, there is no overlap, and all requested surfaces were not previously in use, however, surfaces 17 and 19 were moved within the decoder's frame buffer, preventing a seamless transition.

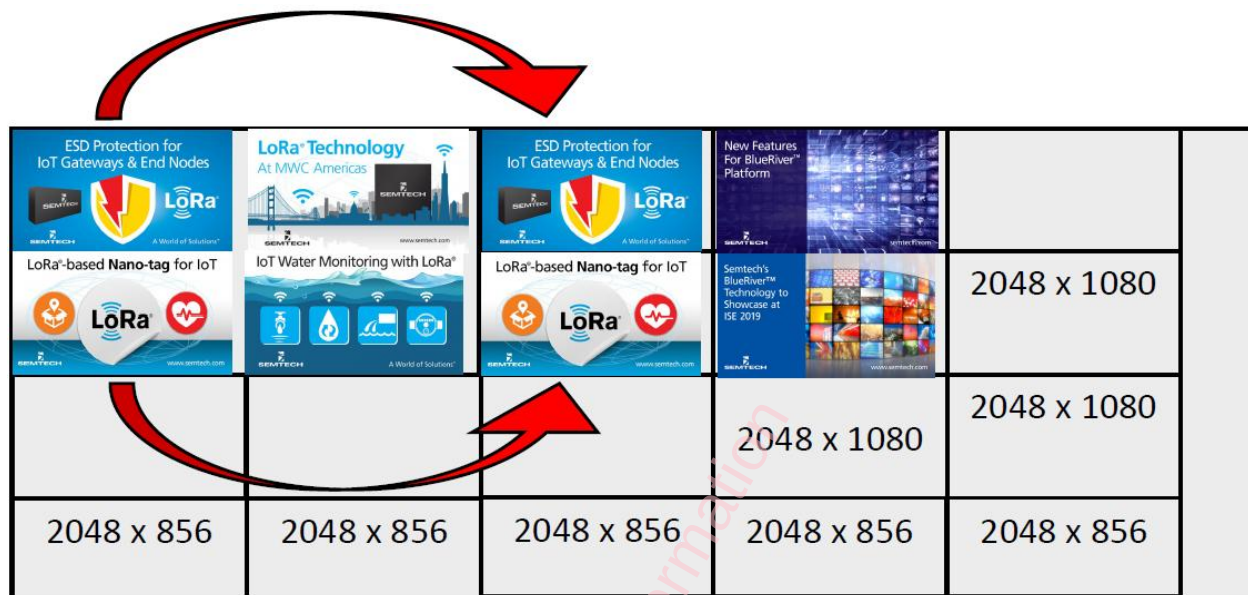


Figure 19: Multiview Transitions Use Case 3 – Illustration of Memory Reallocation

6.13.3.4 Use Case 4: Maximum Layout Size

As previously defined at the beginning of this section, for a seamless multiview transition to occur, the sum of the quantity of tiles in the current and requested layouts must be equal to or less than 255. Furthermore, transitioning to a layout containing more than 128 tiles, results in black being displayed during the layout transition. Refer to 6.13.3 Achieving Seamless Transitions for more information.

6.13.3.5 Use Case 5: Simple Layout Transitions

The following example navigates an AVP2000 decoder through five (5) different multiview layouts, for a total of four (4) layout transitions. These transitions are seamless when the layouts are loaded in the order outlined here, as all the conditions outlined in the previous sections are met. This means the surface allocations are unique and the set multiview request includes the subscription requests.

Initial Layout Loaded

The initial layout in this series is a single window layout.

The summary of the initial active layout `Single_Window`, is as follows:

Total layout size is 3840 2160

Includes a single window:

Window 0 position 0 0 size 3840 2160 target surface 0

The subscription of the decoder (RX) is to stream index 1 and subscription_index (surface) 0.

Example of the decoder output and surface allocation:

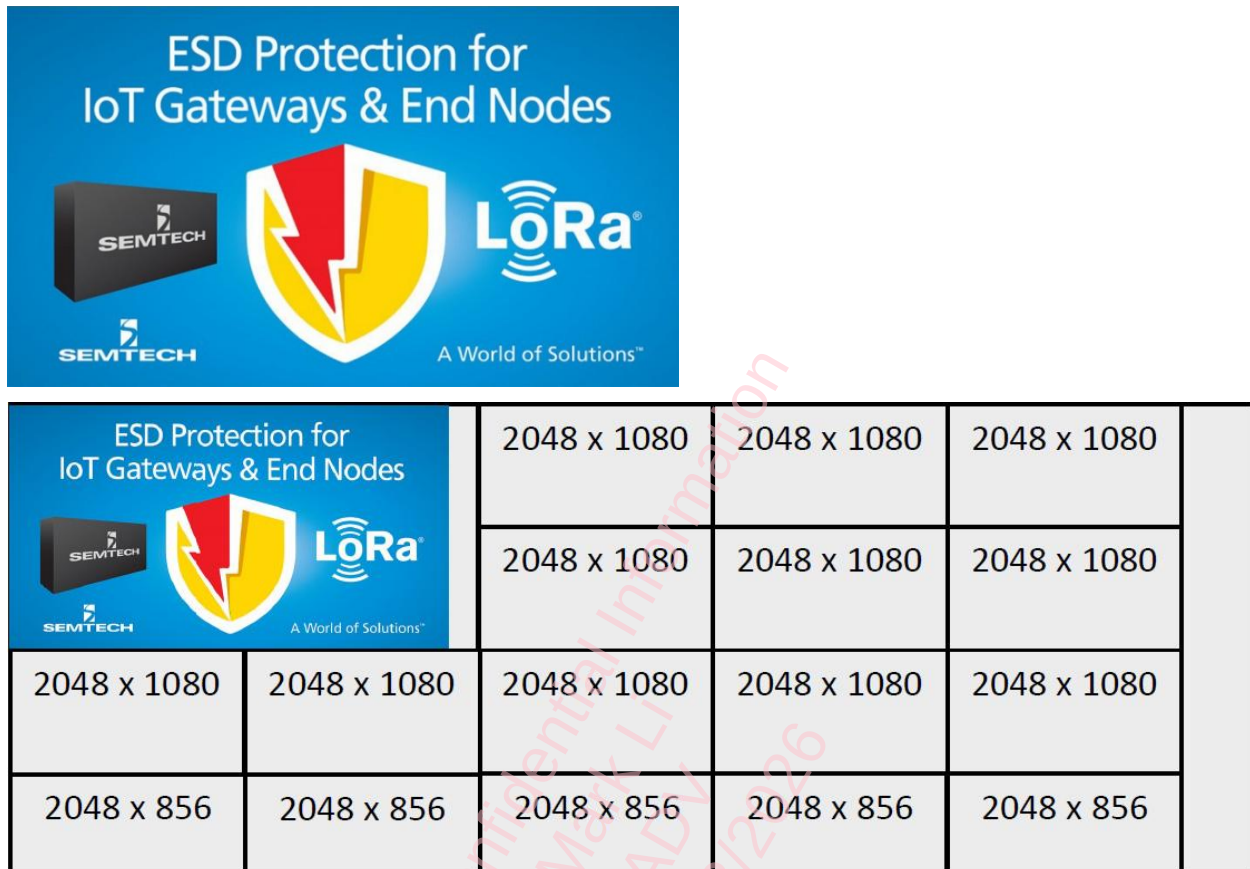


Figure 20: Multiview Transitions Use Case 4 – Current Memory Allocation of Single Layout

PiP Layout Request Loaded

Summary details of the second layout, PiP_Layout:

Total size 3840 2160

Includes two windows as follows:

Window 0 position 2240 1400 size 960 540 target surface 12

Window 1 position 0 0 size 3840 2160 target surface 1

This layout request also includes a surface configuration change:

Surface 1 position 4096 0 size 4096 2160

When the `set multiview` request is sent for the decoder to load the PiP_Layout with total size of 3840 2160 fps 30, the requested subscriptions are as follows:

Encoder B: stream index 1 subscription_index (surface) 1

Encoder C: stream index 1 subscription_index (surface) 12

Example of the decoder output stream for the PiP_Layout:



Illustration of the surface allocation:

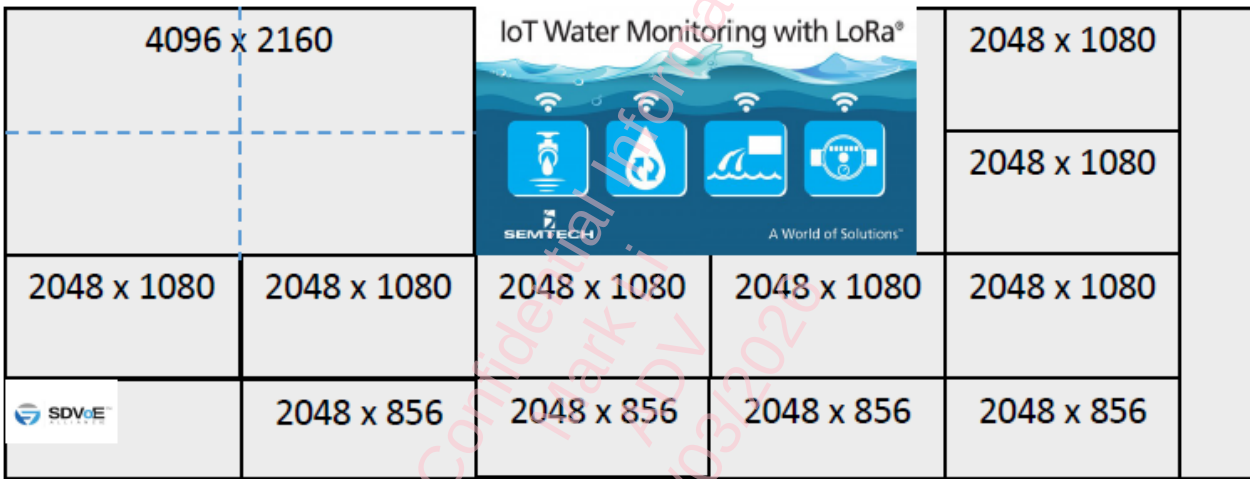


Figure 21: Multiview Transitions Use Case 4 – Memory Allocation of PiP Layout

PiP Corner Window Layout Request Loaded

This third layout overlays four smaller windows over a fifth larger window, creating a form of a PiP layout. For this illustration, this layout is referred to as the PiPCorner_Layout.

Summary details of a third layout, PiPCorner_Layout:

Total size 3840 2160

Includes four windows as follows:

- Window 0 position 3200 0 size 640 360 target surface 13
- Window 1 position 0 0 size 640 360 target surface 14
- Window 2 position 3200 1800 size 640 360 target surface 15
- Window 3 position 0 1800 size 640 360 target surface 16
- Window 4 position 0 0 size 4096 2160 target surface 1

This layout request also includes a surface configuration change:

Surface 1 position 4096 0 size 4096 2160

When the `set multiview` request is sent for the decoder to load the `PiPCorner_Layout` with total size of 3840 2160 fps 30, the requested subscriptions are as follows:

Encoder D: stream index 1 subscription_index (surface) 13
 Encoder E: stream index 1 subscription_index (surface) 14
 Encoder F: stream index 1 subscription_index (surface) 15
 Encoder G: stream index 1 subscription_index (surface) 16

Note that the existing subscription for Encoder B: stream index 1 subscription_index (surface) 1 is not changed.

Example of the decoder output stream for the `PiPCorner_Layout`:



Illustration of the surface allocation:

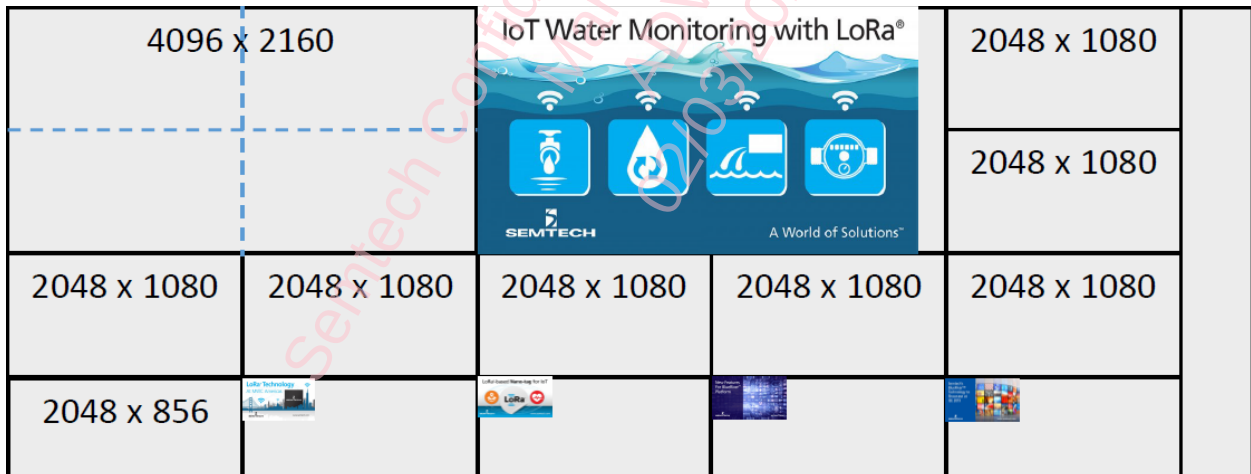


Figure 22: Multiview Transitions Use Case 4 – Memory Allocation of PiPCorner Layout

Quad Layout Request Loaded

Summary details of the fourth layout request, referred to as `Quad5_Layout`:

Total size 3840 2160
 Window 0 position 0 0 size 1920 1080 target surface 7
 Window 1 position 0 1080 size 1920 1080 target surface 8
 Window 2 position 1920 0 size 1920 1080 target surface 9
 Window 3 position 1920 1080 size 1920 1080 target surface 10

When the `set multiviewrequest` is sent to load the new layout `Quad5_Layout`, with the output resolution of `3840 2160 fps 30`, the requested subscriptions are as follows:

```
Encoder H:      stream index 1 subscription_index (surface) 7
Encoder J:      stream index 1 subscription_index (surface) 8
Encoder K:      stream index 1 subscription_index (surface) 9
Encoder L:      stream index 1 subscription_index (surface) 10
```

Example of the decoders output stream for the `Quad5_Layout`:



Illustration of the surface allocation:

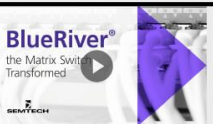
4096 x 2160		2048 x 1080	2048 x 1080	2048 x 856	
		2048 x 1080	2048 x 1080	2048 x 1080	
				2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 23: Multiview Transitions Use Case 4 – Memory Allocation of Quad5 Layout

Layout request PaP

The fifth layout in this sequence is a Picture-and-Picture layout, referred to as `PaP_Layout`.

This layout contains two windows which are configured side by side and use two surface allocations.

Details of this layout are as follows:

```
Total size 3840 2160
Window 0 position 0 540 size 1920 1080 target surface 3
Window 1 position 1920 540 size 1920 1080 target surface 6
```

When the `set multiview` request is sent to load the new layout `PaP_Layout`, it will request the output resolution of 3840 2160 fps 30 and establish subscriptions as follows:

Encoder M: stream index 1 subscription_index (surface) 3
Encoder N: stream index 1 subscription_index (surface) 6

Example of the decoder output stream for the `PaP_Layout`:



Illustration of the surface allocation:

4096 x 2160		2048 x 1080	2048 x 1080	
		2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 24: Multiview Transitions Use Case 4 – Memory Allocation of PaP Layout

Summary: This series of layout changes transition cleanly as the constraints outlined previously are avoided and the update of required subscriptions are established when the new layout is loaded as part of the `set multiview` request. Refer to section 6.8.6.1 Managing Decoder Subscriptions for details on managing stream subscriptions.

6.13.3.6 Use Case 5: Simultaneous Source Change

It is possible to simultaneously change more than one video subscription by applying the same multiview layout configuration while specifying different subscriptions.

This is particularly useful when a visually sequential transition is not acceptable. For example, when the video stream subscription requests are sent individually using a `join` request, each change takes an interval of approximately 150-200ms.

Looking at an example 3x3 layout, which requires 9 stream subscriptions, when multiplied by 200ms the total time is approximately 2 seconds. In comparison, when the same subscriptions are completed in a single request, the total time is approximately 500ms (0.5 seconds).

The following illustration outlines loading layouts which include stream subscription changes. These transitions would be seamless as different surface allocations are used and the subscriptions are established in a single request using the set multiview command.

The summary for the initial layout Quad6_Layout loaded on the decoder (RX), is as follows

```
Total size 3840 2160
Window 0 position 0 0 size 1920 1080 target surface 17
Window 1 position 0 1080 size 1920 1080 target surface 18
Window 2 position 1920 0 size 1920 1080 target surface 19
Window 3 position 1920 1080 size 1920 1080 target surface 20
```

When the set multiview request is sent to load the Quad6_Layout, it requests the output resolution of 3840 2160 fps 30 and establishes subscriptions as follows:

```
Encoder A:    stream index 1 subscription_index (surface) 17
Encoder B:    stream index 1 subscription_index (surface) 18
Encoder C:    stream index 1 subscription_index (surface) 19
Encoder D:    stream index 1 subscription_index (surface) 20
```

Example of the decoder multiview output for the initially loaded layout, Quad6_Layout:



Surface Allocation of the Quad6_Layout:

		2048 x 1080	2048 x 1080	2048 x 1080
		2048 x 1080	2048 x 1080	2048 x 1080
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 25: Multiview Transitions Use Case 5 – Memory Allocation of Initial Quad Layout

The second layout in this series, Quad7_Layout, again has new surface allocations and subscriptions.

Details of the layout itself are as follows:

Total size 3840 2160

Window 0 position 0 0 size 1920 1080 target surface 1

Window 1 position 0 1080 size 1920 1080 target surface 2

Window 2 position 1920 0 size 1920 1080 target surface 4

Window 3 position 1920 1080 size 1920 1080 target surface 5

When the `set multiview request` is sent to load the above Quad7_Layout, it requested the output resolution of 3840 2160 fps 30 and established subscriptions as follows:

Encoder E: stream index 1 subscription_index (surface) 1

Encoder F: stream index 1 subscription_index (surface) 2

Encoder G: stream index 1 subscription_index (surface) 4

Encoder H: stream index 1 subscription_index (surface) 5

Example of the decoder output stream:



Illustration of the requested surface allocations:

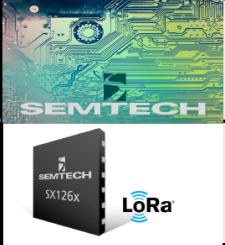
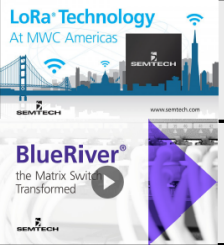
4096 x 2160				2048 x 1080
				2048 x 1080
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 26: Multiview Transitions Use Case 5 – Memory Allocation of Quad7 Layout

The next layout, Quad8_Layout, changes two of the four current surface allocations and subscriptions.

Details of the layout itself are as follows:

Total size 3840 2160
Window 0 position 0 0 size 1920 1080 target surface 1
Window 1 position 0 1080 size 1920 1080 **target surface 17**
Window 2 position 1920 0 size 1920 1080 target surface 4
Window 3 position 1920 1080 size 1920 1080 **target surface 19**

When the set multiview request is sent to load the Quad8_Layout, it requests the output resolution of 3840 2160 fps 30 with the following subscriptions:

Encoder E: stream index 1 subscription_index (surface) 1
Encoder J: stream index 1 **subscription_index (surface) 17**
Encoder G: stream index 1 subscription_index (surface) 4
Encoder K: stream index 1 **subscription_index (surface) 19**

Example of the decoder output stream, note the windows 1 and 3 subscriptions were modified:



Illustration of the requested surface allocations for the Quad8_Layout:

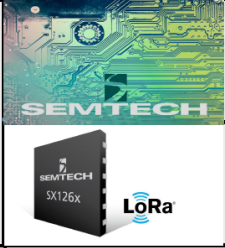
			2048 x 1080	2048 x 1080
			2048 x 1080	2048 x 1080
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856

Figure 27: Multiview Transitions Use Case 5 – Memory Allocation of Quad8 Layout

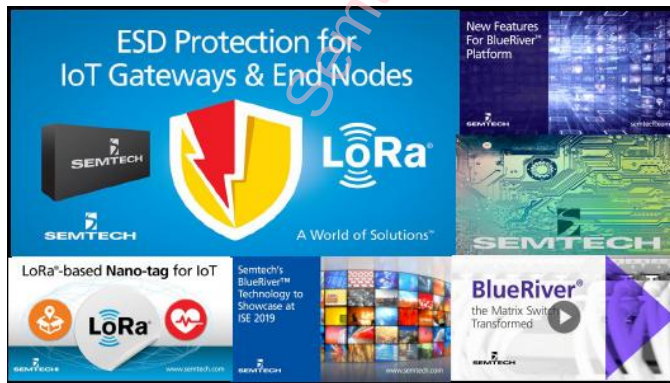
6.13.4 Transition Corner Cases & Limitations

In this section, some specific corner cases, and limitations of the multiview mode are reviewed that should be considered when creating and applying multiview layouts. This information is of particular importance when planning applications where seamless transitions are preferred and avoiding these types of scenarios will prevent outputting black during transition periods.

6.13.4.1 Scaled Stream Reconfiguration (Stream 1)

A decoder (RX) cannot be expected to synchronize an active multiview layout application with the live scaling reconfiguration of an encoder (TX) video source to which it is currently subscribed. For example, if the scaled stream resolution is modified, the layout windows do not automatically resize. The two events happen asynchronously on two distinct and separate devices. However, there are solutions that have the potential to offer the desired system behavior.

For the following examples, the initial video output of the decoder is as illustrated below:



The requested video output:



Solution #1 – Sequential Reconfiguration

As the name implies, this solution consists of sending a series of requests that, in the order shown, perform the following operations:

1. Unsubscribe the decoder from the first video stream that is to be rescaled using a `leave` request.
2. Unsubscribe the decoder from the second video stream to be rescaled also using a `leave` request.
3. For the first video stream, send a request to the appropriate encoder (TX) using a `set:scaler` command to reconfigure its scaled stream (stream 1).
4. Then for the second video stream, send another `set:scaler` request to reconfigure the scaled stream (stream 1) of the second encoder (TX).
5. Subscribe to the first video that was rescaled.
6. Subscribe to the second video that was rescaled.

Visually, the complete transition will look as depicted below:



Figure 28: Illustration of Solution 1

Solution #2 – Dual Video Stream

The second solution requires the usage of both the native (stream 0) and scaled (stream 1) streams from the required encoder devices (TX). For this solution to be successfully applied, both stream 0 and stream 1 must be configured at the required pre-defined resolutions.

TIP: The frame rate divider can be applied to either the native and/or scaled streams, as outlined earlier this option is useful in managing the output bandwidth of an encoder (TX).

An example of this use case is illustrated below.

Initial Layout

The initial layout in this example is referred to as `LShape` and it contains six windows. The summary for this initial layout is as follows:

```
Total layout size is 3840 2160
Window 0 position 0 0 size 2560 1440 target 0
Window 1 position 0 1440 size 1280 720 target 12
Window 2 position 1280 1440 size 1280 720 target 13
Window 3 position 2560 1440 size 1280 720 target 14
Window 4 position 2560 0 size 1280 720 target 15
Window 5 position 2560 720 size 1280 720 target 16
```

When the `set_multiviewrequest` is sent to load the new layout `LShape_Layout`, it requests the output resolution of 3840 2160 fps 30 and establishes subscriptions as follows:

```
Encoder A:    stream index 0 subscription_index (surface) 0
Encoder B:    stream index 1 subscription_index (surface) 12
Encoder C:    stream index 1 subscription_index (surface) 13
Encoder D:    stream index 1 subscription_index (surface) 14
Encoder E:    stream index 1 subscription_index (surface) 15
Encoder F:    stream index 1 subscription_index (surface) 16
```

Example of the decoder output stream for the layout, `LShape`:

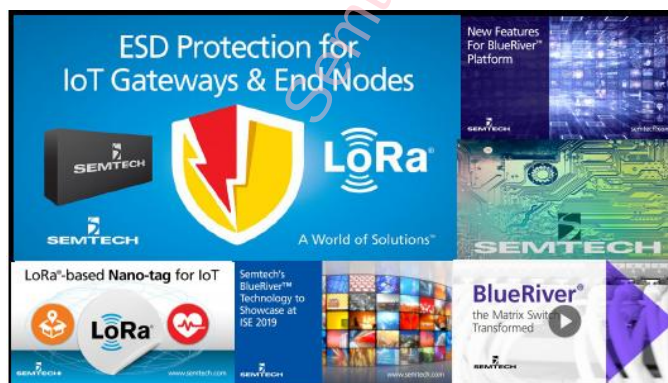


Illustration of the surface allocation:

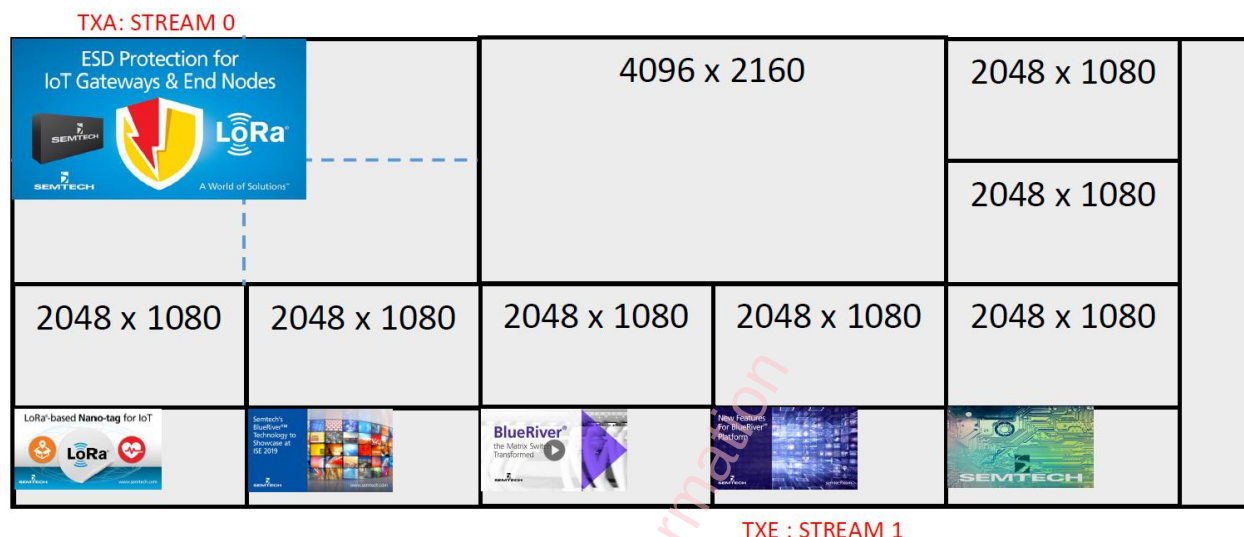


Figure 29: Illustration Initial Layout of Solution 2

New Layout Request

A new layout configuration is requested for the LShape layout loaded above. Note that windows 0 and 4 target index assignments are modified as well as their associated subscriptions and that surface 1 is also being modified.

Total layout size is 3840 2160

Window 0 position 0 0 size 2560 1440 target 1

Window 1 position 0 1440 size 1280 720 target 12

Window 2 position 1280 1440 size 1280 720 target 13

Window 3 position 2560 1440 size 1280 720 target 14

Window 4 position 2560 0 size 1280 720 target 10

Window 5 position 2560 720 size 1280 720 target 16

Surface 1 position 4096 0 size 4096 2160

When the set multiview request is sent to load the new LShape_Layout layout, it requests the output resolution of 3840 2160 fps 30 and establishes the following subscriptions:

Encoder A: **stream index 1 subscription_index (surface) 10**

Encoder B: stream index 1 subscription_index (surface) 12

Encoder C: stream index 1 subscription_index (surface) 13

Encoder D: stream index 1 subscription_index (surface) 14

Encoder E: **stream index 0 subscription_index (surface) 1**

Encoder F: stream index 1 subscription_index (surface) 16

Example of the decoder output stream for the second LShape layout configuration:



Illustration of the surface allocation for the new layout request:

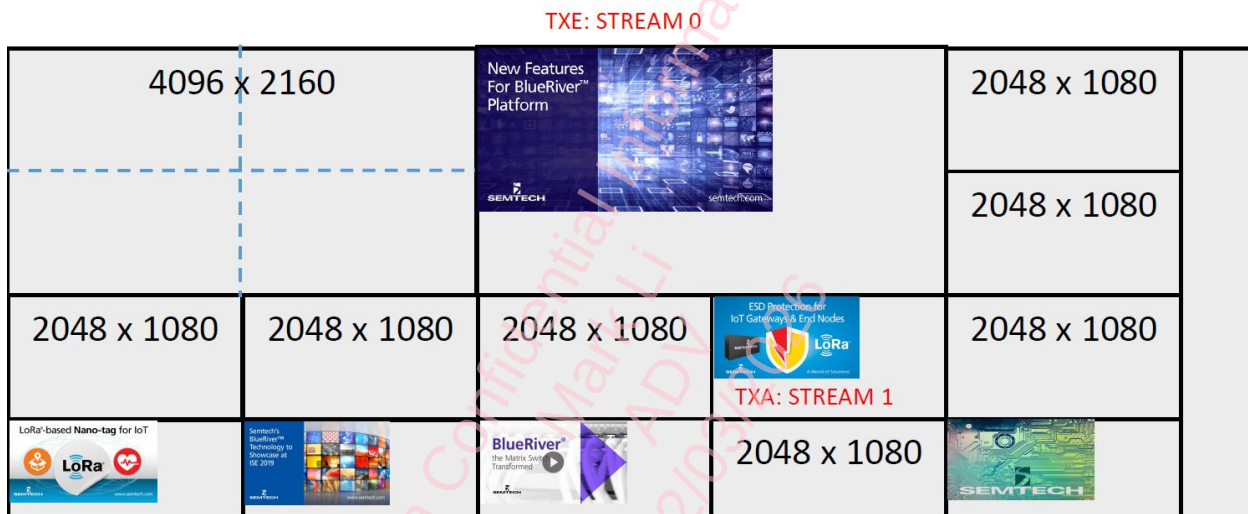


Figure 30: Illustration of Second Layout Configuration for Solution 2

6.13.4.2 Layout Changes Requiring More Than 32 Surfaces

Due to the surface reuse constraint, any layout transition that requires more than 32 shared surfaces between the current and requested layout cannot be completed seamlessly.

For example, when transitioning from a 5X5 layout to a 3X3 layout with new subscriptions, the 5X5 layout requires 25 surfaces and the 3X3 layout requires 9 for a total of 34 surfaces. In this case, the reuse of surfaces cannot be avoided.

6.13.4.3 Layout Change with Window Reshuffling

To ensure a seamless transition, when loading an alternate layout with new subscriptions, they must be attached to unused surfaces. Moving and/or rotating subscriptions between active windows must be completed by moving the window placement of the multiview layout, not through reassigning currently active subscriptions to active surfaces. This section will review some proposed solutions for the issue described above.

Initial layout active the decoder (RX):



Shuffling of the layout window positions:



Figure 31: Illustration of Window Reshuffling

Solution – Window Repositioning + Single Added Subscription

A solution to this problem is illustrated below using a quad layout as an example.

Initial Layout

The summary of the currently loaded layout on the decoder (RX) is as follows:

Total size 3840 2160

Includes four windows as follows:

```
Window 0 position 0 0 size 1920 1080 target surface 17
Window 1 position 0 1080 size 1920 1080 target surface 18
Window 2 position 1920 0 size 1920 1080 target surface 19
Window 3 position 1920 1080 size 1920 1080 target surface 20
```

The decoder running the current quad layout described above is set to an output resolution of 3840 2160 fps 30 and has the following active subscriptions:

```
Encoder A: stream index 1 subscription_index (surface) 17
Encoder B: stream index 1 subscription_index (surface) 18
Encoder C: stream index 1 subscription_index (surface) 19
Encoder D: stream index 1 subscription_index (surface) 20
```


Example of the decoder output stream for the initial layout:



Surface allocation of the initial layout:

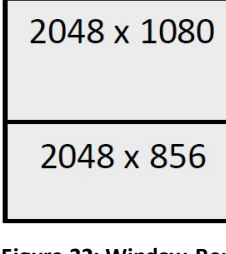
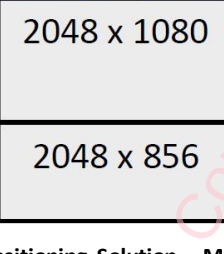
		2048 x 1080	2048 x 1080	2048 x 1080	
		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 32: Window Repositioning Solution – Memory Allocation of Current Layout

New Layout Request

Summary details of the new layout to be loaded:

Total size 3840 2160

Includes four windows as follows:

Window 0 position 0 1080 size 1920 1080 target surface 17

Window 1 position 1920 1080 size 1920 1080 target surface 18

Window 2 position 0 0 size 1920 1080 target surface 19

Window 3 position 1920 0 size 1920 1080 target surface 1

When the `set multiview` request is sent for the decoder to load the new layout, the total output resolution is 3840 2160 fps 30 and the requested subscriptions are as follows:

Encoder A: stream index 1 subscription_index (surface) 17

Encoder B: stream index 1 subscription_index (surface) 18

Encoder C: stream index 1 subscription_index (surface) 19

Encoder E: stream index 1 subscription_index (surface) 1

Example of the decoder output stream for the new quad layout:



Illustration of the surface allocation for this layout:

			2048 x 1080	2048 x 1080	
		2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	2048 x 1080	
2048 x 856	2048 x 856	2048 x 856	2048 x 856	2048 x 856	

Figure 33: Window Repositioning Solution – Memory Allocation of New Layout

7 BlueRiver Device Video Wall Mode

SDVoE endpoint devices that implement the AV Processing Engine support creating video wall grids. To assist a developer with programming this functionality into a custom application, this section reviews details of implementing and managing a video wall.

Specifically:

- Reviews the wall mode in relation to the SDVoE feature.
 - Comparison of the Genlock and Fast Switch Wall modes
- Reviews implementing a Source Independent Wall, including:
 - Requirements
 - API request and associated arguments
 - Review of the SI Wall node.
- Configuring a Custom Video Wall:
 - Provides an example of implementing a simple video wall grid.
 - Review the “set” command and its associated video wall arguments.
 - Bezel compensation.
 - Example calculations to determine the values required for API requests.
 - Review of managing video walls with different source versus wall aspect ratios.
 - Applying viewports, including calculation and application.

This section is of interest to software designers, system integrators and others who are involved in implementing a video wall using SDVoE technology.

7.1 Introduction to Video Wall Mode

The Video Wall mode renders a single video source across multiple displays.

- The SDVoE solution supports video wall capabilities without the use of additional equipment.
- Each display is connected to a decoder (RX).
- The video source is connected to an encoder (TX) that sends the source video to each of the subscribed decoders (RX) that assimilate the wall grid.
- Each decoder (RX) crops an “Area of Interest” according to its position in the wall grid and then uses the scaling engine to generate an output raster to display the cropped area on screen.
- A black pillarbox or letterbox can be applied to keep the aspect ratio of the source intact.

When in Wall mode, a decoder (RX) uses multiple processing blocks inside the AV processing engine. Refer to Figure 34: Block View of Decoder Processing below for illustration. The decoder’s cropping engine is used to crop the Area of Interest (AOI) from the initial source. The AOI is presented to the scaler to display the cropped region.

To compensate for the display bezels, cropping regions are adjusted, virtually hiding the source pixels. The goal of bezel compensation is to make the source image appear to continue behind the bezels, as if the bezels are part of a window frame that is overlaid on top of the displayed source image. Bezel compensation is reviewed in more detail in section 7.3.5 Managing Bezel Compensation.

The chipset's scaler receives the AOI from the cropping engine and then generates an output raster from the image data inside the AOI. The scaler has no intelligence; it simply scales the Area of Interest (AOI) to fill the output raster unless the viewport option is applied.

To avoid geometric distortion, one of two scenarios applies:

1. The Area of Interest (AOI) and output raster must have the same geometry (aspect ratio); or
2. The black generator can be programmed to create a viewport with same aspect ratio as the Area of Interest (AOI).
 - a. The black generator is used to create a viewport inside the output raster.
 - b. Source data inside the AOI is then displayed inside the viewport.
 - Outside the viewport, the output raster is filled with black pixels.
 - The viewport is used when the source and wall aspect ratios differ.

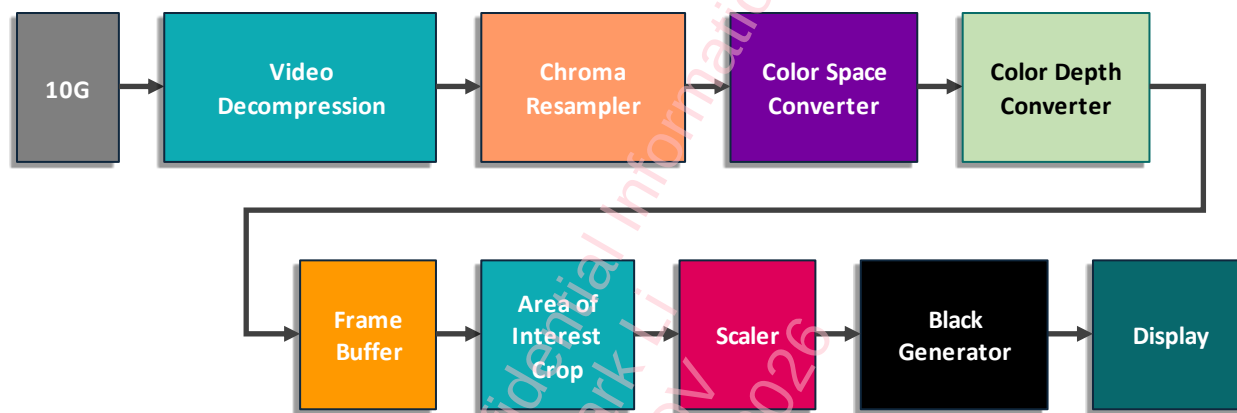


Figure 34: Block View of Decoder Processing

Two video wall modes are available:

- Genlock Wall (synchronized)
- Fast Switch Wall (asynchronous)

The same programming and control options are available for both wall modes, including cropping, scaling and viewports, as well as bezel and overlap management.

The Genlock and Fast Switch Wall video modes can coexist within the same system, although only one mode can be assigned to a single decoder device (RX).

Regardless of the wall mode applied, the output of a decoder (RX) operating in wall mode is always in RGB 8-bit format and BT.709 color space.

7.1.1 Genlock Wall Mode

When set in Genlock wall mode, the decoders (RX) are all synchronized (time-locked) to the source and therefore to one another.

Benefits:

- i. Deterministic latency. First row is 1 frame behind the source, then each subsequent row is slightly delayed reducing screen tearing such that the bottom of last row is full frame behind the beginning of the first row.
- ii. Each display is synchronized (genlocked) to the source and thus to each other.
- iii. Video quality of a genlock wall is better overall when compared to a fast switch wall.

Limitations:

- i. Genlock wall mode has been validated for three network hops.
 - When more than three network hops are required, synchronization issues could occur. A suggested workaround is to arrange the system such that the required source and all wall displays are connected to the same network switch (switch segment).
 - The other option is to implement the video wall using the Fast Switch Wall mode.
- ii. Fast switching between different sources is not possible. Due to the nature of Genlock, switching from once source to another source requires the decoder (RX) and its attached display to resynchronize to the new source.
- iii. Frame rate conversion is not available in this mode.
 - For a decoder (RX), this means that the frame rate of the RX must match the frame rate of the source to which it is subscribed. For example, if the source is 60 fps then the output frame rate must also be 60 fps.
 - For an encoder (TX), the divide frame rate by two is not supported.

7.1.2 Fast Switch Wall Mode

The Fast Switch wall mode is an asynchronous video wall. It does not have the limitations of the Genlock video wall in regard to transporting streams through more than three network hops or changing sources on-the-fly.

While the Genlock video provides better overall visual quality, a Fast Switch video wall is comparable to the video wall functionality offered by most matrix switches (not including dedicated video wall processors). The Fast Switch Wall mode uses the chipset's frame buffer, allowing for fast switching between sources and for signals to cross multiple switches but does result in individual screens that are not in sync with each other.

Benefits:

The primary benefits of a Fast Switch wall are:

- i. Using the built-in scaler means that each display is always synchronized to the decoder (RX) and the display does not have to resynchronize when switching between sources.
- ii. The option of a video wall is always available regardless of network topology (single switch versus multiple switches).
- iii. Source changes occur quickly and cleanly (Fast Switch).
- iv. Frame rate conversion is available in this mode.
 - For a decoder (RX), this means that the frame rate can be different than the frame rate of the source it is subscribed to.
 - For an encoder (TX), the divide frame rate by two is supported and can be applied to either the native stream (stream 0) and/or scaled stream (stream 1).

Limitations:

The main limitation of this mode is that displays are not in sync with each other by up to 1 frame. This implementation uses an independent frame buffer on each receiver, which generates latency that is between 1-2 frames.

7.1.3 Genlock & Fast Switch Wall Modes Summary

The key difference between Genlock and Fast Switch Wall modes is the pros and cons. The best wall mode to apply is dependent on the actual use case.

- Genlock Wall pros are that all screens are in sync with the source therefore with each other, reduced screen tearing and a fixed latency.
- Genlock Wall cons are if the source changes, the switch takes time, therefore it is not seamless. Also, genlock mode synchronization may be impacted if there are more than three network switch hops.
- Fast Switch Wall pros include that it can quickly and seamlessly switch between sources. This mode can also be used when more than 3 switch hops are required.
- Fast Switch cons are latency is variable and that up to 1 frame of screen tearing can occur.

7.2 Implementing Source Independent Video Wall

The “Source Independent Video Wall” or “SI Wall” for short, deals with common resolution changes so that control clients do not need to. For AVP2000(T) endpoint devices, the required SI Wall values automatically adjust as a function of the input source resolution.

Since the SI Wall configuration supports configuring video walls without the need to calculate the cropping region of the source resolution, it simplifies both the software design process and implementation of video wall applications. This allows AV integrators to implement wall grids quicker and easier when compared to configuring a video wall manually. This option is applicable to both Wall modes, the Fast Switch Wall mode and the Genlock Wall mode.

Why not base calculations on the source resolution? When implementing a video wall manually, defining cropping regions is dependent on the source resolution, so each time a resolution change occurs it is required to recalculate the wall values. Source resolution changes can occur not only when the source itself is changed but also when a display is operating in menu mode or when a different presentation laptop or device is connected, such as in a meeting or presentation/conference room.

7.2.1 Source Independent Video Wall Requirements

The following are required when configuring an SI Wall:

- The decoder (RX) device contains either an AVP2000 or AVP2000T chipset.
Reminder: Since an encoder (TX) device does not manage the video display mode, the source can originate from any encoder regardless of type, such as an AVP1000 or NT1000/NT2000 device.
- The wall is rectangular.
- The video source vertical resolution is always even.
- The video source horizontal resolution divided by two is always even.
- Video source pixels are assumed to always be square (1:1) (This is currently the case in Genlock Scaling and Fast Switch modes).
- Monitor display pixels are always assumed to be square (1:1).
- All displays are identical, including display bezel borders and output resolution.

Reminder, a SI Wall API request is applicable to BlueRiver AVP2000/AVP2000T chipsets. It is also backwards compatible with video walls previously configured manually. However, a wall must be applied using either the API requests for the SI Wall or a Custom Wall but not a combination of both.

When configuring an SI Wall through an API request, all required arguments along with their appropriate values are specified in a single API request. This includes the following arguments:

- `aspect_ratio mode`
- `grid width`
- `grid height`
- `grid index` (horizontal and vertical)
- `bezel` (horizontal and vertical)
- `wall name`

When planning the configuration of an SI Wall, the following information is required:

- **Wall Information:**
 - The grid size (wall grid) is measured in units of displays, such as 2x2, 3x2, 3x3, etc. This number is always a positive integer value above zero.
 - Bezel compensation to apply.
 - Aspect Ratio (Keep, Stretch or Crop) to apply.
 - A wall name can be configured allowing the control software to determine which devices are part of a given wall configuration.
 - With respect to SI Wall information, all displays that are part of a given wall are configured using the specified display values, including the bezel, output resolution and framerate.
 - This includes the specified output resolution, whether it is a resolution or VIC/HDMI VIC code, it is applied to all displays comprising the SI Wall.
- **Display Information:**
 - The Grid Index defines the location (column/row) of a specific display within the wall.
 - With respect to the “area of interest” of original source for each display location, as well as the viewport to apply (if appropriate), each API request applies the appropriate values based on a display’s Grid Index.
 - Grid Index values:
 - Are positive integer values above 0 and may go up to a maximum value equal to the grid size dimensions.
 - They are calculated increasing grid size within the wall display from left to right and from top to bottom.
 - Values are 1-based, with the column being specified first, followed by the row. For example, a 2x2 wall display has values 1-1, 1-2, 2-1 or 2-2.
 - The `wall_name` argument was added in API 3.5 allowing for control software to determine which devices are part of a given wall configuration. Useful for applications where multiple video wall configurations exist.

TIPS

- i. Spaces are allowed, trailing space will be included in `wall_name`.
- ii. To clear the `wall_name` it is required to send the API wall request without the `wall_name` argument.
- iii. For the wall processing mode, when the `wall_name` is being set using a TCI command it must be the last argument specified in the API request.

- **Bezel Compensation to Apply:**

- Bezel defines the number of output pixels between the end of the last active pixel of a monitor and the start of the first active pixel of the following monitor.
- Bezels measured in units of pixels.
- Horizontal and Vertical Bezels are specified independently.
- Must be a positive integer value equal to or greater than 0.
- Refer to section 7.2.1.1 Calculating Bezel Compensation below for details on calculating the vertical and horizontal bezels to apply.

- **Aspect Ratio to Apply:**

Specifying the Aspect Ratio mode defines how the aspect ratio is to be managed when the Source aspect ratio differs from the Video Wall aspect ratio.

The supported Aspect Ratio modes are:

KEEP	When required black bars are added to preserve the aspect ratio.
STRETCH	The video is stretched to cover the entire screen without regards for preserving the aspect ratio.
CROP	Aspect ratio preserved by cropping the video.

The figure shown below is an illustration of the grid indexes for a Video Wall of 3x3 displays.

1,1	2,1	3,1
1,2	2,2	3,2
1,3	2,3	3,3

Figure 35: Illustration of Grid Indexes for 3x2 Video Wall

7.2.1.1 Calculating Bezel Compensation

Wall bezels are determined by measuring the display bezel and then calculating the display pixel density.

- For example, if the display's bezel equals 0.2 inches and the display pixel density is 120 pixels per inch, then the bezel thickness would be: $120 \times 0.2 = 24 \text{ pixels}$
- Pixel density varies depending on the display manufacturer and model. Be sure to confirm the correct pixel density to apply when calculating the bezel thickness.
- Measure the display borders located within the inside of the wall grid. Also, keep in mind the thickness of a display border could vary. For example, the bottom border of a display could be wider in relation to the top border if it has a manufacturer logo present.

7.2.1.2 Applying Wall Aspect Ratios

Following are illustrations of the supported aspect ratios applied to an example video wall with a grid of 3 x 2 grid.



Figure 36: Original 16:9 Source Example

Illustration of the **KEEP** aspect ratio applied to a 3x2 Video Wall grid. Advantage is that the original source is shown in its entirety without distortion. Black pillar bars have been added.



Figure 37: SI Wall KEEP Illustration

The second example illustrates the **STRETCH** option where the wall content is stretched to fill the wall without regard for maintaining the aspect ratio. The advantage is that the entire wall displays video content, however the user experience may be impacted by the distortion that is present.

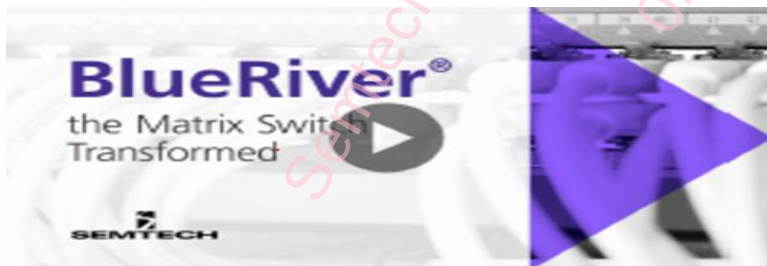


Figure 38: SI Wall STRETCH Illustration

The third example illustrates the CROP option, note that some of the original source pixels are hidden, so some of the original content is lost. It does however, fill the entire wall with content.

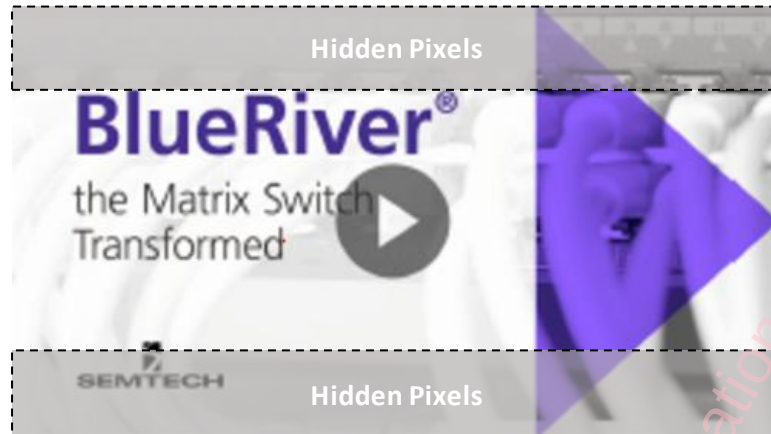


Figure 39: SI Wall CROP Illustration

7.2.2 Additional Configuration of Source Independent Video Wall

When decoders (RX) are operating in Genlock Wall mode with a Source Independent Video Wall configuration applied, changes to the wall configuration or video source result in a black, glitch-less transition on the decoder devices constituting the wall display until the entire video path is stable. That is the decoders and displays resynchronize to the source change or configuration settings loaded.

With respect to the Fast Switch Wall mode, there are two different reactions that could be noted during a transition, depending on the argument values impacted by the change.

1. When decoders (RX) are operating in Fast Switch Wall, changes to the following elements result in a black, glitch-less transition on the decoder devices constituting the wall display until the entire video path is stable:

- Video source change (timings/resolution)
- Monitor Resolution
- Monitor Framerate

Note: This is the same behavior as outlined earlier for the Genlock Wall mode with respect to transitions. For details refer to section 7.1.1 Genlock Wall Mode.

2. When decoders (RX) are operating in Fast Switch Wall, changes to the following elements do not result in black, instead a seamless, glitch-less transition occurs for the decoders (RX) that constitute the wall display:

- Grid Size
- Bezel
- Grid Index
- Aspect Ratio Mode
- Wall Name
- Video Subscription

Note: For the second set of arguments, this is the same behavior as outlined earlier for the Fast Switch Wall mode with respect to transitions.

When either the “KEEP” or “STRETCH” aspect ratio option is applied, the SI Wall shall display:

1. The first and last rows of pixels of the video source in the top and bottom wall rows of the displays.
2. The first and last pixels of each video source line in the leftmost and rightmost wall columns of the displays.

Note: This is only true when the video source vertical resolution is an even number, and the video source horizontal resolution evenly divided by two.

When the “CROP” aspect ratio is applied, then the SI Wall shall display either:

1. The first and last rows of pixels of the video source in the top and bottom wall rows of monitors, when cropped horizontally.
2. The first and last pixels of each video source line in the leftmost and rightmost wall columns of monitors, when cropped vertically.

Note: This is also only true when the video source vertical resolution is an even number, and the video source horizontal resolution is evenly divided by two.

7.2.3 Source Independent Video Wall API Request

A `set video` request is used to set the video display mode that a decoder device (RX) is to operate in. Regarding the video wall mode, two modes are supported the Fast Switch Wall (`WALL_FAST_SWITCHED`) and the Genlock Wall (`WALL_GENLOCKED`) mode. The API arguments supported for these wall modes are the same, the difference in the API request is whether the wall command being issued is for a manual wall configuration or a source independent wall configuration.

In this section, the source independent wall configuration is reviewed. As mentioned earlier it is necessary to enter the required arguments for an SI Wall in a single API request.

Note: For information on implementing a custom video wall refer to section 7.3 Configuring Custom Video Wall.

The POST API request includes the following arguments:

```
POST /api/device/{RX_DEVICE_ID}
{
  "op" : "set:video",
  "display_mode" : WALL_FAST_SWITCHED|WALL_GENLOCKED,
  "video" : {
    "aspect_ratio_mode" : [KEEP|CROP|STRETCH],
    "grid_width" : [COLUMNS] (integer),
    "grid_height" : [ROWS] (integer),
    "grid_index_horiz" : [COLUMNS] (integer),
    "grid_index_vert" : [ROWS] (integer),
    "bezel_horiz" : [THICKNESS_H] (integer),
    "bezel_vert" : [THICKNESS_V] (integer),
    "wall_name" : ["wall_name"] (string)
  }
}
```

Note: The `wall_name` option supports a maximum 19 characters. All displayable characters in the ASCII character set are allowed, meaning a to z, A to Z, 0 to 9, space, as well as symbols.

Below is an example of an API request for a 2x2 Source Independent Video Wall to operate in Fast Switch Wall mode with an output resolution of 3840x2160 and framerate of 60Hz. The aspect ratio to apply is KEEP and the bezel compensation for both horizontal and vertical is 24 pixels.

```
POST /api/device/D88039636626
{
  "op" : "set:video",
  "display_mode" : "WALL_FAST_SWITCHED",
  "video" : {
    "width" : 3840,
    "height" : 2160,
    "fps" : 60,
    "aspect_ratio_mode" : KEEP,
    "grid_width" : 2,
    "grid_height" : 2,
    "grid_index_horiz" : 2,
    "grid_index_vert" : 2,
    "bezel_horiz" : 24,
    "bezel_vert" : 24
  }
}
```

It is also possible to combine the set video SI Wall arguments with a join request, the data required for this type of request is:

```
POST /api/device/{target}
...
{
  "op" : "join",
  "source_device" : "device_id",
  "stream_type" : "HDMI",
  "stream_index" : [0|1],
  "subscription_index" : [0|1],
  "display_mode" : "[WALL_FAST_SWITCHED|WALL_GENLOCKED]",
  "video" : {
    "video" : {
      "aspect_ratio_mode" : "[KEEP|CROP|STRETCH]",
      "grid_width" : [COLUMNS] (integer),
      "grid_height" : [ROWS] (integer),
      "grid_index_horiz" : [COLUMNS] (integer),
      "grid_index_vert" : [ROWS] (integer),
      "bezel_horiz" : [THICKNESS_H] (integer),
      "bezel_vert" : [THICKNESS_V] (integer)
    }
  }
}
```

Example of creating a Genlock Wall combining a join request and set video request with the SI Wall arguments applied this example also configures a video wall name. The decoder (RX) is the device ID 001ec0f04d9c while the source encoder device ID is 001ec0f03668. The wall example is a 2x2 wall grid.

```
POST /api/device/001ec0f04d9c HTTP/1.1
...
{
  "op" : "join",
  "source_device" : "001ec0f03668",
  "stream_type" : "HDMI",
  "stream_index" : 0,
  "subscription_index" : 0,
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "aspect_ratio_mode" : "KEEP",
    "grid_width" : 2,
    "grid_height" : 2,
    "grid_index_horiz" : 2,
    "grid_index_vert" : 2,
    "bezel_horiz" : 24,
    "bezel_vert" : 30,
    "wall_name" : "abcdefg"
  }
}
```

7.2.4 Frame Buffer Node Related to SI Wall (AUTO_WALL)

Below is an example return of the FRAME_BUFFER node type, outlining details of the wall settings currently applied to the encoder (RX) D88039636626.

```
get D88039636626 settings
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    [...]
    "nodes" : [
      {
        "type" : "FRAME_BUFFER",
        "index" : 0,
        "configuration" : {
          "display_mode" : "WALL_FAST_SWITCHED",
          "width" : 3840,
          "height" : 2160,
          "horiz_offset" : 0,
          "vert_offset" : 0,
          "frames_per_second" : 30,
          "keep_width" : 700,

```

```

        "keep_height" : 524,
        "viewport_horiz" : 960,
        "viewport_vert" : 0,
        "viewport_width" : 2880,
        "viewport_height" : 2160,
        "aspect_ratio_mode" : "KEEP",
        "has_auto_wall_configurations" : true,
        "auto_wall_configurations" : {
            "grid_width" : 2,
            "grid_height" : 2,
            "grid_index_horiz" : 1,
            "grid_index_vert" : 2,
            "bezel_horiz" : 5,
            "bezel_vert" : 5,
            "wall_name" : "abcdefg"
        },
        [...]
    }
}

```

Semtech Confidential Information
 Mark Li
 ADV
 02/03/2026

7.3 Configuring Custom Video Wall

If the endpoint device is not based on a BlueRiverAVP2000 or AVP2000T chipset or the video wall is not rectangular in nature, it is still possible to create a Video Wall without additional equipment. The only stipulation is that the endpoint device must include the AV Processor Engine. For these applications the Video Wall values are calculated and applied using a different set of API arguments. The following sub-sections review details on building a video wall manually.

7.3.1 Wall Mode Terms & Concepts

The following is terms and concepts referred to in Semtech documentation relating to the Wall mode.

Table 5: Wall Mode Terms and Concepts

Term/Concept	Definition
Frame Rate	The frequency (rate) that consecutive images, called frames, appear on a display. Expressed in frames per second (fps).
Area of Interest (AOI)	Represents the source Area of Interest (AOI) for a decoder (RX) to output. The AOI is defined using two arguments: Offset defining the upper left corner of the AOI (horizontal/vertical). Size defines the width/height of the AOI.
Display Bezel	This is the area of a display (frame) that surrounds the screen.
Bezel Compensation	Hides a portion of the source image behind the bezel. This creates a better viewing experience for the end user.
Viewport	A defined rectangular area of a display, where source AOI is shown. - Used when displaying a source on a wall that has a different aspect ratio between the source aspect ratio and the wall aspect ratio. Viewport is also defined in display pixel coordinates using two parameters: - Viewport offset defining upper left corner of viewport (horizontal/vertical). - Viewport size of viewport (width/height).
Output resolution	Refers to the resolution received by a display connected to a decoder (RX). It is specified per decoder (RX).
Columns and rows	In this document, the wall grid is referred to in columns and rows: x-axis of wall is defined as columns. y-axis of the wall is defined as rows. - The number of columns refers to number of horizontal (x-axis) displays that make up a given video wall. - The number of rows refers to number of vertical displays (y-axis) that make up a given video wall.
Wall tearing	Horizontal visual artifact seen between wall rows. Refer to Figure 40: Illustration of Wall Tearing for an example of wall tearing and to Figure 41: "Per Row Delay" to Reduce Wall Tearing for the solution applied by BlueRiver chipsets to reduce it.

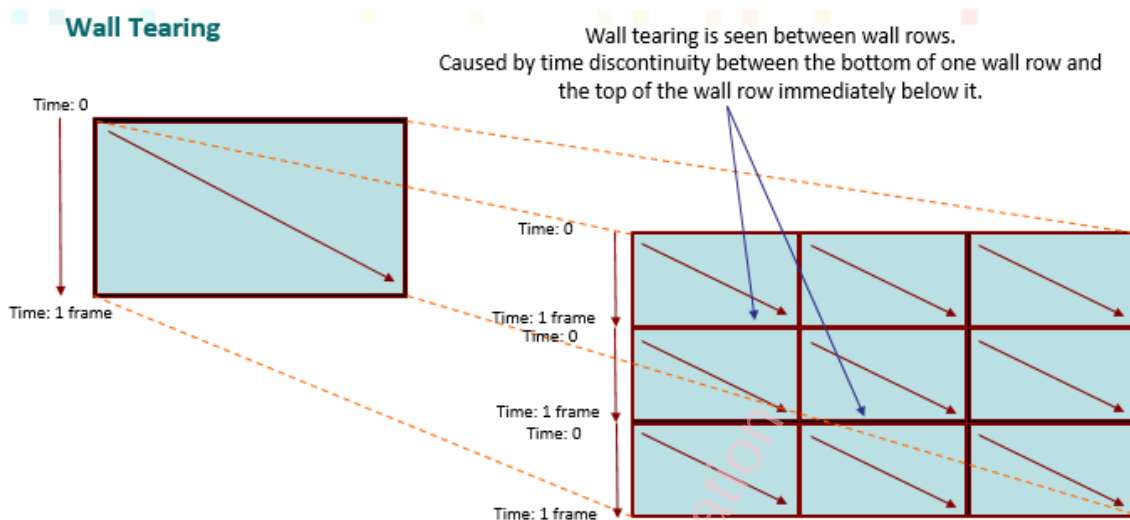


Figure 40: Illustration of Wall Tearing

Decoders (RX) apply “per row delay” to reduce Wall Tearing. This advanced function is usually found only in dedicated high end video wall processors.

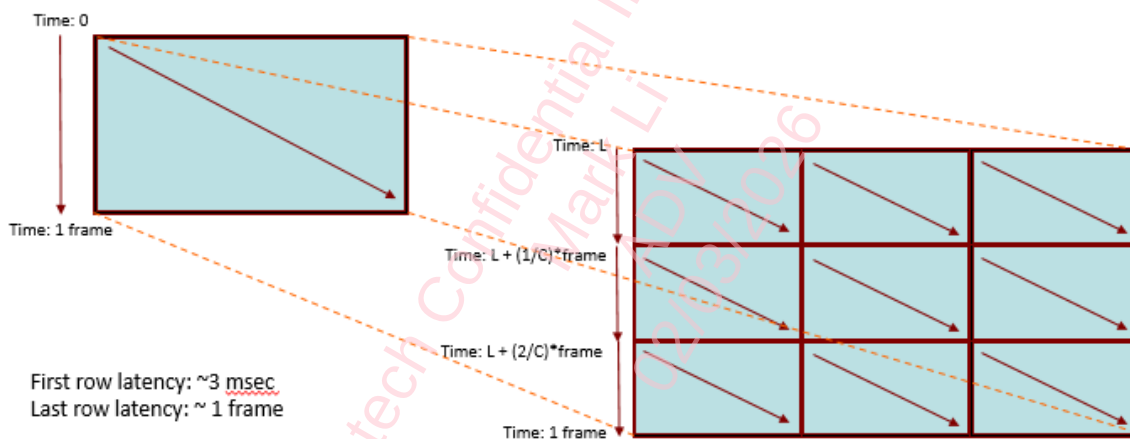


Figure 41: "Per Row Delay" to Reduce Wall Tearing

7.3.2 Configure Video Wall Tasks

There are three main tasks required to create a Video Wall using the API:

1. Configure appropriate decoders (RX) into Wall mode.
 - a. Calculate the values to apply. Refer to section 7.3.9.3 Wall Parameter Computation.
 - b. The `set video` command is used to configure mode, wall size, fps, read offset, keep region size, as well as if applicable to set the viewport.
 - c. Requires the device ID of each of the decoders included in the wall grid.
2. Start the encoder (TX) video stream that is to feed the wall.
 - a. The API start command is used to enable the source stream.
3. Subscribe the appropriate decoders (RX) to the encoder (TX) that is to feed the display wall.
 - a. The API `join` command is used.

7.3.3 Planning a Custom Video Wall

When planning the implementation of a video wall, it is necessary to have the following data available:

- The source resolution and fps.
- Number of displays, as well as the number of columns and rows in the grid. Also the position of each decoder (RX) and to which display it is attached.
- The output resolution of each display that is to be incorporated in the video wall grid.
TIP: Typically, this is the same for each display assuming the same model/size of display is attached to each decoder (RX).
- The bezel width for the displays in relation to the inside borders. This topic is covered in section 7.3.5 Managing Bezel Compensation.
- In regard to issuing the API command, the following device ID are required:
 - The encoder (TX) supplying the wall source.
 - Each decoder (RX) that is incorporated in the wall grid.

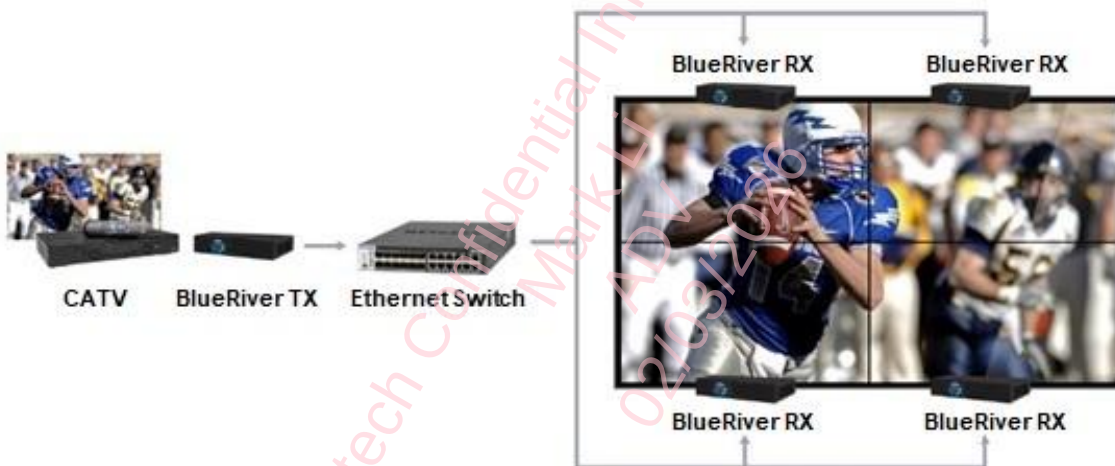


Figure 42: Example of 2x2 Wall Grid

WARNING! It is critical that the correct Area of Interest (AOI) offset value be assigned to the appropriate decoder. If applied incorrectly, it will result in an incorrect position of an image segment in the displayed video wall grid.

7.3.4 API Wall Parameters

In this section, the following assumptions are made:

1. The wall is rectangular.
2. All displays are identical, and all are outputting the same resolution.
3. To preserve the aspect ratio, no display is required to be completely black (this excludes extreme wall configurations such as 5x2 or sources that have an unusual aspect ratio).

7.3.4.1 Custom Wall API Set Video Requests

The API command, `set video` is used to configure each of the decoders (RX) individually. The API provides a series of arguments that support the creation of a wall grid.

The structure of an API request for `set video` in relation to the wall mode:

```
POST /api/device/{target}

{
  "op" : "set:video",
  "display_mode" : "{WALL_GENLOCKED/WALL_FAST_SWITCHED}",
  "video" : {
    "width" : integer,
    "height" : integer,
    "fps" : integer,
    "horiz_offset" : integer,
    "vert_offset" : integer,
    "keep_width" : integer,
    "keep_height" : integer,
    "viewport_horiz" : integer, (optional)
    "viewport_vert" : integer, (optional)
    "viewport_width" : integer, (optional)
    "viewport_height" : integer (optional)
  }
}
```

The table below describes the arguments used by decoders (RX) operating in wall mode.

Table 6: set video Arguments Applicable to Wall Mode

Argument	Description
device_ID	The decoder ID that the specified arguments are to be applied to, including the offset, keep and if appropriate, the viewport. Reminder: A separate set video request is sent to each decoder (RX) of the wall, specifying the appropriate values for the part of the grid.
display_mode	Specify the display mode that the decoder is to operate in. As described earlier in the wall mode introduction, two wall modes are supported, genlock (WALL_GENLOCKED) and fast switch (WALL_FAST_SWITCHED). Refer to section 7.1 Introduction to Video Wall Mode for more information. Note: The genlock wall mode is used in the case studies provided.
width/height fps	Specifies the output resolution (size) of a specified decoder (RX). fps is the frame rate per second to be applied to the specified decoder (RX). - Can be either integer (e.g. 60) or real number (e.g. 60.000). - Optionally if it is a fractional frame rate (i.e. multiplied by 1000/1001) follow it with letter "m" (e.g. 60m or 30.00m). Reminder: In genlock wall mode, the frame rate specified must be equal to source's frame rate. Alternately, a VIC or HDMI_VIC code can replace the size and fps arguments. TIP: Refer to the SDVoE Developers API Reference Guide for a list of the supported VIC and HDMI_VIC codes.

Argument	Description
<code>horiz_offset</code> <code>vert_offset</code>	<code>offset</code> specifies the location/offset (horizontal/vertical) of the area of interest (AOI) that is to be displayed on the specified decoder (RX). The <code>horiz_offset</code> and <code>vert_offset</code> respectively specify (in source pixels) the horizontal/vertical offset of the displayed area (AOI) within the video source. The offset must be even number (i.e. multiple of two). Independently specified for each decoder (RX) used in the wall grid. Applies only to the Wall mode and is not applicable to other display modes.
<code>keep_width</code> <code>keep_height</code>	<code>keep</code> specifies the size (width/height) of the area of interest (AOI) to display on the specified decoder (RX). The <code>keep_width</code> and <code>keep_height</code> respectively specify in source pixels the width/height portion of the source video that is to be displayed (i.e. "kept"). The <code>keep_width</code> argument must be even (i.e. multiple of two). Independently specified for each decoder (RX) used in the wall grid. Applies only to the Wall mode and is not applicable to other display modes.
<code>viewport</code>	The position and size of the region on the display where the video is displayed. Outside this region the display is black.

The figure below illustrates an example of a 2x2 wall grid for the decoder in lower left corner, showing the area of interest as well as the set video keep, offset and size arguments for a 3840 x 2160 source.

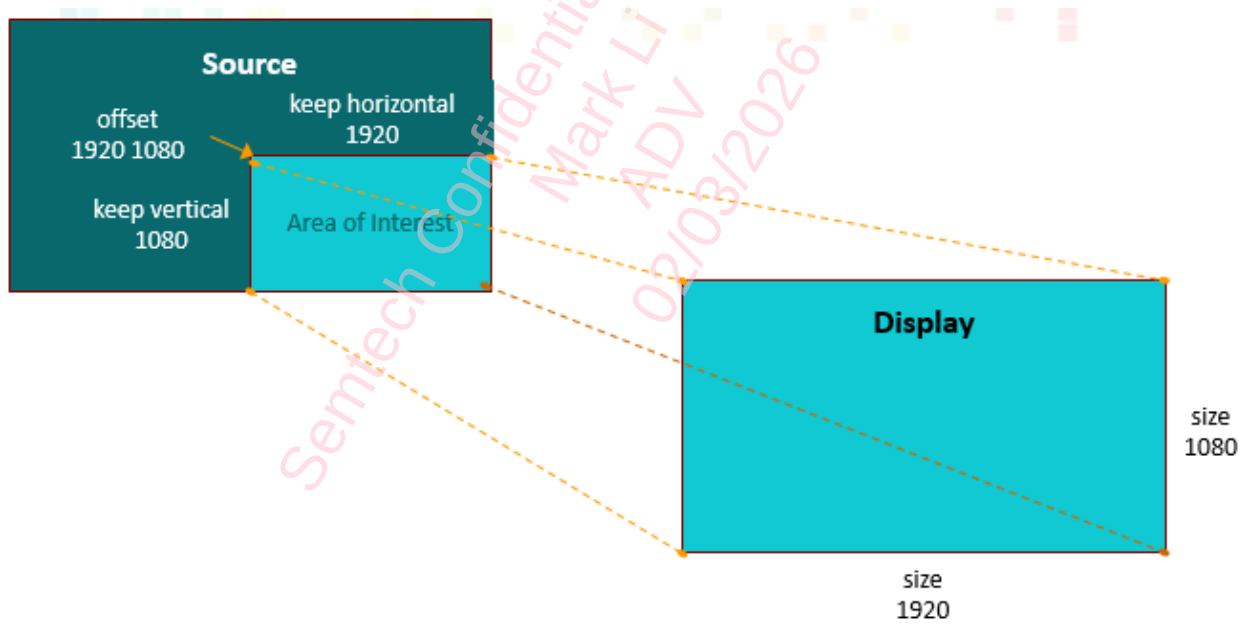


Figure 43: Illustration of set video Arguments Applied to RX4 of 2x2 Wall

7.3.5 Managing Bezel Compensation

Displays typically have a physical border, known as a “bezel”. These borders/bezels vary in thickness depending on the display type and manufacturer.

When a source is extended over multiple displays, these borders can result in the content not being displayed ‘naturally’. Rather, the image may appear misaligned. Refer to the figure below “without bezel compensation”. Note the discrepancy highlighted in the figure between the left and right displays. The tracks and building are misaligned.

To compensate for this discrepancy, bezel compensation can be applied. It will account for the thickness of the display bezel allowing a more realistic experience for the viewer.

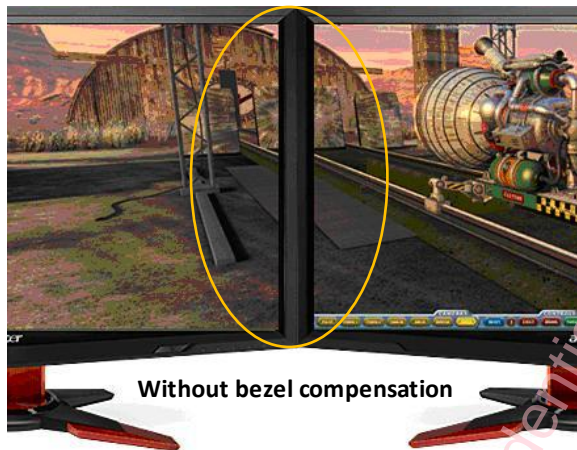


Figure 44: Example Without Bezel Compensation

To resolve the discrepancy to the video caused by the display bezels while maintaining the aspect ratio of the source, the adjustment of the bezel is applied to all “inside” borders of the video wall grid.

Bezel compensation is supported by using the set video command to adjust the size (keep) and/or location (offset) of the area of interest (AOI) for each of the decoders (RX) involved in a given wall grid. The adjustment applied compensates for the pixel thickness of these borders.

Reminder: The area of interest (AOI) is defined in source pixels.

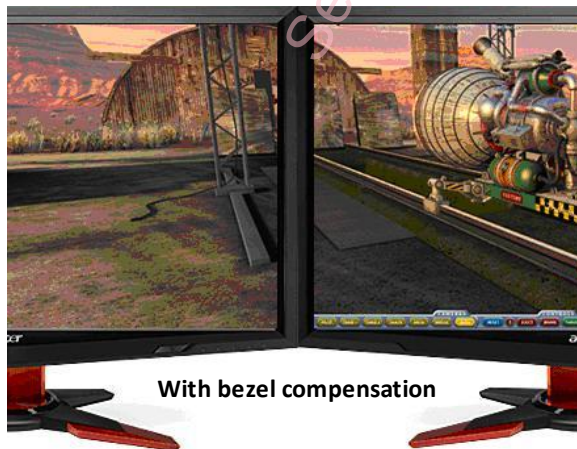
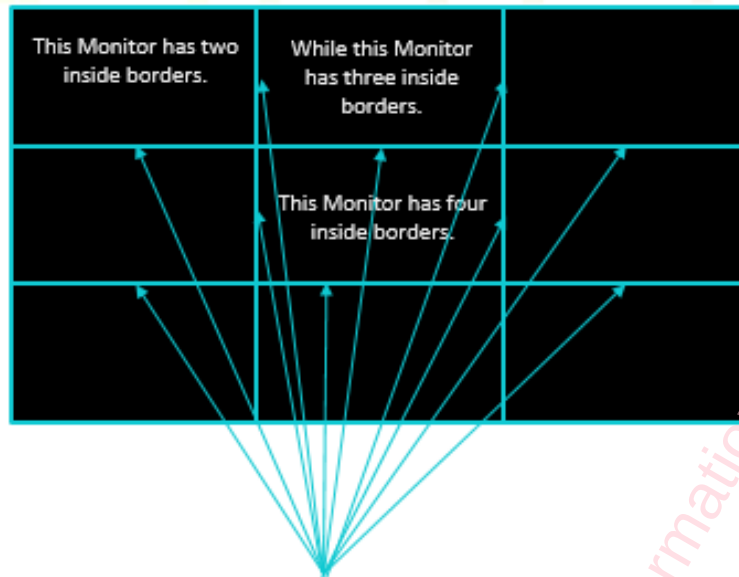


Figure 45: Example with Bezel Compensation



9 displays mounted in a 3x3 array highlighting the inside borders

Figure 46: Example of Bezel Borders to Consider for 3x3 Wall Grid

Tip: Wall bezels are determined by measuring the display bezel, then calculating and applying the display pixel density.

For example, if the display's bezel equals 0.2 inches and the display pixel density is 120 pixels per inch, then the bezel thickness would be: $120 \times 0.2 = 24$ pixels

Pixel density varies depending on the display manufacturer and model. Be sure to confirm the correct pixel density to apply when calculating the bezel thickness.

Measure the display borders located within the inside of the wall grid. Also, keep in mind the thickness of a display border could vary. For example, the bottom border of a display could be wider in relation to the top border if it has a manufacturer logo present.

The figure below shows an example of bezel compensation using a 2x2 wall, with the source resolution of 3840x2160 and applying a 20-pixel bezel compensation to both the horizontal and vertical borders.

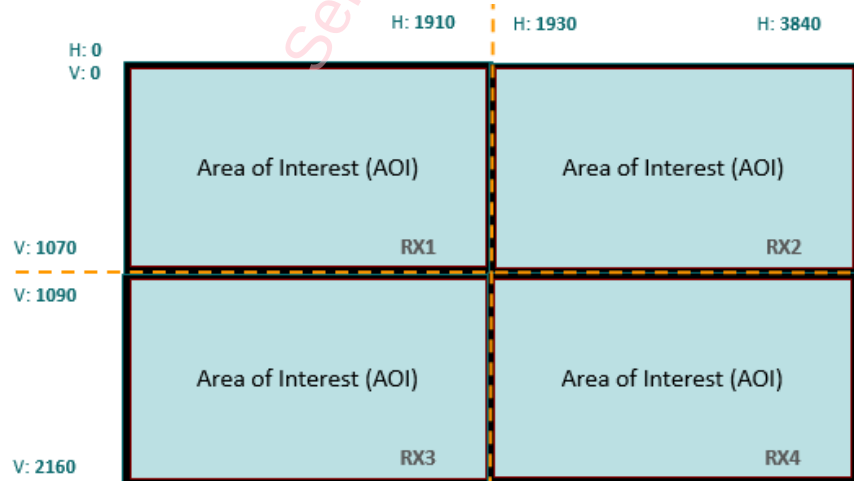


Figure 47: 2x2 Example of Rectangular Wall Grid with Bezel Compensation Applied

The horizontal and/or vertical resolutions are impacted. Specifically, this means the `offset` and/or `keep` values being applied using a `set video` request are adjusted for each individual decoder that is part of the video wall.

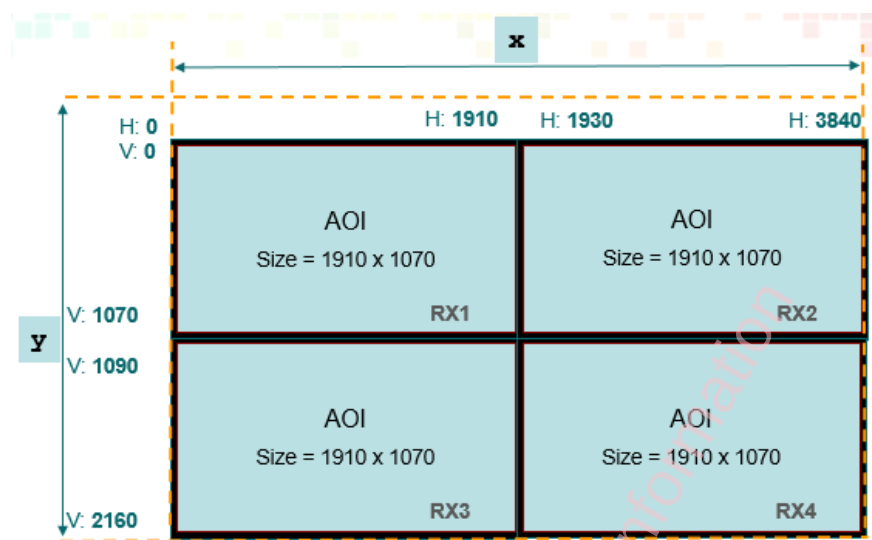


Figure 48: Example with AOI Size and x/y Coordinates Applied

7.3.6 Managing Different Source & Wall Aspect Ratios

What if the source and wall aspect ratios differ?

For example:

4:3 source and 16:9 wall.

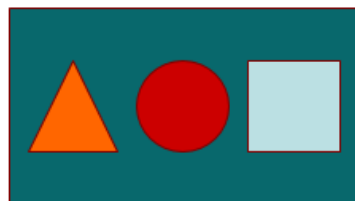
16:9 source and 16:10 wall.

Number of display rows and columns are not equal, for example, 3x2 wall.

BlueRiver chipsets, which implement the AV Processing Engine manage scenarios such as these, without the addition of expensive equipment.

There are three potential options available to deal with a scenario where the source and wall aspect ratios are different:

- Stretch the source to fit the wall geometry.
- Crop the source horizontally or vertically to match the wall aspect ratio.
- Use the Viewport option to maintain the source aspect ratio.



Source



Wall

Figure 49: Example of Source Different than Wall Aspect Ratio

7.3.7 Stretch Source to Fit Wall Geometry

If the impact to the area of interest (AOI) size and offset settings are not taken into consideration when the source and wall have different aspect ratios, the result is a distorted image being displayed. The output from the decoder (RX) would be stretched either vertically or horizontally to fit the wall. Refer to figure below, Figure 50: Horizontal Example of Stretching Source to Fit Wall.

This example shows what occurs when the wall aspect is larger than the source aspect ratio. Notice how the source is stretched horizontally, the circle becomes an oval and the square a rectangle. This is not likely a desired output, as a viewer's experience would be negatively impacted.

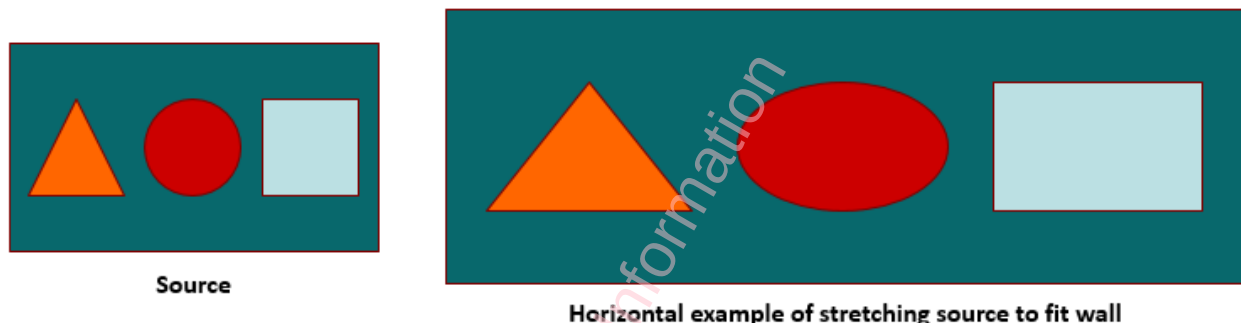


Figure 50: Horizontal Example of Stretching Source to Fit Wall

7.3.8 Crop Source Horizontally or Vertically to Match Wall Aspect Ratio

A second option to managing a different source and wall aspect ratio, is to crop the source either horizontally or vertically to match the wall's aspect ratio. With this option, there is no distortion present, but part of the image is not displayed.

In the example below, the shaded area, outlined by an orange dashed line, is the only part of the source that is displayed. The lighter teal area is hidden. This approach is useful when it is desired to fill the entire wall, for example to show a 4:3 source on a 16:9 wall.

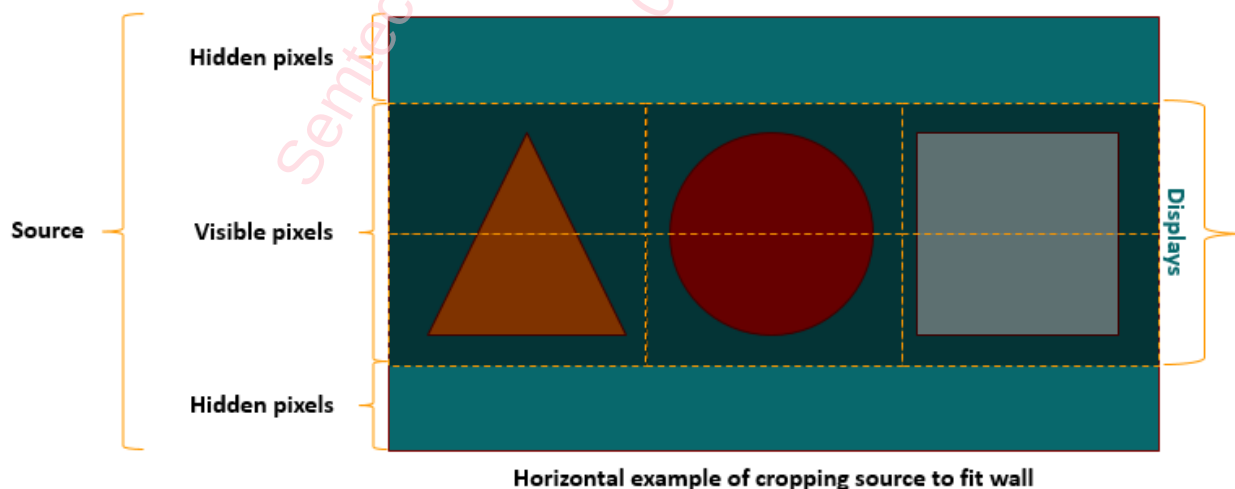


Figure 51: Example of Cropping the Source

In this section, details of computing the parameters needed to configure a device in wall mode when the source picture is cropped to preserve the picture aspect ratio is reviewed.

If cropping is not desirable and black borders are to be added instead, refer to section 7.3.9 Applying Viewport to Maintain Source Aspect Ratio.

The following assumptions are made:

- The wall is rectangular; and
- All displays are identical, and they all output the same resolution.

7.3.8.1 Tasks Involved to Crop Source

To apply the source cropping option to a wall grid involves the following tasks:

- Calculate the total wall size.
- Calculate the horizontal and vertical scaling ratios.
- Calculate the scaling ratio and area of interest (`offset/keep` values).
- Apply the values calculated to the appropriate decoder (RX) using the API wall arguments of a `set video` request, refer below for details of the request syntax.
- Repeat above steps for each individual decoder (RX) included in the wall grid.

7.3.8.2 Wall Parameters & API Request

The `set video` API command is used to configure each decoder (RX) individually.

The syntax for `set video` request in relation to cropping the source is as follows:

```
POST /api/device/{target}
{
  "op" : "set:video",
  "display_mode" : "{WALL_GENLOCKED/WALL_FAST_SWITCHED}",
  "video" : {
    "width" : integer,
    "height" : integer,
    "fps" : integer,
    "horiz_offset" : integer,
    "vert_offset" : integer,
    "keep_width" : integer,
    "keep_height" : integer,
  }
}
```

The following parameters need to be applied to each of the decoder (RX) devices that are incorporated into the wall grid:

- The width and height (`size`) of the output resolution, specified in pixels.
- The frame rate (`fps`).
Reminder: It must be equal to the source's frame rate when the decoder is operating in genlock wall mode.
- The position of origin (top-left corner) of the area of interest (AOI) for a decoder (RX) in source coordinates (`horiz_offset` and `vert_offset`), specified in pixels.
- The size (`width` and `height`) of the area of interest for the specified decoder (RX) in the source coordinates (`keep`), specified in pixels.

As an example, taking the bottom-left display in a 2x2 wall grid that is outputting 3840x2160 at 60 frames per second, with a 1080p60 source, without bezel compensation, assuming a device ID of 010203040506, the set video request to apply would be:

```
POST /api/device/010203040506
{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 3840,
    "height" : 2160,
    "fps" : 60,
    "horiz_offset" : 0,
    "vert_offset" : 540,
    "keep_width" : 960,
    "keep_height" : 540,
  }
}
```

7.3.8.3 Wall Parameters Computation

It is assumed that the number of columns and rows of displays is known for the wall grid, as well as the output resolution and bezel correction required for each display. In addition, it is also assumed that the source width and height resolution along with the frame rate of the source is known.

The table provided below outlines some of the definitions referred to in the sample formulas provided.

Table 7: Definition of Parameters Used in Scaling Wall Calculations

Parameter	Description
C	Represents the number of columns of displays (horizontal) in the wall grid.
R	Represents the number of rows of displays (vertical) in the wall grid.
W	Represents each display's horizontal output resolution (aka. output width).
H	Represents each display's vertical output resolution (aka. output height).
B _w	Represents the width of the bezel, in equivalent pixels of the output resolution.
B _h	Represents the height of the bezel, in equivalent pixels of the output resolution.
S _h	Represents the horizontal scaling ratio.
S _v	Represents the vertical scaling ratio.
W _s	Represents the horizontal resolution of the source.
H _s	Represents the vertical resolution of the source.
F _s	Represents the frame rate of the source.
W _{tot}	Represents the total width of the wall, including all displays and their bezel.
H _{tot}	Represents the total height of the wall, including all displays and their bezel.
S	The scaling ratio.
i	The row in which a display is located in the wall grid, with the displays in the first row being numbered i=0
j	The column in which a display is located in the wall grid, with the displays in the first column being numbered j=0
K _w	The width of a display's area of interest (AOI) in the source's coordinates system (i.e. the horizontal keep)

Parameter	Description
K_h	The height of a display's area of interest (AOI) in the source's coordinates system (i.e. the horizontal <code>keep</code>)
O_h	The horizontal position of a display's area of interest (AOI) origin in the source's coordinates system (i.e. the horizontal <code>offset</code>)
O_v	The vertical position of a display's area of interest (AOI) origin in the source's coordinates system (i.e. the vertical <code>offset</code>)
C_w	Width, in pixels, of the portion that is cropped at the left of the source picture.
C_h	Height, in pixels, of the portion that is cropped from the top of the source picture.

Step 1: Total Wall Size

First step is calculating the total width (W_{tot}) and total height (H_{tot}) of the wall grid, including all the displays and their bezels. These are computed using the following formulas:

$$W_{tot} = C * W + (C - 1) * B_w$$

$$H_{tot} = R * H + (R - 1) * B_h$$

Step 2: Horizontal & Vertical Scaling Ratios

It is required to determine if it is necessary to preserve the aspect ratio, that is whether and where the source picture needs to be cropped. Before determining if cropping of the source is required, first need to determine the scaling ratio, as this value is used in the next set of calculations.

The following formulas are used to compute the horizontal and vertical scaling ratios:

$$S_h = W_{tot} / W_s$$

$$S_v = H_{tot} / H_s$$

Step 3: Scaling Ratio & Area of Interest

In this step, the scaling ratio and area of interest (AOI) are computed.

Calculating `offset` Values

To calculate the `offset` parameters, the following intermediate results are required to first be computed, specifically the column width, column height and scaling ratio.

There are three cases to consider when computing the cropped width (C_w), cropped height (C_h) and scaling ratio. The intuition behind this is that, if the horizontal and vertical scaling factors are different, the one to use is the one that "stretches" the image the most (i.e. the largest). When the other dimension is "stretched" by the same amount, the source image needs to be cropped accordingly for this dimension so that it fits in the wall.

Case 1: $S_h = S_v$

If $S_h = S_v$ then

$$S = S_h = S_v$$

$$C_w = 0$$

$$C_h = 0$$

Case 2: $S_h < S_v$

If $S_h < S_v$ then

$$S = S_v$$

$$C_w = (W_s - W_{tot} / S) / 2$$

$$C_h = 0$$

Case 3: $S_v < S_h$

If $S_v < S_h$ then

$$S = S_h$$

$$C_w = 0$$

$$C_h = (H_s - H_{tot} / S) / 2$$

Using the appropriate values for the applicable case, the offset parameters are calculated using the following formulas:

$$O_h = C_w + j * (W + B_w) / S$$

$$O_v = C_h + i * (H + B_h) / S$$

Calculating keep Values

It is also necessary to calculate the keep as part of the area of interest, using the following formulas:

$$K_w = W / S$$

$$K_h = H / S$$

7.3.8.4 Step 4: Apply Values to Decoders (RX) to Crop Image

Once the above calculations are completed for each of the displays that incorporate the wall grid, then it is necessary to create and apply an API request to each of the associated decoder (RX) devices with the appropriate values entered.

Below summarizes the set video command syntax using the parameters referencing the formulas shown above:

```
POST /api/device/010203040506
{
  "op" : "set:video",
  "display_mode" : "{WALL_GENLOCKED/WALL_FAST_SWITCHED}",
  "video" : {
    "width" : W,
    "height" : H,
    "fps" : F_s,
    "horiz_offset" : O_h,
    "vert_offset" : O_v,
    "keep_width" : K_w,
    "keep_height" : K_h
  }
}
```

7.3.9 Applying Viewport to Maintain Source Aspect Ratio

The third option to manage a different source aspect ratio versus wall aspect ratio is to apply the `viewport` argument to API set video request.

With this option, the entire source can be displayed, adding either a black pillarbox (left and/or right) or a black envelope (top and/or bottom) to the appropriate displays of the wall grid. The advantage to this approach is that no distortion is present and the full source image is displayed. The viewport argument defines the rectangular region for each display where the area of interest (AOI) is rendered.

7.3.9.1 Viewport Option Overview



Figure 52: Example of Viewport Option Applied

Each decoder has a set of arguments added to the API set video wall request to manage not only the size, keep and offset (AOI) but also to specify the viewport. Adding the viewport argument prevents sections of the wall from being stretched incorrectly. Instead, the AOI video is contained in the viewport region and the areas outside of the viewport are filled with black pixels.



Figure 53: Viewport Regions Illustrated for Example 3x2 16:9 Wall Grid

The API set command syntax with the `viewport` arguments added:

```
POST /api/device/{target}
{
  "op" : "set:video",
  "display_mode" : "{WALL_GENLOCKED/WALL_FAST_SWITCHED}",
  "video" : {
    "width" : integer,
    "height" : integer,
    "fps" : integer,
    "horiz_offset" : integer,
    "vert_offset" : integer,
    "keep_width" : integer,
    "keep_height" : integer,
```

```

        "viewport_horiz" : integer, (optional)
        "viewport_vert" : integer, (optional)
        "viewport_width" : integer, (optional)
        "viewport_height" : integer (optional)
    }
}

```

As mentioned, a viewport is defined per decoder/display.

The `viewport` arguments specifically define the area where the Area of Interest (AOI) is to be placed within a display. It is used in conjunction with the `offset` and `keep` arguments of a `set video` request. As mentioned, areas on a display outside the specified region are displayed as black.

The following applies when the source and wall aspect ratios differ and a viewport is being utilized:

- `viewport_horiz` and `viewport_vert` respectively specify the location of the upper/left (top/left) viewport corner within the display.
 - These values must be even (multiple of two).
- `viewport_width` and `viewport_height` respectively specify the output size in pixels (width/height) of the viewport.
- A viewport cannot extend outside the output resolution of the display.
- This argument is applicable to Wall mode only and not applicable to other display modes.

7.3.9.2 Calculating Wall Parameters

The viewport calculation varies per video wall configuration and is dependent on multiple factors, including the following:

- Source aspect ratio.
- Display aspect ratio.
- Number of rows and columns.
- Horizontal and vertical bezel thickness.

This section provides a deeper understanding of individual viewports to be calculated, as well as information required to calculate them.

Using the earlier figure of six wall displays mounted in a 3x2 grid as an example, a series of tasks are involved to calculate the values, including the size, keep, offset and viewport. Then it is necessary to apply them to the appropriate decoders (RX) using a `set video` request.

When planning a wall grid, there are five items to calculate. These specifically are:

- Total Wall Size.
- Horizontal and Vertical Scaling Ratios.
- Scaling Ratio and Borders.
- Viewport.
- Area of Interest.

In the figure below, the total wall size for the horizontal and vertical coordinates are illustrated.

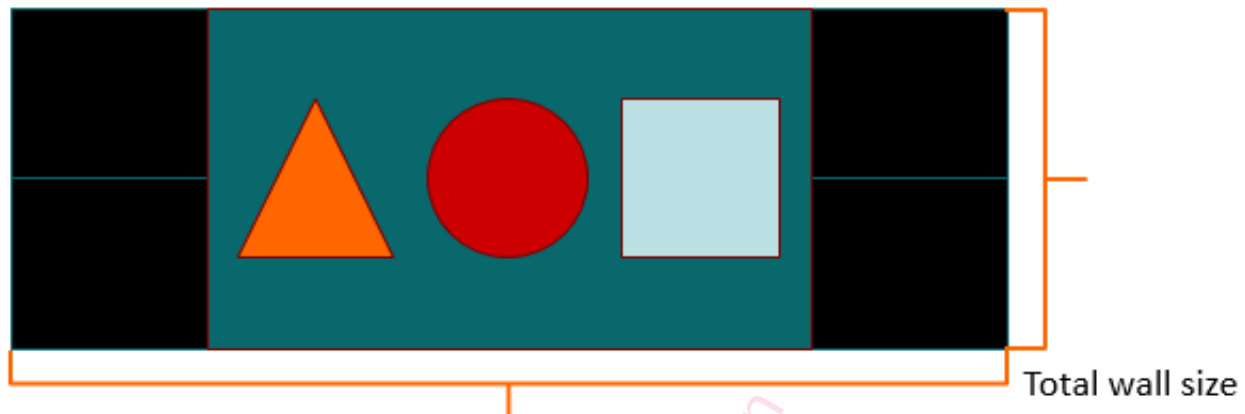


Figure 54: Viewport Wall Size

Before proceeding to investigate calculating the required argument values, it is first necessary to know some information related to a wall grid.

Note: It is assumed some of this data is known, such as number of columns and rows as well as the source and output resolutions.

The table provided below outlines some of the definitions referred to in the sample formulas provided.

Table 8: Definition of Parameters Used in Wall Calculations

Parameter	Description
C	Represents the number of columns of displays (horizontal) in the wall grid.
R	Represents the number of rows of displays (vertical) in the wall grid.
W	Represents each display's horizontal output resolution (aka. output width).
H	Represents each display's vertical output resolution (aka. output height).
W_{tot}	Represents the total width of the wall, including all displays and their bezel.
H_{tot}	Represents the total height of the wall, including all displays and their bezel.
B_w	Represents the width of the bezel, in equivalent pixels of the output resolution.
B_h	Represents the height of the bezel, in equivalent pixels of the output resolution.
S_h	Represents the horizontal scaling ratio.
S_v	Represents the vertical scaling ratio.
W_s	Represents the horizontal resolution of the source.
H_s	Represents the vertical resolution of the source.
S	The scaling ratio.
W_B	The width, in pixels, of each of the black bars on both sides of the wall that are added to preserve the aspect ratio, if applicable.
H_B	The height, in pixels, of each of the black bars on the top and bottom of the wall, if applicable.
V_{hp}	The horizontal position of the viewport on a specific display.
V_{vp}	The vertical position of the viewport on a specific display.
V_w	The width of the viewport on a specific display.
V_h	The height of the viewport on a specific display.
F_s	Represents the frame rate of the source.

Note: For both the width and height (B_w and B_h) bezel compensations, the separation or inside borders between the active parts of two adjacent displays equals the border/bezel. The bezels surrounding the outside of the wall are not relevant in relation to a wall grid. Refer to section 7.3.5 Managing Bezel Compensation for more information.

The figures below illustrate the location in relation to the arguments that need to be calculated:

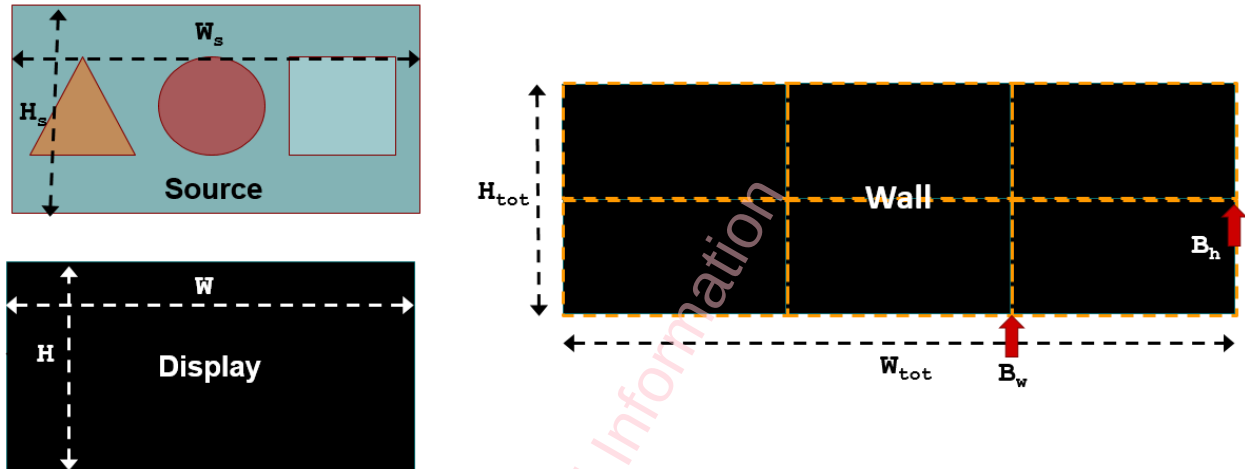


Figure 55: Illustration of Location for Viewport Parameters

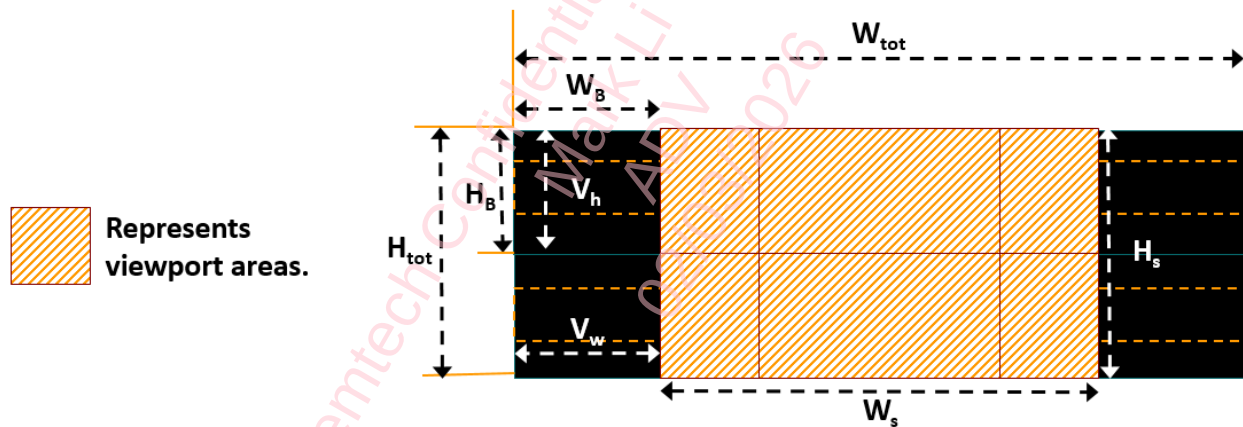


Figure 56: Illustration of Addition Viewport Parameters

To gain a solid understanding of the concepts of viewports and the area of interest (AOI) using the top row of the example 3x2 wall grid shown earlier, RX1, RX2 and RX3, we'll look closer at the regions that apply to a video wall. Refer to Figure 57: Individual Viewport Areas Highlighted for Sample 3x2 Wall. The following concepts shown for row one would apply to the second row of displays as well.

In this example, the wall aspect ratio is larger than the source aspect ratio ($A_w > A_s$), therefore black pillars are added to the left and right displays.

Reminder: With the exception of size, each set of values to be applied to video set command is calculated for each individual decoder (RX). The offset, keep and viewport values are different for each display.

In the following figure, each viewport area is outlined by orange dashes, for this sample 3x2 wall grid there is six decoders (RX) involved that require their values to be calculated.

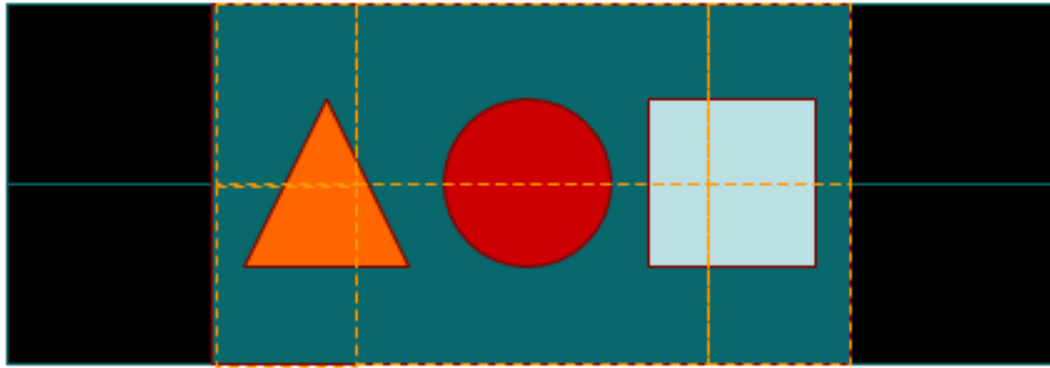


Figure 57: Individual Viewport Areas Highlighted for Sample 3x2 Wall Grid

For the first row looking at the display located in the upper left corner, the figures below show the viewport and area of interest (AOI) for decoder 1 (RX1). The source is displayed to the right and a black pillarbox fills the remaining pixels of the display on the left.

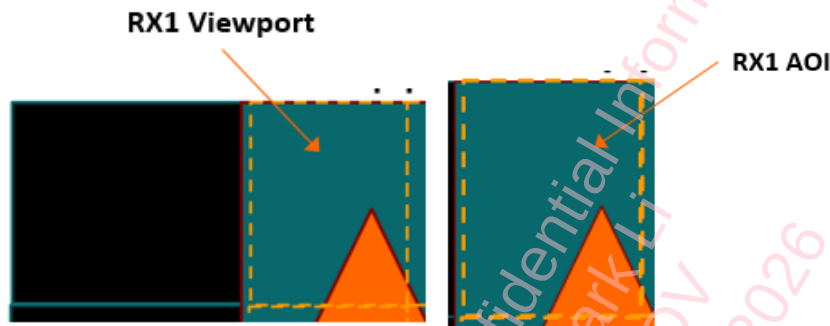


Figure 58: Example of Decoder 1 (RX1) Viewport and AOI

For the second (middle) display of the first row, the viewport covers the entire display.

Therefore, it is possible to either:

- Not specify the viewport arguments in the set command (recommended).
- Specify the viewport with the size equal to the display size and viewport offset set to 0.

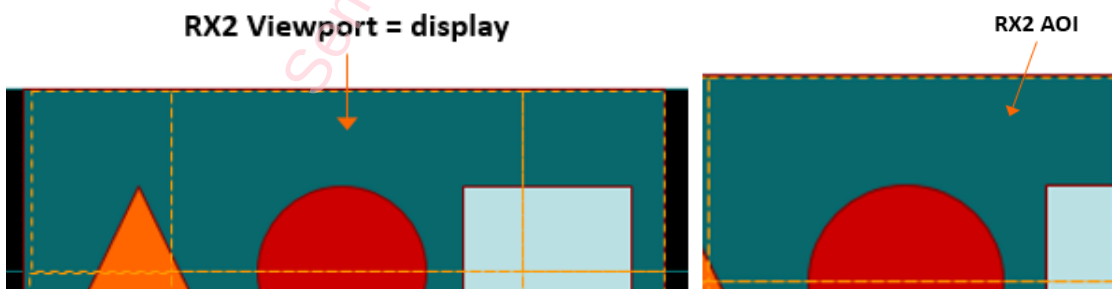


Figure 59: Example of Decoder 2 (RX2) Viewport and AOI

For the third display (RX3) on the top row of the sample wall grid of 3x2, the source is displayed on the left side of display with the black pillarbox filling the remaining pixels to the right. Therefore, there is no viewport offset adjustment required.

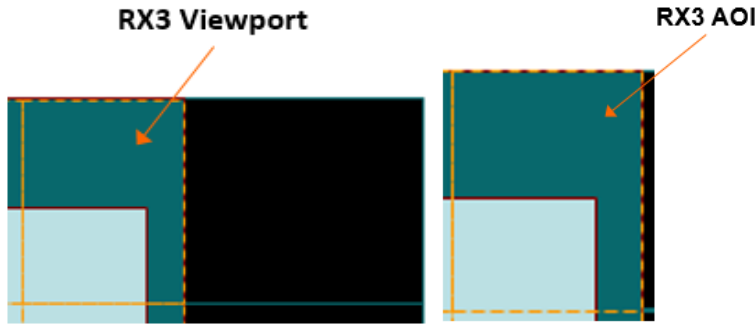


Figure 60: Example of Decoder 3 (RX3) Viewport and AOI

Now that we've introduced the concepts involved in applying a viewport, example computations of the wall values required for a set video request in relation to the wall mode will be reviewed.

7.3.9.3 Wall Parameter Computations

The following sub-sections review calculating each of the required five items: wall size, scaling ratios, borders, viewport and area of interest.

Once these values are calculated, then the final step is to apply the values to the API set video wall request arguments for each of the decoders (RX) used in the wall grid.

Note: These are sample formulas only and not necessarily the only formulas available to calculate the values needed for the video set command arguments.

Step 1: Total Wall Size

It is first necessary to calculate the total wall size. This can be done using the formulas shown below to compute the total wall size for the width and height respectively.

$$W_{tot} = C * W + (C - 1) * B_w$$

$$H_{tot} = R * H + (R - 1) * B_h$$

Step 2: Horizontal & Vertical Scaling Ratios

Although not yet known, it is possible that the aspect ratio will need to be preserved. That is black borders, pillars or envelope, will need to be added to the wall grid.

This is determined in the next step but in the event black borders are required, proceed to compute the following horizontal and vertical scaling ratios:

Calculate the horizontal and vertical scaling ratios as follows:

$$S_h = W_{tot} / W_s$$

$$S_v = H_{tot} / H_s$$

Reminder: Refer to Table 8: Definition of Parameters Used in Wall Calculations for description of parameters used in these formulas.

Step 3: Scaling Ratio & Borders

To calculate if black borders are required, black pillar box or an envelope, the scaling ratio needs to be calculated along with the width or height of the black borders, in pixels. If calculation result equals 0, then no border is required.

As outlined below, three cases need to be considered when performing these calculations.

The intuition behind this is that, if the horizontal and vertical scaling factors are different, the one to apply is the one that “squeezes” the image the most (i.e. the smallest). When the other dimension is “squeezed” by the same amount, black bars need to be added.

Case 1: $S_h = S_v$

If $S_h = S_v$ then:

$$S = S_h = S_v$$

$$W_B = 0$$

$$H_B = 0$$

Case 2: $S_h < S_v$

If $S_h < S_v$ then:

$$S = S_h$$

$$W_B = 0$$

$$H_B = (H_{tot} - S * H_s) / 2$$

Case 3: $S_v < S_h$

If $S_v < S_h$ then:

$$S = S_v$$

$$W_B = (W_{tot} - S * W_s) / 2$$

$$H_B = 0$$

Step 4: Viewport

Once the total wall size is calculated and it has been determined if black bars are required, it is then necessary to calculate the viewport for each of the decoders (RX) incorporating the wall grid.

There are five potential scenarios that can be applied to a given wall grid. They are outlined below.

Special Case (Horizontal): First Column

If a display is on the first column of the wall grid, the following applies:

$$V_{hp} = W_B$$

$$V_w = W - W_B$$

Special Case (Horizontal): Last Column

If a display is on the last column of the wall grid, the following applies:

$$V_{hp} = 0$$

$$V_w = W - W_B$$

Special Case (Vertical): First Row

If a display is on the first row of the wall grid, the following applies:

$$V_{vp} = H_B$$

$$V_h = H - H_B$$

Special Case (Vertical): Last Row

If a display is on the last row of the wall grid, the following applies:

$$V_{vp} = 0$$

$$V_h = H - H_B$$

Default Case

If neither of the two horizontal special cases listed above applies, the following applies instead:

$$V_{hp} = 0$$

$$V_w = W$$

If neither of the two vertical special cases listed above applies, the following applies instead:

$$V_{vp} = 0$$

$$V_h = H$$

Step 5: Area of Interest

The final calculations required is to calculate the Area of Interest.

Table 9: Values to Calculate for Area of Interest

Parameter	Description
i	The row in which a display is located in the wall grid, with the displays in the first row being numbered $i=1$.
j	The column in which a display is located in the wall grid, with the displays in the first column being numbered $j=1$.
O_h	The horizontal position of a display area of interest origin in the source coordinates system. That is the horizontal offset.
O_v	The vertical position of a display area of interest origin in the source coordinates system. That is the vertical offset.
K_w	The width of a display area of interest in the source coordinates system. That is the horizontal keep.
K_h	The height of a display area of interest in the source coordinates system. That is the vertical keep.

Using the row and column index, it is necessary to compute the following:

horizontal keep (width)

vertical keep (height)

horizontal offset

vertical offset

The keep parameters are calculated as follows:

$$K_w = V_w / S$$

$$K_h = V_h / S$$

When calculating the keep, the appropriate viewport value (V_w or V_h) must be inserted based on the special case column, row or default that applies. Refer to the Step 4: Viewport.

When computing the `offset` parameters, there are three cases that need to be considered. They are outlined below.

Special Case: First Column

$$O_h = 0$$

Special Case: First Row

$$O_v = 0$$

Default Case

If the "first column" special case above doesn't apply, the following applies:

$$O_h = ((j-1) * (W + B_w) - W_B) / S$$

If the "first row" special case above doesn't apply, the following applies:

$$O_v = ((i-1) * (H + B_h) - H_B) / S$$

7.3.9.4 Applying Viewport Values to API Request

Final step is to apply the calculated values to each of the decoders (RX) that incorporate the wall grid using set video wall API requests.

As a reminder, the `set video` in relation to `wall` mode with the optional viewport is shown below:

```
POST /api/device/{target}
{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : W,
    "height" : H,
    "fps" : F_s,
    "horiz_offset" : O_h,
    "vert_offset" : O_v,
    "keep_width" : K_w,
    "keep_height" : K_h,
    "viewport_horiz" : V_hp,
    "viewport_vert" : V_vp,
    "viewport_width" : V_w,
    "viewport_height" : V_h
  }
}
```

Note: The above request will set the target decoder to operate in the Genlock Wall mode. If the Fast Switch Wall mode is to be applied to the decoder, the `display_mode` option would be replaced with the `WALL_FAST_SWITCHED` option.

7.3.10 Genlock Wall Case Studies

To allow for a deeper understanding of calculating the values required for the API set video requests in relation to the Wall mode, two case studies will be reviewed.

7.3.10.1 Case Study 1

For Case Study 1, the values for the `set video wall` mode arguments are calculated for a 3x2 wall grid, assuming a source of 3840x2160@60 and output resolution of 1920x1080@60. Also, assume a bezel thickness equivalent of 24 pixels for all inside display borders incorporating the wall.

Based on the steps outlined in the previous section, the following calculations would be completed.

- First step is to calculate the total wall size. First we will calculate the total wall width then the total wall height.

The formula to use to calculate the total wall width is:

$$\begin{aligned}W_{tot} &= C * W + (C - 1) * B_w \\&= 3 * 1920 + (3 - 1) * 24 \\&= 5760 + 2 * 24 \\&= 5760 + 48 \\&= \mathbf{5808}\end{aligned}$$

To calculate the total wall height:

$$\begin{aligned}H_{tot} &= R * H + (R - 1) * B_h \\&= 2 * 1080 + (2 - 1) * 24 \\&= 2160 + 1 * 24 \\&= 2160 + 24 \\&= \mathbf{2184}\end{aligned}$$

- Next calculate the horizontal and vertical scaling factor for the sample wall grid:

$$\begin{aligned}S_h &= W_{tot} / W_s \\&= 5808 / 3840 \\&= 1.5125\end{aligned}$$

$$\begin{aligned}S_v &= H_{tot} / H_s \\&= 2184 / 2160 \\&= 1.0111\end{aligned}$$

- Now we'll proceed to calculate if black borders are required for the video wall.

In Step 3: Scaling Ratio & Borders, it was noted that the formula to use varies depending on the scaling ratio calculated.

For our example, $S_v < S_h$ therefore, the following formulas are used:

$$\begin{aligned}S &= S_v \\&= \mathbf{1.0111}\end{aligned}$$

$$\begin{aligned}
W_B &= (W_{tot} - S * W_s) / 2 \\
&= (5808 - 1.0111 * 3840) / 2 \\
&= (5808 - 3882.66) / 2 \\
&= 1925.33 / 2 \\
&= \mathbf{962.66} \rightarrow \mathbf{962} \text{ (Round to the closest even integer)}
\end{aligned}$$

$$H_B = 0$$

The result shows that vertical black borders (pillar bars) of 962 pixels need to be added to this case study's wall grid.

- We are now ready to calculate the Viewport for each of the decoders of the wall grid.

In Step 4: Viewport, it was shown that there are five potential scenarios that can apply to a given wall grid.

Reminder for this sample 3x2 wall grid, there are three columns and two rows. Therefore, the following formulas apply:

Horizontal Special Case: First Column

For the displays located on the first column of the sample wall grid, RX1 and RX4, the following formulas applies:

$$\begin{aligned}
V_{hp} &= W_B \\
&= 962 \\
V_w &= W - W_B \\
&= 1920 - 962 \\
&= 958
\end{aligned}$$

Horizontal Special Case: Last Column

For the displays located on the last column of the sample wall grid, RX3 and RX6, the following formulas applies:

$$\begin{aligned}
V_{hp} &= 0 \\
V_w &= W - W_B \\
&= 1920 - 962 \\
&= 958
\end{aligned}$$

Vertical Special Case: First Row

For the displays located on the first row of the sample wall grid, RX1, RX2 and RX3, the following formulas applies:

$$\begin{aligned}
V_{vp} &= H_B \\
&= 0 \\
V_h &= H - H_B \\
&= 1080 - 0 \\
&= 1080
\end{aligned}$$

Vertical Special Case: Last Row

For the displays located on the last row of the sample wall grid, RX4, RX5 and RX6, the following formulas applies:

$$\begin{aligned}V_{vp} &= 0 \\V_h &= H - H_B \\&= 1080 - 0 \\&= 1080\end{aligned}$$

Default Case

If neither of the horizontal special cases outlined above apply, the following applies instead:

$$\begin{aligned}V_{hp} &= 0 \\V_w &= W \\&= 1920\end{aligned}$$

If neither of the two vertical special cases listed above apply, the following does instead:

$$\begin{aligned}V_{vp} &= 0 \\V_h &= H \\&= 1080\end{aligned}$$

- The final calculation is the Area of Interest (AOI) for each decoder (RX) that incorporates the sample 3x2 wall grid.

For a full description of the list of parameters referred to in this step, refer to Table 9: Values to Calculate for Area of Interest in Step 5: Area of Interest.

In this step, the row and column index are used to calculate the following values:

horizontal keep (width)
vertical keep (height)
horizontal offset
vertical offset

- The `keep` argument of the API set video wall mode command is calculated using the same formula for both the special cases and the default case but the appropriate viewport value must be inserted, that is V_w or V_h based on the special case column or row that applies.

To calculate the `keep` for Special Case (Horizontal): First column, RX1 and RX4, the following values would be applied:

$$\begin{aligned}K_w &= V_w / S \\&= 957 / 1.0111 \\&= 946 \text{ (Round to the closest even integer)} \\K_h &= V_h / S \\&= 1080 / 1.0111 \\&= 1068.13 \rightarrow 1068 \text{ (Round to the closest even integer)}\end{aligned}$$

In relation to the Special Case (Horizontal): Last column, RX3 and RX6, the following values would be applied:

$$\begin{aligned}
 K_w &= V_w / S \\
 &= 957 / 1.0111 \\
 &= 946 \text{ (Rounded to the closest even integer)} \\
 K_h &= V_h / S \\
 &= 1080 / 1.0111 \\
 &= 1068.13 \rightarrow 1068 \text{ (Round to the closest even integer)}
 \end{aligned}$$

Note: The results for both special cases are the same.

For the default case, if neither of the above special cases apply, using the same formula calculate the keep values for width and height. For this case study, these calculations apply to the middle column which contains RX2 and RX5:

$$\begin{aligned}
 K_w &= V_w / S \\
 &= 1920 / 1.0111 \\
 &= 1898.90 \rightarrow 1898 \text{ (Round to the closest even integer.)} \\
 K_h &= V_h / S \\
 &= 1080 / 1.0111 \\
 &= 1068.13 \rightarrow 1068 \text{ (Round to the closest even integer)}
 \end{aligned}$$

- To compute the offset arguments for the sample 3x2 wall grid, there are three cases to be considered.

TIP: Refer to earlier steps for more information on determining what applies to which decoders (RX) of a given wall grid.

Special Case: First Column

Applies to RX1 and RX4.

$$O_h = 0$$

Special Case: First Row

Applies to RX1, RX2 and RX3.

$$O_v = 0$$

Default Case

For the sample 3x2 wall grid, this default case applies to calculating the horizontal offset (O_h) for RX2, RX3, RX5 and RX6.

$$O_h = ((j-1) * (W + B_w) - W_B) / S$$

Reminder: j equals the column

Horizontal offset (O_h) for RX2

$$\begin{aligned} &= ((2-1) * (1920 + 24) - 962) / 1.0111 \\ &= (1944 - 963) / 1.0111 \\ &= 981 / 1.0111 \\ &= 970.23 \rightarrow \mathbf{970} \text{ (Rounded to closest even integer.)} \end{aligned}$$

Horizontal offset (O_h) for RX3

$$\begin{aligned} &= ((3-1) * (1920 + 24) - 962) / 1.0111 \\ &= (2*1944 - 963) / 1.0111 \\ &= (3888 - 963) / 1.0111 \\ &= 2925 / 1.0111 \\ &= 2892.88 \rightarrow \mathbf{2892} \text{ (Rounded to closest even integer.)} \end{aligned}$$

Horizontal offset (O_h) for RX5

$$= \mathbf{970}$$

Horizontal offset (O_h) for RX6

$$= \mathbf{2892}$$

Note: The same calculations are used for RX2 and RX5, as well as RX3 and RX6, therefore only the calculations for RX2 and RX3 are shown.

For the sample 3x2 wall grid, this default case applies to calculating the vertical offset (O_v) for RX4, RX5 and RX6.

Reminders:

- Special Case: First Row applies to RX1, RX2 and RX3, therefore the $O_v = 0$.
- i equals the row

Similar to the horizontal offset, the vertical offset of the default case applies to RX4, RX5 and RX6, calculations are shown below:

$$O_v = ((i-1) * (H + B_h) - H_B) / S$$

Vertical offset (O_h) for RX4

$$\begin{aligned} &= ((2-1) * (1080 + 24) - 0) / 1.011 \\ &= (1*1104-0) / 1.011 \\ &= 1104 / 1.011 \\ &= 1091.88 \rightarrow \mathbf{1092} \text{ (Round to the closest even integer.)} \end{aligned}$$

Vertical offset (O_h) for RX5

$$= \mathbf{1092}$$

Vertical offset (O_h) for RX6

$$= \mathbf{1092}$$

- The last step is to create the API requests using the values calculated for each of the decoders (RX), that is RX1 through RX6, that are used to incorporate this sample 3x2 wall grid.

Reminder that in relation to the formulas used, the syntax of the API request is as follows:

```
POST /api/device/{target}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : W,
    "height" : H,
    "fps" : Fs,
    "horiz_offset" : Oh,
    "vert_offset" : Ov,
    "keep_width" : Kw,
    "keep_height" : Kh,
    "viewport_horiz" : Vhp,
    "viewport_vert" : Vvp,
    "viewport_width" : Vw,
    "viewport_height" : Vh
  }
}
```

Reminder: There are no line breaks when issuing the set video API command for each of the decoders (RX), so the command is to be entered as a single line command.

Below are examples where the calculated values have been entered for the API set video requests for each of the decoders (RX) involved in Case Study 1.

TIP: The appropriate device ID would need to be inserted for each of the examples provided.

API request for RX1:

```
POST /api/device/{RX1}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "horiz_offset" : 0,
    "vert_offset" : 0,
    "keep_width" : 946,
    "keep_height" : 1068,
    "viewport_horiz" : 962,
    "viewport_vert" : 0,
    "viewport_width" : 958,
    "viewport_height" : 1080
  }
}
```

API request for RX2:

```
POST /api/device/{RX2}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "horiz_offset" : 970,
    "vert_offset" : 0,
    "keep_width" : 1898,
    "keep_height" : 1068,
    "viewport_horiz" : 0,
    "viewport_vert" : 0,
    "viewport_width" : 1920,
    "viewport_height" : 1080
  }
}
```

API request for RX3:

```
POST /api/device/{RX3}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
    "horiz_offset" : 970,
    "vert_offset" : 0,
    "keep_width" : 946,
    "keep_height" : 1068,
    "viewport_horiz" : 0,
    "viewport_vert" : 0,
    "viewport_width" : 958,
    "viewport_height" : 1080
  }
}
```

API request for RX4:

```
POST /api/device/{RX4}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1920,
    "height" : 1080,
    "fps" : 60,
```

```

        "horiz_offset" : 0,
        "vert_offset" : 1092,
        "keep_width" : 946,
        "keep_height" : 1068,
        "viewport_horiz" : 962,
        "viewport_vert" : 0,
        "viewport_width" : 958,
        "viewport_height" : 1080
    }
}

```

API request for RX5:

```

POST /api/device/{RX5}
{
    "op" : "set:video",
    "display_mode" : "WALL_GENLOCKED",
    "video" : {
        "width" : 1920,
        "height" : 1080,
        "fps" : 60,
        "horiz_offset" : 970,
        "vert_offset" : 1092,
        "keep_width" : 1898,
        "keep_height" : 1068,
        "viewport_horiz" : 0,
        "viewport_vert" : 0,
        "viewport_width" : 1920,
        "viewport_height" : 1080
    }
}

```

API request for RX6:

```

POST /api/device/{RX6}
{
    "op" : "set:video",
    "display_mode" : "WALL_GENLOCKED",
    "video" : {
        "width" : 1920,
        "height" : 1080,
        "fps" : 60,
        "horiz_offset" : 2892,
        "vert_offset" : 1092,
        "keep_width" : 946,
        "keep_height" : 1068,
        "viewport_horiz" : 0,
        "viewport_vert" : 0,
        "viewport_width" : 958,
        "viewport_height" : 1080
    }
}

```

7.3.10.2 Case Study 2

For Case Study 2, we will calculate the values for the set video wall mode arguments also using a sample 3x2 wall grid with a source of 1920x1080@60 but this time assume an output resolution of 1280x720@60 and a bezel thickness of 20 pixels for the display.

Based on the steps outlined in section 7.3.9.3, the following calculations would be completed.

- First calculate the total wall size for this second case study. Remember the total wall width and the total wall height are calculated individually.

Formula to calculate the total wall width:

$$\begin{aligned}W_{tot} &= C * W + (C - 1) * B_w \\&= 3 * 1280 + (3 - 1) * 20 \\&= 3840 + 2 * 20 \\&= 3840 + 40 \\&= \mathbf{3880}\end{aligned}$$

Formula to calculate the total wall height:

$$\begin{aligned}H_{tot} &= R * H + (R - 1) * B_h \\&= 2 * 720 + (2 - 1) * 20 \\&= 1440 + 1 * 20 \\&= 1440 + 20 \\&= \mathbf{1460}\end{aligned}$$

- Next calculate the horizontal and vertical scaling factor for the sample wall grid:

$$\begin{aligned}S_h &= W_{tot} / W_s \\&= 3880 / 1920 \\&= 2.0208 \\S_v &= H_{tot} / H_s \\&= 1460 / 1080 \\&= 1.3519\end{aligned}$$

- Now proceed to calculate if black borders are required for the video wall and if so whether they will be pillar bars or letterbox/envelope.

In Step 3: Scaling Ratio & Borders, we noted that the formula used will vary depending on the scaling ratio calculated.

For this case study $S_v < S_h$, so the following formulas are used:

$$\begin{aligned}S &= S_v \\&= 1.3519 \\W_B &= (W_{tot} - S * W_s) / 2 \\&= (3880 - 1.3519 * 1920) / 2 \\&= (3880 - 2596) / 2 \\&= 1284 / 2 \\&= \mathbf{642}\end{aligned}$$

$$H_B = 0$$

The result shows that vertical black borders (pillar bars) of 642 pixels need to be added to this case study's wall grid.

- Proceed to calculate the viewport for each of the decoders incorporating the wall grid.

In Step 4: Viewport, it was shown that there are five potential scenarios that can apply to a given wall grid. Since this second case study has three columns and two rows, the following formulas apply:

Horizontal Special Case: First Column

For the displays located on the first column of the sample wall grid, RX1 and RX4, the following formulas applies:

$$\begin{aligned} V_{hp} &= W_B \\ &= 642 \\ V_w &= W - W_B \\ &= 1280 - 642 \\ &= 638 \end{aligned}$$

Horizontal Special Case: Last Column

For the displays located on the last column of the sample wall grid, RX3 and RX6, the following formulas applies:

$$\begin{aligned} V_{hp} &= 0 \\ V_w &= W - W_B \\ &= 1280 - 642 \\ &= 638 \end{aligned}$$

Vertical Special Case: First Row

For the displays located on the first row of the sample wall grid, RX1, RX2 and RX3, the following formulas applies:

$$\begin{aligned} V_{vp} &= H_B \\ &= 0 \\ V_h &= H - H_B \\ &= 720 - 0 \\ &= 720 \end{aligned}$$

Vertical Special Case: Last Row

For the displays located on the last row of the sample wall grid, RX4, RX5 and RX6, the following formulas applies:

$$\begin{aligned} V_{vp} &= 0 \\ V_h &= H - H_B \\ &= 720 - 0 \\ &= 720 \end{aligned}$$

Default Case

If neither of the horizontal special cases (RX2 and RX5 for this case study) outlined above apply, the following formula is used:

$$\begin{aligned}V_{hp} &= 0 \\V_w &= W \\&= 1280\end{aligned}$$

If neither of the two vertical special cases listed above apply, then the following formula is used:

$$\begin{aligned}V_{vp} &= 0 \\V_h &= H \\&= 720\end{aligned}$$

For this use case, we have two rows, so the default case is not applicable, as the Vertical Special Case: First Row and Vertical Special Case: Last Row apply.

- The final calculation is the Area of Interest (AOI) for each decoder (RX) that incorporates this case study's sample 3x2 wall grid.

For a full description for the list of parameters referred to in this step, refer to Table 9: Values to Calculate for Area of Interest in Step 5: Area of Interest.

In this step, the row and column index are used to calculate the following values:

horizontal offset

vertical offset

horizontal keep (width)

vertical keep (height)

- The `keep` argument of the API set video wall mode command is calculated using the same formula for both the special cases and default, but the appropriate viewport value must be inserted, that is V_w or V_h based on the special case column or row that applies.

To calculate the `keep` for Special Case (Horizontal): First column, specifically RX1 and RX4, the following values would be applied:

$$\begin{aligned}K_w &= V_w / S \\&= 638 / 1.3519 \\&= 471.92 \rightarrow \mathbf{470} \text{ (Round to the closest even integer)} \\K_h &= V_h / S \\&= 720 / 1.3519 \\&= 532.584 \rightarrow \mathbf{532} \text{ (Rounded to the closest even integer)}\end{aligned}$$

In relation to the Special Case (Horizontal): Last column, specifically this would be RX3 and RX6, the following values would be applied:

$$\begin{aligned}K_w &= V_w / S \\&= 638 / 1.3519 \\&= 470 \text{ (Round to the closest even integer)}\end{aligned}$$

$$\begin{aligned}
K_h &= V_h / S \\
&= 720 / 1.3519 \\
&= 532.584 \rightarrow \mathbf{532} \text{ (Rounded to the closest even integer)}
\end{aligned}$$

Note: The result is the same for both special cases.

For the default case, if neither of the above special cases apply, using the same formula the following would be the calculations. For this case study, these calculations apply to the middle column which contains RX2 and RX5:

$$\begin{aligned}
K_w &= V_w / S \\
&= 1280 / 1.3519 \\
&= 946.815 \rightarrow \mathbf{946} \text{ (Round to the closest even integer)} \\
K_h &= V_h / S \\
&= 720 / 1.3519 \\
&= 532.584 \rightarrow \mathbf{532} \text{ (Round to closest even integer)}
\end{aligned}$$

- To compute the `offset` arguments for the sample 3x2 wall grid, there are three cases that need to be considered.

Reminder: Refer to earlier steps for more information on determining what applies to which decoders (RX) of a given wall grid.

Special Case: First Column

Applies to RX1 and RX4.

$$O_h = 0$$

Special Case: First Row

Applies to RX1, RX2 and RX3.

$$O_v = 0$$

Default Case

For the sample 3x2 wall grid, this default case applies to calculating the horizontal offset (O_h) for RX2, RX3, RX5 and RX6.

$$O_h = ((j-1) * (W + B_w) - W_B) / S$$

Reminder: j equals the column

Horizontal offset (O_h) for RX2

$$\begin{aligned}
&= ((2-1) * (1280 + 20) - 642) / 1.3519 \\
&= (1300 - 642) / 1.3519 \\
&= 658 / 1.3519 \\
&= 486.722 \rightarrow \mathbf{486} \text{ (Round to the closest even integer)}
\end{aligned}$$

Horizontal offset (O_h) for RX3

$$\begin{aligned}
&= ((3-1) * (1280 + 20)) - 642) / 1.3519 \\
&= 2*1300-642 / 1.3519 \\
&= 2600-642 / 1.3519
\end{aligned}$$

$$= 1958 / 1.3519$$

$$= 1448.331 \rightarrow \mathbf{1448} \text{ (Round to the closest even integer)}$$

Horizontal offset (O_h) for RX5

$$= \mathbf{486}$$

Horizontal offset (O_h) for RX6

$$= \mathbf{1448}$$

Note: The same calculations are used for RX2 and RX5, as well as RX3 and RX6, therefore only the actual calculations for RX2 and RX3 are shown.

For the sample 3x2 wall grid provided in this case study, the default case applies to calculating the vertical offset (O_v) for RX4, RX5 and RX6.

Reminder: i equals the row

Similar to the horizontal offset, the vertical offset is the same for RX4, RX5 and RX6, therefore only the calculations for RX4 are shown.

$$O_v = (i * (H + B_h) - H_B) / S$$

Vertical offset (O_h) for RX4

$$= ((2-1) * (720 + 20) - 0) / 1.3518$$

$$= 1*740-0 / 1.3518$$

$$= 740 / 1.3518$$

$$= 547.378 \rightarrow \mathbf{546} \text{ (Round to the closest even integer)}$$

Vertical offset (O_h) for RX5

$$= \mathbf{546}$$

Vertical offset (O_h) for RX6

$$= \mathbf{546}$$

- The last step is to create the API requests using the values calculated for each of the decoders (RX), RX1 through RX6, used to create this sample 3x2 wall grid.

Note: The appropriate device ID would need to be inserted into each of the command examples provided.

API request for RX1:

```
POST /api/device/{RX1}
{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1280,
    "height" : 720,
    "fps" : 60,
    "horiz_offset" : 0,
    "vert_offset" : 0,
```

```

        "keep_width" : 470,
        "keep_height" : 532,
        "viewport_horiz" : 642,
        "viewport_vert" : 0,
        "viewport_width" : 638,
        "viewport_height" : 720
    }
}

```

API request for RX2:

```

POST /api/device/{RX2}
{
    "op" : "set:video",
    "display_mode" : "WALL_GENLOCKED",
    "video" : {
        "width" : 1280,
        "height" : 720,
        "fps" : 60,
        "horiz_offset" : 486,
        "vert_offset" : 0,
        "keep_width" : 946,
        "keep_height" : 532,
        "viewport_horiz" : 0,
        "viewport_vert" : 0,
        "viewport_width" : 1280,
        "viewport_height" : 720
    }
}

```

API request for RX3:

```

POST /api/device/{RX3}
{
    "op" : "set:video",
    "display_mode" : "WALL_GENLOCKED",
    "video" : {
        "width" : 1280,
        "height" : 720,
        "fps" : 60,
        "horiz_offset" : 1448,
        "vert_offset" : 0,
        "keep_width" : 470,
        "keep_height" : 532,
        "viewport_horiz" : 0,
        "viewport_vert" : 0,
        "viewport_width" : 638,
        "viewport_height" : 720
    }
}

```

API request for RX4:

```
POST /api/device/{RX4}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1280,
    "height" : 720,
    "fps" : 60,
    "horiz_offset" : 0,
    "vert_offset" : 546,
    "keep_width" : 470,
    "keep_height" : 532,
    "viewport_horiz" : 642,
    "viewport_vert" : 0,
    "viewport_width" : 638,
    "viewport_height" : 720
  }
}
```

API request for RX5:

```
POST /api/device/{RX5}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1280,
    "height" : 720,
    "fps" : 60,
    "horiz_offset" : 486,
    "vert_offset" : 546,
    "keep_width" : 946,
    "keep_height" : 532,
    "viewport_horiz" : 0,
    "viewport_vert" : 0,
    "viewport_width" : 1280,
    "viewport_height" : 720
  }
}
```

API request for RX6:

```
POST /api/device/{RX6}

{
  "op" : "set:video",
  "display_mode" : "WALL_GENLOCKED",
  "video" : {
    "width" : 1280,
    "height" : 720,
    "fps" : 60,
```

```

        "horiz_offset" : 1448,
        "vert_offset" : 546,
        "keep_width" : 470,
        "keep_height" : 532,
        "viewport_horiz" : 0,
        "viewport_vert" : 0,
        "viewport_width" : 638,
        "viewport_height" : 720
    }
}

```

7.4 Video Wall Mode Summary

To summarize Wall mode functionality, it is the ability to display a single source across multiple displays, each showing an assigned section of the source defined with an Area of Interest (AOI) and when appropriate, fill part of a display with black bars.

Two wall modes are supported, the `genlock wall` and `fast switch wall` modes.

1. When Genlock Wall mode is implemented, the decoders (RX) are synchronized to the source, therefore to each other.
2. Fast Switch Wall mode is an alternative when fast switching so a clean switch between sources is needed or when signals need to cross multiple network switches. Keep in mind this may result in individual screens that are not in sync with each other.

Bezel compensation equaling the inside borders of the displays should be applied to calculations to provide a more realistic viewing experience for end users.

If the Wall aspect ratio is different when compared to the source aspect ratio, black bars referred to as “viewports” are available to show the entire source without geometric distortion.

A wall configuration can be applied using a Source Independent Wall API request that automatically builds the wall configuration for a AVP2000(T) decoder (RX) device. Refer to section 7.2 Implementing Source Independent Video Wall for details. Alternately a video wall can be custom configured, refer to section 7.3 Configuring Custom Video Wall for details.

Tips for when entering values into a `set video` request:

- i. The video mode of a decoder (RX) is set to `WALL_GENLOCKED` or `WALL_FAST_SWITCHED`.

Reminder: The `set video` arguments use for wall mode are not applicable to other processing modes.

- ii. Be sure the appropriate values are applied to the `set video` request in relation to the position of the decoder (RX) outputting to a display mounted in the wall grid.
 - a. For instance, if the wrong values are applied in relation to the position of a display, the video output would be located in an incorrect position on the video wall.
 - b. If the bezel compensation is not accurate, the viewing experience would be unrealistic in that the video would not align cleanly between the displays.
- iii. In regard to the output frame rate, when operating in Genlock Wall mode the frame rate (fps) must be equal to the source frame rate.

TIP: If a different output frame rate is needed, the Fast Switch wall mode can be applied.

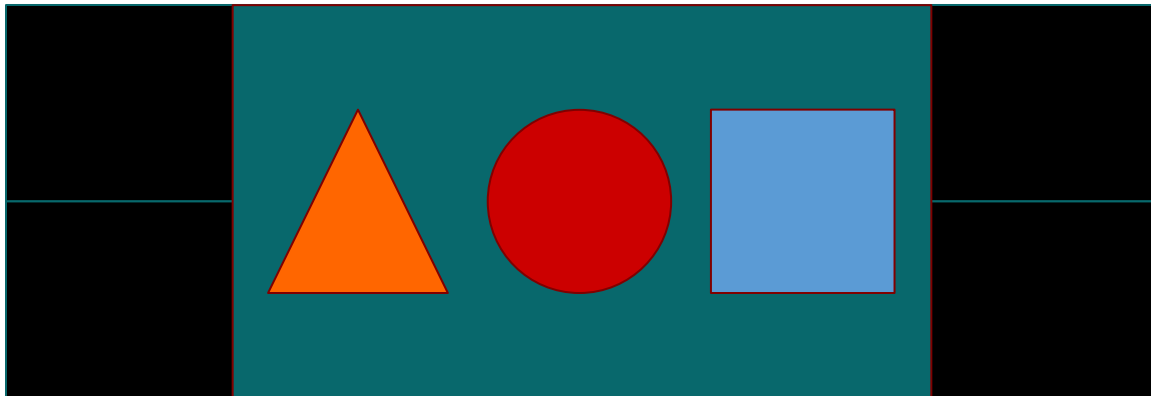


Figure 61: 3x2 Wall Grid 16:9 Source Shown with Viewport Applied

7.5 Creating a Video Wall with Multiple Sources

If it is desired to create a wall that is to display multiple sources, such as the example shown below, this is possible using SDVoE technology. There are two approaches available to complete this scenario, with both approaches outlined in this section, including advantages and disadvantages.

This section will use the following 2x2 wall grid as an example.

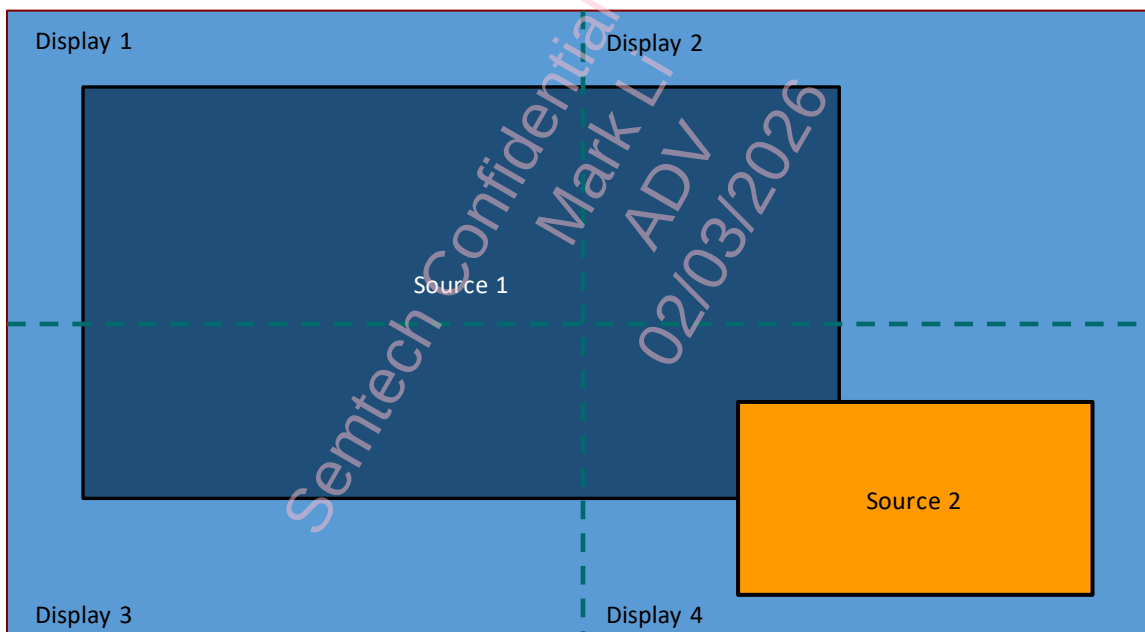


Figure 62: Example of 2x2 Wall Grid and Two Source Layout

The Multiview and Wall mode sections of this application note library supply details on the use of these respective processing modes. An understanding of these sections is a prerequisite to applying either of the scenarios outlined below.

7.5.1 Scenario 1: Multiview Layout as Wall Source

For this first scenario, a multiview layout is created and then it is fed as the source to the wall grid.

The advantage to this scenario is that the sources remain synchronized and the calculations to apply match those outlined for the multiview, and the wall modes outlined in this application library in their respective sections.

Note: The wall mode is synchronized when the Genlock wall mode is applied to the decoders that are feeding the wall grid. If the Fast Switch wall mode is implemented, then the wall behaves as outlined in section 7.1 Introduction to Video Wall Mode.

The disadvantage of this scenario is that an extra encoder and decoder are required in comparison to scenario 2. Also, the video quality is slightly impacted as the source is converted once extra when compared to scenario 2.

For our above example that is using two sources and four output displays, in total three (3) encoders (TX) and five (5) decoders (RX) are required.

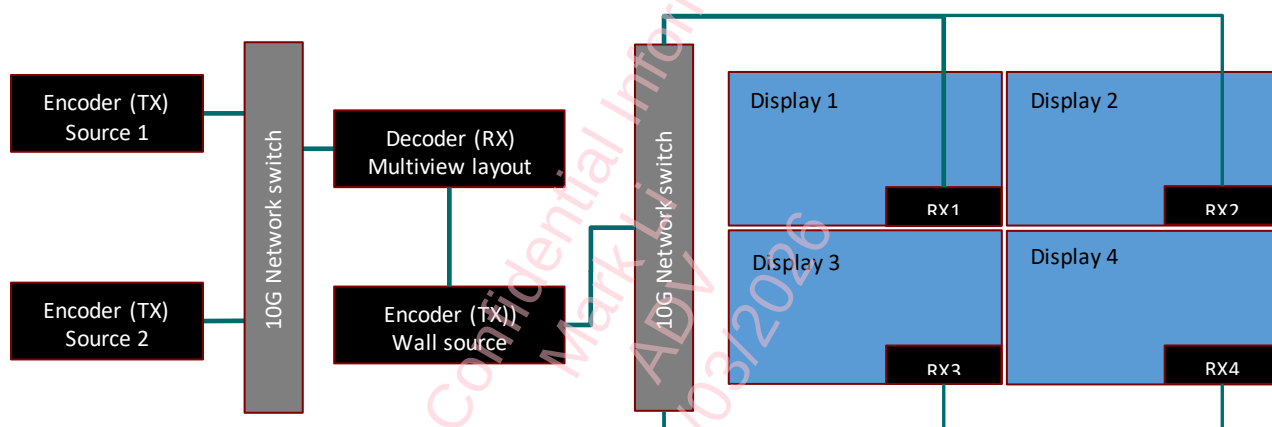


Figure 63: Scenario 1 Multiview Layout as Wall Source

The figure above illustrates our example, where the decoder (RX) labeled in the above figure as “Multiview layout” subscribes to the two sources assigned in the multiview layout. Based on the API multiview layout requests issued, it creates the output multiview composite.

This decoder's output is then fed to the encoder labeled as “Wall source”.

The four decoders that are involved directly in the wall grid then subscribe to this stream. API requests are then applied to each of the decoders (RX) specifying the area of interest (AOI) that each is to display.

7.5.2 Scenario 2: Multiview Offset Argument Modified

The second option would utilize the multiview mode and apply a unique layout to each individual decoder (RX). Each decoder that is part of the wall grid will operate in multiview mode and each will load a layout displaying the source(s) with the offset location (area of interest) specified.

When creating and managing the layouts, an offset (location) is calculated and applied to the “layout window” for each decoder (RX). The offset values applied are unique for each decoder.

An advantage of this approach is that it does not require extra equipment. The source is also scaled only once so the video quality may be better when compared to scenario 1. In addition, if the source is changed the fast switch option applies. This means that the source change occurs quickly, and no glitches will be noted.

The disadvantage is that the displays are **not** synchronized to the sources, so therefore are not synchronized to each other. It is also more complicated in regard to calculating the horizontal and vertical offset values.

Looking at Figure 62: Example of 2x2 Wall Grid and Two Source Layout, Figure 64 below shows the hardware setup that would be involved with this scenario.

Summary: Each decoder (RX) is operating independently, each is set to multiview mode, and each is displaying a specific layout displaying the source(s) applicable to the display it is connected to. Although it appears as a wall grid, it is actually four multiview layouts.

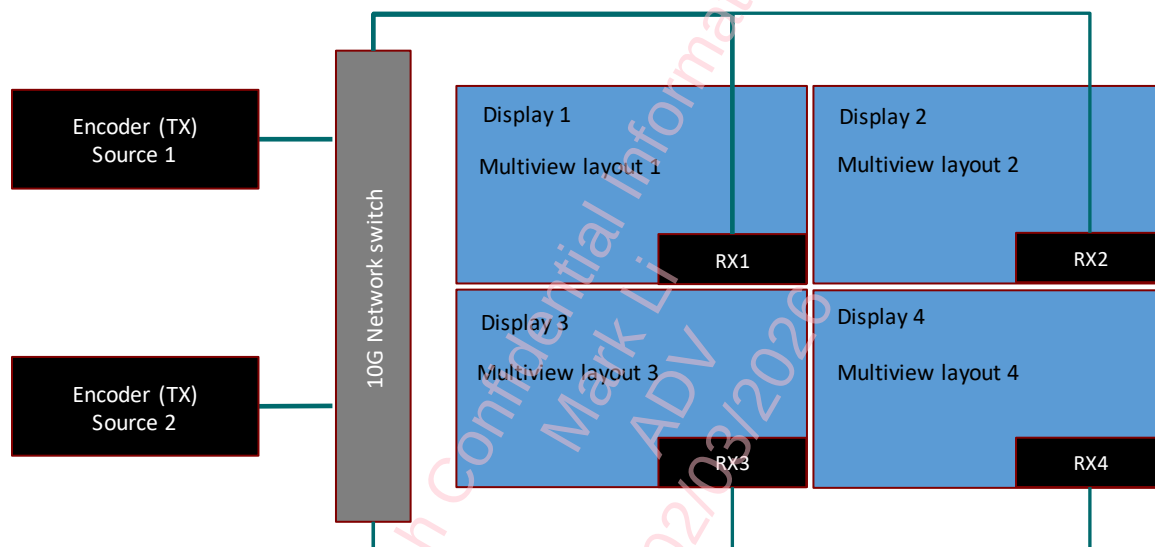


Figure 64: Scenario 2 Multiview Layouts Applied

The setup above illustrates that the two HDMI sources required for our example are fed to two encoder devices (TX). As mentioned previously, the decoder devices (RX) must be operating in multiview mode and be joined/subscribed to the appropriate sources.

Each decoder is sent a multiview API request containing details on the layout it is to display, including the size, offset and position information appropriate for the layout. The source will then be cropped to the values specified, resulting in the wall grid being output as shown in Figure 62: Example of 2x2 Wall Grid and Two Source Layout.

8 Set Property Command Example Use Cases

The following section provides details and examples of some use cases that involve the management of device properties using a `set property` request.

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : String,
  "value" : Varies (see text)
}
```

Refer to the SDVoE Developers API Reference Guide for information on the `set property` command and a full list of configurable properties.

8.1 Device Mode Property

In the case of either an AVP endpoint device, it is possible to modify the active device mode of an endpoint device that has been deployed in the field, allowing a device to behave specifically as a transmitter (TX) or receiver (RX). It is required however, that the endpoint device have both an HDMI Input connector and an HDMI Output connector. It is also required that the hardware manufacturer enabled this ability for the device, this can be confirmed by retrieving the device capabilities using a `get device` request. Refer to the SDVoE Developers API Reference Guide for details on `get` requests.

The device mode is managed using a `set property` request along with the property key `"configuration.device_mode"`. The value entered is `TRANSMITTER`, `RECEIVER` or `TRANSCEIVER`. The `target` is the device ID of endpoint whose mode is to be changed.

Syntax for a `set property` request to set the device mode:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : "configuration.device_mode",
  "value" : "TRANSMITTER/RECEIVER/TRANSCEIVER"
}
```

Table 10: Valid Device Mode Changes

Chipset	Hardware**	TRANSMITTER	RECEIVER	TRANSCEIVER
AVP2000T	Video In/Out	Transmitter	Receiver	Transceiver
	Video In	Transmitter	Invalid, no change	Invalid, no change
	Video Out	Invalid, no change	Receiver	Invalid, no change
AVP2000/AVP1000	Video In/Out	Transmitter	Receiver	Invalid, no change
	Video In	Transmitter	Invalid, no change	Invalid, no change
	Video Out	Invalid, no change	Receiver	Invalid, no change

** Important to note that **hardware** refers to the HDMI connectors present on the BlueRiver AVP endpoint device, possible that hardware will have only a Video In or Video Out for transmitter and receiver devices, respectively.

Example of setting a transceiver device with the device ID of 0016c04e1c0c, so a device with both a physical HDMI Input connector and HDMI Output connection, to operate in TRANSMITTER mode.

```
POST /api/device/0016c04e1c0c
{
  "op" : "set:property",
  "key" : "configuration.device_mode",
  "value" : "TRANSMITTER"
}
{
  "status": "PROCESSING",
  "request_id": 9,
  "result": null,
  "error": null
}
```

8.2 Managing Device Name

The default Device Name is the device id (MAC address) of the endpoint device, it is often useful to modify this to something more meaningful to the end users. This is done using a set property request with the property key, configuration.device_name.

Starting in API version 3.5 the device name may be used when issuing an API request in place of the device ID (MAC address). Also, worth noting that since BlueRiver AVP firmware 1.5, the device name is also persistent following a factory reset, that is it does not revert back to the devices MAC address as in previous firmware versions but keeps the updated device name.

Syntax for a set property request to set the device mode:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : "configuration.device_name",
  "value" : "specify device name"
}
```

Note: When using the device name to issue an API request, it is case sensitive.

Warning! When issuing an API request that has a device name containing a special reserved character or unsupported character in the URL of the request, the application must encode the character(s) with a % escape sequence. For example a space is encoded as %20. To illustrate the URL of an API request that includes a space in the device name, Peter Pan, would be encoded as follows:

<http://localhost:8080/api/device/Peter%20Pan/>

Note: This article assumes that the reader is familiar with the issuing commands using encoded characters. The table below provides a list of commonly used reserved characters for reference purposes.

Table 11: RFC3986 Reserved Characters

Symbol	Escape Sequence	Description
	%20	Space
!	%21	Exclamation Mark
#	%23	Pound Sign
\$	%24	Dollar Sign
%	%25	Percent
&	%26	And Sign
'	%27	Single Quote
(	%28	Left Parenthesis
)	%29	Right Parenthesis
*	%2A	Star
+	%2B	Plus Sign
,	%2C	Comma
/	%2F	Forward Slash
:	---	Not supported, see note below ⁽¹⁾ .
;	%3B	Semi Colon
=	%3D	Equal Sign
?	%3F	Question Mark
@	%40	At Sign
[]	%5D	Square Bracket

- ⁽¹⁾ Colon character (:) is not supported as part of the "Device Name" as a colon is often used to separate individual parts of an API request command. For example when managing the frame rate divider of the native stream (stream 1) a colon (:) is used in property key.

```
streams[stream_type:0].configuration.source.value
```

Looking at the earlier example device name, `Peter:Pan` would not be allowed. When an unsupported character is detected, the set name command will fail with an `ILLEGAL_ARGUMENT` error containing the message: Device name contains unsupported character.

API Warning Logged

If there are multiple MAC addresses mapped to the same Device Name, the API logs a warning. For example, if `Peter Pan` was assigned to two devices with the MAC addresses of `d8803962e132` and `d88039e7a4ee`, the API would log the warning:

```
WARNING: Device name 'Peter Pan' is mapped to multiple mac
addresses: d8803962e132 d88039e7a4ee
```

Generally a "Device Name" is mapped to a single device, that is to a single device ID (MAC address). If more than one is mapped, the first one registered is used by default.

For the illustrated example shown above where two endpoint devices have mac addresses (1) `d8803962e132` and (2) `d88039e7a4ee`, respectively. If both are named "Peter Pan", when an API request is sent to "Peter Pan", it will be sent only to mac address (1) `d8803962e132`.

Device Name Errors

When a device is mapped to a unique name, should that device go offline the device is reported as `disconnected` and the device name is displayed.

For example, if a device name request was sent to change the name of device ID `d8803962e132` to `Peter Pan`, the device name change will fail with the following message:

```
"error": [
  {
    "device_id": "d8803962e132",
    "device_name": "PETER PAN",
    "reason": "DEVICE_DISCONNECTED",
    "message": "Device is disconnected from the
network"
  }
]
```

When a device is in a disconnected state, the device name is kept.

When a new device connects to the API server and this device is configured with the same name as a disconnected device, the name of the disconnected device is set to `NULL`. This allows for the use case where a device is replaced in the network. The new device uses the device name in place of the disconnected device.

When the name of a currently connected device is changed to the same name as a disconnected device, the name of the disconnected device is set to `NULL`, allowing for a device to be replaced in the network.

8.2.1 Resetting Device Name to MAC Address

If it is desired to reset an endpoint device name to its MAC address this can be done using an API `set property` request with the following arguments:

```
Key:      configuration.device_name
Value:    mac_address
```

When an endpoint device(s) receives a `mac_address` request, it overwrites the current device name with its MAC address for AVP devices and with its MAC address and prefix `USB_` for UEX devices.

The `target` argument can be either the device ID of a single device or the name of a device group.

The allowed device groups are:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All receiver devices
<code>ALL_TX</code>	All transmitter devices

REST Syntax:

```
POST /api/device/<target>
{
  "op" : "set:property",
  "key" : "configuration.device_name",
  "value" : "mac_address"
}
```


For example to reset the endpoint device 801f124307b7 to its MAC address the API request is:

```
POST /api/device/801f124307b7

{
  "op" : "set:property",
  "key" : "configuration.device_name",
  "value" : "mac_address"
}
```

8.3 Managing Frame Rate Divider

Applying a frame rate divider to either the native stream (`stream 0`) and/or scaled stream (`stream 1`) is a useful tool when managing bandwidth. A `set property` request is used to apply the frame rate converter to either or both HDMI input streams of an encoder (TX) device. Each input stream is managed independently and have different property nodes and keys. The figure below illustrates the paths for each the native and scaled streams of an encoder.

Reminder: The frame rate divider is available to devices that support the AV processing engine, specifically devices based on the AVP2000, AVP2000T and NT2000 chipset series.

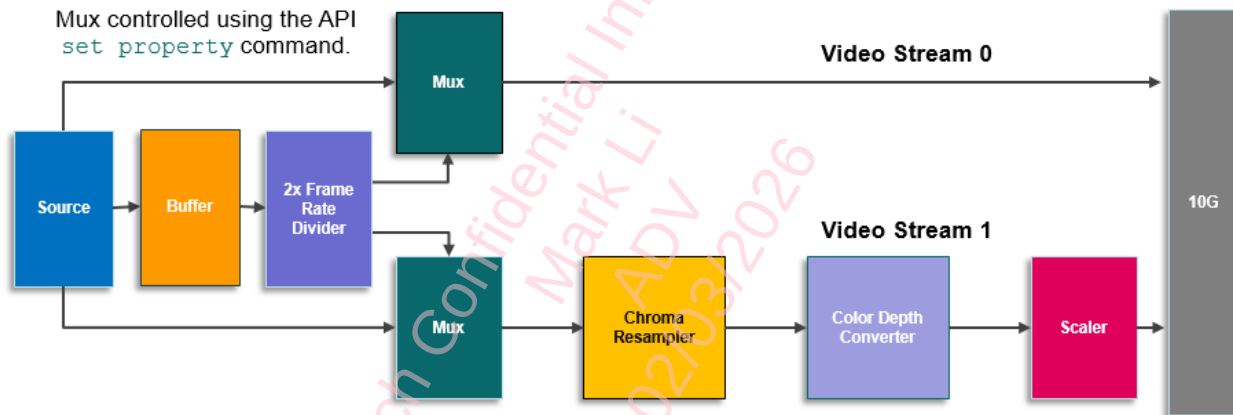


Figure 65: Block Diagram of Encoder (TX) Illustrating Frame Division

8.3.1 Enabling Frame Rate Divider on Native Video Stream

In the case of the native stream (`stream 0`), the `set property` request uses the property key `streams[HDMI:0].configuration.source.value`, with 0 entered to disable the feature and 1 to enable it.

The example shown here is applied to an individual encoder (TX) with the device ID of `d8803962e132` as the target and the value 1 was entered to enable the frame rate divider.

TIP: It is possible to apply the frame divider to all encoder devices simultaneously using the `ALL_TX` device group.

The command example shown returned a `PROCESSING` status and was assigned the request ID 9.

```
POST /api/device/801f124307b7

{
  "op" : "set:property",
  "key" : "streams[HDMI:0].configuration.source.value",
  "value" : 1
}
```

```

}

{
  "status" : "PROCESSING",
  "request_id" : 9,
  "result" : null,
  "error" : null
}

```

A `SETTINGS_CHANGED` event is received from the API to indicate that the first devices settings have been modified. Also, a `REQUEST_COMPLETED` event is generated once the request is complete. If other API clients are connected to the API server, monitoring `SETTINGS_CHANGED` events allows clients to stay up to date with device settings.

```

{
  "status": "SUCCESS",
  "request_id": null,
  "result": {
    "events": [
      {
        "device_id": "801f124389e7",
        "event_id": 1,
        "event_type": "DEVICE_CONNECTED",
        "timestamp": 1610735915,
        "request_id": 1
      },
      {
        "device_id": "801f124389e7",
        "event_id": 2,
        "event_type": "DEVICE_INFO_AVAILABLE",
        "timestamp": 1610735915,
        "request_id": 2
      },
      {
        "device_id": "801f124307b7",
        "event_id": 3,
        "event_type": "DEVICE_CONNECTED",
        "timestamp": 1610735916,
        "request_id": 3
      },
      {
        "device_id": "801f124307b7",
        "event_id": 4,
        "event_type": "DEVICE_INFO_AVAILABLE",
        "timestamp": 1610735916,
        "request_id": 4
      },
      {
        "device_id": "801f124307b7",
        "event_id": 7,
        "event_type": "SETTINGS_CHANGED",
        "timestamp": 1610736876,

```

```

        "request_id": 8
      },
      {
        "device_id": null,
        "event_id": 9,
        "event_type": "REQUEST_COMPLETE",
        "timestamp": 1610737063,
        "request_id": 9
      }
    ]
  },
  "error": null
}

```

TIP: If a device does not support frame rate conversion, there is an entry in the error array of the result with a reason of `DEVICE_TYPE`, indicating that the request failed for the device because it does not support this feature. For example, the device does not include a chipset such as an AVP2000T or NT2000 supporting the feature or the device ID entered was a decoder.

8.3.2 Disabling Frame Rate Divider on Native Video Stream

Frame rate conversion is also disabled using the `set property` command. For our earlier example, using the device ID of `801f124307b7`, the command issued is similar but the value entered is now “0” to disable/turn off the frame rate divider.

```

POST /api/device/801f124307b7
{
  "op" : "set:property",
  "key" : "streams[HDMI:0].configuration.source.value",
  "value" : 0
}

```

8.3.3 Enabling/Disabling Frame Rate Divider on Scaled Video Stream

Similarly, the `set property` command is used to select the frame rate converted video as the source of the video scaler. Enter the value 0 to enable or 1 to disable the frame rate divider for the scaled stream (stream 1).

There is currently only one scaler node supported, therefore the `SCALER:index` is always 0.

Example of an API request to enable the frame rate divider for the scaled stream (stream 1) of the decoder `801f124307b7`.

```

POST /api/device/801f124307b7
{
  "op" : "set:property",
  "key" :
    "nodes[SCALER:0].inputs[main:0].configuration.source.value"
  ,
  "value" : 0
}

```

```

{
  "status" : "PROCESSING",
  "request_id" : 39,
  "result" : null,
  "error" : null
}

{
  "status" : "NOTIFICATION",
  "request_id" : null,
  "result" : {
    "events" : [
      {
        "device_id" : null,
        "event_id" : 40,
        "event_type" : "REQUEST_COMPLETE",
        "timestamp" : 1489090702,
        "request_id" : 39
      }
    ]
  },
  "error" : null
}

```

Example of the result, retrieved with the request command and request_id 9, showing the changes have been applied.

```

request 39
{
  "status" : "SUCCESS",
  "request_id" : 39,
  "result" : {
    "devices" : [
      {
        "device_id" : "d8803962e132",
        "identity" : {
          "engine" : "PLETHORA",
          "firmware_version" : "3.2.1",
          "firmware_rc" : "6",
          "is_receiver" : false,
          "is_transmitter" : true
        },
        "configuration" : {
          "device_name" : " d8803962e132"
        },
        "status" : {
          "active" : true,
          "point_to_point" : false
        },
        "streams" : [
          (...)
        ],
        "subscriptions" : [
          (...)
        ],
      }
    ]
  }
}

```

```

        "nodes" : [
            (...)
            {
                "type" : "SCALER",
                "index" : 0,
                "configuration" : {
                    "width" : 0,
                    "height" : 0
                },
                "status" : {
                    "video" : {
                        "width" : 0,
                        "height" : 0,
                        "frames_per_second" : 0,
                        "color_space" : "RGB",
                        "bits_per_pixel" : 8,
                        "scan_mode" : "PROGRESSIVE"
                    }
                },
                "inputs" : [
                    {
                        "name" : "main",
                        "index" : 0,
                        "configuration" : {
                            "source" : {
                                "value" : 1
                            }
                        },
                        "status" : {
                            "source" : {
                                "ref_class" : "NODE",
                                "ref_type" : "FRAME_RATE_CONVERTER",
                                "ref_index" : 0
                            }
                        }
                    }
                ]
            },
            (...)
        ],
        "error" : []
    },
    "error" : null
}

```

8.4 Compression of HDMI:0 Always On

When lowering the bandwidth usage is of concern, it is possible to compress the native stream (stream 0) of an encoder(s), even if the bandwidth of the stream does not surpass 10Gbps. A `set` property request is used to enable compression.

The API request is as follows:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : "
    nodes[VIDEO_COMPRESSOR:index].configuration.always_compress",
  "value" : true|false
}
```

The `target` argument is the device ID of a single encoder (TX) device or the all transmitters device group. The supported device groups are:

ALL	All devices
ALL_TX	All transmitter devices

The key for this property is

"nodes[VIDEO_COMPRESSOR:index].configuration.always_compress", where the `index` is always 0 as this property is applicable to the native stream only (stream 0). The `value` is either `true` or `false`, where `true` enables the feature and `false` disables it.

Example of sending a request to enable HDMI compression for the encoder (TX) 801f124307b7:

```
POST /api/device/801f124307b7
{
  "op" : "set:property",
  "key" : "
    nodes[VIDEO_COMPRESSOR:index].configuration.always_compress",
  "value" : true
}
```

For more information on the video compressor node, refer to the SDVoE Developers API Reference Guide, specifically the `set` property and Video Compressor Node Configuration sections.

8.4.1 Verifying Status & Video Compression Version

A `get device` request can be used to retrieve details of the video compressor node, including the compression version applied and status in regard to if the `always compress` option is enabled.

The node example shown below has been retrieved from an AVP2000T transceiver (TR) device. Note that the `version_support` indicates it is set to version 2, while the `always_compress` option indicates it is not currently enabled as it is reported as `false`. If this option was enabled then the status would be set to `true`.

```

{
    type: VIDEO_COMPRESSOR,
    index: 0,
    configuration: {
        always_compress: false,
        version: 1
    },
    status: {
        version_support: 2
    },
    inputs: []
},

```

8.5 Removing Audio from HDMI Video Stream

By default, the original HDMI audio is carried within the native HDMI video stream (`stream 0`) coming from an encoder (TX) as auxiliary data. However, only a decoder (RX) running in Genlock mode can make use of this auxiliary data. When a decoder is operating in any other video display mode, the HDMI source audio is received twice. Once as the auxiliary audio embedded in the HDMI video stream (`stream 0`) and again when a decoder (RX) is subscribed/joined separately to the HDMI audio stream.

The benefit of enabling this option is removing the auxiliary audio data from the HDMI video stream saves bandwidth usage. Estimated bandwidth savings is ~200 Mbps in the case of eight LPCM channels at 192 kHz. Saving such bandwidth can be crucial for certain use cases, such as multiview layouts using the native source video, USB or 1GbE heavy applications, even for AV distribution applications living on either a 2.5 Gbps or 5 Gbps network (NBASE-T) infrastructure.

However should be noted that when this property is applied decoders (RX) operating in any video mode, including Genlock, need to subscribe/join to the HDMI audio stream for playout as the source audio.

The API request for this `set property` option is as follows:

```

POST /api/device/{target}

{
    "op" : "set:property",
    "key" : "streams[HDMI:index].configuration.remove_audio",
    "value" : true|false
}

```

The `target` argument must be the device ID of a single encoder (TX) device or the all transmitters devices group. The supported device groups are:

ALL	All devices
ALL_TX	All transmitter devices

The `key` argument identifying the configurable property is `"streams[HDMI:index].configuration.remove_audio"` where the `index` is always 0 as this property is applicable to the native stream only (`stream 0`). The value is either `true` or `false`, where `true` enables the feature and `false` disables it.

For more information regarding feature, refer to the SDVoE Developers API Reference Guide, the `set property` command, and Configurable Device Properties > Stream Configuration section.

8.6 Black Video Generator

For applications where the decoder (RX) is to drive audio only at the HDMI output, it is possible to tie the audio to a black video output. When an endpoint device includes the AV Processor Engine, such as an NT2000 or AVP2000(T), this can be done by setting the output display mode to `fast_switch` mode and subscribing the decoder to the appropriate audio source.

However, if the endpoint device includes an NT1000 or AVP1000 chipset, it does not include the AV Processor Engine and so a black output stream is generated by enabling the `Black Video Generator` property. This feature allows a dummy black video signal to be generated, it enables the selected audio stream to be embedded into a dummy stream for HDMI playout. For example, audio player functionality or for transitions/temporary audio transitions.

The black generator supports two modes:

- Maintain Output
- Force Output

When `set_property key nodes[COLOR_GENERATOR:0].configuration.force_output` is specified with the value set as `true`, then a blank video output with a uniform color and a fixed resolution video signal of 720P60 is output.

The API request to enable the forced output is as follows:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" :
    "nodes[COLOR_GENERATOR:0].configuration.force_output",
  "value" : true
}
```

When the black generator is disabled, set to `false`, value equal to `false`, the device returns to the default video display mode, Genlock.

Reminder: This feature supported only for devices that do not support the AV Processor Engine, such as AVP1000 based devices.

The `target` argument is either the device ID of a single decoder (RX) or the receiver device group. The supported device groups are:

ALL	All devices
ALL_RX	All receiver devices

For more information on color generator node, refer to the SDVoE Developers API Reference Guide, the `set_property` command, Color Generator Node (Type `COLOR_GENERATOR` and Color Generator Node Configuration sections are relevant for this property.

8.7 HDCP Encryption

On decoder (RX) devices operating in fast switch or multiview mode changing from one HDCP encryption level to another, causes either a glitch in the sink or a short black period during the authentication of the HDCP keys.

To limit such glitches and black periods, it is best to authenticate at the highest HDCP level supported by the sink. For example, if the sink supports HDCP 2.2 and the authentication with the sink is done with HDCP 2.2 then switching between NO HDCP and/or HDCP 1.4 sources to a source with HDCP 2.2 does not cause a glitch. If the sink only supports HDCP 1.4, it is still best to output HDCP 1.4 when possible to prevent glitches when switching from a source with NO HDCP to a source with HDCP 1.4. Note that in such a case, sources with HDCP 2.2 will fail the authentication with the sink and black is displayed on the sink.

The HDCP output modes outlined in this section are only applicable to decoders (RX) operating in one of the following video processing modes:

Fast Switch
Multiview
Fast Switch Wall

There are three (3) HDCP output modes supported:

- Follow Sink 1

If the monitor supports HDCP 2.2, this mode tries to negotiate HDCP 2.2. If HDCP 2.2 authentication fails, it automatically tries HDCP 1.4 or NO HDCP based on the sink capabilities.

This mode is the default HDCP encryption mode of operation and covers most use cases.

- Follow Sink 2

This mode is similar to the “Follow Sink 1” outlined above but limits the HDCP level requirement to HDCP 1.4 when possible.

It is applicable in special cases when there is equipment located between the decoder and an output display, such as an AVR that does not expose the HDCP capability of the display. For example, the AVR is HDCP 2.2 capable but is connected to a display capable only of HDCP 1.4.

- Follow Source

In this mode, the output encryption follows the HDCP mode of the source.

In all HDCP output modes, the HDCP encryption level at the output cannot be lower than the HDCP encryption level of the source (which in multiview is the highest HDCP encryption level out of all current subscriptions). If it surpasses the maximum capability of the sink, the video output is black.

Note that the HDCP output mode selected with the API command is available through the I²C registers for the External MCU to properly manage the external HDMI chip output level encryption.

Command usage:

```
POST /api/device/{target}

{
    "op" : "set:property",
    "key" :
        "nodes[HDMI_ENCODER:0].configuration.hdcp_output_mode",
    "value" : ("FOLLOW_SINK_1"|"FOLLOW_SINK_2"|"FOLLOW_SOURCE")
}
```

In regard to the HDCP output mode, the target argument is either the device ID of a single device or the name of a device group. The supported device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

The `key` argument identifies a configurable property of a device and the `value` argument is a value of a type appropriate for that property. For this property, the applicable `key` argument is `"nodes[HDMI_ENCODER:0].configuration.hdcp_output_mode"`. For `value`, the options (described above) are `FOLLOW_SINK_1`, `FOLLOW_SINK_2` and `FOLLOW_SOURCE`.

For details on the HDMI Encoder Node refer to the `set property` command and the HDMI Encoder Node (Type `HDMI_ENCODER`) sections in the SDVoE Developers API Reference Guide.

8.8 BlueRiver Receiver (HDMI TX 5V) API Control

8.8.1 Overview of HDMI TX 5V Control

The SDVoE API supports controlling the configuration of the HDMI TX 5V output mode of the `HDMI_ENCODER` node.

Reminder: The HDMI transceiver IC acts as a transmitter on a BlueRiver decoder (RX) endpoint device; therefore, this command is applicable only to BlueRiver decoder (RX) and transceiver (TR) devices but not to BlueRiver encoder (TX) devices.

The status of the `hdmi_tx_5v` is provided in the `HDMI_ENCODER` configuration node:

```
"type": "HDMI_ENCODER",
"index": 0,
"configuration": {
    "hdcp_output_mode": "FOLLOW_SINK_1",
    "hdmi_tx_5v": <API control value>
},
```

The `hdmi_tx_5v` parameter will be either `AUTO` or `LOW`.

`AUTO` indicates the argument is currently set to operate in "normal" mode, meaning:

- In `switch` mode, the HDMI TX 5V output is always `HIGH`.
- In `loopback` mode, it follows 5V loopback mode rules, refer to section 8.8.2 HDMI TX 5V Mirroring below for details.
- In `P2P` mode, it follows 5V mirroring rules, refer to section 8.8.2 HDMI TX 5V Mirroring below for details.

LOW indicates the argument is set to forced remain LOW, overriding any other mode. Therefore, regardless of the operating mode (switch, P2P or loopback), when the configuration is set to LOW the HDMI TX 5V is always LOW.

An API set property request is used to modify the HDMI TX 5V output mode as following:

```
POST /api/device/{target}
{
  "op" : "set:property",
  "key" : "nodes[HDMI_ENCODER:0].configuration.hdmi_tx_5v",
  "value" : "AUTO|LOW"
}
```

The default value is AUTO. Note also that this set property option is volatile and is reset on reboot, power up and factory default.

8.8.2 HDMI TX 5V Mirroring

HDMI TX 5V mirroring applies only when the HDMI_TX_5V property is set to AUTO.

When the decoder (RX) is operating in a video processing mode (Fast Switch, Fast Switch Wall or Multiview), the HDMI TX 5V output is always high. This is regardless of the network link state or 10GbE connection type (switch or point-to-point).

When the decoder (RX) is not operating in a video processing mode, so is operating in Genlock, Genlock Wall or Genlock Scaling:

- When operating in loopback mode (applicable to transceiver (TR) devices only), the HDMI TX 5V output follows the input 5V state from HDMI RX side (i.e. video source HDMI 5V output value).
- When not in loopback:
 - If the 10GbE interface is down, the HDMI TX 5V output is set low (same as if the HDMI cable was disconnected).
 - If connected to a switch, when the 10GbE interface link goes up, the HDMI TX 5V output is set high.
 - If connected in point-to-point, the HDMI TX 5V output follows the point-to-point remote HDMI RX 5V state (i.e. remote video source HDMI 5V output value). However, if the remote P2P device does not support this feature (i.e. older firmware version), the HDMI TX 5V output value is set to high for legacy compatibility.

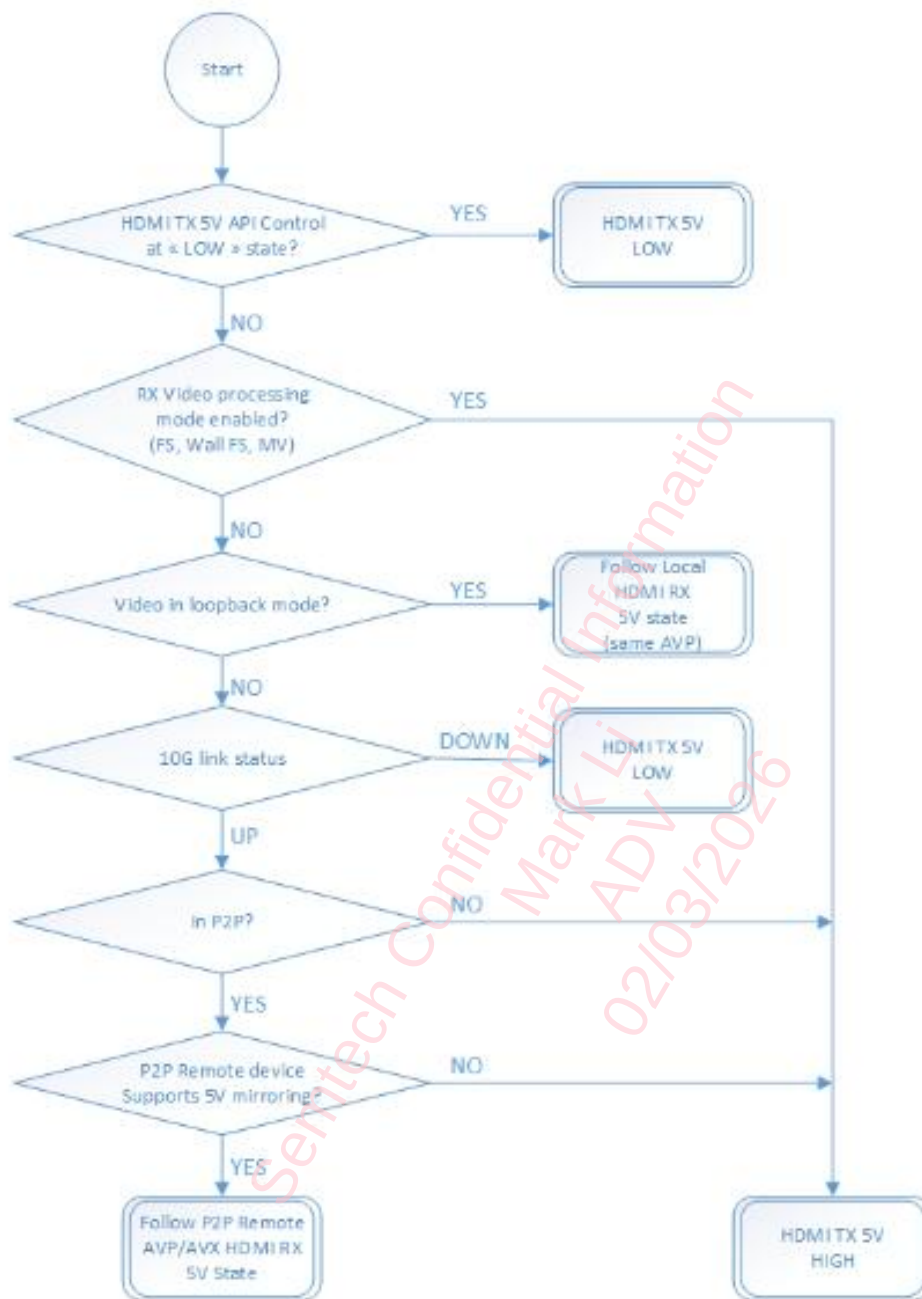


Figure 66: Flow Chart of HDMI TX 5V Behavior

9 BlueRiver Device Audio Routing

Audio routing is supported by BlueRiver chipsets in regard to HDMI, Stereo and Multichannel audio.

Note some settings and values are different for AVP2000T transceiver (TR) devices than for encoder (TX) and decoder (RX) devices. The system filters out settings that are not applicable based on the device and chipset type. Refer to the “Properties” section of the SDVoE Developers API Reference Guide for details regarding audio nodes.

The diagram below illustrates how audio inputs can be routed to the audio outputs on a single device via a number of muxes. Each audio output has its own mux, and a `set property` command is used to set the audio source on the mux.

Audio inputs are divided into two categories:

- Local inputs (connectors on the BlueRiver device).
- Network inputs (received through the 10GbE interface).

Network inputs are in fact remote, which means they are local to another device. When the audio input selected is a network input, a `join` request is required to subscribe to the network data. The `set property` request sets the path from the network input to the audio output, while the `join` request feeds the network input with data from the network.

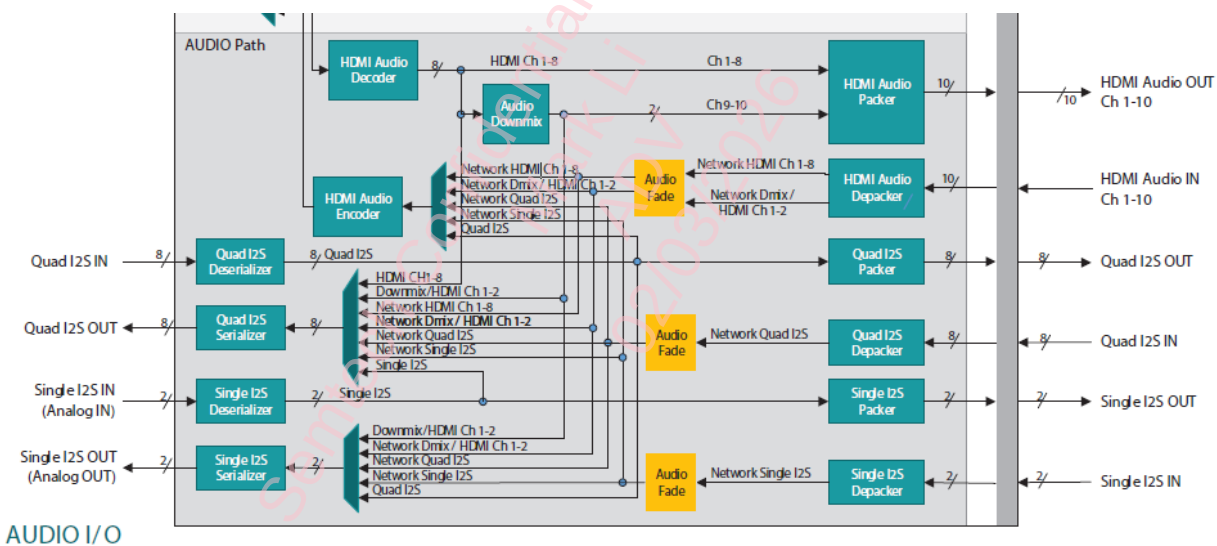


Figure 67: Available Audio Routing for AVP2000T

Reminder that when managing audio routing one needs to consider the hardware settings for the device being managed as well as the audio type to be routed. The table below summarizes the supported audio routing for BlueRiver ASIC devices, referencing the audio indexes for encoders (TX), decoders (RX) and transceivers (TR). A black cell indicates the audio input in the left-most column is not available for the that device type, therefore routing is not possible. For example, the local HDMI audio input cannot be routed to the stereo output on a decoder (RX) device. This is because a decoder (RX) does not have a local HDMI audio input.

Note: It is possible to verify the capabilities of a device using a “`get capabilities`” request.

Table 12: Summary of Audio Output Routing

Summary of Audio Output								
Audio Source	Transmitter (TX)		Receiver (RX)			Transceiver (TR)		
	Stereo	Multi-channel	Stereo	Multi-channel	HDMI	Stereo	Multi-channel	HDMI
Net HDMI aux					2			2
Net HDMI		14		7	7		7	7
Net HDMI Downmix	15	15	2	6	6	2	6	6
Net Stereo	13	13	3	3	3	3	3	3
Net Multichannel	8	8	8	8	8	8	8	8
Local HDMI		7					11	
Local HDMI Downmix	2	6				12	12	
Local Stereo		3		10			10	
Local Multichannel	9		9		9	9		9

9.1 Audio Routing Use Cases

In this section we review some specific use cases regarding the routing of audio and the appropriate API requests required to successfully output audio.

Notes:

- The presence of device interfaces is managed through the factory settings applied by the hardware manufacturer.
- Reminder, as mentioned above, it is possible to verify the capabilities of a device using a “get capabilities” request.
- Refer to the “Command `join`” section in the SDVoE Developers API Reference Guide for full details on the use of the `join` command regarding managing video and audio subscriptions.
- Similarly, refer to the reference guide for details on using the “Command `set property`”.

9.1.1 Routing Audio to HDMI Output

This use case applies to decoder (RX) and transceiver (TR) devices. As per Table 12: Summary of Audio Output Routing, the allowed routings for both decoders and transceivers are shown in the `HDMI` column. Note that all network audio sources can be routed to the HDMI OUT but the only local source that can be routed to the HDMI OUT is the multichannel input.

- The first step is to select the desired source using a `set property` request.
- If the source is a Network source, then a second step is required to subscribe the decoder (RX) to the network stream. A `join` request is used to create this subscription. If the source is local (multichannel) then no `join` command is required. Possible network sources are HDMI audio, Stereo and Multichannel.

Reminder: Refer to the SDVoE Developers API Reference Guide for details on the `join` command.

TIP: For the `join` request to succeed, the `source` stream must currently be started/running. Refer to the SDVoE Developers API Reference Guide, section “Command `start`”, for details on starting device streams.

9.1.1.1 HDMI Audio Routing Use Case

As mentioned above to route audio to the HDMI OUT, it is necessary to potentially complete two steps:

1. Specify the audio source as per Table 12: Summary of Audio Output Routing using a `set property` command.
2. In the case of a network HDMI audio source, subscribe to the `HDMI_AUDIO` source stream using a `join` request.

For the use case example illustrated below, the decoder (RX) device `0016c04e05fd` is being subscribed to the Network HDMI audio. The HDMI audio stream (`HDMI_AUDIO`) is being taken from the encoder (TX) device `0016c04e1c0c`.

Note: The subscription index for HDMI audio is 0.

1. For this use case the first step to complete is to use a `set property` request to set the mux of the HDMI audio output.

Reminder: For details on the `set property` command refer to the SDVoE Developers API Reference Guide, section `Command set property`.

The key to be used for the `set property` command for this use case is:

```
nodes[HDMI_ENCODER:index].inputs[audio:0].configuration.source.value.
```

An encoder node may support none, some, or all of the following values:

- 2 – HDMI audio (original audio from video subscription in genlock mode)
- 3 – Stereo audio
- 6 – HDMI audio (stereo downmix)
- 7 – HDMI audio (all available channels)
- 8 – Multichannel audio subscription
- 9 – Multichannel audio local loop out

So based on the above list since this use case is route the HDMI audio, the value to specify in the API request is the HDMI audio which is 7. The `HDMI_ENCODER` and `audio` indexes are both 0.

The syntax for the API request for this use case is therefore:

```
POST /api/device/0016c04e05fd
{
  "op" : "set:property",
  "key" :
    "nodes[HDMI_ENCODER:0].inputs[audio:0].configuration.
    source.value",
  "value" : 7
}
```

2. Since for this use case we are not using the local multichannel audio but a network input, the second step is required. That is a `join` request to subscribe the decoder (RX) to the HDMI audio stream.

The `join` request for this use case would be:

```
POST /api/device/0016c04e05fd
{
  "op" : "join",
  "source_device" : 0016c04e1c0c,
  "stream_type" : "HDMI_AUDIO",
  "stream_index" : 0,
  "subscription_index" : 0,
}
```

Reminder: For details on sending a `join` request refer to the SDVoE Developers API Reference Guide, section `Command join`.

9.1.2 Routing Audio to Stereo Output

This second use case applies to BlueRiver devices that are equipped with a Stereo Audio Output or Stereo Audio Input/Output interface.

As outlined in Table 12: Summary of Audio Output Routing the supported routings are shown in the `STEREO` column for each of the BlueRiver device types (transmitter/receiver/transceiver) and the value of the supported network streams is indicated. Note, that a black cell indicates that the audio source is not supported. Note that HDMI can be routed to the Stereo OUT only if the device supports the Downmix capability and that the only local source that can be routed to the Stereo OUT is multichannel.

Reminder: It is possible to verify device capabilities device using a “`get capabilities`” request.

Refer to the SDVoE Developers API Reference Guide, section `Stereo Audio Input/Output Node Configuration` for details on supported audio input options. Specifically, refer to the table “`Outputs Supported for the Audio Output Node`”.

To successfully route audio to Stereo OUT, it is necessary to potentially complete three steps:

1. Set direction through a `set property` request to `output`.

TIP: If the `OUTPUT` node is not listed in the return of `get device` request under `capabilities`, it is likely because the `STEREO` direction is currently set to `INPUT`.

Note: If you have questions about the hardware design capabilities of the endpoint device itself, contact the hardware manufacturer for details on the interfaces included in the design, as well as the default hardware settings applied.

-
2. Specify the audio source as per Table 12: Summary of Audio Output Routing. A `set` property request is required to specify the desired source.

For a receiver device none, some or all of the following may be supported:

- 2 – HDMI audio subscription (stereo downmix)
- 3 – Stereo audio subscription
- 8 – Multichannel audio subscription
- 9 – Local Multichannel audio

Note: Refer to the SDVoE Developers API Reference Guide section Stereo Audio Output Node (Type `STEREO_AUDIO_OUTPUT`) for details on the node.

3. If the source is network HDMI audio, an additional action is required, that is to send a `join` request to subscribe to the network stream.

Possible audio network sources are:

<code>STEREO_AUDIO</code>	Stereo audio stream
<code>HDMI_AUDIO</code>	HDMI audio stream
<code>MULTICH_AUDIO</code>	Multichannel audio stream

Notes:

- i. Reminder, if the source is local (multichannel) then a `join` request is not required.
- ii. Refer to the SDVoE Developers API Reference Guide for details on the `join` command.

9.1.2.1 Stereo Audio Output Use Case

Based on the steps outlined above, a use case will be illustrated to configure the Stereo Audio Output. This use case uses the Device ID `0016c04e1c0c` for the encoder (TX) source device and `0016c04e05fd` for the decoder (RX) device.

It is assumed that the `STEREO` port is currently set as an `INPUT`, therefore it is required to first set it as an `OUTPUT`. The second step shown specifies the audio type to be output from the decoder (RX). For this use case the `Net HDMI Downmix` will be used, so the value to specify is 2. The third step is to subscribe (`join`) the receiver to the network stream to output the audio.

Reminder: Although the default settings for BlueRiver hardware interfaces are set by the hardware manufacturer, the direction of the Stereo audio port can be modified. This is done using an API `set` property request with the property key:

```
nodes[STEREO_AUDIO_INPUT_OUTPUT:index].configuration.direction
```

The value for the direction is specified as either `INPUT` to set the stereo audio port as an input or `OUTPUT` to set it as an output.

So to summarize up to three API requests are required to send Net HDMI Downmix audio to the Stereo port. For this use case the API requests are as follows:

1. Set the Stereo port as an OUTPUT on the decoder (RX).

```
POST /api/device/0016c04e05fd
```

```
{
  "op": "set:property",
  "key":
    "nodes[STEREO_AUDIO_INPUT_OUTPUT:index].configuration.direction",
  "value" : "OUTPUT"
}
```

Reminder: Presence of device interfaces are managed by the hardware manufacturer.

2. Specify the audio source to be sent to the Stereo Output port.

For this use since the Net HDMI Downmix audio of the encoder (TX) is to be used, the value to specify is 2. Both the STEREO_AUDIO_OUTPUT index and the inputs[main] index are set to 0.

Therefore, the API syntax for this use case is:

```
POST /api/device/0016c04e05fd
```

```
{
  "op" : "set:property",
  "key" :
    "nodes[STEREO_AUDIO_OUTPUT:0].inputs[main:0].configuration.source.value",
  "value" : 2
}
```

3. For this use case as the source is network HDMI audio, it is necessary to complete an API join request.

Specifically, we will subscribe the decoder (RX) device 0016c04e05fd to the HDMI_AUDIO of the encoder (TX) device 0016c04e1c0c. The stream_index is being set to 0, as is the subscription_index.

The API request for this use case therefore would be as follows:

```
POST /api/device/0016c04e05fd
```

```
{
  "op" : "join",
  "source_device" : 0016c04e1c0c,
  "stream_type" : "HDMI_AUDIO",
  "stream_index" : 0,
  "subscription_index" : 0,
}
```

Note: For details on the join command, refer to the SDVoE Developers API Reference Guide.

9.1.3 Routing Audio to Multichannel Output

This use case applies to BlueRiver devices that are equipped with a Multichannel Audio Output interface. As outlined in Table 12: Summary of Audio Output Routing allowed routings are shown in the `MULTICHANNEL` column for each of the device types (transmitter/receiver/transceiver), the supported network sources that can be routed to the Multichannel OUT are indicated.

Note that the only local source that can be routed to the Multichannel OUT of a decoder (RX) is `Stereo`. On an encoder (TX) or transceiver (TR) device the local HDMI audio can also be routed.

Reminder: It is possible to verify device capabilities device using a “`get capabilities`” request. For details on the hardware design and configuration of your device(s) contact the hardware manufacturer.

There are potentially two steps required to output audio through the Multichannel Output.

1. A `set property` request is required to specify the desired audio source.

Options for the audio output routing vary depending on the device type, refer to Table 12: Summary of Audio Output Routing for full list. In regard to a decoder (RX) the following are the available audio sources

A decoder (RX) may support none, some, or all of the following:

- 3 – Stereo audio subscription
- 6 – HDMI audio subscription (stereo downmix)
- 7 – HDMI audio subscription
- 8 – Multichannel audio subscription
- 10 – Local Stereo audio

Note: Not all audio types are available to all endpoint devices, supported audio is based on chipset type and hardware design.

Refer to the SDVoE Developers API Reference Guide, section Multichannel Audio Output Node Configuration for more information on this node.

2. If the source is a Network stream, then a `join` request is required to subscribe to the network stream. Possible network sources are `HDMI_AUDIO`, `STEREO_AUDIO` or `MULTICH_AUDIO`. If the source is local stream (Stereo or local HDMI audio) then a `join` request is not required.

Note: Refer to the SDVoE Developers API Reference Guide for details on the `join` command.

9.1.3.1 Multichannel Output Use Case

Based on steps outlined above, a use case will be illustrated to configure the Multichannel Audio Output. This use case uses the Device ID `0016c04e1c0c` for the encoder (TX) as the source device and `0016c04e05fd` for the decoder (RX) device that will send the audio out on its multichannel audio output interface.

Example of the API requests for this use case to send output audio through the Decoder Multichannel audio port are as follows:

1. As mentioned above, the first step is to specify the audio source to output from the decoder (RX) multichannel output port.

The property key to set the source of the multichannel output node is:

```
nodes[MULTICH_AUDIO_OUTPUT:index].inputs[main:0].configuration.source.value
```

For this use case the Network HDMI source of an encoder (TX) is to be used, so the value to specify is 7.

Therefore the command syntax for our example request is:

```
POST /api/device/0016c04e05fd
```

```
{
  "op" : "set:property",
  "key" : "
nodes[MULTICH_AUDIO_OUTPUT:0].inputs[main:0].configuration
.source.value",
  "value" : 7
}
```

2. For this use case since the source needs to be fed by a network stream the second step is required to subscribe (join) the receiver to the appropriate audio source to successfully output the audio.

Reminder: This step is not required for a local Stereo source.

Specifically, we need to subscribe (join) the decoder (RX) 0016c04e05fd to the HDMI_AUDIO of the encoder (TX) 0016c04e1c0c.

For this use case the API request would be:

```
POST /api/device/0016c04e05fd
{
  "op" : "join",
  "source_device" : 0016c04e1c0c,
  "stream_type" : "HDMI_AUDIO",
  "stream_index" : 0,
  "subscription_index" : 0,
}
```

Reminder: Refer to the SDVoE Developers API Reference Guide for details on the join command.

10 Sending Wake-on-LAN Packet via BlueRiver Devices

This section summarizes the use of an API command to send a Wake-on-Lan (WoL) packet to an AVP device, for the AVP device to relay it on the network (acts as a WoL proxy) by broadcasting the message to the local subnet to wake a powered-off target machine.

10.1 Wake-on-LAN Standard Overview

The Wake-On-LAN standard is an Ethernet de facto standard that allows computers to be turned on or awakened when receiving a specific Ethernet message, called the “magic packet”. The target computer requires its network adapter to be in standby mode (consuming minimal power) in order to detect the WoL message. The target machine will power-up upon detection of the WoL message, as if the power button had been pressed.

10.1.1 How to Enable Wake-On-LAN on Your Computer

To enable the Wake-On-LAN on a Windows 10 computer, you need to enable the feature both in the BIOS and in the Network Adapter settings.

Each manufacturer has a different way to enable the Wake-On-LAN feature in its BIOS. Once you entered the BIOS, locate the Wake-On LAN setting (e.g. for a Lenovo Thinkpad it is in Config/Network/Wake On LAN) and set the appropriate option to enable it (e.g. for a Lenovo Thinkpad you should set either “AC ONLY” or “AC and battery” to enable WoL when computer is plugged-in and optionally when on battery). Then save the changes and exit the BIOS.

Run “Device Manager” and double-click on the specific Network Adapter. In the “Power Management” tab, select the “Allow this device to wake the computer” property and click OK. In the “Advanced” tab, select the “Wake on Magic Packet” property, make sure it is set to “Enabled” and click OK.

10.1.2 Magic Packet

The target machine should react when receiving the “magic packet” anywhere within the Ethernet packet payload (the magic packet doesn’t need to be present at a specific byte offset in the message). The message consists of 6 bytes set to 0xFF, followed by 16 repetitions of the target’s MAC address (also 6 bytes), for a total of $17 \times 6 = 102$ bytes. The target machine will awake only when the MAC address in the Magic Packet matches its MAC address.

10.1.3 SecureOn™ Password

Some computers support a 4-byte or 6-byte password called “SecureOn”, which can be set in the BIOS. This optional password is appended immediately following the “magic packet”. If this option is supported by the target machine, then the password must match in order for the machine to awake.

10.1.4 WoL Transport

Since the magic packet is only scanned for the string above, and not actually parsed by a full protocol stack, it could be sent as payload of any network and transport-layer protocol, although it is typically sent as a UDP datagram to port 0, 7 or 9, or directly over Ethernet using EtherType 0x0842 (but EtherType 0x0842 is not registered with IANA as of August 2023). When UDP is used for transporting the WoL messages, they are typically broadcasted to the local subnet.

Below is an example of a Wake-on-LAN Wireshark capture:


```

> Frame 87030: 150 bytes on wire (1200 bits), 150 bytes captured (1200 bits) on interface \Device\NPF_{8DDFAD09-5B3B-43F9-9047-9E16E9D51313}, id 0
> Ethernet II, Src: Semtech_4c:3d:a3 (00:16:c0:4c:3d:a3), Dst: Broadcast (ff:ff:ff:ff:ff:ff)
> Internet Protocol Version 4, Src: 169.254.35.14, Dst: 255.255.255.255
> User Datagram Protocol, Src Port: 49153, Dst Port: 9
v Wake On LAN, MAC: LCFChFe_59:31:a0 (68:f7:28:59:31:a0), password: de:ad:be:ef:ca:fe
  Sync stream: ffffffff
  > MAC: LCFChFe_59:31:a0 (68:f7:28:59:31:a0)
  Password: de:ad:be:ef:ca:fe

```

```

0000  ff ff ff ff ff ff 00 16 c0 4c 3d a3 08 00 45 00  ....L=...E:
0010  00 88 00 66 00 00 ff 11 ed f2 a9 fe 23 0e ff ff  ...f....#...
0020  ff ff c0 01 00 09 00 74 da 46 ff ff ff ff ff ff  ....t..F.....
0030  68 f7 28 59 31 a0 68 f7 28 59 31 a0 68 f7 28 59  h.(Y1.h.(Y1.h.(Y
0040  31 a0 68 f7 28 59 31 a0 68 f7 28 59 31 a0 68 f7  1.h.(Y1.h.(Y1.h.
0050  28 59 31 a0 68 f7 28 59 31 a0 68 f7 28 59 31 a0  (Y1.h.(Y1.h.(Y1.
0060  68 f7 28 59 31 a0 68 f7 28 59 31 a0 68 f7 28 59  h.(Y1.h.(Y1.h.(Y
0070  31 a0 68 f7 28 59 31 a0 68 f7 28 59 31 a0 68 f7  1.h.(Y1.h.(Y1.h.
0080  28 59 31 a0 68 f7 28 59 31 a0 68 f7 28 59 31 a0  (Y1.h.(Y1.h.(Y1.
0090  de ad be ef ca fe .....

```

10.2 Sending WoL Packet via AVP from API Command

The `send_wol` API command is used for the AVP device to broadcast a WoL packet (UDP destination port 9) to the local subnet only, to awake a target machine with known MAC address (and optional Wake-on-LAN password). The WoL packet is only sent via the AVP devices 10G network interface (not the 1G).

The command syntax is as follows:

```

POST /api/device/{target}

{
  "op" : "send_wol",
  "wol_mac_address" : String,
  "wol_password" : String (optional)
}

```

The `target` argument must be the device ID of a single device.

The `wol_mac_address` argument is a 6 bytes hexadecimal string representing the mac address of the machine to wake up. The leftmost byte is transmitted first in the WoL packet.

The `wol_password` argument is a 4- or 6-bytes hexadecimal string. The leftmost byte is transmitted first in the WoL packet. This argument is optional.

For details on the `send_wol` API request, refer to the SDVoE Developers API Reference Guide.

Example of sending a WoL packet via AVP from an API command:

```

POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "send_wol",
  "wol_mac_address" : "010203040506",
  "wol_password" : "0708090A0B0C"
}

HTTP/1.1 201 Created
Location: /api/request/16
{
  "status" : "PROCESSING",
  "request_id" : 16,
  "result" : null,

```

```
    "error" : null
  }
GET /api/request/16 HTTP/1.1
HTTP/1.1 200 OK
...
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "send_wol" : [
      {
        "device_id" : "001ec0f03668",
        "device_name" : "my-AVP-03668",
      }
    ],
    "error" : []
  },
  "error" : null
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

11 Securing Control Communication

This section summarizes the use of API tools available to secure communication between the API servers and endpoint devices. Endpoint device authentication can be implemented through the use of API `auth` requests to apply industry-standard cryptography techniques (ECDH, AES128-CTR and POLY1305-AES) to protect communication between the API and endpoint devices.

The `claim` feature is also another useful tool when planning the implementation of an SDVoE solution where segmentation of API servers and endpoint devices is desired, such as a conference rooms, operating rooms or learning centers. This approach allows devices existing within the same network to communicate directly with a specified API instance.

Although these API commands are documented in the Developers API Reference guide there is information to be aware of that is useful when planning to deploy these security features, this section covers this information.

The table below outlines terms referred to in the following section.

Table 13: Terms and Definitions

Terms	Definition
<code>bound-authenticated</code>	When a device is "bound-authenticated", it communicates with a specific API server, securing communication between the API server and the device.
<code>friend API servers</code>	A pair of API servers configured with the same secure API key. When one API server is bound-authenticated to an endpoint device(s), the other API server is also authenticated to the same endpoint device(s).
<code>auth</code>	An API request that enables an endpoint device(s) to be bound-authenticated to the API server that the request initiated from.
<code>unauth</code>	Disables a device from being bound-authenticated to the API server.
<code>claim</code>	Useful to implement when it is desired to segment devices within a given network, such as group resources for a classroom or operating room. Also, prevents an adversary from using an unauthorized API server instance — for example running on a laptop connected to a network port in an installation—to control devices. When a device is claimed, it is controlled exclusively by the API server instance from which the request was issued.
<code>release</code>	Used to release an endpoint device to an unclaimed state.

11.1 Secure Control Communication Overview

Secure communication allows two entities to communicate without a third-party listening in, allowing information to be shared in a way that is unsusceptible to eavesdropping or interception. The API provides the ability to pass data securely through authentication and/or encryption as it passes between registered endpoint devices.

Securing communication between the API server and endpoints is accomplished using an `auth` request, which initiates a security protocol involving standard algorithms and best practices, such as ECDH, AES128-CTR, POLY1305-AES, SHA256 and anti-replay attack counters.

11.2 Introduction to API Authentication & Encryption

When authentication is enabled, the API server reports which devices are bound to it as well as those that are not bound-authenticated. Devices that are bound-authenticated to another non-friend API instance are not reported.

Note: A USB HID stream is only encrypted when the endpoint device is configured to use an encryption key for the given API server.

When an API server and an endpoint device are bound-authenticated, both ignore messages received that have not been authenticated or have been previously processed (to prevent replay).

Authentication is enabled through an API request. In regard to USB HID authentication, the endpoint device must have this option enabled in the firmware configuration settings.

Notes:

- i. The hardware manufacturer manages firmware configuration settings, contact the manufacturer for information on the default factory settings.
- ii. For details on the USB HID, refer to section 12 BlueRiver Device , as well as the USB HID Protocol Application Note Library (PDS-062253).

Messages ignored by the built-in anti-replay mechanism, or that failed authentication are logged by the API server to allow such occurrences to be monitored. Data reported includes the endpoint device ID that generated the error.

Important! A `factory` request resets an endpoint device back to the default factory configuration settings, disabling bound authentication. Therefore, following a factory reset, it is necessary to reestablish bound authentication to the appropriate API server using an `auth` request.

When a device is bound-authenticated to an API server, only the messages exchanged between the API server and the endpoint device are authenticated. Messages sent between paired endpoint devices, such as RS232, Infrared or USB, are not covered by this feature. If it is desired to encrypt USB HID data, this is supported through the API USB HID secure argument of the command `set usb`, refer to the SDVoE Developers API Reference guide for details. Broadcast packets originating from an endpoint device, such as `HELLO` packets, are also not authenticated.

When an endpoint device is bound-authenticated, the AVP Serial Update Tool cannot be used to complete a firmware update. This tool is generally only used by devices operating in a point-to-point configuration which do not have API server access; therefore, does not impact chipset updates completed through an API request.

11.3 Key Exchanges

The bound-authentication process dynamically exchanges a security key between the API server and its authenticated endpoints. When bound authentication is enabled by an API `auth` request, the security key is saved on endpoint devices in nonvolatile memory. If the option is disabled using an API `unauth` request or a factory reset is applied to an endpoint device, authentication needs to be re-enabled.

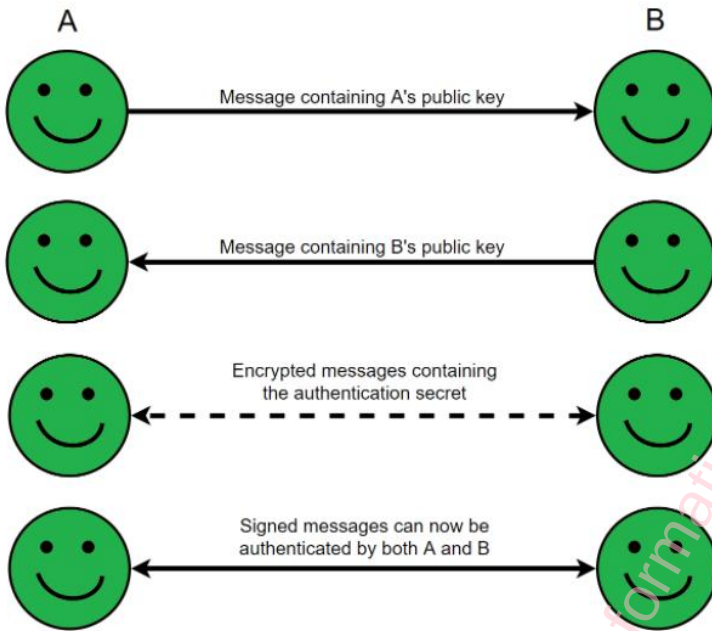


Figure 68: Communication Between API Server and Bound-Authenticated Endpoint Device

The figure below shows the relationship of an endpoint device that has been bound-authenticated and when a device has had an API `unauth` or `factory` request applied.

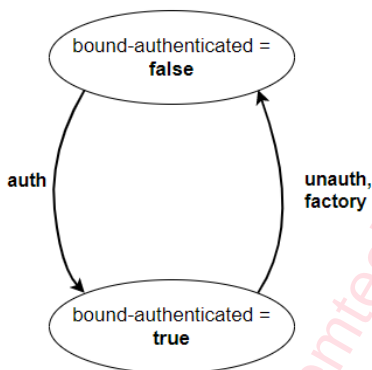


Figure 69: Endpoint Device Authentication

11.4 API Server Configuration

To allow an API server to bound-authenticate devices, a unique key is added to the API server configuration file. This key is added to the configuration file to ensure that, should the API server be restarted, it applies the same identifier after the restart as was set previously.

Server key (`secure_comm_secret`) best practices:

1. This configuration option can be set to any alphanumeric string (0-9, the letters A-Z, uppercase and lowercase, no special character, no space) but must be unpredictable.
2. A randomly generated string of 24 to 64 alphanumeric ASCII characters and symbols is a good choice.
3. Maximum of 64-characters can be used, if the string entered is longer than 64-characters it is internally truncated to 64 characters.

4. Do not share this value keep it secret. If it is leaked, an adversary could potentially run its own API server instance, impersonating the authorized server, negating this security feature.
5. Keep a record of this string. If you are replacing the API server in an existing installation, set the same secret as the original API server to let the replacement server control devices claimed by the original.

For more details refer to the SDVoE Control Server Administrators Guide.

The `secure_comm_secret` key is the basis for creating an authentication key that is unique for each endpoint device; it is forged using device data combined with the authentication key. This allows for another API server, referred to as `Friend API Server`, to be configured with the same security key and thus be bound-authenticated to the same pool of devices.

Warning! As the API server configuration file contains security information, special attention should be given to the access rights of this file.

11.5 auth / unauth API Requests

An API request is used to enable and disable bound authentication between the API server and an AVP endpoint device, where `auth` is used to enable the feature and `unauth` to remove the authentication.

Note: The API `auth` command is not applicable to NT1000/NT2000 devices; however, the `claim` command can be implemented on all SDVoE endpoint devices regardless of the chipset.

When an `auth` or `unauth` request is successful, the endpoint device(s) triggers a `DEVICE_DISCONNECTED` event followed by a `DEVICE_CONNECTED` event as the device communication is reset. For details on event types and tracking notifications, refer to the Developers Introduction to the SDVoE API User Guide. An `auth` request also only succeeds if the device is not currently bound to another API server instance.

The command syntax for `auth` (enabling of bound authentication) is as follows:

```
POST /api/device/{target}
{
  "op" : "auth"
}
```

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

For details on the `auth/unauth` API requests, refer to the SDVoE Developers API Reference Guide and for details related to the security key refer to the SDVoE Control Server Administrators Guide.

Example of enabling bound authentication for a single device with the target ID of 001ec0f03668.

```
POST /api/device/001ec0f03668
{
  "op" : "auth"
}
```

```

HTTP/1.1 201 Created
Location: /api/request/144
...
{
  "status" : "PROCESSING",
  "request_id" : 144,
  "result" : null,
  "error" : null
}

```

The command for removing/disabling bound authentication is `unauth`, the syntax for this is:

```

POST /api/device/{target}
{
  "op" : "unauth"
}

```

Example of disabling bound authentication for a single device with the target ID of 001ec0f03668.

```

POST /api/device/001ec0f03668
{
  "op" : "unauth"
}
HTTP/1.1 201 Created
Location: /api/request/145
{
  "status" : "PROCESSING",
  "request_id" : 145,
  "result" : null,
  "error" : null
}

```

11.5.1 API Configuration Node

If authentication has been enabled, the API device configuration return member “authenticated” indicates `TRUE`. If the member displays `FALSE`, the device has not had this option enabled. Example shows device settings return for a configuration node with `authenticated` member reporting `true`:

```

"configuration": {
  "device_name": "0016C04C3D9F",
  "locate_mode": false,
  "secure_av": "default",
  "resume_streaming": true,
  "claimed": true,
  "authenticated": true,
  "device mode": "TRANSCEIVER"
}
...

```

For information on the device mode and associated members, refer to the Developers Introduction to the SDVoE API User Guide and for details on retrieving device information refer to the SDVoE Developers API Reference Guide “Command `get`” section.

11.6 Authentication Use Cases

This section reviews use cases where authentication could be useful if implemented. Two use cases are reviewed: the first is a backup API deployment example and the second is resource segmentation. Prior to proceeding with review of use cases, a high-level look at normal communication is reviewed.

11.6.1 Single API Server Deployment

Normal communication between a client, single API server and endpoint devices is demonstrated in the figure below. When an API request is issued, it is sent to the API server instance specified in the request. The API forwards the request to the appropriate endpoint device(s), who proceeds to complete it.

A single API server instance can manage all transceiver (TR), encoder (TX) and decoder (RX) devices on a given subnet.

Warning! Only one API server instance should control an endpoint device at a given time, multiple API instances are not aware of each other. Therefore, using multiple instances to control signal routing can result in routing and multicast address conflicts, although this scenario can be avoided by implementing the claim feature. Refer to section 11.6.3 Segmented Deployment with Shared Authentication for an example use case.

Multiple client instances can connect and simultaneously communicate with an API server. An API server uses a FIFO to manage requests from multiple client sessions.

When an endpoint device completes a request, it issues a message notifying the API server. Under normal conditions all API server instances currently running on the network segment will receive the message notification. Each of the API servers will then take appropriate action, generating an event and asynchronous message notification, which is then forwarded to online client applications potentially generating unnecessary network traffic.

When bound authentication is enabled, the endpoint sends the message result to the API server to which it is bound, allowing data transfer to this single API instance not only applying the encryption between the API server and the endpoint, but also limiting the generation of event notifications.

Reminder: Details on event types, polling and event notifications are provided in the Developers Introduction to the SDVoE API User Guide.

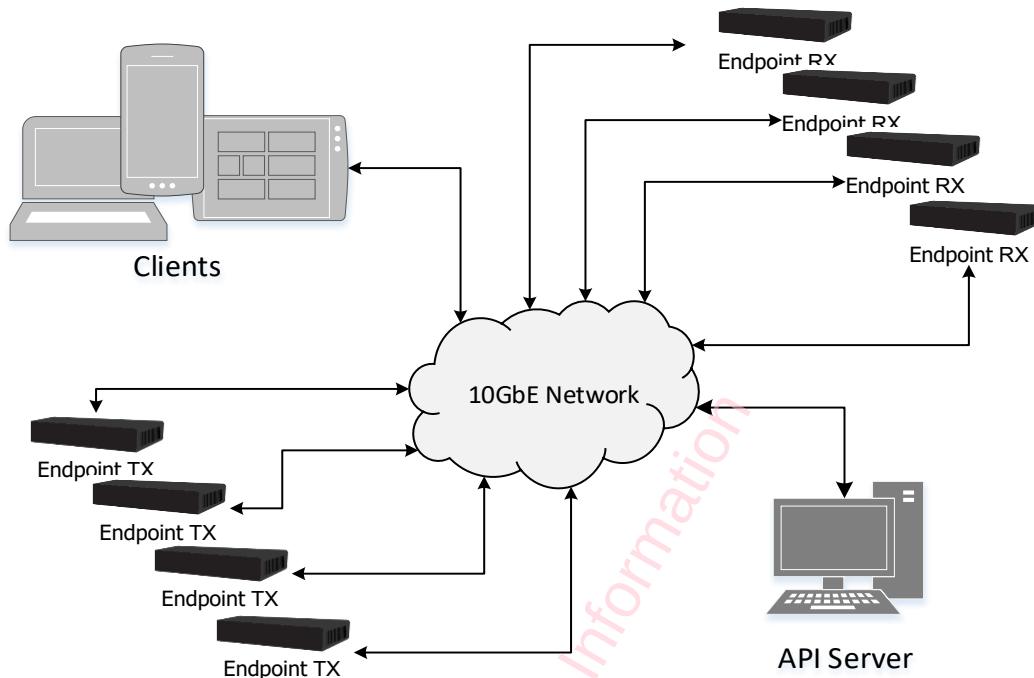


Figure 70: Network Setup with Single API Server Instance

11.6.2 Primary-Backup API Server Deployment

Some SDVoE systems include a primary API server instance running with a backup API available. As mentioned previously, it is possible for both API servers to be configured with the same security key. In the event the primary server is unavailable, and the backup becomes the active API, the shared security data allows the backup API server to exchange messages with the same pool of authenticated endpoint devices as the original primary API server, without additional authentication API requests.

In the case of a primary-backup scenario, the primary API server manages the polling of endpoint devices, managing command requests and monitoring for event notifications while the Friend API server is a dormant backup API server. Should the primary server become unreachable, since the two servers are using the same secure key data, the backup can take over control the endpoint devices originally handled by the primary server.

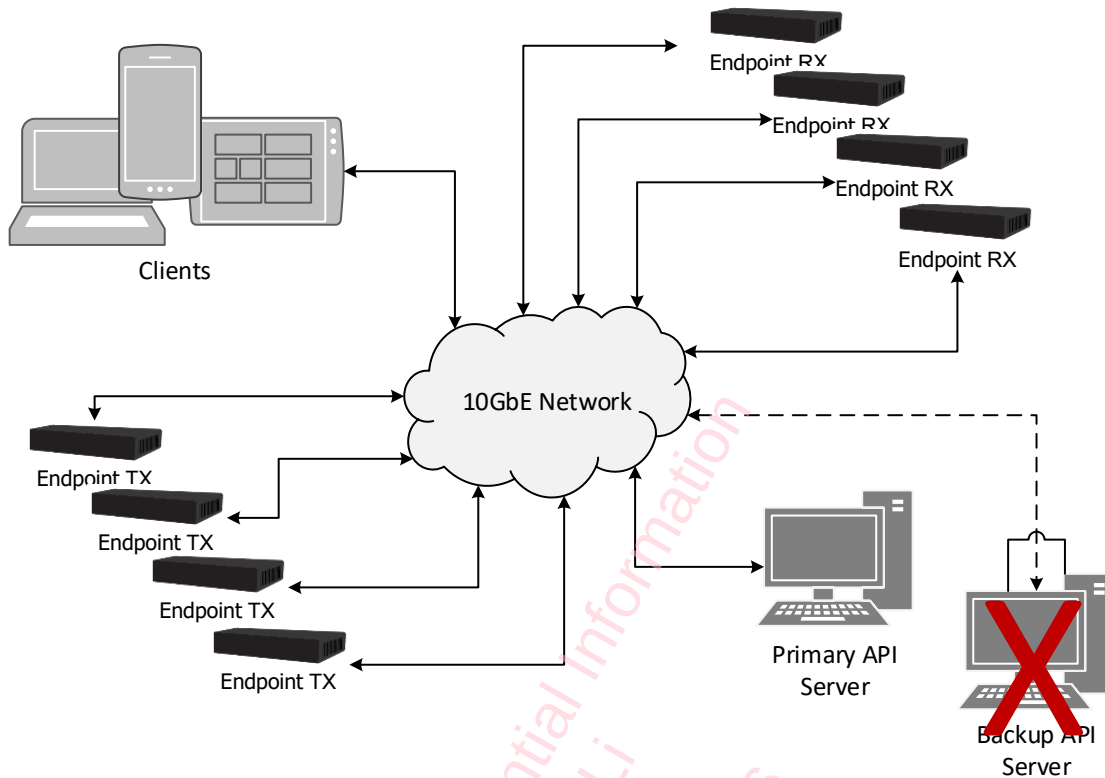


Figure 71: Use Case – Backup API Server

11.6.3 Segmented Deployment with Shared Authentication

In a large installation as well as some specific use cases, such as conference rooms, operating rooms and lecture halls, it is often desired to group resources. For API servers and endpoint devices, they can be grouped by sharing the same security authorization key and implementing the API claim feature. This allows end users to segment API servers and devices into groups, often referred to as “pools”. Sharing a bound-authentication security key among all segments allows for moving and sharing of endpoint devices to be more easily managed, while registering devices to a specific API server allows for them to be controlled by the specified API instance.

Implementing bound authentication would be done using an API `auth` request, refer to section 11.5 `auth/unauth` API Requests for example. Below the use of the `claim` API request is reviewed.

Note: For details on `auth/unauth` or `claim/release` API requests, refer to the SDVoE Developers API Reference Guide and the SDVoE Control Server Administrators Guide.

Reminder: When a device is claimed, it is controlled exclusively by the API server instance from which the command was issued.

- The `claim` command only succeeds if the device(s) is/are currently unclaimed or already claimed by the API server issuing the command.
- The `release` command returns the device(s) to unclaimed state. It only succeeds if the device(s) is/are claimed by the current API server or already unclaimed.

- When a `claim` or `release` request is successful, the endpoint device(s) triggers a `DEVICE_DISCONNECTED` event followed by a `DEVICE_CONNECTED` event as the device communication is reset. For details on event types and tracking notifications, refer to the Developers Introduction to the SDVoE API User Guide.

11.6.3.1 Claim Use Case

The command syntax for `claim` and `release` is as follows:

```
POST /api/device/{target}
{
  "op" : "claim"
}
```

```
POST /api/device/{target}
{
  "op" : "release"
}
```

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are:

ALL	All devices
ALL_TX	All transmitter devices
ALL_RX	All receiver devices

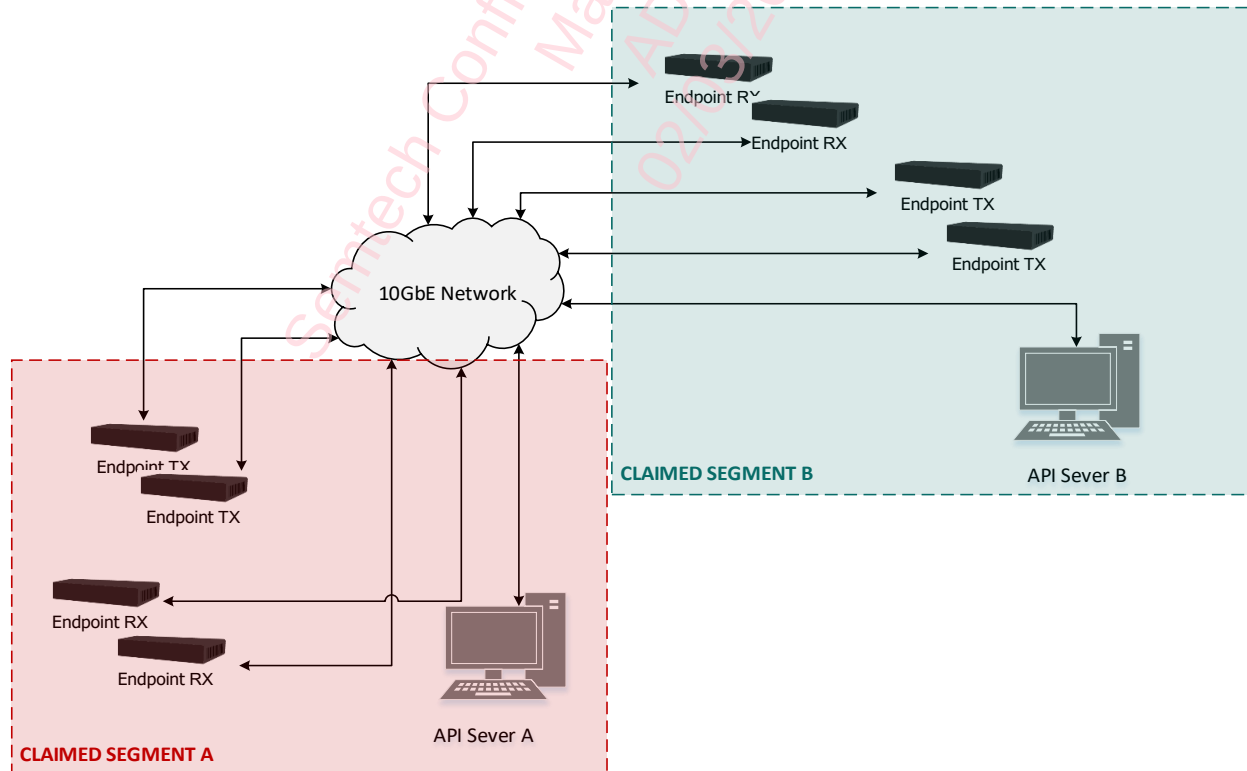


Figure 72: Example of API Server/Endpoint Device Segmentation using API Claim Feature

For the use case demonstrated above there are two segments, segment A (pink) and segment B (teal), there are two API servers running in the example network with multiple encoder (TX) and decoder (RX) devices. An API `claim` request is issued from the API server to each of the endpoint devices that it is to control. For this example, the four-endpoint devices shown grouped with API server A would be claimed through API requests sent to this API, while the four-endpoint devices shown grouped with API server B would be claimed through API requests sent to API server B.

In addition, an API `auth` request could be employed to add encryption security between the API and endpoint devices, as outlined previously.

Example of issuing an API `claim` request to an individual endpoint device, 001ec0f03668:

```
POST /api/device/001ec0f03668 HTTP/1.1
...
{
  "op" : "claim"
}
HTTP/1.1 201 Created
Location: /api/request/42
...
{
  "status" : "PROCESSING",
  "request_id" : 42,
  "result" : null,
  "error" : null
}
```

11.6.3.2 API Configuration Node

If a device has been registered to a specific API server using an API claim request, the API device configuration return member `"claimed"` indicates `TRUE`. If the member displays `FALSE`, the device has not had this option enabled.

Example of the device settings return for a configuration node where the `claimed` member is reporting `true`:

```
"configuration": {
  "device_name": "0016C04C3D9F",
  "locate_mode": false,
  "secure_av": "default",
  "resume_streaming": true,
  "claimed": true,
  "authenticated": true,
  "device mode": "TRANSCIEVER"
}
...
```

For information on the device mode and associated members, refer to the Developers Introduction to the SDVoE API User Guide, and for details on retrieving device information refer to the SDVoE Developers API Reference Guide "Command get" section.

11.6.3.3 Managing Multicast IP Range for Multiple API Servers

System integrators may find it useful to modify the multicast IP range used by an API server, segmenting the network communication. When applying "claim/release", it is possible to run multiple API instances on the same subnet mask as this feature creates a communication tunnel between a specific API and its registered endpoint devices.

However, the following needs to be considered:

- The API servers must be able to reach the same endpoint devices.
- Otherwise, one API server may stop streams of a device managed by another server erroneously as it assumes that the multicast IP stream is available.
- To avoid potential multicast IP assignment conflicts, customers can do either one or both of the following:
 - i. In the API server configuration file, specify a specific multicast IP range for each API server. Refer below for details on completing this task.
 - ii. Also, change the default multicast IP for one of the servers for the control data (Infrared/RS-232) when issued to ALL receiver or transmitter devices.

Note: Refer to the SDVoE Control Server Administrators Guide for details on managing the API configuration file (`controlserver.conf`).

- When starting a video or audio stream, specify the multicast IP to be assigned.

Modifying Default Multicast Range of API Server

Setting the multicast IP range is done by modifying the assigned multicast IP range of the API server in its configuration file (`controlserver.conf`).

The default range assigned in the factory configuration file is [224.1.1.1 - 224.1.3.255]. When an AV stream is started, the encoder (TX) device uses a multicast IP address within the range specified in the API server configuration file.

1. Locate the configuration file `controlserver.conf` and open it using a text editor such as Notepad+ (Windows) or vim (Linux).
2. Under the section `Multicast Addresses`, modify the start and/or end range to reflect the range that to apply.

```
#####  
# Multicast Addresses  
#####  
# All IP addresses in this section must be in the [224.0.2.0,  
# 239.255.255.255] range.  
  
# IP address range to use for multicast channel allocations  
multicast_range_start = 224.1.1.1  
multicast_range_end = 224.1.3.255
```

3. Save the `controlserver.conf` file.
4. Stop and relaunch the API server to apply the changes.

Modifying the Reserved Multicast IP Addresses

For the transfer of RS-232 and Infrared data to a group of devices (ALL_RX or ALL_TX), there are two multicast IP addresses reserved in the API server configuration file. In the case of multiple API server instances, it is necessary to modify these to be within the assigned multicast IP range of the API server and each instance should use a unique set of reserved addresses.

1. If not currently open, locate the configuration file, `controlserver.conf`, as outlined previously and open it using a text editor such as Notepad+ (Windows) or vim (Linux).
2. Under the section `Multicast Addresses`, modify the encoder (TX) and/or decoder (RX) IP addresses to reflect the addresses to apply.

```
# The all TX and all RX addresses are reserved multicast IP
#addresses that are used for device
# control and cannot be used for multicast channel allocations
#(i.e. to transport video or audio).
# These two addresses can be anywhere in the [224.0.2.0,
#239.255.255.255] range (inside or outside
# the allocations range).
#
# The legacy values for these addresses are the following:
#   all TX: 224.1.1.253
#   all RX: 224.1.1.254
# It is permissible to use these legacy values but they cannot be
#swapped (i.e. the all TX address cannot be configured to the
#legacy all RX address and vice versa).
#
# The legacy all TX and all RX addresses are always reserved
#(in addition to the configured ones if
# they differ).
multicast_all_tx = 224.1.1.253
multicast_all_rx = 224.1.1.254
```

3. Save the `controlserver.conf` file.
4. Stop and relaunch the API server to apply the changes.

12 BlueRiver Device USB-HID Encryption

It is possible to add a level of security to USB HID streams through by API encryption, implementation of this feature is reviewed in this section.

Reminder: USB HID functionality is only supported for the AVP chipset series, not applicable to NT1000/NT2000 endpoint devices.

12.1 Activate USB HID Encryption Tasks

To activate USB HID encryption, there are multiple tasks that need to be completed:

1. Set `secure_comm_secret` in the API server configuration file. Refer to section 11.4 API Server Configuration for example of configuring this key.
2. Set the `usb_hid_encryption_default_key` in the API server configuration file.
3. Restart or if not currently running start the API server instance for whom the above security data was modified.
4. Enable secured communication between the API server and the AVP endpoint device(s) using an `auth` request. Refer to section 11.5 `auth` / `unauth` API Requests for details completing this task, as well as the SDVoE Developers API Reference Guide.
5. Set the USB HID security level for the appropriate AVP devices using a `set usb security level` API request.
6. Set the USB-HID security key using a `set security_key` API request for each of the appropriate endpoint devices.

Note: For this feature to successfully be enabled, the feature must be enabled by the hardware manufacturer in the hardware configuration settings for the endpoint device. Contact the manufacturer for more information.

12.2 Configuring USB HID Encryption Key

The default USB HID encryption key is defined in the secure communication section of the API server configuration file (`controlserver.conf`).

This secret key shares the same characteristics and best practices as described for the authentication feature but is referred to as the `usb_hid_encryption_default_key`.

This configuration option can be set to any alphanumeric string (0-9, the letters A-Z, uppercase and lowercase, no special characters or spaces) but must be unpredictable. A random string that contains at least 24 to 32 characters is a good choice with maximum of 64 characters. If the string is longer than 64 characters, it will be internally truncated to 64 characters. Refer to the SDVoE Control Server Administrators Guide for more details.

Warning! As the API server configuration file contains security information, special attention should be given to the access rights of this file.

Example of the secure communication settings from the API server configuration file with the `secure_comm_secret` and `usb_hid_encryption_default_key` bolded for emphasis:

```
#####
# Secure Communication
#####
# Default secret encryption key phrase to be shared between instances of
# secured API server installations.
#
# To enable encrypted communication between the API and network devices,
# uncomment the line below and set the value to a unique, unpredictable,
# randomly-generated string.
# All ASCII characters are allowed. For maximum security, set to a random
# string of 64 characters that contains at least lower case and upper
# case letters as well as numbers.
#
# Important notes:
# * Keep this value secret. If it is leaked, an adversary can run its own
#   API server and have it impersonate this server, which allows it to
#   control secured devices.
# * Keep a record of this string. If you lose it and need to replace the
#   API server (e.g. because of a failed hard disk), then the only way to
#   get back control of the devices secured by the failed server is to
#   physically perform a factory reset on each device.
# * It is highly recommended to enable this feature as unsecured devices
#   on your network can be secured by an adversary running their own
#   API server.
#secure_comm_secret = Tq8AEXDDPS49qrDuK5X!%lkj<LBBMR

# Default secret encryption key phrases to be used to encrypt network
# data streams between devices supporting such data streams.

#usb_hid_encryption_default_key = nRAENMxzHhzZe!!22sr<`DTejvCT%z
```

12.3 Configuring USB HID Security Level

An API set `usb` request is to configure the security settings regarding USB HID device IP communication, it is also used to configure as the role of a device USB HID extender.

Notes:

- i. Applicable to BlueRiver AVP series chipsets, this includes the AVP1000, AVP2000 and AVP2000T.
- ii. Only supported on a device if the device JSON object for that device reports a `USB_HID` node.

If this security option is implemented it must be applied to all appropriate endpoint devices, that is both the LOCAL and REMOTE USB HID extenders of paired devices.

Note: To enable USB HID encryption, this feature must be enabled in the product configuration settings by the hardware manufacturer.

The command syntax for `auth` (enabling of bound authentication) is as follows:

```
POST /api/device/{target}
{
  "op" : "set:usb",
  "role" : String (optional)
  "security" : {
    "level" : "security_level_value" (optional)
  }
}
```

The following examples, show the syntax for API requests enabling and disabling USB HID encryption, as well an example setting the role and encryption through a single API request.

Reminder: It is required that this option be enabled by the hardware manufacturer in the product configuration. Contact the manufacturer for more information.

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "security" :
    "level" : "ENABLED"
}
```

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "security" :
    "level" : "DISABLED"
}
```

Request below sets the extender role to `REMOTE` and enables USB HID security encryption for a specific device.

```
POST /api/device/010203040506
{
  "op" : "set:usb",
  "role" : REMOTE
  "security" : {
    "level" : "ENABLED"
  }
}
```

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are:

<code>ALL</code>	All devices
<code>ALL_TX</code>	All transmitter devices
<code>ALL_RX</code>	All receiver devices

The `role` argument sets role of device when acting as a USB HID extender, can be one of the following:

<code>DISABLED</code>	USB HID support is disabled.
<code>OTG</code>	The device automatically detects the type of the connected USB HID device and set its role to <code>LOCAL</code> or <code>REMOTE</code> accordingly.
<code>LOCAL</code>	The device acts as USB device that connects to a USB host (e.g. a PC).
<code>REMOTE</code>	The device acts as USB host to which USB HID devices connect (keyboard, mouse, etc.)

The `security_level_value` argument is used to enable or disable the USB HID security of the device when acting as a USB extender. It can be one of the following:

- DISABLED USB HID IP communication encryption is disabled.
- ENABLED USB-HID IP communication encryption for all HID devices (keyboard, mouse, touchpad, etc.).

For more details on the API requests refer to the SDVoE Developers API Reference Guide and for details related to the security key refer to the SDVoE Control Server Administrators Guide.

12.3.1 API USB HID Node

Example below is a USB HID extender node JSON object returned as part of a `get device` request.

```
{
  "type" : "USB_HID",
  "index" : 0,
  "configuration" : {
    "role" : "HOST"
  },
  "status" : {
    "usb_devices" : [
      {
        "status" : "CONNECTED"
      },
      {
        "status" : "UNSUPPORTED",
      },
      {
        "status" : "DISCONNECTED",
      }
    ],
    "usb_host" : {
      "status" : "CONNECTED"
    },
    "role" : "REMOTE"
  },
  "inputs" : {}
}
```

This example is of the USB HID extender node with IP communication security enabled:

```
{
  "type" : "USB_HID",
  "index" : 0,
  "configuration" : {
    "role" : "REMOTE",
    "security" : {
      "level" : "ENABLED",
      "encryption key type" : "DEFAULT"
    }
  },
  "status" : {
    "usb_devices" : [
      {

```

```
        "status" : "CONNECTED"
    },
    {
        "status" : "UNSUPPORTED",
    },
    {
        "status" : "DISCONNECTED",
    }
],
"usb_host" : {
    "status" : "CONNECTED"
}
"role" : "REMOTE",
}
},
"inputs" : {}
}
```

For information on the device mode and associated members, refer to the Developers Introduction to the SDVoE API User Guide and for details on retrieving device information refer to the SDVoE Developers API Reference Guide “Command get” section.

13 Managing BlueRiver Device's 802.1X

13.1 Overview

IEEE 802.1X is a standard for port-based network access control. It provides an authentication mechanism for devices attaching to a LAN or WLAN. The network's authentication server verifies the credentials of SDVoE devices (BlueRiver AVP) that wish to connect to the network, allowing network access only to those devices that have been successfully identified.

The SDVoE devices implement IEEE 802.1X with the EAP-TLS authentication method, which utilizes digital certificates. The endpoint device is not pre-loaded with a default certificate from the factory. Therefore, the customer must issue a device certificate and configure it into the SDVoE device before it will have access to the 802.1X controlled network.

During the 802.1X TLS handshake, digital certificates are exchanged as certificate chains. The authentication server sends its certificate chain to the SDVoE device (the client), and the device sends its certificate chain to the authentication server. A certification chain is defined as an ordered list of certificates, where the sender's leaf certificate (the end-entity certificate) comes first. Each of the following certificates in the list certifies the one immediately preceding it. The last certificate in the chain is the root certificate, which is a self-signed certificate. During the authentication process, the server verifies the validity and integrity of the SDVoE devices certificate list.

The SDVoE device supports a TLS cipher suite which requires the usage of ECDSA certificates as leaf certificates, on both the server side and client side. The SDVoE device supports the SECG secp256r1 elliptical curve which is an alias for the NIST P-256 curve or the ANSI prime256v1 curve.

TIPS

- i. API `get device` and `get capabilities` returns include a "dot1x" flag in the "capabilities" section. This flag is true if the device supports 802.1X, false otherwise.
- ii. API `get device` and `get capabilities` returns include a "dot1x_certificate" flag, in the "configuration" section. When true indicates an 802.1X certificate is configured into the device, shows false otherwise.

Table 14: 802.1X Feature List

Feature	Description
Authentication Method	EAP TLS
TLS Version	1.2
TLS Cipher Suite	TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
TLS Ephemeral Key Exchange	Elliptical Curve secp256r1 (NIST P-256)
Device Digital Certificate (leaf certificate)	ECDSA using the secp256r1 key (NIST P-256)
Intermediate Certificates	Optional
Mixed Certificate Chains	Allowed. Intermediate and Root certificates can be either RSA or ECDSA.
Device Certificate Store	4K bytes
Supported File Format (Key File and Certificate Files)	Base-64 "PEM" format
Password-protected Key File	Optional
Mutual Authentication	Not supported
Periodic Reauthentication	Supported

13.2 Certificate Requirements

802.1X with TLS requires the creation of two certificate chains. One chain is the authentication server chain, and the other is the SDVoE device chain. Both certificate chains must be created by the customer – the SDVoE API server doesn't provide services for certificate creation. The exact certificate chain configuration is based on the customer's requirements. One constraint is that the end-entity certificates (the leaf certificates) must be ECDSA certificates created with the secp256r1 elliptical curve (P-256).

Each certificate forming the SDVoE device (AVP) certificate chain must be provided in individual files in base-64 (PEM) format. The key file (private key file) associated to the SDVoE device leaf certificate must be provided separately in base-64 (PEM) format. The BlueRiver chipset does NOT support certificate bundles where multiple certificates and keys are regrouped in a single file.

Following applies to the certificate:

- i. A certificate file in PEM format is base-64 data enclosed by the "-----BEGIN CERTIFICATE-----" header and the "-----END CERTIFICATE-----" footer.
- ii. A key file in PEM format is also base-64 encoded but using the "-----BEGIN PRIVATE KEY-----" and "-----END PRIVATE KEY-----" header and footer.
- iii. The key file can be password-protected, in which case the header and footer will be "-----BEGIN ENCRYPTED PRIVATE KEY-----" and "-----END ENCRYPTED PRIVATE KEY-----".

The figure below illustrates an example of two certificate chains. One chain is used as the authentication server chain and consists of the server leaf certificate, which is signed by an intermediate certificate. The intermediate is signed by the root certificate.

Similarly, the second chain consists of the SDVoE device leaf certificate, signed by an intermediate certificate, the latter signed by the root certificate. We chose to include an intermediate here but this is optional. We could have signed the server and SDVoE device leaf certificates directly from the root certificate. One could also include more than one intermediate certificate in the chain. Both leaf certificates are ECDSA with secp256r1 keys. The root certificate and any intermediate certificates can be RSA or ECDSA with any key length.

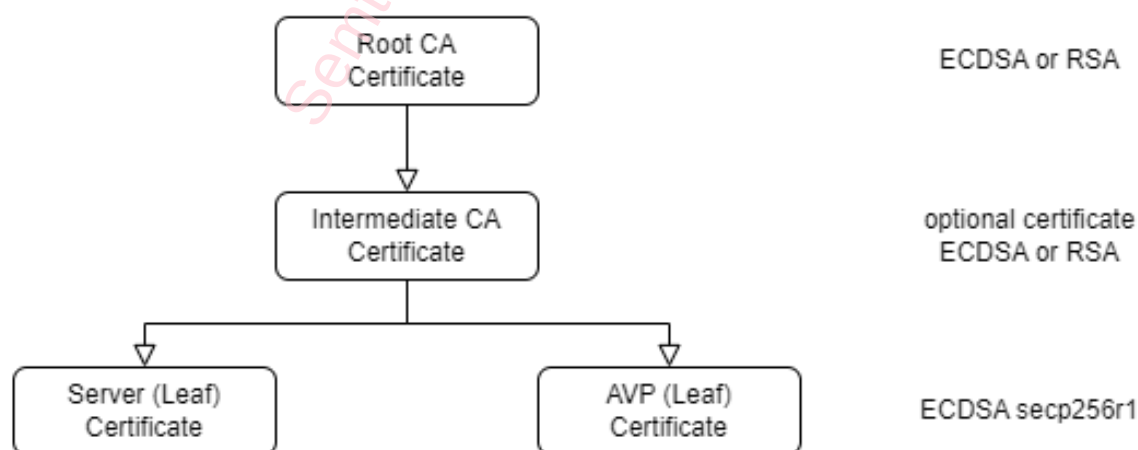


Figure 73: Example Leaf Certificate Structure

13.3 Uploading Device Certificate

Notes:

- i. If an `API factory` request is applied to an SDVoE device the 802.1X certificate chain is erased from the device, as well as its private key.
- ii. How to renew the 802.1X certificate chain? Issue the `set:dot1x_certificates` request originally issued to overwrite the existing certificate chain and install a new certificate chain into the SDVoE device. However, note that it is not possible to overwrite a single certificate in the existing chain; the complete chain must be reprogrammed as a whole.
- iii. If a `firmware update` is applied the 802.1X certificate chain is persistent, it is not removed from the SDVoE device.

Below outlines uploading a device certificate to an SDVoE AVP device.

1. The `dot1x_certificates_path` needs to be modified in the API server configuration file (`controlserver.conf`) to specify the path to the directory where the certificates are stored. This path specifies the directory containing the 802.1X customer-provided certificate files. If a relative path is provided, it is relative to the API server's working directory.

```
dot1x_certificates_path = <802.1X directory path>
```

Example:

```
dot1x_certificates_path = dot1x_certificates
```

TIP: Certificates must be copied to this directory before a `set dot1x_certificates` request is sent.

2. The SDVoE device(s) must be bound-authenticated with the API server using the following API command.

```
POST /api/device/{target}
{
  "op": "auth"
}
```

Refer to section 11.5 `auth / unauth` API Requests for information on using the `auth` command.

3. The complete certificate chain must be uploaded to the device using a single API `set:dot1x_certificates` request.

Warning! This command will overwrite any previously uploaded chain, there can be only one certificate chain loaded into the SDVoE device.

```
POST /api/device/{target}
{
  "op": "set:dot1x_certificates",
  "password": "<password for the key file>",
  "file_names": "<key file_name>,<cert file_name>,<cert file_name>, ..."
}
```

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are:

ALL - All devices | ALL_TX - All transmitter devices | ALL_RX - All receiver devices

The `password` specifies the password set for the key file. It is only required when the key file is encrypted.

The `file_names` is a list of file names. Starts with the key file, followed by the leaf certificate and the intermediate certificates, in this order. A comma (,) separates each `file_name` in the string.

Note: Using the https protocol is highly recommended because there is a password in the command, but http is also possible.

An example is illustrated below, where the `client.key` is the private key file which has been password-protected using the specified password. The end-entity certificate is "`client.pem`", which was signed by "`intermediate.pem`". The latter was signed by "`root.pem`".

Reminder: The order is important. The first two files are the key file and the end-entity certificate. Then each certificate in the list certifies (signs) the one immediately preceding it. The four files must reside in the directory specified in the `dot1x_certificates_path` above.

```
POST /api/device/0016c0123456
{
  "op" : "set:dot1x_certificates",
  "password": "semtech",
  "file_names" : "client.key, client.pem, intermediate.pem,
  root.pem"
}
```

Since certificate validation requires that the root certificate reside on the authentication server, it may be omitted from the chain. In this example, the "`file_names`" could be specified as "`client.key,client.pem,intermediate.pem`". It is recommended to always include the intermediate certificate in the TLS conversation.

4. Once the certificate chain has been loaded into the SDVoE device, it is ready to be connected to a controlled network.

Note: The device LED "`status`" will blink at a slow pace if there is an authentication failure while attempting the connection.

13.4 Reading Back Device Certificate

It is possible to read back a previously uploaded certificate chain from an SDVoE device. An API `get` request is used with the subset `dot1x_certificates` specified. The certificates are read-back in the same order as they are specified in an API `set:dot1x_certificates` request. That is beginning with the end-entity leaf certificate and progressing towards the root certificate.

Note: The private key cannot be read from the device.

```
POST /api/device/{target}
{
  "op" : "get",
```

```
    "subset" : "dot1x_certificates"
}
```

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are:

ALL - All devices | ALL_TX - All transmitter devices | ALL_RX - All receiver devices

Example of the data returned:

```
{
  "status": "SUCCESS",
  "request_id": <id>,
  "result": {
    "dot1x_certificates": [
      {
        "device_id": "0016c0123456",
        "device_name": "0016c0123456",
        "certificates": [
          {
            "certificate": "-----BEGIN CERTIFICATE-----
<base-64 data> -----END CERTIFICATE-----"
          },
          {
            "certificate": "-----BEGIN CERTIFICATE-----
<base-64 data> -----END CERTIFICATE-----"
          },
          {
            "certificate": "-----BEGIN CERTIFICATE-----
<base-64 data>-----END CERTIFICATE-----"
          }
        ]
      }
    ],
    "error": []
  },
  "error": null
}
```

Semtech Confidential Information 02/03/2026

The following is an example of a certificate in PEM format:

-----BEGIN CERTIFICATE-----

```
MIICjDCCAhKgAwIBAgIJASNFZ4mrze8WMAoGCCqGSM49BAMDMIGNMQswCQYDVQQGE
wJDQTEPMA0GA1UECAwGUXVlYmVjMREwDwYDVQQHDAhNb250cmVhbDEQMA4GA1UECg
wHVGVzdCBDQTERMA8GA1UECwwIRlcgZ3JvdXAxEDAOBgNVBAMMB1Rlc3QgQ0ExIzA
hBgkqhkiG9w0BCQEWFg1qYWNxdWVzQHNlbXRlY2guY29tMB4XDTIzMDQwNTE5MTMl
MVoXDTIzMDcwNDE5MTMlMVowXTELMakGA1UEBhMCQ0ExDzANBgNVBAGMB1F1ZWJlY
zERMA8GA1UEBwwITW9udHJlYWwxFDASBgNVBAoMC1Rlc3QgQ2xpZW50MRQwEgYDVQ
QDDAtUZXN0IENSaWVudDBZMBMGBYqGSM49AgEGCCqGSM49AwEHA0IABO2Tk9adSbl
jhZhwdXArn3CbsfEl55h4JIH6ttCBdifBsrs9gDx/6Fm//9/pYCqWXUg1F5I9772r
nrFlvek7duaJgYkgwYYwHQYDVR0OBBYEFB8CF6QvKaBujgTxac6dwz1295WtMB8GA
1UdIwQYMBaAFAvriXkQieLCzwPZm1WfIoDBD/NpMAkGA1UdEwQCMAAwCwYDVR0PBA
QDAgWgMCwGCWCGSAGG+EIBDQQfFh1PcGVuU1NMIEdlbmVyYXRlZCBZDZXJ0aWZpY2F
0ZTAKBgqhkiG9w0BQDAwNoADBlAjEArbRPXhpPHD4hE5eJ5iPHnWuT1bMtJ71Treqs
XlemrOMaYYdXO5WWJ9HyabO1yboEAjAI2CDNJB9eGZM16N3VcrG1xqFMFEowuKT+2
XPpUlnyrqKuhb4EwvYS9aBdfqunQzA=
```

-----END CERTIFICATE-----

Semtech Confidential Information
Mark Li
ADV
02/03/2026

14 BlueRiver Error Codes

The BlueRiver chipset has a built-in set of error codes that if they occur are reported through the API server, as well as if present the 7-segment display. If one of these error codes are reported, contact the hardware manufacturer for assistance in further troubleshooting.

Table 15: Error Codes

Code Number	Description
E00	Box is going back to factory settings.
E01	Device running from golden firmware image.
E04	Error writing into flash memory.
E07	Answer settings throttling.
E08	Invalid flash request.
E09	Key storage failure.
E12	USB chip did not initialize correctly.
E20	HDMI IC did not initialize correctly.
E21–E24	Impossible to communicate with HDMI IC (read).
E25–E28	Impossible to communicate with HDMI IC (write).
E29	Impossible to Flash HDMI IC.
E30	10G PHY IC did not initialize correctly.
E31–E34	Errors when accessing the 10G PHY
E37	Fall back to golden firmware for the 10GBASE-T PHY successful.
E40	EDID checksum does not match calculated checksum.
E51	Impossible to initialize MAC IC.
E52	MAC is busy.
E53	Impossible to communicate with MAC IC (write).
E54	Impossible to communicate with MAC IC (read).
E60	SFP+ did not initialize correctly.
E61	Impossible to communicate with SFP+.

Example of an error contained within a JSON return array:

```
"status" : {  
  "active" : true,  
  "point_to_point" : false,  
  "point_to_point_peer" : "",  
  "boot_status" : "PRIMARY",  
  "update_in_progress" : false,  
  "igmp_reporting_mode" : "QUERY",  
  "error_status" : {  
    "has_error_code" : true,  
    "error_code" : 51  
  }  
},
```

Note, that when an error has been reported the `has_error_code` field indicates `true` and the `error_code` field reports the actual error, in this example it was 51.

15 Managing Semtech UEX or Icron USB 2.0 Extender

To enable USB 2.0 routing and extension over IP:

- BlueRiver endpoint devices can incorporate one or two Semtech UEX (UEX1000 or UEX2000 attached to AVP) or one third-party Icron ExtremeUSB (attached to AVP or NT1000/NT2000) USB extender technology into their designs.
- The Semtech UEX2000 USB 2.0 extender can be used as a standalone endpoint (not attached to AVP) for USB 2.0 only products without BlueRiver.

In both cases, the API acts as an abstraction layer between a control client application and the Semtech UEX or Icron USB extender, providing the ability to detect, pair/unpair and configure the network interface of Semtech UEX (attached to AVP or standalone) or Icron (attached to AVP or NT1000/NT2000) USB extenders mounted within an endpoint device. Note that if a MAC address of an Icron USB extender exists in a given subnet that was not reported by a BlueRiver endpoint device, the API server ignores it.

The API is also used to manage the extender mode of the Semtech UEX or Icron subsystem that is if the endpoint device is to act as a Local Extender (LEX) or Remote Extender (REX). Specifically, through the API, the user can:

- Manage the USB extender role, `LOCAL` or `REMOTE`, using a `set property` request. For Semtech UEX, the role can also be set to `DISABLED`.
- Pair Semtech UEX or Icron USB extenders using `switch` requests and specifying the stream type as `USB_EXTENDER`.
- Also, possible using a `pair` request to pair a remote Semtech UEX or up to seven remote Icron USB extenders to a single Icron USB local extender, acting as a virtual hub. This feature known as Simultaneous User Interactions (SUI).
- Unpair Semtech UEX or Icron USB extender(s) using `unpair` request(s).
- Configure the network interfaces of both the BlueRiver chipset and the Semtech UEX or Icron USB extender using a `set ip` request.
- Retrieve the status of Semtech UEX or Icron USB extenders using a `get` request.

The following subsections review details of the above use cases.

15.1 Setting USB 2.0 Extender Role

A default USB 2.0 extender role is configured for devices during production by the hardware manufacturer, refer to manufacturer documentation for details on the default role applied to endpoint devices. That is if for example an encoder is set as Local Extender and decoders as Remote Extender. This data can also be retrieved from the API using a `get` request.

The assigned extender role can be managed using an API request. This is done using a `set property` request and the USB Extender key specifying the role to be assigned, either `LOCAL` or `REMOTE`. For Semtech UEX, the role can also be set to `DISABLED`.

Request syntax to set the role of the `USB_EXTENDER` extender role:

```
POST /api/device/{target}
{
  "op" : "set:property",
```

```

    "key" : nodes[USB_EXTENDER:index].configuration.role,
    "value" : [LOCAL|REMOTE|DISABLE]
}

```

The *target* is the device ID of the associated BlueRiver AVP endpoint device if using UEX attached to AVP or Icron and is the device ID of the UEX itself if the UEX is used standalone (not attached to AVP). The target can be the device ID of a specific endpoint device or a device group, ALL, ALL_RX or ALL_TX.

The USB_EXTENDER index is always 0 for Icron as only one Icron USB extender is supported per device, but can be either 0 or 1 for UEX as up to 1 UEX devices can be attached to a single AVP.

Refer to the SDVoE Developers API Reference Guide for additional information on the set property command and USB Extender Node.

15.2 Pairing & Unpairing USB Extenders

15.2.1 Simple Pairing using “switch”

The API includes the *switch* command, which provides the ability to pair two Semtech UEX (attached to AVP or standalone) or one Icron (attached to AVP or NT1000/NT2000) USB extenders that can then route data directly.

To create bidirectional routing, two API commands must be issued, once from the local extender to the remote extender and then again from remote extender to local extender. If an extender is currently paired, it is unpaired and then paired as per the API request.

Note: For new software application clients, it is suggested to consider implementing the API *pair* option to manage Semtech UEX or Icron extender pairings rather than *switch*. This is because the pair option supports both direct pairing of Semtech UEX or Icron modules and the implementation of the Icron Simultaneous User Interaction (SUI) feature. Refer to Icron documentation for details on the SUI feature.

15.2.1.1 Using “switch” Requests

The API *switch* command supports the pairing of Semtech UEX (attached to AVP or standalone) or Icron (attached to AVP or NT1000/NT2000) USB subscriptions of two (2) endpoint devices using the stream type USB_EXTENDER. Possible to pair either a Local Extender (LEX) to a single Remote Extender (REX) or a single Remote Extender (REX) to a Local Extender (LEX) using a switch request. The identification of the source and destination uses the following *switch* request:

```

POST /api/device/{source_device}
{
    "op" : "switch:device",
    "stream_type" : "USB_EXTENDER",
    "stream_index" : 0 (or 1),
    "dest_device" : "destination_device",
    "subscription_index" : 0 (or 1)
}

```

Where *source_device* and *destination_device* are the associated device ID of the endpoint devices (BlueRiver endpoint if attached to USB Extender or UEX endpoint if standalone). The USB_EXTENDER index can either be 0 or 1 as up to 2 UEX USB extenders is supported per the AVP

device (but if Icron, index is always 0).

The `STREAM_TYPE` for the USB 2.0 is `USB_EXTENDER`.

Note: Other device stream types can be paired using an API `switch` request including RS-232, Infrared, CEC and USB HID, refer to the SDVoE Developers API Reference Guide for details on these pairing these stream types.

With respect to `USB_EXTENDER` streams, the API does not allow the pairing of two USB extenders with the same assigned role that is two Local Extenders or two Remote Extenders nor does the API allow an Semtech UEX or Icron USB to be paired with a BlueRiver AVP USB HID extender. These types of requests return an error message.

WARNING! When a `switch` request is sent, it removes existing pairings. The impact of such a request is that if a local extender (LEX) is using the SUI feature, all current pairings of remote extenders (REX) are removed. This applies whether the LEX is specified as the destination or the source device. A `pair` request should be used if it is desired to preserve current Simultaneous User Interaction (SUI) pairings.

Two `switch` requests are used to pair USB 2.0 extenders bidirectional communication, first in one direction then in the opposite, as shown below:

```
switch <device1_stream> <device2_subscription>
switch <device2_stream> <device1_subscription>
```

The first `switch` request establishes a route from device1 to device2 and the second request establishes a route from device2 to device1.

“switch” Example

To pair two Semtech UEX or Icron USB extenders where device1 has device ID of 0016c035566f and device2 0017c0355675, the `switch` requests to pair the extenders in bidirectionally are as follows:

```
POST /api/device/0016c035566f
{
  "op" : "switch:device",
  "stream_type" : "USB_EXTENDER",
  "stream_index" : 0,
  "dest_device" : "0017c0355675",
  "subscription_index" : 0
}

POST /api/device/0017c0355675
{
  "op" : "switch:device",
  "stream_type" : "USB_EXTENDER",
  "stream_index" : 0,
  "dest_device" : "0016c035566f",
  "subscription_index" : 0
}
```

Reminder: If an extender is currently paired, the API will automatically unpair it and then pair it again based on the new request. For example, if device1 is currently paired to device3, a `switch` request to pair device1 to device2, first will unpair device1 from device3 and

then proceed to pair device1 to device2, this is a seamless transaction.

15.2.1.2 Using “stop” Request

To unpair an Semtech UEX or Icron USB extender, an API `stop` request can be used. Similar to the pairing USB 2.0 extenders, an API request is required to first unpair the two USB extenders in one direction, specifying the `stream_type` as `USB_EXTENDER` and then a second `stop` request to unpair the extenders in the other direction.

Note: If an extender is not currently paired, an error message is returned for the unpair request.

The `stop` command syntax is as follows:

```
POST /api/device/{target}
{
  "op" : "stop",
  "stream_type" : String,
  "stream_index" : Integer
}
```

Reminder: The `stream_index` can either be 0 or 1 for a `USB_EXTENDER` stream.

When a `stop` request is issued, it removes the current pairing, as follows:

- If issued to an endpoint device set as a remote extender (REX), it is unpaired from its current local extender (LEX) peer. No impact on other existing pairings for the specified local extender (LEX), current pairings remain.
- If however, the `stop` request is issued to an endpoint device that is set as a local extender (LEX), all current remote extender (REX) pairings are removed.

“stop” Example

For the switch example shown earlier, where device1 has a MAC address of 0016c035566f and device2 has a MAC address of 0017c0355675 to remove the bidirectional communication for the extenders in question, the `stop` requests would be:

```
POST /api/device/0016c035566f
{
  "op" : "stop",
  "stream_type" : USB_EXTENDER,
  "stream_index" : 0
}

POST /api/device/0017c0355675
{
  "op" : "stop",
  "stream_type" : USB_EXTENDER,
  "stream_index" : 0
}
```

The first `stop` request unpairs in the direction of device1 to device2, while the second `stop` request unpairs in the direction device2 to device1.

15.2.2 Simultaneous User Interaction (SUI) Pairing

Icron extenders can be configured to act as a virtual hub. This Icron feature is called “Simultaneous User Interaction” or SUI for short and supports up to seven remote extenders (REX) to be paired to a local extender (LEX). Simultaneous User Interactions is also sometimes referred to as VHub, or Virtual Hub

Note: Mass Storage Acceleration (MSA) enables faster data transfers from Mass Storage devices from remote extenders (REX) to local extenders (LEX). Important to note that this feature is incompatible with SUI.

15.2.2.1 SUI Guidelines & Limitations

1. The Simultaneous User Interaction (SUI) configuration should not be changed during runtime. The API reads and returns the configuration presented by the device but it doesn't report any changes made to the active configuration during runtime.
2. Mass Storage Acceleration (MSA) must be disabled on Icron extenders.
3. Simultaneous User Interaction (SUI) must be enabled on local extenders (LEX).
4. The first USB device connected to a remote extender (REX) must be a USB 2.0 hub. The USB 2.0 hub could be internal or external, confirm with the hardware manufacturer if you are unsure if your design includes an internal USB 2.0 hub.
5. Both SUI-enabled and disabled remote extenders can be paired with SUI-enabled local extenders, so long as all other requirements have been met.

When a remote extender (REX) is MSA-enabled and a paired with an SUI-enabled local extender, the API does not prevent the operation and logs the following warning message:

WARNING: ICRON EXTENDER: All extenders should be MSA-disabled in SUI configuration.

Simultaneous User Interaction (SUI) functionality pairing information is reported as part of the USB_EXTENDER node JSON array under a section called "sui_peers". Devices that include an Icron IC running firmware version N12 or higher and that support SUI shall display their active pairing configurations, up to seven LEX with a REX. Other devices shall present an empty JSON array.

Simultaneous User Interaction (SUI):

```
"type": "USB_EXTENDER",
  "index": 0,
  "configuration": {
    "role": "LOCAL",
    "program_mode": "NONE",
    "rs232_port": 0,
    "peers": [
      {
        "avp_attached": {
          "device_id": "AAAAAAAAAAAA",
          "device_name": "AVP_AAAAAAAAAA"
        },
        "mac_address": "0016c03556a4",
      },
      {
        "avp_attached": {
```

```

        "device_id": "BBBBBBBBBBBB",
        "device_name": "AVP_BBBBBBBBBBBB"
    }
    "mac_address": "123456789abc",
}
]
},
"status": {
    "chip_present": true,
    "mac_address": "0016c035567c",
    "chipset_type": "ICRON",
    "reachable": true,
    "revision": "1.9.4",
    "ip_mode": "DHCP",
    "ip_address": "169.254.41.9",
    "link_status_support": true,
    "sui_enabled": true
    "peers": [
        {
            "linked": false
        },
        {
            "linked": false
        }
    ]
},
"inputs": []

```

15.2.3 Pairing using “pair”

Pairing USB Extenders

API `pair` command is used when it is desired to pair one Semtech UEX or Icron USB extenders (REX) or multiple Icron remote extenders (REX) to a single local extender (LEX). A request is sent for each pair to be paired to the local extender.

```

POST /api/device/{target}
{
    "op": "usb_extender:pair",
    "stream_index": index (0 or 1), (optional, 0 if omitted)
    "dest_device": "device_id"
    "subscription_index": index (0 or 1), (optional, 0 if omitted)
}

```

The *target* is the associated device ID of the endpoint device (BlueRiver if attached, UEX if standalone) whose USB extender is to be paired, can be either a transmitter (TX), receiver (RX) or transceiver (TR) device.

- When the *target* is associated to a local extender (LEX) and SUI is enabled on the Icron extender, then multiple pairings are allowed. Reminder, each pairing request requires a separate command.
- When the *target* is the endpoint device associated to a remote extender (REX) then only a

single pairing is allowed.

- Regarding the `dest_device` argument, when pairing extenders the device ID is the device (BlueRiver if attached, UEX if standalone) of either a local extender (LEX) or remote extender (REX) depending on the extender type of the target specified.

TIPS:

- i. If the error “Device d88039e83991: (INVALID_STATE) Icron extender is not reachable” is received add a two-second delay between pairing commands.
- ii. The error message then received will then be, “Device d88039e83991: (INVALID_STATE) The icron extender did not acknowledge the 'pairing' process”, indicating the pairing request was unsuccessful.

Unpairing USB Extenders

The API `unpair` command is used to unpair a remote extender (REX) from a local extender (LEX).

```
POST /api/device/{target}
{
  "op": "usb_extender:unpair",
  "stream index": index (0 or 1), (optional, 0 if omitted)
  "dest_device": "device_id"
  "subscription index": index (0 or 1) (optional, 0 if omitted)
}
```

The *target* is the associated device ID of the endpoint device (BlueRiver if attached, UEX if standalone) whose Semtech UEX or Icron extender is to be unpaired.

- When the *target* specified is associated to a local extender (LEX) and SUI is enabled on the Icron extender, then multiple unpairings are allowed.
- If the *target* specified is associated to a remote extender (REX) then only single unpairing is completed.

The `dest_device` argument is the device ID of the endpoint (BlueRiver if attached, UEX if standalone) that the local extender (LEX) or remote extender (REX) is to be unpaired from.

- When the *target* specified is associated to a local extender (LEX) and the `dest_device` is a remote extender (REX) the pairing of the specified local extender is unpaired.
- When the *target* specified is associated to a local extender (LEX) and the `dest_device` is not specified, then all current existing pairings to remote extenders (REX) are removed.
- When the *target* specified is associated to a remote extender (REX) then the `dest_device` must be of the paired local extender (LEX) or omitted from the command and in both cases only this pairing is removed.

If a pairing request is sent and the endpoint device has previously reached its maximum allotted pairings, for example seven pairs for Icron, an error is generated. Below is an error example for Icron:

```
ICRON EXTENDER: Icron device does not support more than 7
simultaneous peer(s). Consider unpairing from previous peer(s).
```

Extender Type	SUI	Number of Supported Pairings
Remote (REX)	Enabled	1
Remote (REX)	Disabled	1
Local (LEX)	Enabled	7 for Icron, 1 for Semtech UEX

Extender Type	SUI	Number of Supported Pairings
Local (LEX)	Disabled	1

Notes:

- When a `factory reset` is issued to a device, all current pairings are removed for the device(s) being reset to default settings.
- A `stop` request with the stream type set to `USB_EXTENDER` will unpair all current Semtech UEX or Icron pairings.
- If a `switch` request is used to establish a pairing between a local extender (LEX) and remote extender (REX), if currently a pairing exists, it is removed and the new pairing established.

Examples

The following is an example illustrating a `pair` request. For this use case assume a local extender (LEX) d88039e83991 to remote extender (REX) 0016c04c46d7:

```
POST /api/device/d88039e83991
{
  "op": "usb_extender:pair",
  "dest_device": "0016c04c46d7",
}
```

This example performs an unpairing of the previous paired extenders:

```
POST /api/device/d88039e83991
{
  "op": "usb_extender:unpair",
  "dest_device": "0016c04c46d7",
}
```

15.3 USB_EXTENDER Status Node

```
{
  "chip_present" : Boolean,
  "device_id" : String,
  "device_name" : String,
  "mac_address" : String,
  "chipset_type" : String,
  "reachable" : Boolean,
  "revision" : String,
  "ip_mode" : String,
  "ip_address" : String,
  "link_status_support" : Boolean,
  "sui_enabled" : Boolean
  "peers" : Description of Peers
  [ {
    "linked" : Boolean
  } ],
  "role" : String,
}
```

Table 16: Local USB Extender Status Node Object Members

Member Name	Since	Type	Description
chip_present	3.0	Boolean	TRUE indicates that an Semtech UEX or Icron module is detected. Property is FALSE if USB Extender enabled in the hardware configuration but has not been populated.
device_id	3.0	String	Device ID of the Semtech UEX USB extender. If it has not been detected, this field is not there. This field is not there if the USB extender is an Icron module.
device_name	3.0	String	Device name of the Semtech UEX USB extender. If it has not been detected, this field is not there. This field is not there if the USB extender is an Icron module.
mac_address	3.0	String	MAC address of the Semtech UEX or Icron module. If the module has not been detected, this field is blank ("").
chipset_type	3.8	String	The USB Extender device model: UEX2000 for Semtech UEX2000 USB Extender UEX1000 for Semtech UEX1000 USB Extender ICRON for Icron USB Extender
reachable	3.2	Boolean	Indicates that the USB extender network interface is reachable (TRUE) or not (FALSE) with an IP address.
revision	3.3	String	The revision level of the Semtech UEX or Icron USB Extender firmware.
ip_mode	3.3	String	The active IP mode, either DHCP, AUTO_IP or MANUAL (static).
link_status_support	3.3	Boolean	Indicates whether the Semtech UEX or Icron supports the linked attribute.
sui_enabled	3.4	Boolean	Indicates if the Icron module has Simultaneous User Interaction (SUI) enabled. TRUE indicates it is enabled and FALSE that it is not.
peers linked	3.8	Boolean	Indicates if bidirectional communication is currently established. Devices need to negotiate the link before they can successfully link. TRUE indicates this negotiation successful. If extenders are not currently paired or a Semtech UEX or an Icron module is not detected, the status is FALSE. The field is also FALSE if the

Member Name	Since	Type	Description
			link_status_support field reports FALSE.
role	3.8	String	The role status of the USB extender: LOCAL for a local extender (LEX). REMOTE for a remote extender (REX). DISABLED (only applicable for Semtech UEX).

The example provided shows the USB extender node JSON object returned as part of a `get device` request of a UEX1000 device attached to an AVP. Note that in the case of an Icron LEX with SUI enabled, the `sui_enable` field would be true and the additional peers (up to 7) would appear in the configuration and status `peers` array.

```

"device_id" : "0016c04c3d38",
...
{
  "type" : "USB_EXTENDER",
  "index" : 0,
  "configuration" : {
    "role" : "LOCAL",
    "program_mode" : "NONE",
    "rs232_port" : 0,
    "peers" : [ {
      "avp_attached": {
        "device_id": "0016c04c3da3",
        "device_name": "AVP_0016c04c3da3",
      }
    }
  ],
  "mac_address": "0016c03556a4",
  "device_filtering" : {
    "block_iso_transfers": false,
    "class_filtering": "ALLOW_ALL",
    "allow_api_commands": false,
    "allow_i2c_control": false,
  },
  "remote_vbus_ctrl" : "AUTO",
  "msa_enable" : {
    "bot_protocol": true,
    "uas_protocol": true
  }
},
"status" : {
  "chip_present" : true,
  "device_id" : "0016c35571d ",
  "device_name" : "USB_0016c35571d ",
  "mac_address" : "0016c35571d",
  "chipset_type" : "UEX1000",
  "reachable" : true,
  "revision" : "1.2.0.0",

```

```

        "ip_mode" : "DHCP",
        "ip_address" : "192.168.100.22",
        "link_status_support" : true,
        "sui_enabled": false,
        "peers": [
            {
                "linked": true,
            }
        ],
    },

```

15.4 USB Extender Node Network Configuration

The API `set ip` command

- with BlueRiver device as the target supports the configuration of both the BlueRiver chipset's network interface node and if attached, the Semtech UEX or Icron device(s) network interface node.
- With UEX device as the target, only allowed for UEX2000 used standalone, supports the configuration of the UEX network interface node.

Both DHCP, AUTO_IP and STATIC (manual) IP modes are supported.

For DHCP mode, the following API request is used:

```

POST /api/device/{target}
{
    "op" : "set:ip:DHCP",
}

```

Both the BlueRiver chipset and the Semtech UEX or Icron module request IP addresses from the detected DHCP server.

To configure a static IP address of a BlueRiver device and its attached USB extender(s), the following API request with BlueRiver as the target is used:

```

POST /api/device/{target}
{
    "op" : "set:ip:static",
    "address" : String(conditional),
    "usb_extender0_address" : String(optional),
    "usb_extender1_address" : String(optional),
    "mask" : String(conditional),
    "gateway" : String(conditional)
}

```

To configure a static IP address of a UEX2000 used standalone (not attached to AVP), the following API request with UEX as the target is used:

```

POST /api/device/{target}
{
    "op" : "set:ip:static",
    "address" : String(conditional),
    "mask" : String(conditional),
}

```

```
    "gateway" : String(conditional)
}
```

For AUTO_IP mode, the following API request is used:

```
POST /api/device/{target}
{
    "op" : "set:ip:AUTO_IP",
}
```

This is the Force Auto-IP mode, when selected the Auto IP mechanism implements Automatic IP assignment, whether a DHCP server is present or not. This is not the recommended mode, should be implemented with caution.

Supported only for the BlueRiver AVP chipset series, not supported for the NT1000/NT2000 chipset series.

WARNING! Not applied to the Semtech UEX nor Icron modules.

For example, to specify a static IP address of 10.1.1.5 for the BlueRiver chipset and 10.1.1.6 for its attached Semtech UEX or Icron module for the endpoint device 001ec0f03668, where the subnet mask is 255.255.255.0 and the gateway is 10.1.1.1 would enter the following API request:

```
POST /api/device/001ec0f03668
{
    "op" : "set:ip:static",
    "address" : 10.1.1.5,
    "usb_extender0_address" : 10.1.1.6,
    "usb_extender1_address" : 10.1.1.6,
    "mask" : 255.255.255.0,
    "gateway" : 10.1.1.1
}
```

If the IP mode is set to DHCP, the extender is configured to operate in DHCP mode. If the IP mode is set to static and two or three addresses are provided, the first address (address) is assigned to the BlueRiver chipset's network interface node and the second (usb_extender0_address) is assigned to the first USB_EXTENDER node and the third (usb_extender1_address) assigned to the second USB_EXTENDER node (this third one is only applicable if 2 UEX are attached to the AVP). The configured subnet mask and gateway are applied to both network interface nodes.

Regarding the IP address arguments, if only the usb_extender0/1_address address is specified, since the BlueRiver chipset address is required an error is returned indicating the missing data. If both AVP and USB extender fields are specified but the USB_EXTENDER node(s) show the chip_present as false, the usb_extender_address0/1 is ignored.

15.5 USB Extender Node Settings Filter

The API supports an argument to send a `get settings` request filtering the data returned only for the `USB_EXTENDER` node type. The result lists the details of the `USB_EXTENDER` device node for one or more endpoint devices. For details on the settings filters, refer to the `event` command in the SDVoE Developers API Reference Guide.

```
POST /api/device/target
{
  "op" : "get",
  "subset" : "settings",
  "node_type" : "usb_extender"
}
```

The `target` argument can be the device ID of a single device or the name of a device group. The supported device groups are as follows:

ALL	All devices
ALL_RX	All decoder/receiver devices
ALL_TX	All encoder/transmitter devices

The subset of the `get` request is `settings` and the node type is set to `usb_extender`.

Example return of a `get settings` request filtered for the `USB_EXTENDER` node type for the paired AVP endpoint devices 0016c035566e and 0016c035566f:

```
{
  "status" : "SUCCESS",
  "request_id" : null,
  "result" : {
    "device_nodes" : [
      {
        "device_id" : "0016c035566e",
        {
          "type" : "USB_EXTENDER",
          "index" : 0,
          "configuration" : {
            "role" : "REMOTE",
            "program_mode" : "NONE",
            "rs232_port" : 0,
            "peers" : [ {
              "avp_attached" : {
                "device_id" : "0016c035566f ",
                "device_name" : "AVP_0016c035566f ",
              }
            },
            "device_id" : "0016c0355661",
            "device_name" : "USB_0016c0355661",
            "mac_address" : "0016c0355661"
          }
        }
      ]
    },
    "device_filtering" : [ {
      "block_iso_transfers": false,
      "class_filtering": "ALLOW_ALL",
      "allow_api_commands": false,
    }
  ]
}
```

```

        "allow_i2c_control": false,
    },
    "remote_vbus_ctrl" : "AUTO",
    "msa_enable" : {
        "bot_protocol": true,
        "uas_protocol": true
    },
},
"status" : {
    "chip_present" : true,
    "device_id" : "0016c0355660",
    "device_name" : "USB_0016c0355660",
    "mac_address" : "0016c0355660",
    "chipset_type" : "UEX1000",
    "reachable" : true,
    "revision" : "1.2.0.0",
    "ip_mode" : "DHCP",
    "ip_address" : "192.168.100.20",
    "link_status_support" : true,
    "linked" : true,
    "sui_enabled" : false,
    "peers" : [
        {
            "linked": true,
        }
    ],
},
"inputs" : []
},
{
    "device_id" : "0016c035566f",
    {
        "type" : "USB_EXTENDER",
        "index" : 0,
        "configuration" : {
            "role" : "LOCAL",
            "program_mode" : "NONE",
            "rs232_port" : 0,
            "peers" : [ {
                "avp_attached" : {
                    "device_id" : "0016c035566e",
                    "device_name" : "AVP_0016c035566e",
                }
            },
            {
                "device_id" : "0016c0355660",
                "device_name" : "USB_0016c0355660",
                "mac_address" : "0016c0355660"
            }
        ]
    }
    "device_filtering" : [ {
        "block_iso_transfers": false,
        "class_filtering": "ALLOW_ALL",
        "allow_api_commands": false,
    }

```

```

        "allow_i2c_control": false,
    },
    "remote_vbus_ctrl" : "AUTO",
    "msa_enable" : {
        "bot_protocol": true,
        "uas_protocol": true
    },
},
"status" : {
    "chip_present" : true,
    "device_id" : "0016c0355661",
    "device_name" : "USB_0016c0355661",
    "mac_address" : "0016c0355661",
    "chipset_type" : "UEX1000",
    "reachable" : true,
    "revision" : "1.2.0.0",
    "ip_mode" : "DHCP",
    "ip_address" : "192.168.100.20",
    "link_status_support" : true,
    "sui_enabled" : false
    "peers" : [
        "linked" : true
    ]
},
"inputs" : []
}
},
"error" : []
},
"error" : null
}

```

15.6 Recovery of Unreachable USB 2.0 Extenders

It is possible for a Semtech UEX or an Icron USB extender to become unreachable on the network if any of the following conditions occur:

- i. The IP address of the BlueRiver device is on a different subnet than the Semtech UEX or Icron USB extender.
- ii. The specified IP is in use by another device, creating an IP conflict.
- iii. The extender was not able to retrieve an IP address from a DHCP server.

If one of the above situations occurs, it is still possible to detect a Semtech UEX or an Icron USB extender and communicate with it through UDP multicast messages. However, pairing will fail since UDP unicast streaming is used to establish a pairing.

To allow a Semtech UEX or an Icron USB extender to be able to process configuration requests as well as to be visible, the API requests `Use DHCP` and `Use static IP` are sent as broadcast UDP messages. Upon pairing, a `Pair to Device` request is sent as a unicast UDP message to ensure the IP address is reachable and that the extender can receive a USB stream. In the event that the extender is unreachable, the API returns an error message. If an error occurs, the user should reconfigure the network interface of the endpoint device to ensure that the Semtech UEX or Icron USB extender has or can be assigned an IP address.

15.6.1 Visibility of USB 2.0 Extenders IP Address

The IP address of a Semtech UEX or an Icron extender is stored in the API server as part of the device info. This internal information is visible to the control application through the API as it is returned as part of the USB Extender (previously Icron Extender) Node information. The API server also provides a reachable/unreachable status, paired status, the current IP mode and MAC address of the extender(s) to which it is paired. Refer to the settings filter example provided in section 15.5 USB Extender Node Settings Filter for an example.

15.6.2 Resetting an Unreachable Icron Module

The `usb_extender:reset` (previously `icron:reset`) is used to reset an Icron USB extender module that has been incorporated into a BlueRiver endpoint device, for example if the modules reported state is unreachable.

```
POST /api/device/0016c035566e
{
  "op" : "usb_extender:reset",
  "index" : 0 (optional)
}
```

The `target` argument can be the device ID of a single device or the name of a supported device group. The supported device groups are as follows:

ALL	All devices
ALL_RX	All decoder/receiver devices
ALL_TX	All encoder/transmitter devices

The `index` argument represents the Icron module, since there is only one Icron module mounted the index value is always 0. If the index is not specified the default is 0.

As an example, the API request below resets the Icron module of the endpoint device `0016c035566e`.

```
POST /api/device/0016c035566e
{
  "op" : "usb_extender:reset",
  "index" : 0
}

HTTP/1.1 201 Created
Location: /api/request/101
...
{
  "status" : "PROCESSING",
  "request_id" : 101,
```

```
"result" : null,  
"error" : null  
}
```

Semtech Confidential Information
Mark Li
ADV
02/03/2026

16 BlueRiver NT1000/NT2000 Chipset Features

The following sections outline features that are available only to endpoint devices that include BlueRiver NT1000 and/or NT2000 chipsets.

- Section 16.1 NT2000 Bitmap Overlay Feature
- Section 16.2 Thumbnail Preview

The Bitmap Overlay and Thumbnail Preview features are not applicable to endpoint devices based on BlueRiver AV Processor (AVP1000/AVP2000/AVP2000T) chipsets.

16.1 NT2000 Bitmap Overlay Feature

This section provides information regarding the Bitmap Overlay feature and is applicable **only** to devices based on BlueRiver NT2000 chipsets. The target audience for this section is software developers, project managers and others that are involved in the implementation of the Bitmap Overlay feature.

The Bitmap Overlay feature allows a monochrome (two-color) bitmap to be overlaid on video. Specifically, it is supported:

- On a decoder (RX) devices HDMI video output; and
- On an encoder (TX) devices scaler video stream (HDMI stream 1).

The bitmap overlay can be positioned arbitrarily inside the picture. The foreground and background colors are configurable to arbitrary RGB values. The transparency level is also configurable independently for both the foreground and background colors.

If required, pixel replication by a factor of 2 or 4 is also supported. This provides a rudimentary form of scaling to allow the size of the overlay to be adapted to the picture resolution.

Whenever the picture resolution changes, the bitmap overlay is disabled, which provides the application an opportunity to modify its parameters before re-enabling the overlay. In this context, the picture resolution refers to the scaler resolution on an encoder (TX) or the output resolution on a decoder (RX).

The source bitmap is a file in BMP format that is encoded into a hexadecimal string and specified directly as a part of the relevant API command.

16.1.1 Bitmap Overlay Guidelines & Limitations

The following section outlines limitations that apply to the bitmap overlay feature.

1. Only devices including a BlueRiver NT2000 chipset support this feature.
2. On a decoder (RX) device, the bitmap overlay feature cannot be applied when the device is operating in Genlock mode, as this feature requires the AV Processor Engine.

Reminders:

- i. A decoder running in Genlock mode does not implement the AV Processor Engine.
 - ii. A bitmap overlay is available to NT2000 decoders operating in an advanced processing mode, that is Fast Switch, Genlock Scaling, Wall and Multiview modes.
3. Only one bitmap overlay is supported per device.

However, since encoder (TX) devices support the bitmap overlay feature this limitation does not preclude a multiview application where each video source is identified with a bitmap overlay that specifies e.g. a source number or a short name.

-
4. The width of the bitmap must be an even number of pixels. This also applies to the horizontal position of the overlay.
 5. Subject to the previous requirement, the bitmap overlay can be positioned arbitrarily as long as it is completely within the picture.
 - a. However, a bitmap overlay cannot be positioned in such a way that it requires cropping due to lying outside the video window.
 - b. In this situation, both the `size` of the bitmap image as well as the `pixel replication factor` need to be taken into account.
 6. The `maximum total number of pixels of the bitmap image` is 65536.

For example, 256x256, 4096x16, 64x1024 and 64x64 bitmaps are all acceptable bitmap sizes, but 260x256 is not.
 7. The specified bitmap must be a valid BMP monochrome bitmap file; that is a BMP file with a 1-bit per pixel representation.
 - Multiple variations of the BMP file format exist with different versions of the bitmap header.
 - The following versions of the bitmap header are supported, which in practice covers all formats supported by modern applications:

`BITMAPCOREHEADER` / `OS21XBITMAPHEADER` (header size: 12 bytes)

`BITMAPV4HEADER` (header size: 108 bytes)

`BITMAPV5HEADER` (header size: 124 bytes)

For more information on each of these headers, refer to:

<https://docs.microsoft.com/en-us/windows/desktop/gdi/bitmap-header-types>

16.1.2 Generating Bitmap Overlay

If an application is to generate bitmaps on the fly rather than load existing files, for example text with a specific font, it is recommended to generate the variant with the `BITMAPCOREHEADER` bitmap header as it is the simplest.

For more details on this header, the `BITMAPFILEHEADER` that precedes it and the general structure of a BMP file, refer to:

<https://docs.microsoft.com/en-us/windows/desktop/gdi/bitmap-header-types>

Implementation Tips:

- Although the start of all versions of the bitmap header look similar, should be noted that the width and height fields are 16-bits in `BITMAPCOREHEADER`, while it is 32-bits in the other versions.
- As expected, the height field is unsigned in `BITMAPCOREHEADER` but should be noted that it is a signed value in other versions.
- A positive value for the height indicates that the lines are stored bottom first, while a negative value indicates lines are stored top first.
- `BITMAPCOREHEADER` bitmap files are always bottom first.
- In a bitmap file, the length of each image line must be a multiple of four bytes, with padding added as necessary to meet this requirement.

- For BITMAPCOREHEADER bitmap files, the color table that follows the bitmap header uses three bytes per entry, i.e. one per color component (red, green and blue). Bitmap files with other header versions have four bytes per entry.

16.1.3 API Bitmap Requests

The API `set` command supports the bitmap overlay feature as follows:

- The `set overlay` command allows for the configuration of the bitmap overlay.
- The `BITMAP_OVERLAY` node reports the current bitmap overlay configuration, when applicable.
 - An NT2000 device supporting the bitmap overlay feature will include the `BITMAP_OVERLAY` node.
 - A device that does not support this feature will not have this node present.

16.1.3.1 Applying set overlay

```
POST /api/device/{target}
{
  "op" : "set:overlay:on",
  "horiz_position" : Integer (conditional),
  "vert_position" : Integer (conditional),
  "scaling_factor" : Integer (optional),
  "fg_color" : String (conditional),
  "bg_color" : String (conditional),
  "fg_transparency" : Integer (conditional),
  "bg_transparency" : Integer (conditional),
  "bmp_hex" : String (optional),
  "invert" : Boolean (conditional)
}

POST /api/device/{target}
...
{
  "op" : "set:overlay:off"
}
```

Two forms of the command are supported with the first being used to enable and configure the bitmap overlay and the second to disable it. A bitmap overlay is supported on the scaled stream (`stream 1`) of an encoder and on the HDMI output of a receiver device.

Reminders:

- This command is only applicable to endpoint devices utilizing BlueRiver NT2000 chipsets and that include the AV Processor Engine.
- Refer to the following sources for information about the BMP file format:

Bitmaps: <https://docs.microsoft.com/en-ca/windows/desktop/gdi/bitmaps>

BMP file format: https://en.wikipedia.org/wiki/BMP_file_format

-
- iii. If the overlay bitmap is generated “on-the-fly”, such as by a text generator, instead of loading from an actual BMP file, the `BITMAPCOREHEADER` variant of the file format is recommended, as it is the simplest to implement.

Arguments

The `target` argument must be either the device ID of a single device or the name of a device group. The supported device groups are as follows:

<code>ALL</code>	All devices
<code>ALL_RX</code>	All decoder/receiver devices
<code>ALL_TX</code>	All encoder/transmitter devices

Note: Many bitmap arguments are optional and if omitted the corresponding parameters remain unchanged. This allows an overlay to be re-enabled without having to fully reconfiguring it.

However, a complete configuration must be applied the first time the overlay is enabled on a given device. The API does not report an error if this requirement is not followed. Important to note, failing to initially apply a complete configuration could lead to undefined behavior.

The `horiz_position` and `vert_position` arguments respectively specify the horizontal and vertical position of the overlay, in pixels. This is the position of the top-left corner of the overlay relative to the top-left corner of the picture. If either is specified then both must be. If omitted, then they are left unchanged from the previous call of this command on the target device.

The `scaling_factor` argument specifies the factor by which the overlay bitmap is to be scaled before being overlaid. This argument can be 1 (no scaling), 2 or 4. Scaling is performed by simple pixel replication. If omitted, the scaling factor remains unchanged.

The `fg_color` and `bg_color` respectively specify the foreground and background colors. Each must be a string with format `RRGGBB` where each of `RR`, `GG` and `BB` is a pair of hexadecimal digits that represent the 8-bit value of the red, green and blue components of the specified color. If either is specified, then both must be. If omitted, these parameters are left unchanged.

The `fg_transparency` and `bg_transparency` respectively specify the level of transparency of the foreground and background of the overlay. Each is an integer between 0 (completely opaque) and 100 (completely transparent). If either is specified, then both must be. If omitted, these parameters are unchanged.

The `bmp_hex` argument (optional) is a monochrome that is a 1-bit per pixel, bitmap file in BMP format encoded as a hexadecimal string. The size of the overlay bitmap is implicit from this argument. If not specified the bitmap remains unchanged, which is useful for disabling/re-enabling or repositioning the overlay without reloading the bitmap.

If the `invert` Boolean argument is set to `true`, the colors are inverted when interpreting the source bitmap file. Example, white becomes the background and black the foreground. Otherwise, default is applied, which is black or the darkest color is the background and white or lightest color is foreground.

Note: In an API request, the `invert` argument is specified when the `bmp_hex` argument is and is not specified if the `bmp_hex` is not.

The picture size and the overlay position (`horiz_position` and `vert_position` arguments), bitmap size and scaling factor (`scaling_factor` argument) must be such that the overlay is at least partially within the picture. In this context, the picture size refers to either:

- The output resolution for a receiver device; or
- The scaler resolution (HDMI stream 1) for a transmitter device.

Refer to the SDVoE Developers API Reference Guide for more information including errors and return value.

Examples

Example of configuring and enabling a set overlay:

```
POST /api/device/d80102030405 HTTP/1.1
...
{
  "op" : "set:overlay:on",
  "horiz_position" : 320,
  "vert_position" : 200,
  "scaling_factor" : 2,
  "fg_color" : "00695B",
  "bg_color" : "000000",
  "fg_transparency" : 0),
  "bg_transparency" : 70,
  "bmp_hex" : "424d621600000000000000820000...ffffffff000000",
  "invert" : false
}
HTTP/1.1 201 Created
Location: /api/request/123
...
{
  "status" : "PROCESSING",
  "request_id" : 123,
  "result" : null,
  "error" : null
}
```

Enabling a previously configured bitmap overlay:

```
POST /api/device/d80102030405 HTTP/1.1
...
{
  "op" : "set:overlay:on"
}
HTTP/1.1 201 Created
Location: /api/request/124
...
{
  "status" : "PROCESSING",
  "request_id" : 124,
```

```

    "result" : null,
    "error" : null
}

```

Disabling an active bitmap overlay:

```

POST /api/device/d80102030405 HTTP/1.1
...
{
    "op" : "set:overlay:off"
}
HTTP/1.1 201 Created
Location: /api/request/125
...
{
    "status" : "PROCESSING",
    "request_id" : 125,
    "result" : null,
    "error" : null
}

```

16.1.3.2 BITMAP_OVERLAY Node

The table below shows the BITMAP_OVERLAY node configuration members.

Table 17: BITMAP_OVERLAY Node Configuration Members

Member Name	Type	Description
background_color	String	Background color in format RRGGBB.
background_transparency	Integer	Background transparency level in percent.
enable	Boolean	Whether bitmap overlay is enabled or disabled.
foreground_color	String	Foreground color in format RRGGBB.
foreground_transparency	Integer	Foreground transparency level in percent.
height	Integer	Height of the overlay bitmap.
horiz_position	Integer	Horizontal position, in pixels, of the overlay
scaling_factor	Integer	Scaling factor (1, 2 or 4)
vert_position	Integer	Vertical position, in pixels, of the overlay
width	Integer	Width of the overlay bitmap.

16.2 Thumbnail Preview

The SDVoE solution supports the ability to stream an unencrypted source as a thumbnail from a BlueRiver NT2000 or NT1000 encoder device (TX) to a host computer.

For details on setup of this feature please refer to section 16.2.2 Thumbnail Configuration & Setup which includes setting up GStreamer which can be used as the pipeline-based multimedia framework to implement thumbnails as well as the configuration and setup of thumbnail streams through the API. Should be noted that the thumbnail streams propagate through the 10GbE port.

Reminder: This feature is available to encoder devices that include either a BlueRiver NT1000 or NT2000 chipset only, it is not available to devices based on the AVP chipset series.

16.2.1 Important Customer Notice

It is your responsibility to comply with all laws, rules, and regulations applicable to the operation of this product, including the processing and distribution of electronic content and, if you use Semtech's thumbnail preview feature, the display of such content in a thumbnail format viewable on a monitor.

You are responsible for entering into, performing, and obtaining any necessary authorizations with content providers whose electronic content you opt to process, distribute or display. All interaction between you and third parties providing electronic content or other third-party products or services is between you and the third party. Semtech assumes no liability for any of your activities involving your use of this product for the processing, distribution or display of third-party electronic content. IN NO EVENT SHALL SEMTECH BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES IN CONNECTION WITH OR ARISING OUT OF THE USE OF THIS PRODUCT REGARDLESS OF WHETHER SEMTECH WAS ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Rev. 042019

16.2.2 Thumbnail Configuration & Setup

This section reviews the installation and demonstration of the thumbnail preview feature.

Note: This document uses GStreamer, so it is necessary to install this tool prior to configuring a preview stream. A different tool can be used in place of GStreamer, in such cases refer to the appropriate resources for the configuration of the tool.

The following characteristics apply to the Thumbnail Preview feature:

- Thumbnail streams:
 - Are an RTP stream that can be received using open-source multimedia frameworks, such as GStreamer.
 - The RTP payload type is fixed to 112 uncompressed video.
 - The RTP synchronization source (SSRC) identifier is configurable and must be provided by the user.
 - Always YCbCr 4:2:2 8-bit uncompressed video.
 - Always started through the API server.
 - Contains video only.
 - Are not encrypted.
- Thumbnails must be enabled in the product configuration, done during production by the hardware manufacturer.
- The width applied is the largest width appropriate to the maximum supported width of 320 pixels; the original picture width is always divided evenly.
- Frame rate per second (fps) control:
 - FPS is controllable and can range from 0.5 fps to full source frame rate.
 - The maximum frame rate of the thumbnail stream is the original frame rate of the video source.

- Thumbnail streams can be sent to a unicast IP address, as well the UDP port is configurable.
 - If left unspecified, the default port applied is 6972.
- For thumbnail streams sent to a multicast IP address, as well the UDP port is also configurable.
 - If left unspecified, the default port applied is 6972.

Notes:

- i. If the original video source is HDCP-encrypted a thumbnail stream of the video source is available. However, important to note to enable this type of source an API `start` request is required to be issued to the appropriate encoder(s), specifying the stream type as `THUMBNAIL`. Refer to the 16.2.3.3 HDCP Content for more information.
- ii. Also, refer to section 16.2.1 Important Customer Notice to review the important customer notice.

16.2.3 GStreamer Installation & Configuration

This section reviews the configuration of GStreamer in relation to the thumbnail preview streams on a 64-bit Windows host computer.

Note: GStreamer is a pipeline-based multimedia framework that links together a wide variety of media processing systems to complete complex workflows.

Download and install GStreamer to the client computer that will be outputting the thumbnails. Select the typical installation:

`gststreamer-1.0-mingw-x86_64-XXX.msi`

GStreamer can be downloaded from the following link:

<https://gststreamer.freedesktop.org/data/pkg/windows/>

1. Install GStreamer as Typical installation, default settings are acceptable for purpose of demonstrating thumbnail previews.
2. Once the installation is complete, it is necessary to add GStreamer executable binaries directory to the System variables "Path" in Environment Variables of the host Windows computer:

`C:\gststreamer\1.0\x86_64\bin\`

Refer to the figure below for example.

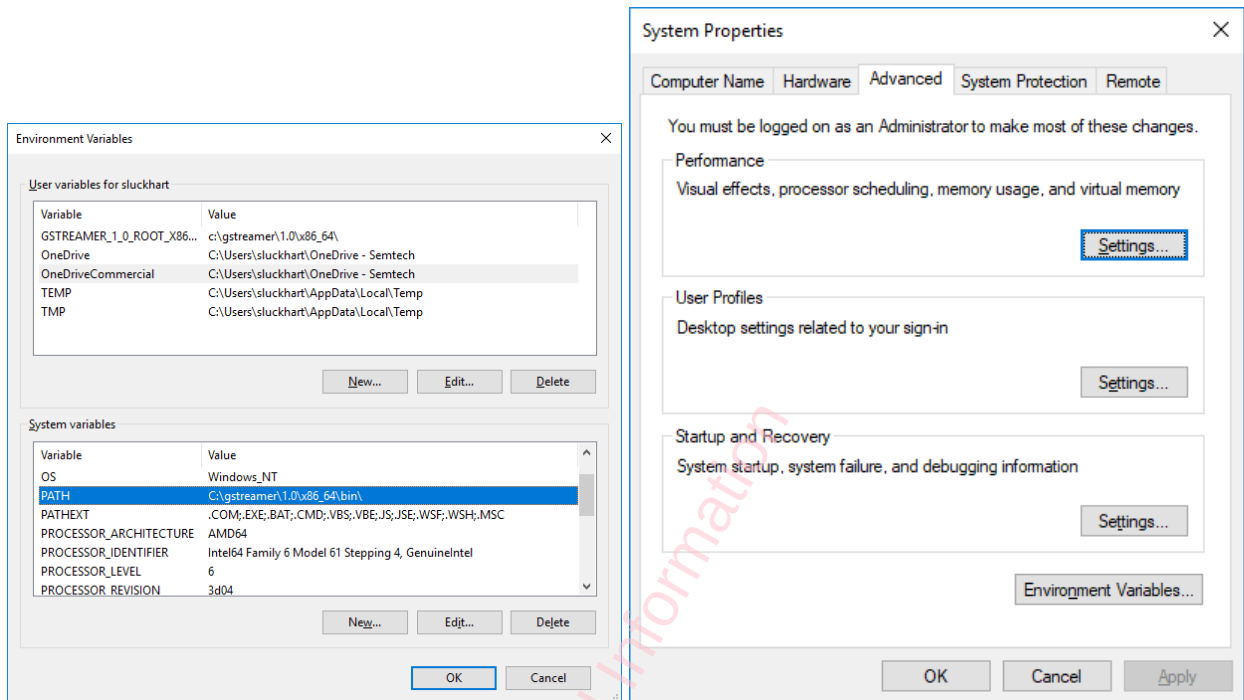


Figure 74: GStreamer System Variable

3. Once the installation is completed and the variable is configured, verify that the `gst-launch-1.0.exe` executable can be launched from any directory location using a Windows command line.

To verify the installation:

- a. Open a Windows command line.
- b. At the prompt type:
`gst-launch-1.0.exe -help`
- c. The `gststreamer` help should then be displayed.

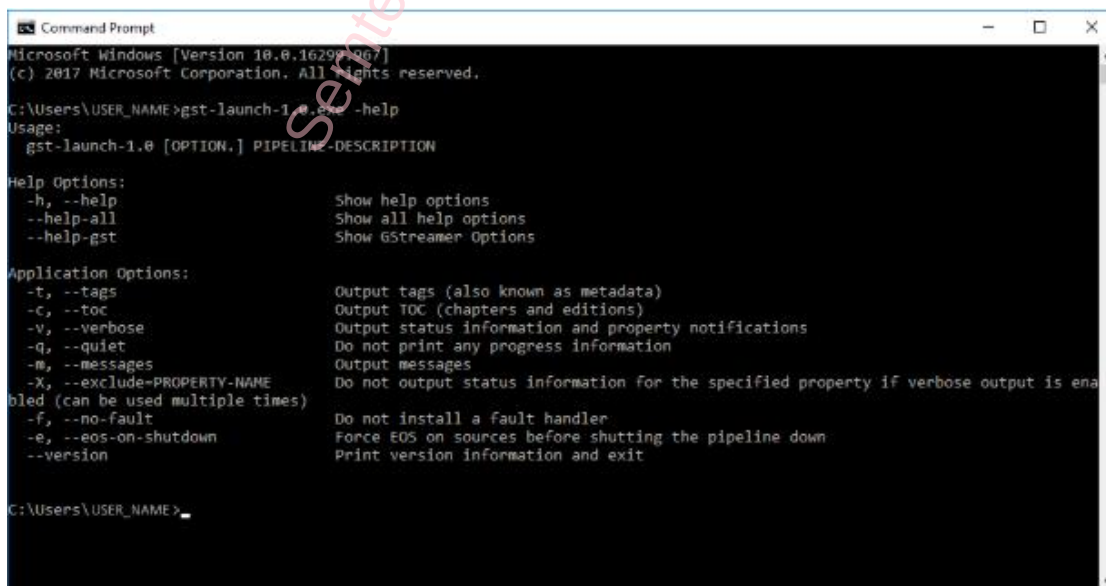


Figure 75: Example of Verifying GStreamer

Helpful Tips:

- i. For information on installing GStreamer on an alternate Operating System than Windows, refer to the web page provided below to be directed to the GStreamer official web site:
<https://gstreamer.freedesktop.org/documentation/installing/index.html>
- ii. If it is experienced that a thumbnail video is not received when an IGMP request (version 3) is sent from a client computer to a Netgear M4300 8X8F network switch, this issue may be resolved by activating the IGMP Querier on the VLAN.

16.2.3.1 Thumbnail Stream Video Resolutions

The maximum image width size of 320-pixels is used to determine the thumbnail image size. The Video Aspect ratio is always respected. The table below shows supported stream resolutions and the output size for the thumbnail streams.

Table 18: Thumbnail Stream Resolutions and Thumbnail Resolution Output

Video Width	Video Height	Interlaced	Output H Size	Output V Size
SD				
1440	480	X	180	120
720	480		240	160
1440	576	X	180	144
720	576		240	192
HD				
1280	720		320	180
1920	1080	X	320	180
1920	1080		320	180
4K				
3840	2160		320	180
4096	2160		292	154
Other				
640	480		320	240
800	600		266	200
1024	768		256	192
1280	800		320	200
1280	960		320	240
1280	1024		320	256
1360	768		272	153
1366	768		273	153
1400	1050		280	210
1440	900		288	180
1600	900		320	180
1600	1200		320	240
1680	1050		280	175
1920	1200		320	200

16.2.3.2 Configuring Thumbnail Generator

A `set_thumbnail` API request is used to configure the thumbnail preview stream of an encoder (TX) endpoint device(s).

Notes:

- i. The thumbnail generator must be configured before the preview can be streamed.
- ii. To enable HDCP content an API `start` request is used, this is covered below in section 16.2.3.3 HDCP Content. To stop streaming HDCP content, a `stop` request is used,.
- iii. Reminder, the thumbnail generator is applicable only to BlueRiver NT1000 and NT2000 encoder devices. It is not applicable to the AVP chipset series (AVP1000, AVP2000 or AVP2000T).

API request to configure the thumbnail generator of an encoder (TX):

```
POST /api/device/{target}
{
  "op" : "set:thumbnail",
  "fps" : Real or String,
  "ssrc" : Integer,
  "udp" : Integer (optional)
}
```

Arguments

The `target` argument must be either the device ID of a single encoder (TX) or the device group, `ALL_TX` to apply to all encoder (TX) devices.

The `fps` argument specifies the frame rate that the thumbnail stream is to playout at in frames per second.

Note: Important to note that the maximum frame rate that can be played out, is equal to the source's frame rate. For example, if the source frame rate is 60 fps then the thumbnail can playout at a maximum of 60 fps. If 5 fps is set, then thumbnail will playout successfully at 5 fps. However, if the source is 60 fps and 120 fps was specified the thumbnail stream will playout at 60 fps equaling the source's maximum frame rate.

The `ssrc` argument is a 32-bit integer (range 0–4294967295) representing the RTP synchronization source (SSRC) identifier. This value must be unique within an RTP session. Refer to the RFC-3550 documentation, section 8 for details.

The `port` argument is the destination UDP port number. If this argument is omitted, the default port number is used.

Examples

```
POST /api/device/d88001020405 HTTP/1.1
...
{
  "op" : "set:thumbnail",
  "fps" : 0.5,
  "ssrc" : 12236,
  "udp" : 20000
}
```

```
HTTP/1.1 201 Created
Location: /api/request/13133
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 13133,
  "result" : null,
  "error" : null
}
```

```
POST /api/device/d88001020405 HTTP/1.1
```

```
...
```

```
{
  "op" : "set:thumbnail",
  "fps" : "60m",
  "ssrc" : 12236
}
```

```
HTTP/1.1 201 Created
Location: /api/request/13134
```

```
...
```

```
{
  "status" : "PROCESSING",
  "request_id" : 13134,
  "result" : null,
  "error" : null
}
```

16.2.3.3 HDCP Content

To enable the streaming of HDCP content, once the thumbnail generator is configured it is necessary to send an API `start` request to enable streaming of HDCP content. To discontinue the transmission of HDCP content a `stop` request is used. By default, thumbnail support for HDCP sources is disabled.

Warning! Prior to enabling HDCP content to stream, refer to section 16.2.1 Important Customer Notice for a critical legal notification.

For detailed information on `start` and `stop` requests related to other stream types, refer to the SDVoE Developers API Reference Guide.

Enabling HDCP Thumbnail Stream

To enable the streaming of HDCP content, an API `start` request is used.

```
POST /api/device/{target}

{
  "op" : "start",
  "stream_type" : String,
  "stream_index" : Integer,
  "dest_address" : String (optional),
  "hdcp_thumbnail" : Boolean (optional) (NT1000/NT2000 only),
}
```

In relation to thumbnail preview streams, a `start` request modifies the behavior of a `THUMBNAIL` stream in regards to HDCP sources. If a `start` request is issued then no video is streamed when the source is HDCP-protected.

Reminder: Prior to starting a thumbnail preview stream, the thumbnail generator must have been configured.

Arguments

The `target` argument must be either the device ID of a single encoder (TX) or the device group, `ALL_TX` to apply to all encoder (TX) devices. Regarding thumbnail preview streams, the stream type is `THUMBNAIL` and the `stream_index` is 0.

The `ip_address` argument specifies the IP address to be used. It can be either a unicast address or a multicast address within the API stream address range. If not specified, the API automatically assigns the next available multicast address.

For the `hdcp_thumbnail` the option is a Boolean entry of either `TRUE` to enable the HDCP content or `FALSE` to disable passing HDCP content. If this argument is not specified then `FALSE` is the default and HDCP content will not be streamed.

Disabling HDCP Content from Streaming with Thumbnail Stream

Regarding thumbnail streams, to discontinue streaming HDCP content, an API `stop` request is used.

```
POST /api/device/{target}
{
  "op" : "stop",
  "stream_type" : String (optional),
  "stream_index" : Integer (optional),
  "free" : Boolean
}
```

Arguments

The `target` argument must be either the device ID of a single encoder (TX) or the device group, `ALL_TX` to apply to all encoder (TX) devices. In regards to a thumbnail preview stream the stream type entered is `THUMBNAIL` and the `stream_index` is always 0.

If the `free` argument is `true` then the multicast IP address associated with a `THUMBNAIL` stream is freed/unassigned. If the `free` keyword is entered but the stream is a unicast address, not a multicast address, the argument is allowed but ignored.

16.2.3.4 Thumbnail Generator Node

The node object with the type member set to `THUMBNAIL_GENERATOR` provides details of thumbnail configuration settings. Applicable to devices containing a BlueRiver NT1000 or NT2000 chipsets only.

Table 19: Thumbnail Generator Node Object Members

Name	Type	Configuration Member Description
<code>frame_rate</code>	Real	The requested frame rate for the thumbnail stream.
<code>ssrc</code>	Integer	RTP synchronization source (SSRC) identifier. This 32-bit value must be unique for each thumbnail source. Refer to RFC-3550, section 8 for details.
<code>udp_port</code>	Integer	Destination UDP port.

The Thumbnail Generator node has the following status members:

```
{
  "width" : Integer,
  "height" : Integer,
  "frame_rate" : Real,
  "bits_per_comp" : Integer,
  "color_space" : String,
  "colorimetry" : String
}
```

Table 20: Thumbnail Generator Node Status Members

Name	Type	Status Member Description
width	Integer	Width of each thumbnail.
height	Integer	Height of each thumbnail.
frame_rate	Real	Actual frame rate for the thumbnail stream.
bits_per_comp	Integer	Value 8 indicates that there are 8-bits per color component.
color_space	String	String YCBCR_422 indicates thumbnail stream is in the YCbCr color space with 4:2:2 Chroma subsampling.
colorimetry	String	The colorimetry of thumbnail stream. Will be one of the following: BT709 for BT.709 BT601 for BT.601

The example shown below shows a thumbnail generator node JSON object as returned as part of the output of a “get device” request for an NT2000 endpoint device.

```
{
  "type" : "THUMBNAIL_GENERATOR",
  "index" : 0,
  "configuration" : {
    "frame_rate" : 10.00,
    "hdcp" : false,
    "ssrc" : 1234,
    "udp_port" : 6972
  },
  "status" : {
    "width" : 320,
    "height" : 180,
    "frame_rate" : 10.00,
    "bits_per_comp" : 8,
    "color_space" : "YCBCR_422",
    "colorimetry" : "BT601"
  },
  "inputs" : []
}
```

16.2.4 Viewing Thumbnail Preview Streams

It is relatively simple to view the generated thumbnail with GStreamer, following are some examples of generating thumbnails. They are executed in a Windows command line.

16.2.4.1 Unicast Thumbnail

```
gst-launch-1.0 -v udpsrc port=<UDP port number> ! "application/x-rtp,
media=(string)video, clock-rate=(int)90000, encoding-name=(string)RAW,
sampling=(string)YCbCr-4:2:2, depth=(string)8,
width=(string)<thumbnail h size>, height=(string)<thumbnail v size>,
ssrc=(uint)<RTP SSRC id>, payload=(int)112" ! rtpvrawdepay ! queue !
videoconvert ! "video/x-raw,colorimetry=<thumbnail colorimetry value>"
! autovideosink sync=true
```

16.2.4.2 Multicast Thumbnail

```
gst-launch-1.0 -v udpsrc multicast-iface="<Ethernet Interface name>"
multicast-group="<Multicast IP address>" port=<UDP port number> !
"application/x-rtp, media=(string)video, clock-rate=(int)90000,
encoding-name=(string)RAW, sampling=(string)YCbCr-4:2:2,
depth=(string)8, width=(string)<thumbnail h size>,
height=(string)<thumbnail v size>, ssrc=(uint)<RTP SSRC id>,
payload=(int)112" ! rtpvrawdepay ! queue ! videoconvert ! "video/x-
raw,colorimetry=<thumbnail colorimetry value>bt709" ! autovideosink
sync=true
```

16.2.4.3 Video Scaling

```
gst-launch-1.0 -v udpsrc port=<UDP port number> ! "application/x-rtp,
media=(string)video, clock-rate=(int)90000, encoding-name=(string)RAW,
sampling=(string)YCbCr-4:2:2, depth=(string)8,
width=(string)<thumbnail h size>, height=(string)<thumbnail v size>,
ssrc=(uint)<RTP SSRC id>, payload=(int)112" ! rtpvrawdepay ! queue !
videoconvert ! videoscale ! video/x-raw,width=<Output video h
size>,height=<Output video v size> ! autovideosink sync=true
```

16.2.5 Thumbnail Script & Executable Demo

For demonstration purposes, Semtech BlueRiver support can provide a python script and/or executable to make it easy to start a thumbnail stream. Following section provides instructions and guidelines on the use of these tools.

Note: GStreamer needs to be properly installed before using this demo code. Refer to section 16.2.3 GStreamer Installation & Configuration for more information.

16.2.5.1 Using Thumbnail Script & Executable

Command Syntax:

```
thumbnail.exe DEVICE SSRC FPS [-udp_port UDP_PORT] [-api API] [-ip_address IP_ADDRESS] [-text TEXT] [-interface INTERFACE] [-hdcpc]
```

Arguments

The `DEVICE` argument (required) must be the device ID of a single NT1000 or NT2000 encoder (TX).

The `SSRC` argument (required) is a 32-bit integer (range 0–4294967295) that represents the RTP synchronization source (SSRC) identifier. This value must be unique within an RTP session. Refer to RFC-3550, section 8 for details.

The `FPS` argument (required) specifies the maximum frame rate of the thumbnail stream in frames per second.

The `UDP_PORT` argument (optional) is the destination UDP port number. If this argument is omitted, the default port number 6972 is used.

The `API` argument (optional) is the IP address of the machine hosting the API server. If this address is not specified, the loopback IP address 127.0.0.1 is used.

The `IP_ADDRESS` argument (optional) is used to specify a destination IP address to the thumbnail stream. This can be a unicast IP address or an unused multicast address within the range managed by the API server.

The `TEXT` argument (optional) is used to add a text overlay on the GStreamer window.

The `INTERFACE` argument (optional) can be used to specify the Ethernet interface where the IGMP request will be sent. If this argument is not set, IGMP requests will be sent to all available Ethernet ports of the client computer.

The `HDCP` argument (optional) must be used to enable a thumbnail from HDCP video. If it is not specified, then a thumbnail is only sent if the video source is not using HDCP encryption.



Figure 76: Example of Thumbnail Preview

16.2.5.2 Guidelines & Limitations

The purpose of the thumbnail preview demo script is to start thumbnail viewing on an NT1000 or NT2000 encoder (TX) and client computer. Below outlines guidelines and limitations to keep in mind when using the demo script:

- No video encryption or compression is done.
- Video needs to be present and valid on the encoder (TX) device before executing the demo script. The script needs to retrieve the thumbnail stream video dimensions prior to being able to start the GStreamer with the proper window size.
- Once the stream is started, there is no dynamic management of this stream. The script does not monitor for changes in the Thumbnail stream.
- The GStreamer window is kept opened with the same size even if the stream is stopped on the encoder (TX) device.
- The GStreamer window needs to be manually closed. Once the window is closed, the demo script will automatically send the IGMP leave message in case a multicast address was used as a destination address of the thumbnail script.
- The GStreamer window size will remain the same even if the input video size on the encoder (TX) device is changed.
 - In the case of a smaller image size being implemented, a "ghost" image could be present if extra pixels are displayed.
 - For a larger image size, some pixels could be missing, resulting in only a portion of the stream be shown in the GStreamer window.
- Not able to be shown by VLC. The library version used by VLC to read streaming video does not support RTP RAW videos data.

16.2.6 Managing Format Changes

When a video format is changed the following recommendations apply.

Reminder: Thumbnail streams are sent using RTP-4175 for uncompressed video.

TIP: For more information, the following link can be referred to:

<https://tools.ietf.org/html/rfc4175>

In the RTP-4175 standard, only active video is sent. Metadata, such as video dimensions and frame rate, need to be transported separately. All data related to thumbnails is handled by the API.

In the event of a video source size or frame rate change, the encoder (TX) will automatically resize the thumbnail according to the input video source statuses. These stream status changes are automatically reported by the API.

To be more accurate on these changes, it is recommended to register the thumbnail management to the API event change. As soon as a status changes on a device, the manager will be notified and an action can be quickly executed to change the thumbnail viewing window size. Since image dimensions are not embedded in RTP-4175 packets for uncompressed video, the RTP receiver on viewer side needs to be updated with the new video settings.

16.2.6.1 General Management

The flowchart below shows the general flow when a thumbnail resolution change occurs:

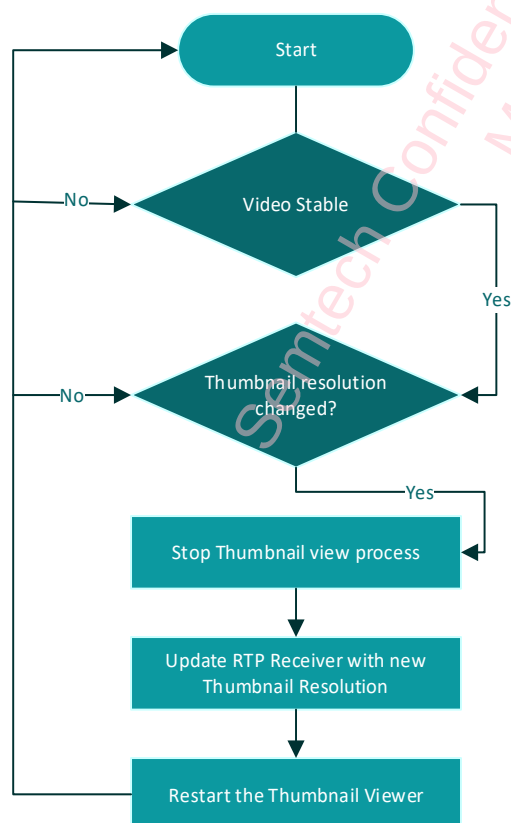


Figure 77: Thumbnail Resolution Change

Video Stable

- Video Stable check can be done by reading the `source_stable` in the `status` section of the `HDMI_DECODER` node in the API. This status value will be "true" or "false".

Thumbnail Resolution Changed

- Before reading Thumbnail resolution, make sure that the `THUMBNAIL` stream status is "STREAMING" and not "STOPPED" or "LINK_DOWN".
- If the thumbnail is streaming, read the "width" and "height" statuses of the `THUMBNAIL_GENERATOR` node from the API.
- If the resolution has changed, execute the next steps, otherwise return to Video Stable check.

Stop Thumbnail View Process

- In the case of a thumbnail resolution change, stop the viewing process.
- Update RTP Receiver with New Thumbnail resolution
- Update the RTP receiver with the appropriate RAW video resolution from API `THUMBNAIL_GENERATOR` statuses.

Restart Thumbnail Viewer

- Restart the thumbnail viewer with new configurations.

16.2.6.2 Using GStreamer as Viewer

If GStreamer is used as the thumbnail viewer, then the steps outlined below apply to execute the management of a thumbnail video size change.

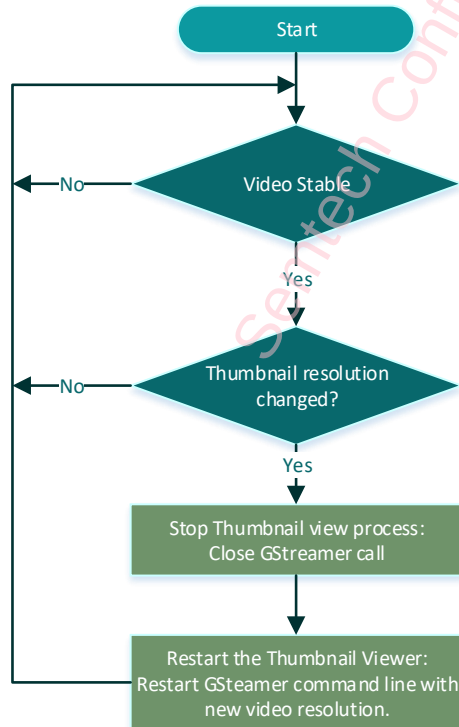


Figure 78: Video Size Changes with GStreamer

Video Stable

- *Video Stable* check can be done by reading the `source_stable` in status of `HDMI_DECODER` node in API. This status value will be "true" or "false".

Thumbnail Resolution Changed

- Before reading Thumbnail resolution, make sure that the `THUMBNAIL` stream status is "STREAMING" and not "STOPPED" or "LINK_DOWN".
- If the thumbnail is streaming, read the "width" and "height" statuses of the `THUMBNAIL_GENERATOR` node from the API.
- If the resolution has changed, execute the next steps, otherwise return to Video Stable check.

Stop Thumbnail View Process

- In the case of a thumbnail resolution change, stop the viewing process. When using GStreamer, close the actual GStreamer console call.

Restart Thumbnail Viewer

- Restart the thumbnail viewer with new configurations. In the case of using GStreamer, call the console command line with new resolution information.

16.2.6.3 Compile Custom GStreamer Pipeline

Instead of using the pre-compiled GStreamer version, it is possible to compile your own GStreamer pipeline using custom code. This provides the opportunity to quickly change the viewing window, set the location where the viewing window will be displayed, as well as customize other details regarding the thumbnail preview.

Details on how to use and compile GStreamer is outside the scope of this document. For more Information on GStreamer and its functionality refer to the link provided here:

<https://gstreamer.freedesktop.org/documentation/tutorials/index.html>

17 Revision History

Version	Date	Revision
2.9	December 2025	Removed USB Icron syntax support - Removed <code>usb_icron</code> node, <code>icron</code> command, <code>usb_icron</code> stream_type, <code>usb_icron</code> subscription_type - Affects capabilities, commands pair, unpair, switch, stop, capabilities, netstat, set property, etc. - See PDS-63049 SDVoE_API_Icron_to_USB_Extender_Migration_Guide. Added support of a second UEX USB 2.0 extender attached to an AVP - Affects <code>get capabilities / identity / usb_tree</code>, <code>USB_EXTENDER</code> node, <code>pair / unpair</code>, <code>set ip mode / property</code>, <code>claim / release</code>, <code>factory</code>, <code>reboot</code>.
2.8	March 2025	Swapped sections 3 Managing the API Server and 4 Managing BlueRiver Device Bandwidth.
2.7	April 2024	Section 13 Managing BlueRiver Device's 802.1X added outlining details of the support for 802.1x certificates. Section 6 BlueRiver Device Multiview Mode added details about the possibility for a decoder (RX) device to scale a multiview layout to a different output resolution than the size of the layout. Section 8.2.1 Resetting Device Name to MAC Address corrected the set property command to remove the <code>streams[HDMI : 0]</code> and only keep <code>configuration.device_name</code> as the set property key.
2.6	September 2023	Section 3.1 Detection of API Server added. Section 5.3.2 added to add the new Quantization Level Override option. Error 37 Fall back to golden firmware success message for the 10GBASE-T PHY added to Table 15: Error Codes. In addition, section 9 BlueRiver Device Audio Routing rewritten, adding details in relation to managing audio routing.
2.5	March 2022	Added section 14 BlueRiver Error Codes outlining the error codes that endpoint devices may report to the API. Icron pairing tip added as well regarding potential errors. The set property option, <code>nodes[HDMI_ENCODER:0].configuration.hdmi_tx_5v</code> added to section use cases in section 8.8 BlueRiver Receiver (HDMI TX 5V) API Control.
2.4	February 2023	Added set property device name example to the section Set Property Command – Example Use Cases.
2.3	November 2022	Clarifications made to Quantization, specifically related to section 5.2.3 Quantization in Multiview Mode.
2.2	May 2022	API version 3.4 released. The <code>get multiview</code> request and <code>MULTIVIEW_SETTINGS</code> event added in new section Retrieving Details of Layout Currently Loaded on Decoder (RX) Device. Also, the Icron SUI feature added to the Icron API Integration in section, specifically Simultaneous User Interaction (SUI) Pairing.
2.1	January 2022	Updated the security section, section "Overview of Secure Control Communication" added.

Version	Date	Revision
2.0	November 2021	Audio Routing section added, detailing information related to routing audio through BlueRiver chipsets, including use cases illustrating the use of the SDVoE API in relation to routing audio. The Multiview limitations and guidelines also updated regarding the AVP2000T transceiver loop back and framerate divider use.
1.1	October 2021	Icron API Integration section added.
1.0	July 2021	Initial release of the SDVoE API Application Note Library.

Semtech Confidential Information
Mark Li
ADV
02/03/2026

IMPORTANT NOTICE

Information relating to this product and the application or design described herein is believed to be reliable, however such information is provided as a guide only. Semtech and SDVoE Alliance (collectively, "Parties") assume no liability for any errors in this document, or for the application or design described herein. The Parties reserve the right to make changes to their respective products or this document at any time without notice. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. The Parties warrant performance of their respective products to the specifications applicable at the time of sale, and all sales are made in accordance with their respective standard terms and conditions of sale.

NEITHER PARTY'S PRODUCTS ARE DESIGNED, INTENDED, AUTHORIZED OR WARRANTED TO BE SUITABLE FOR USE IN LIFE-SUPPORT APPLICATIONS, DEVICES OR SYSTEMS, OR IN NUCLEAR APPLICATIONS IN WHICH THE FAILURE COULD BE REASONABLY EXPECTED TO RESULT IN PERSONAL INJURY, LOSS OF LIFE OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE. INCLUSION OF EITHER PARTY'S PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE UNDERTAKEN SOLELY AT THE CUSTOMER'S OWN RISK. Should a customer purchase or use either Party's products for any such unauthorized application, the customer shall indemnify and hold such Party and their officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs damages and attorney fees which could arise.

Semtech, the Semtech logo and BlueRiver are registered trademarks of Semtech Corporation or its subsidiaries. The SDVoE Alliance name is a registered trademark of SDVoE Alliance. All other trademarks and trade names mentioned may be marks and names of Semtech, SDVoE Alliance or their respective licensors. The Parties reserve the right to make changes to, or discontinue any products described in this document without further notice. The Parties make no warranty, representation, or guarantee, express or implied, regarding the suitability of their respective products for any particular purpose.

© Semtech 2025. All rights reserved.

For more information contact: api@sdvoe.org